



ZERO A MONERO: SEGUNDA EDIÇÃO

UM GUIA TÉCNICO PARA UMA MOEDA DIGITAL PRIVADA;
PARA PRINCIPIANTES, AMADORES E ESPECIALISTAS

TRADUÇÃO PORTUGUESA EM REVISÃO, 22 DE AGOSTO DE
2026

KOE¹, KURT M. ALONSO², SARANG NOETHER³

TRADUÇÃO DE `slave_blocker`⁴

Licença: *Zero a Monero: Segunda Edição* é dedicada ao domínio público.

¹ ukoe@protonmail.com

² kurt@oktav.se

³ sarang.noether@protonmail.com

⁴ dollner@gmx.de

Resumo

Criptografia. A palavra pode sugerir uma matéria reservada a matemáticos e cientistas da computação: obscura, poderosa e, para quem lhe conhece as peças, elegante. Na verdade, muitos dos seus conceitos fundamentais estão ao alcance de qualquer leitor disposto a acompanhar o raciocínio passo a passo.

Uma cripto-moeda usa criptografia para definir e transferir a posse de fundos. Para impedir que a mesma moeda seja gasta duas vezes ou que novas moedas sejam criadas arbitrariamente, as transacções são registadas numa lista de blocos (*block-chain*): um livro de contas público e distribuído que terceiros podem verificar [89]. À primeira vista, essa verificabilidade parece exigir que remetentes, destinatários e montantes fiquem expostos. Monero mostra que não. É possível ocultar esses dados e, ainda assim, permitir que a rede confirme a validade das transacções [124].

Para os leitores experientes: a rede principal usa criptografia sobre a curva Edwards25519 [34], endereços de utilização única, assinaturas em anel CLSAG, compromissos de Pedersen e provas de intervalo Bulletproof+. Cada anel tem 16 membros e as saídas incluem etiquetas de vista. RandomX protege a prova de trabalho; Dandelion++ intervém na difusão inicial de transacções, não nas regras de consenso. Este livro constrói a intuição e a notação necessárias antes de reunir estas peças.

A segunda edição original descreve o protocolo v12. A rede principal actual de Monero usa o protocolo v16, estado conferido na fotografia do código-fonte datada de 14 de Agosto de 2026 [88]. A edição portuguesa está a ser revista progressivamente para esse estado; por isso, os capítulos de protocolo que ainda não tenham sido actualizados devem ser lidos como uma descrição histórica da v12.

Conteúdo

1	Introdução	1
1.1	Objectivos	2
1.2	Âmbito técnico desta edição	2
1.3	Aos leitores	3
1.4	Origens da cripto-moeda Monero	4
1.4.1	Parte 1: “Essenciais”	4
1.4.2	Parte 2: “Extensões”	4
1.4.3	Conteúdo adicional	4
1.5	Aviso	5
1.6	A história de Zero a Monero	5
1.7	Reconhecimentos	6
I	Essenciais	7
2	Conceitos Básicos	8
2.1	Algumas palavras sobre a notação	8
2.2	Aritmética modular	9

2.2.1	Adição e multiplicação modular	10
2.2.2	Exponenciação modular	11
2.2.3	Inversa multiplicativa modular	11
2.2.4	Equações modulares	12
2.3	Criptografia de curva elíptica	13
2.3.1	O que são curvas elípticas?	13
2.3.2	Criptografia de chave pública com curvas elípticas	15
2.3.3	A troca de chaves tipo Diffie-Hellman com curvas elípticas	15
2.3.4	Assinaturas tipo Schnorr e a transformada Fiat-Shamir	16
2.3.5	Assinar mensagens	19
2.4	Curva Edwards25519	21
2.4.1	Representação binária	22
2.4.2	Compressão de ponto elíptico	22
2.4.3	Algoritmo de assinatura EdDSA	23
2.5	Operador binário XOR	25
3	Assinaturas tipo Schnorr avançadas	27
3.1	Igualdade do logaritmo discreto em várias bases	27
3.2	Uma prova com múltiplas chaves privadas	29
3.3	Assinaturas espontâneas anónimas de grupo	30
3.4	Assinaturas em anel ligáveis de Adam Back	33
3.5	Assinaturas em anel ligáveis com múltiplas camadas	36
3.6	Assinaturas em anel ligáveis concisas	39
4	Endereços	44
4.1	Chaves	44
4.2	Endereços ocultos	45
4.2.1	Transacções de múltiplas saídas	48
4.3	Sub-endereços	48
4.3.1	Enviar para um sub-endereço	50
4.4	Endereços integrados	51
4.5	Endereços de multi-assinatura	52

5	Montantes ocultos	54
5.1	Compromissos	54
5.2	Compromissos de Pedersen	55
5.3	Montantes ocultos	56
5.4	Introdução a RingCT	57
5.5	Provas de intervalo	58
5.6	Assinaturas em anel Borromean	58
5.7	Bulletproofs	59
5.7.1	Verificação	63
5.8	Bulletproof+	70
5.8.1	O que fica e o que muda	71
5.8.2	Dos tipos históricos ao Bulletproof+	71
5.8.3	Bulletproof+ na versão 16	72
5.8.4	Verificação	75
6	Transacções Confidenciais em Anel (RingCT)	77
6.1	O caminho activo: RCTTypeBulletproofPlus	78
6.1.1	Compromissos a montantes e taxas de transacção	79
6.1.2	O que nasce em cada saída	79
6.1.3	Uma prova Bulletproofs+ para todas as saídas	80
6.1.4	Um anel: uma entrada real e quinze desvios	81
6.1.5	Assinatura	81
6.1.6	A mensagem que as CLSAG assinam	82
6.1.7	Verificação	83
6.1.8	Evitar o gasto duplo	83
6.1.9	Maturação e tempo de desbloqueio	84
6.1.10	o segredo público γ	84
6.2	Resumo conceptual	88
6.2.1	Requisitos de espaço	89

7	A Blockchain	91
7.1	Moedas digitais	92
7.1.1	Versão de eventos comuns e distribuídos	92
7.1.2	Lista de blocos simples	93
7.2	Dificuldade	94
7.2.1	Mineração de um bloco	94
7.2.2	Velocidade de mineração	94
7.2.3	Consenso: maior dificuldade cumulativa	95
7.2.4	Minerar em Monero	96
7.3	Quantidade monetária	98
7.3.1	Recompensa de bloco	98
7.3.2	Peso dinâmico de bloco	99
7.3.3	Recompensa de bloco com multa	101
7.3.4	Taxas dinâmicas: política, não consenso	102
7.3.5	Cauda de emissão	104
7.3.6	Transacção de mineiro: <code>RCTTypeNull</code>	105
7.4	A estrutura da lista de blocos	106
7.4.1	ID de transacção	106
7.4.2	Árvore de Merkle	107
7.4.3	Blocos	109
II	Extensões	111
8	Provas de conhecimento de transacções	112
8.1	Provas de transacção em Monero	112
8.1.1	Provas de transacção com múltiplas bases	113
8.1.2	Provar a criação de uma entrada de transacção (<code>SpendProofV1</code>)	113
8.1.3	Provar a criação de uma saída de transacção (<code>OutProofV2</code>)	115
8.1.4	Provar a posse de uma saída (<code>InProofV2</code>)	117

8.1.5	Prova de não-gasto de uma saída numa transacção	118
8.1.6	Provar que um endereço tem um saldo mínimo não gasto (ReserveProofV2)	120
8.2	Estrutura de auditoria	121
8.2.1	Provar que um sub-endereço corresponde a um endereço	122
8.2.2	a estrutura de auditoria	122
9	Multi-assinaturas	124
9.1	Comunicação entre co-signatários	125
9.2	Agregação de chaves para endereços	125
9.2.1	Abordagem ingénua	125
9.2.2	Desvantagens da abordagem ingénua	126
9.2.3	Agregação de chave robusta	128
9.3	Assinaturas tipo Schnorr com limite	129
9.4	MLSTAG assinaturas confidenciais em anel de Monero	131
9.4.1	RCTTypeBulletproof2 com multi-assinaturas N-de-N	132
9.4.2	Comunicação simplificada	134
9.5	Recalcular imagens de chaves	136
9.6	$M < N$	137
9.6.1	Agregação de chave 1-de-N	138
9.6.2	Agregação de chave (N-1)-de-N	138
9.6.3	Agregação de chave M-de-N	141
9.7	Famílias de chaves	143
9.7.1	Árvores de família	144
9.7.2	Chaves de multi-assinatura inclusivas	145
9.7.3	Implicações	146
10	Mercados garantidos por terceiros	148
10.1	Características essenciais	149
10.1.1	Sequência de passos na compra	149
10.2	Multi-assinatura Monero	149
10.2.1	Os básicos de uma interacção de multi-assinatura	150
10.2.2	Experiência com garantia para o utilizador	152

11 Transacções juntas (TxTangle)	155
11.1 Construir transacções juntas	156
11.1.1 Canal de comunicação em grupo	156
11.1.2 Rondas de mensagens para construir uma transacção junta	157
11.1.3 Fraquezas	159
11.2 Servidor TxTangle	160
11.2.1 Comunicação básica com um servidor sobre I2P, e outras características . .	160
11.2.2 Um anfitrião como um serviço	162
11.3 Usar um administrador á confiança	163
11.3.1 Processo baseado num administrador	163
Bibliografia	165
Appendices	172
A Espécime histórico: estrutura de transacção RCTTypeBulletproof2	174
B Conteúdo de bloco	180
C Bloco de génese	183
D Uma nota sobre notações	186

CAPÍTULO 1

Introdução

No mundo digital, copiar informação é trivial; alterá-la também. Para que uma moeda digital seja aceite, os seus utilizadores precisam de confiar em duas coisas: a quantidade total de moeda obedece às regras anunciadas e cada unidade só pode ser gasta uma vez. Quem recebe um pagamento tem, portanto, de poder verificar que as moedas não são contrafeitas nem foram já entregues a outra pessoa. Se não quisermos confiar essa tarefa a uma autoridade central, a oferta monetária e a história completa das transacções têm de ser verificáveis pelo público.

As cripto-moedas registam transacções numa lista de blocos (*blockchain*), uma base de dados distribuída cuja história é muito difícil de alterar sem que a rede o detecte [89]. Muitas listas de blocos expõem os intervenientes e os montantes para tornar simples a verificação colectiva. O preço é evidente: perde-se privacidade e, com ela, fungibilidade.¹

Um utilizador de Bitcoin pode tentar ofuscar o percurso dos fundos com endereços intermédios [90]. Ainda assim, ferramentas de análise conseguem muitas vezes estabelecer ligações entre remetentes e destinatários [116, 37, 103, 44].

Monero adopta outra estratégia. Endereços de utilização única escondem o destinatário na lista de blocos; assinaturas em anel tornam ambígua a saída que foi efectivamente gasta; e compromissos criptográficos ocultam os montantes sem impedir a verificação da contabilidade. O resultado é uma cripto-moeda com um elevado grau de privacidade e fungibilidade. Não se trata, porém, de

¹ Um bem é **fungível** quando uma unidade pode substituir outra no uso ou no cumprimento de um contracto [23]. Numa lista de blocos transparente como a de Bitcoin, as moedas podem ser distinguidas pela sua história. Se essa história incluir actores considerados suspeitos, certas moedas podem ser tratadas como “manchadas” [86]; inversamente, houve quem pagasse um prémio por moedas recém-mineradas, sem história anterior [109].

magia: o comportamento do utilizador e a informação disponível fora da lista de blocos podem permitir alguma análise.²

1.1 Objectivos

Monero nasceu em 2014 e conta com uma história continuada de desenvolvimento [118, 12]. Quando a segunda edição deste livro foi preparada, a comunidade e a adopção da moeda continuavam a crescer [49].³

Documentar o sistema é difícil. Existe documentação prática no projecto <https://monerodocs.org/> e em *Mastering Monero* [52], mas partes importantes da construção teórica foram publicadas em textos incompletos, não revistos por pares ou entretanto ultrapassados. Para saber o que uma versão do programa faz, o código-fonte é a referência decisiva; para saber por que uma construção criptográfica é segura, continuam a ser indispensáveis os artigos e as provas que a fundamentam.⁴

Para quem não traz consigo uma formação matemática, até os fundamentos da criptografia de curva elíptica podem parecer uma sucessão de portas fechadas. Uma explicação anterior do funcionamento de Monero [101], por exemplo, não desenvolvia esses fundamentos, era incompleta e ficou obsoleta. Este livro procura abrir as portas pela ordem certa: introduz os conceitos necessários, revê os algoritmos criptográficos e reúne uma descrição construtiva, passo a passo, do funcionamento interno de Monero.

1.2 Âmbito técnico desta edição

A segunda edição original descreve o protocolo v12 e o programa Monero v0.15.x.x. A revisão portuguesa toma esse texto como ponto de partida, mas confere o estado corrente contra a rede principal e a fotografia do código identificada no resumo inicial.⁵

Esta distinção evita três confusões frequentes:

- uma *regra de consenso activa* determina que blocos e transacções a rede principal aceita;
- um *comportamento implementado* pode pertencer à carteira ou à rede entre nós sem fazer parte do consenso;

² Para um exemplo de heurísticas aplicadas às assinaturas em anel, veja-se [56].

³ Como retrato daquele período, Monero ocupava o 14.^o lugar por capitalização de mercado em 14 de Junho de 2018 e o 12.^o em 5 de Janeiro de 2020; veja-se <https://coinmarketcap.com/>. Estes valores não são uma medida técnica nem uma afirmação sobre a posição actual.

⁴ A série *Monero Building Blocks*, de Seguias [114], trata com cuidado as provas de segurança dos esquemas de assinatura. Tal como a primeira edição de *Zero a Monero* [30], concentra-se no protocolo v7.

⁵ Foram conferidos, entre outros, `README.md`, `src/hardforks/hardforks.cpp`, `src/cryptonote_core/blockchain.cpp`, `src/cryptonote_protocol/levin_notify.cpp` e o directório `src/fcmp_pp/`. O repositório completo está em <https://github.com/monero-project/monero>.

- *trabalho experimental* pode existir no código antes de estar completo, estável ou sequer programado para activação.

Na v16, as novas transacções usam CLSAG e Bulletproof+, anéis fixos de 16 membros e etiquetas de vista. RandomX é a prova de trabalho activa desde a v12. Dandelion++ é comportamento implementado para a difusão de transacções na rede pública, não uma regra de consenso. O ramo consultado contém componentes de FCMP++ e tipos associados a Carrot, mas não programa uma versão posterior à v16 na rede principal; são, por isso, trabalho experimental nesta edição.

O “protocolo” é o conjunto de regras contra o qual cada novo bloco é testado antes de entrar na lista. Inclui as regras das transacções, mas não se confunde com o campo de versão da própria transacção. Este último continua a identificar RingCT v2, embora várias regras concretas e o formato RingCT tenham mudado de facto ao longo das versões de consenso.

A actualização é progressiva. Os preliminares, esta introdução e o capítulo de conceitos básicos já obedecem ao âmbito acima; os capítulos seguintes aguardam a passagem editorial completa. Até à sua revisão, as descrições de transacções e da lista de blocos devem ser lidas como documentação histórica da v12. Esquemas descontinuados serão conservados apenas quando ajudarem a interpretar dados históricos; a primeira edição, por sua vez, corresponde ao protocolo v7 e ao programa v0.12.x.x [30].

O código beneficia de revisão pública, mas isso não o torna infalível. O leitor é convidado a conferir as explicações nas fontes. Uma vulnerabilidade séria deve ser comunicada de forma responsável em <https://hackerone.com/monero>; uma correcção não sensível pode ser proposta no repositório do projecto. As propostas Triptych [97], RingCT 3.0 [131], Omniring [75] e Lelantus [64] pertencem ao contexto de investigação da segunda edição, não ao conjunto de regras activas aqui descrito.

1.3 Aos leitores

Prevemos que muitos leitores cheguem a este relatório com pouco ou nenhum contacto com matemática discreta, estruturas algébricas, criptografia ou listas de blocos. Procurámos dar pormenor suficiente para que um leitor atento possa aprender sem depender constantemente de pesquisa externa.⁶

Alguns pormenores matemáticos foram omitidos ou remetidos para notas de rodapé quando prejudicariam a clareza. Também não seguimos cada detalhe de implementação quando ele não é necessário para compreender o mecanismo. O objectivo é caminhar a meio caminho entre a criptografia matemática e a programação: suficientemente formal para não esconder as hipóteses, suficientemente concreto para mostrar como as peças trabalham juntas.⁷

⁶ Uma introdução extensa à criptografia aplicada encontra-se em [38].

⁷ Certas notas de rodapé antecipam capítulos posteriores. Foram pensadas para ganhar utilidade numa segunda leitura, quando os detalhes de implementação já têm onde assentar.

1.4 Origens da cripto-moeda Monero

Monero, inicialmente chamado BitMonero, foi criado em Abril de 2014 a partir da prova de conceito CryptoNote [118]. *Monero* significa “moeda” em esperanto; o plural é *moneroj*.

CryptoNote foi desenvolvido por várias pessoas. O texto de referência que o descreve foi publicado sob o pseudónimo Nicolas van Saberhagen, em Outubro de 2013 [124]. Propunha endereços de utilização única para ocultar o destinatário e assinaturas em anel para tornar ambíguo o remetente. Desde então, Monero reforçou a privacidade com montantes confidenciais, inspirados, entre outros, no trabalho de Greg Maxwell [80] e integrados em RingCT segundo as recomendações de Shen Noether [99]. As provas Bulletproof tornaram depois essa ocultação muito mais eficiente [39]; CLSAG reduziu o custo das assinaturas em anel [60], e Bulletproof+ sucedeu às provas Bulletproof nas transacções activas.

1.4.1 Parte 1: “Essenciais”

Começamos pelas ferramentas criptográficas necessárias. O capítulo 2 desenvolve aritmética modular, curvas elípticas, chaves, assinaturas de Schnorr e a notação usada no resto do livro. O capítulo 3 prolonga essa construção até às assinaturas em anel. No capítulo 4, vemos como os endereços controlam a recepção de fundos e por que uma saída não revela publicamente o destinatário.

O capítulo 5 introduz compromissos e provas de intervalo para ocultar montantes. Reunidas as peças, o capítulo 6 constrói uma transacção RingCT. O capítulo 7 abre por fim a lista de blocos: mineração, dificuldade, emissão, peso e taxas.

1.4.2 Parte 2: “Extensões”

Uma cripto-moeda é mais do que as suas regras de consenso. Esta parte explora aplicações e propostas, várias das quais não estão implementadas ou dependem de funcionalidades experimentais. Uma futura alteração do formato das transacções pode torná-las impraticáveis; cada capítulo deve, portanto, ser lido com o seu estatuto técnico em mente.

O capítulo 8 mostra que informação sobre uma transacção pode ser demonstrada a um observador. O capítulo 9 descreve a abordagem de multi-assinaturas da carteira; na implementação consultada, esta funcionalidade continua marcada como experimental e desactivada por omissão. O capítulo 10 usa multi-assinaturas para desenhar mercados com garantia. O capítulo 11 apresenta TxTangle, uma proposta original e descentralizada para reunir as transacções de várias pessoas numa só.

1.4.3 Conteúdo adicional

O apêndice A dissectiona uma transacção `RCTTypeBulletproof2`; é um exemplo histórico, não o formato que a v16 aceita para novas transacções. O apêndice B explica a estrutura de um bloco,

incluindo o cabeçalho e a transacção de mineiro. O apêndice C fecha o percurso com o bloco gênese de Monero. Em conjunto, ligam a teoria a objectos concretos da lista de blocos.

As notas de margem apontam para locais relevantes no código-fonte. Nos capítulos ainda não revistos, os caminhos referem-se ao programa v0.15.x.x e podem ter mudado; a história completa continua disponível em Git. Uma nota contém normalmente um caminho, como `src/ringct/rctOps.cpp`, e por vezes uma função, como `ecdhEncode()`. Um hífen num caminho estreito pode ser apenas uma quebra tipográfica; os qualificadores de *namespace*, como `Blockchain::`, são geralmente omitidos. isto é útil!

1.5 Aviso

Os esquemas de assinatura, as aplicações de curvas elípticas e os detalhes da implementação aqui apresentados são descritivos. Quem pretenda construir um sistema real deve consultar as fontes primárias, as especificações técnicas e o código correspondente à versão que vai usar.

Um esquema de assinatura precisa de uma prova de segurança bem definida e de revisão competente. Vale aqui a velha regra: não invente a sua própria criptografia. Código que implemente primitivas criptográficas deve ser revisto por especialistas e acumular um historial longo e fiável. As contribuições originais deste documento podem não ter recebido a mesma revisão nem testes; leia-as com o cuidado devido.

1.6 A história de Zero a Monero

Zero a Monero nasceu como expansão da tese de mestrado de Kurt Alonso, “Monero — Privacy in the Blockchain” [29], publicada em Maio de 2018. A primeira edição do livro apareceu em Junho desse ano [30].

Para a segunda edição, os autores melhoraram a introdução às assinaturas em anel (capítulo 3) e reorganizaram a explicação das transacções, acrescentando o capítulo de endereços (4). Modernizaram a comunicação dos montantes de saída (secção 5.3), substituíram as provas Borromean por Bulletproof no percurso corrente da época (secção 5.5), retiraram `RCTTypeFull` do formato corrente e actualizaram o peso dinâmico dos blocos e as taxas dos mineiros.

Foram ainda estudadas provas sobre transacções (capítulo 8), multi-assinaturas (capítulo 9), mercados com garantia (capítulo 10) e a proposta TxTangle (capítulo 11). Essa edição ficou alinhada com o protocolo v12 e o programa v0.15.x.x.⁸

⁸ O código-fonte L^AT_EX das duas edições está em <https://github.com/UkoeHB/Monero-RCT-report>; a primeira edição encontra-se no ramo `ztml`.

1.7 Reconhecimentos

Texto original do autor “koe”.

Este relatório não existiria sem a tese de mestrado de Kurt Alonso [29], a quem o autor deve profunda gratidão. Brandon “Surae Noether” Goodell e o investigador conhecido pelo pseudónimo “Sarang Noether”, do Monero Research Lab, foram fontes constantes de conhecimento durante as duas edições. “moneromooo”, um dos programadores mais prolíficos do projecto, indicou inúmeras vezes o caminho certo através do código. Agradecemos também a todos os contribuidores que responderam a perguntas e aos leitores que enviaram correcções e palavras de encorajamento.

Parte I

Essenciais

CAPÍTULO 2

Conceitos Básicos

2.1 Algumas palavras sobre a notação

Um dos principais objectivos deste texto é reunir, corrigir e harmonizar a informação necessária para compreender os mecanismos internos de Monero. Ao mesmo tempo, queremos apresentar a matéria de forma construtiva: cada símbolo deve conservar o seu papel enquanto acrescentamos uma peça de cada vez.

Salvo indicação em contrário, adoptamos as seguintes convenções:

- letras minúsculas para inteiros, escalares, sequências de bits e valores simples;
- letras maiúsculas para pontos de curva elíptica e objectos compostos.

Tentamos também reservar os mesmos símbolos para os mesmos objectos. Um gerador de curva será normalmente G , a ordem do seu subgrupo será l , e um par de chaves privada e pública será escrito k/K , respectivamente.

A apresentação é deliberadamente *conceptual*. Um leitor de ciência da computação poderá sentir falta da representação exacta de certos objectos ou dos pormenores de uma implementação; um estudante de matemática poderá desejar mais álgebra abstracta. Daremos esses pormenores quando forem necessários ao raciocínio, sem os deixar esconder a construção principal.

Um objecto simples, como um inteiro ou uma sequência de caracteres, pode ser codificado como uma sequência de bits. A ordem dos bytes, ou *endianness*, é essencial para uma implementação

interoperável, mas raramente altera as ideias estudadas neste capítulo; quando importar, será indicada.

Um ponto de curva elíptica escreve-se habitualmente como um par (x, y) , mas isso não obriga a transmiti-lo como dois inteiros completos. Na secção 2.4.2 veremos como uma coordenada e um bit bastam para codificar os pontos que nos interessam. Sempre que contarmos bytes, assumiremos essa compressão.

As funções hash serão por vezes usadas sem declarar um algoritmo específico. Monero emprega uma variante de *Keccak*, cuja construção interna não é necessária para a teoria deste capítulo¹. `src/crypto/keccak.c`

Chamaremos simplesmente *função hash* a uma função de dispersão criptográfica. Ela recebe uma mensagem m de comprimento variável e devolve uma sequência h de comprimento fixo. É determinística: a mesma entrada dá sempre a mesma saída. Pretende-se, contudo, que as saídas se pareçam com escolhas uniformes e imprevisíveis, que seja computacionalmente difícil recuperar uma entrada a partir da sua hash e que seja difícil encontrar duas entradas distintas com a mesma saída. O chamado *efeito avalanche* faz com que uma pequena alteração da entrada possa mudar muitos bits da saída.

Aplicaremos funções hash a inteiros, sequências de caracteres, pontos de curva e combinações desses objectos. Isso significa que primeiro se usa uma codificação binária inequívoca e, quando há vários objectos, uma concatenação inequívoca. Uma hash é originalmente uma sequência de bits; para a usar como escalar ou ponto de curva é ainda necessária uma conversão apropriada, que indicaremos quando for relevante.

2.2 Aritmética modular

A maior parte da criptografia moderna começa com aritmética modular. Fixemos um módulo $n > 1$. Para qualquer inteiro a , a operação

$$c = a \bmod n$$

devolve o único resto c que satisfaz

$$a = qn + c, \quad 0 \leq c < n,$$

para algum inteiro q . O símbolo $\bmod$ designa aqui uma operação. Quando queremos afirmar que dois inteiros deixam o mesmo resto, escrevemos antes a congruência $a \equiv c \pmod{n}$.

Os restos nunca são negativos. Em particular, se $0 < a < n$, então $(-a) \bmod n = n - a$. Nos casos de fronteira convém não abreviar: se $a \bmod n = 0$, também $(-a) \bmod n = 0$. A fórmula que cobre todos os casos é

$$(-a) \bmod n = (n - (a \bmod n)) \bmod n.$$

¹ O algoritmo Keccak serve de base ao standard NIST *SHA-3* [24].

2.2.1 Adição e multiplicação modular

Ao implementar estas operações, reduzir resultados intermédios evita inteiros desnecessariamente grandes. Suponhamos, por exemplo, que queremos calcular $(29 + 87) \bmod 99$, mas que não desejamos formar primeiro 116. Para restos $0 \leq a, b < n$, pomos $x = n - a$:

- se $b < x$, então $(a + b) \bmod n = a + b$;
- se $b \geq x$, então $(a + b) \bmod n = b - x$.

No exemplo, $x = 70$ e obtemos directamente $87 - 70 = 17$. A igualdade no segundo caso é importante: quando $a + b = n$, o resto é zero.

Podemos repetir esta adição segura para calcular uma multiplicação modular. O algoritmo chama-se *duplica-e-adiciona*. Por exemplo,

$$(7 \cdot 8) \bmod 9 = (8 + 8 + 8 + 8 + 8 + 8 + 8) \bmod 9 = 2.$$

Agrupando parcelas iguais,

$$(8 + 8) + (8 + 8) + (8 + 8) + 8$$

e depois

$$[(8 + 8) + (8 + 8)] + (8 + 8) + 8,$$

podemos reutilizar cada duplicação em vez de repetir todo o trabalho. ²

Para calcular $c = (a \cdot b) \bmod n$, escrevemos o multiplicador a em binário e percorremos os seus bits do menos para o mais significativo.

No nosso exemplo, seja $A = [0111]$, com os índices 3, 2, 1, 0. ³ Assim, $A[0] = 1$ é o bit menos significativo. Inicializamos o acumulador com $r = 0$ e a parcela corrente com $s = b \bmod n$. Depois:

1. Para $i = 0, \dots, |A| - 1$:
 - (a) se $A[i] = 1$, faça $r = (r + s) \bmod n$;
 - (b) faça $s = (s + s) \bmod n$.
2. O resultado é $c = r$.

Para $(7 \cdot 8) \bmod 9$, a sequência é:

1. $i = 0$
 - (a) $A[0] = 1$, portanto $r = (0 + 8) \bmod 9 = 8$;
 - (b) $s = (8 + 8) \bmod 9 = 7$.

² A vantagem torna-se clara para multiplicadores grandes: a adição repetida exige um número de operações proporcional ao multiplicador, enquanto duplica-e-adiciona exige apenas um número proporcional ao seu comprimento em bits.

³ Esta numeração é conhecida como “LSB 0”, do inglês *least significant bit*: o bit menos significativo tem o índice zero. Usá-la-emos no resto do capítulo quando tornar a ordem dos bits mais clara.

2. $i = 1$

(a) $A[1] = 1$, portanto $r = (8 + 7) \bmod 9 = 6$;

(b) $s = (7 + 7) \bmod 9 = 5$.

3. $i = 2$

(a) $A[2] = 1$, portanto $r = (6 + 5) \bmod 9 = 2$;

(b) $s = (5 + 5) \bmod 9 = 1$.

4. $i = 3$

(a) $A[3] = 0$, portanto r não muda;

(b) $s = (1 + 1) \bmod 9 = 2$.

5. O resultado é $r = 2$.

2.2.2 Exponenciação modular

Tal como a multiplicação é adição repetida, a exponenciação é multiplicação repetida. Por exemplo,

$$8^7 \bmod 9 = (8 \cdot 8 \cdot 8 \cdot 8 \cdot 8 \cdot 8 \cdot 8) \bmod 9.$$

O análogo de duplica-e-adiciona chama-se *eleva-ao-quadrado-e-multiplica*. Para calcular $a^e \bmod n$, com $e \geq 0$, escrevemos e em binário no vector A , do mesmo modo que acima, e inicializamos $r = 1 \bmod n$ e $m = a \bmod n$:

1. Para $i = 0, \dots, |A| - 1$:

(a) se $A[i] = 1$, faça $r = (r \cdot m) \bmod n$;

(b) faça $m = (m \cdot m) \bmod n$.

2. O resultado é o valor final de r .

As condições iniciais tratam também o expoente zero: nenhum bit provoca uma multiplicação do acumulador e o resultado é $1 \bmod n$.

2.2.3 Inversa multiplicativa modular

Por vezes precisamos de dividir por a em aritmética modular. Isso significa procurar uma inversa multiplicativa a^{-1} , isto é, um resto que, ao ser multiplicado por a , dê a identidade 1. Ao contrário do que sucede com números reais, nem todo o resto diferente de zero tem inversa para qualquer módulo.

Existe uma inversa módulo n se, e somente se, a e n são relativamente primos, ou seja, se $\gcd(a, n) = 1$.⁴ Nesse caso há um único resto c tal que

$$\begin{aligned} c &\equiv a^{-1} \pmod{n}, & 0 \leq c < n, \\ ac &\equiv 1 \pmod{n}. \end{aligned}$$

Dois inteiros relativamente primos não têm qualquer divisor comum além de 1; por exemplo, a fracção a/n já estaria na forma irredutível.

Quando n é primo e $a \not\equiv 0 \pmod{n}$, podemos calcular a inversa com eleva-ao-quadrado-e-multiplica graças ao *pequeno teorema de Fermat*:⁵

$$\begin{aligned} a^{n-1} &\equiv 1 \pmod{n} \\ a \cdot a^{n-2} &\equiv 1 \pmod{n} \\ a^{n-2} &\equiv a^{-1} \pmod{n}. \end{aligned}$$

Para um módulo composto, e em geral de forma mais directa, o algoritmo estendido de Euclides [6] encontra a inversa sempre que $\gcd(a, n) = 1$; se o máximo divisor comum não for 1, confirma que a inversa não existe.

2.2.4 Equações modulares

As reduções modulares podem ser feitas antes ou depois de adições, subtracções e multiplicações. Mais precisamente, se $\circ \in \{+, -, \cdot\}$, então

$$(A \circ B) \bmod n = [(A \bmod n) \circ (B \bmod n)] \bmod n.$$

Por exemplo, para $c = (3 \cdot 4 \cdot 5) \bmod 9$, tomamos $A = 3 \cdot 4$, $B = 5$ e $n = 9$:

$$\begin{aligned} (3 \cdot 4 \cdot 5) \bmod 9 &= [(3 \cdot 4) \bmod 9] \cdot (5 \bmod 9) \bmod 9 \\ &= (3 \cdot 5) \bmod 9 \\ &= 6. \end{aligned}$$

Em particular, a subtracção pode ser tratada como a soma de um valor negativo:

$$\begin{aligned} (A - B) \bmod n &= (A + (-B)) \bmod n \\ &= [(A \bmod n) + ((-B) \bmod n)] \bmod n. \end{aligned}$$

Uma potência com expoente inteiro não negativo obedece à mesma regra por ser uma sucessão de multiplicações. A divisão, porém, não pode ser introduzida sem mais: substituir uma divisão pela multiplicação por uma inversa só é válido quando essa inversa existe. Assim, uma expressão como

$$x = (a - bcd)^{-1}(ef + g^h) \bmod n$$

só está definida desta forma se $h \geq 0$ e $\gcd(a - bcd, n) = 1$. Esta condição será importante sempre que dividirmos num corpo finito.

⁴ Na expressão $a \equiv b \pmod{n}$, dizemos que a é *congruente* com b módulo n . Isso equivale a $a \bmod n = b \bmod n$, ou a afirmar que n divide $a - b$.

⁵ Se $ac \equiv b \pmod{n}$ e $\gcd(a, n) = 1$, a congruência linear tem a solução única $c \equiv a^{-1}b \pmod{n}$. Só podemos voltar a multiplicar por c^{-1} se também c for invertível. Veja-se [9].

2.3 Criptografia de curva elíptica

2.3.1 O que são curvas elípticas?

Quando q é primo, podemos representar o corpo finito $\mathbb{F}_q$ pelo conjunto de restos $\{0, 1, \dots, q-1\}$. **fe:** field element
A soma, a subtração e a multiplicação são seguidas de redução módulo q ; cada elemento não nulo tem ainda uma inversa multiplicativa. É precisamente esta última propriedade que faz do conjunto um *corpo*, e não apenas um conjunto de restos.

Por exemplo, em $\mathbb{F}_{29}$, escrevemos $17 + 20 = 8$, pois $(17 + 20) \bmod 29 = 8$, e $-13 = 16$, pois $(-13) \bmod 29 = 16$. Uma fracção como u/v significa $u \cdot v^{-1}$ no corpo e só está definida quando $v \neq 0$.

Para $q > 3$, uma curva elíptica pode ser apresentada na forma de *Weierstrass* [62] como o conjunto dos pontos (x, y) que satisfazem

$$y^2 = x^3 + ax + b, \quad a, b, x, y \in \mathbb{F}_q,$$

juntamente com um elemento de identidade. Exige-se ainda que a curva não seja singular; nesta forma, isso equivale a $4a^3 + 27b^2 \not\equiv 0 \pmod{q}$.⁶

Monero trabalha com uma curva na forma de Edwards torcida (*Twisted Edwards*) [34]. Para parâmetros não nulos e distintos $a, d \in \mathbb{F}_q$, esta forma é

$$ax^2 + y^2 = 1 + dx^2y^2, \quad a, d, x, y \in \mathbb{F}_q.$$

Usaremos esta representação daqui em diante. Além de admitir fórmulas de adição particularmente regulares, permite implementar certas primitivas com menos operações aritméticas [36].

Sejam $P_1 = (x_1, y_1)$ e $P_2 = (x_2, y_2)$ dois pontos da curva. A sua soma $P_3 = P_1 + P_2 = (x_3, y_3)$ é definida por⁷

$$x_3 = \frac{x_1y_2 + y_1x_2}{1 + dx_1x_2y_1y_2},$$

$$y_3 = \frac{y_1y_2 - ax_1x_2}{1 - dx_1x_2y_1y_2},$$

onde todas as operações, incluindo as divisões, têm lugar em $\mathbb{F}_q$. Para os parâmetros usados mais adiante, esta lei de adição é completa: os denominadores não se anulam para pontos da curva. As mesmas fórmulas duplicam um ponto quando $P_1 = P_2$. O inverso aditivo de $P = (x, y)$ é $-P = (-x, y)$ [34]; portanto, $P_1 - P_2 = P_1 + (-P_2)$, e não uma duplicação. A coordenada $-x$ significa, naturalmente, $(-x) \bmod q$.

A soma permanece na curva. Com esta operação, os pontos formam um grupo abeliano: há um elemento de identidade, cada ponto tem um inverso, a soma é associativa e a ordem das parcelas não importa. Na forma de Edwards, a identidade é o ponto afim

$$0 = (0, 1).$$

⁶ A expressão $a \in \mathbb{F}$ significa que a é um elemento do corpo finito $\mathbb{F}$.

⁷ Por razões de eficiência, as implementações representam habitualmente os pontos em coordenadas projectivas, evitando assim inversões no corpo durante muitas operações [130].

Na forma de Weierstrass, o mesmo papel é desempenhado pelo chamado ponto no infinito.⁸

Somando um ponto P repetidamente, obtemos o subgrupo cíclico que ele gera,

$$\langle P \rangle = \{0, P, 2P, \dots, (u-1)P\}.$$

A sua ordem u é o menor inteiro positivo para o qual $uP = 0$. Por exemplo, se $u = 5$, o subgrupo contém $0, P, 2P, 3P, 4P$, e $5P = 0$, donde $5P + P = P$.

Qualquer ponto gera, por definição, um subgrupo cíclico. Se a ordem desse subgrupo for prima, todos os seus pontos não nulos são também geradores. No exemplo de ordem 5, os múltiplos de $2P$ percorrem

$$2P, 4P, 6P, 8P, 10P = 2P, 4P, P, 3P, 0.$$

Isto muda quando a ordem é composta. Num subgrupo de ordem 6,

$$2P, 4P, 6P, 8P, 10P, 12P = 2P, 4P, 0, 2P, 4P, 0;$$

logo $2P$ gera apenas um subgrupo de ordem 3.

À quantidade total de pontos do grupo chamamos a *ordem da curva*, N . O elemento de identidade está incluído nessa contagem. Pelo teorema de Lagrange, a ordem de qualquer subgrupo divide N . No exemplo anterior, 3 divide 6, como seria necessário.

Conhecendo N , podemos encontrar a ordem u de um ponto P :

1. Calcula-se N , por exemplo com o *algoritmo de Schoof*.
2. Enumeram-se os divisores positivos de N por ordem crescente.
3. Para cada divisor n , calcula-se nP .
4. O primeiro n para o qual $nP = 0$ é u .

As curvas escolhidas para criptografia têm frequentemente $N = hl$, onde l é um primo grande e $h = N/l$ é um pequeno *cofactor*.⁹ Escolhe-se então um ponto G de ordem l como gerador do subgrupo primo. Cada ponto P desse subgrupo tem uma representação $P = nG$ para um único escalar $n \in \{0, 1, \dots, l-1\}$; o valor $n = 0$ corresponde à identidade.

Calcular a *multiplicação escalar* nP é fácil: trata-se de somar P a si próprio n vezes, com $0P = 0$. O percurso inverso é muito diferente. Dados um ponto não nulo P_2 , que nesse subgrupo primo é um gerador, e outro ponto P_1 , pretende-se encontrar n tal que

$$P_1 = nP_2.$$

Este é o *problema do logaritmo discreto* (PLD) em curvas elípticas. Para curvas e parâmetros bem escolhidos, não se conhece um algoritmo clássico que o resolva eficientemente. Nesse sentido, a multiplicação escalar funciona como uma operação de sentido único: fácil de calcular, mas impraticável de inverter com os recursos conhecidos. Mais adiante encontraremos um escalar γ

⁸ Uma definição concisa de grupo abeliano encontra-se em <https://brilliant.org/wiki/abelian-group/>.

⁹ Um cofactor pequeno simplifica o controlo dos pequenos subgrupos, mas não permite ignorá-los; as verificações de subgrupo continuam a fazer parte da segurança de muitos protocolos [34].

central nos compromissos de Monero, cujo valor ninguém deve conhecer e cuja recuperação exigiria resolver um PLD (secção 5.3).

Para não efectuar $n - 1$ adições, podemos adaptar o algoritmo duplica-e-adiciona da secção 2.2.1. Escrevemos um inteiro não negativo n em binário no vector A , do bit menos significativo para o mais significativo, e queremos obter $R = nP$:

1. Inicialize $R = 0$, o elemento de identidade, e $S = P$.
2. Para $i = 0, \dots, |A| - 1$:
 - (a) se $A[i] = 1$, faça $R = R + S$;
 - (b) faça $S = S + S$.
3. O resultado é o valor final de R .

Como G tem ordem prima l , os escalares do seu subgrupo são elementos de $\mathbb{F}_l$. Podemos, portanto, reduzi-los módulo l : para qualquer inteiro n , $nG = (n \bmod l)G$. A aritmética entre esses escalares é feita em $\mathbb{F}_l$, enquanto as coordenadas dos pontos pertencem a $\mathbb{F}_q$. Esta distinção entre l e q acompanhar-nos-á no resto do livro.

2.3.2 Criptografia de chave pública com curvas elípticas

Escolhamos uniformemente um escalar k em $\{1, \dots, l-1\}$. Este escalar é a *chave privada*, também chamada chave secreta. A chave pública correspondente é o ponto

$$K = kG.$$

Usaremos de modo consistente uma minúscula para a chave privada k e a maiúscula correspondente para a chave pública K . Qualquer pessoa pode conhecer G e K , mas recuperar k requer resolver o PLD no subgrupo de ordem l . Esta assimetria permite publicar K sem publicar o segredo que a controla.

2.3.3 A troca de chaves tipo Diffie-Hellman com curvas elípticas

Alice e Bob podem usar uma troca de chaves de tipo *Diffie-Hellman* [47] para obter o mesmo segredo sem enviarem esse segredo pela rede:

1. Alice gera (k_A, K_A) e Bob gera (k_B, K_B) . Trocam as chaves públicas e conservam as privadas. Numa implementação, cada um deve ainda validar a codificação e o subgrupo da chave pública que recebeu.

2. Alice calcula $S_A = k_A K_B$, e Bob calcula $S_B = k_B K_A$. Os dois pontos coincidem, pois

$$S_A = k_A K_B = k_A k_B G = k_B k_A G = k_B K_A = S_B.$$

Podemos, portanto, chamar S a esse ponto partilhado.

3. Para obter material de chave, ambos codificam S e aplicam uma função de derivação, que representaremos conceptualmente por $h = \mathcal{H}([S])$. Alice pode então cifrar uma mensagem m com um esquema simétrico autenticado sob a chave derivada h , e Bob pode decifrá-la com o mesmo h . Uma aplicação real não deve confundir esta derivação com a simples soma de uma hash à mensagem.

Um observador passivo conhece G , K_A e K_B , mas não os escalares privados. Calcular $S = k_A k_B G$ apenas a partir desses pontos é o *problema computacional de Diffie-Hellman* (PCDH), que se supõe difícil no grupo escolhido.¹⁰ O PLD, por sua vez, impede a recuperação directa de k_A ou k_B a partir das chaves públicas.

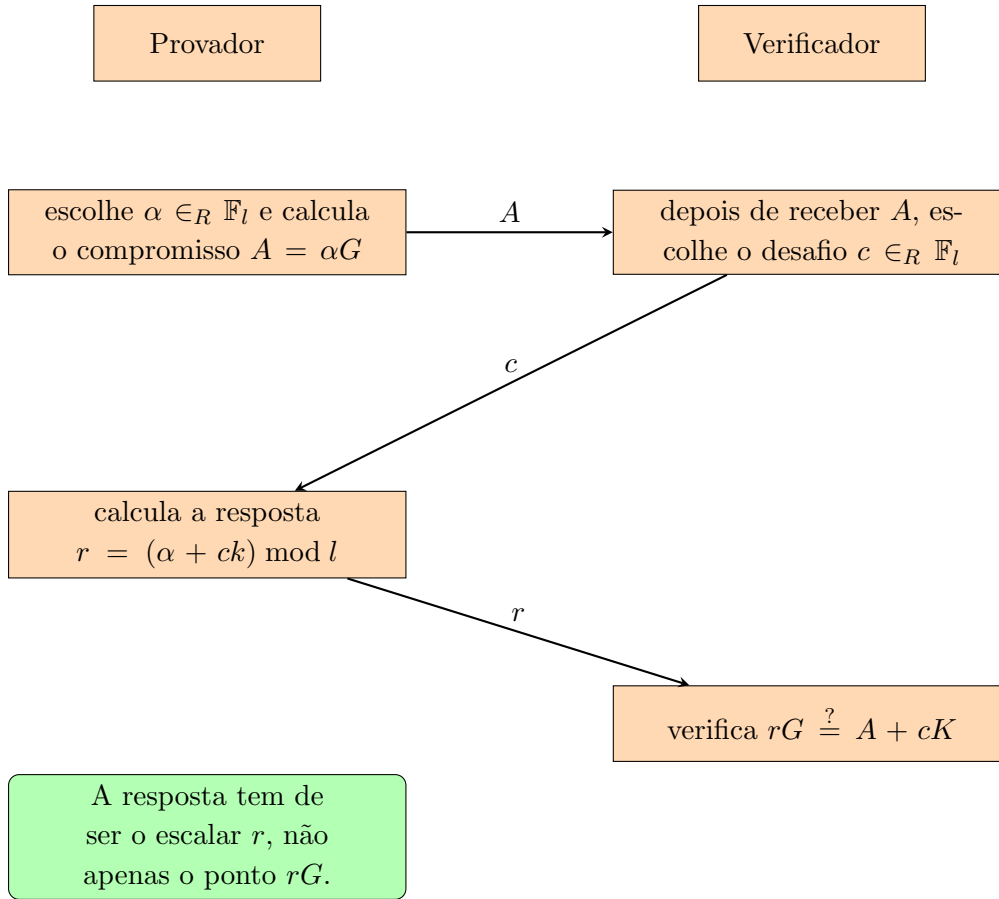
Esta troca, por si só, não autentica os interlocutores: um atacante activo pode substituir as duas chaves públicas e estabelecer um segredo diferente com cada lado. Quando a identidade de Alice ou Bob importa, as chaves trocadas têm de ser autenticadas, por exemplo com assinaturas.

2.3.4 Assinaturas tipo Schnorr e a transformada Fiat-Shamir

Em 1989, Claus-Peter Schnorr publicou um protocolo interactivo de identificação que se tornou fundamental [112]; Maurer generalizou esta família de provas em 2009 [78]. O protocolo permite a um provador demonstrar que conhece o escalar k correspondente à chave pública $K = kG$, sem acrescentar ao verificador informação utilizável sobre k [80].

Ambas as partes conhecem o grupo primo de ordem l , o gerador fixo G e a chave pública K . Usaremos $\mathcal{H}_l$ para uma função hash cujo resultado é convertido num escalar de $\mathbb{F}_l$. A interacção tem três mensagens:

¹⁰ A direcção da redução é importante: quem souber resolver o PLD pode recuperar k_A e, portanto, resolver o PCDH. Logo, o PCDH não é mais difícil do que o PLD; a equivalência das dificuldades não está demonstrada em geral [28].



Um provedor honesto é sempre aceite, porque

$$rG = (\alpha + ck)G = \alpha G + cK = A + cK.$$

O desafio só é escolhido depois de o compromisso A ficar fixo. O registo completo da interacção (o *transcript*) contém as três mensagens (A, c, r) ; sem A , a equação de verificação nem sequer fica determinada. Responder apenas com o ponto rG seria inútil, pois qualquer pessoa poderia copiar o ponto já calculável $A + cK$. O trabalho do provedor consiste em dar o seu logaritmo discreto r .

Se α for uniforme, então, para valores fixos de c e k , a resposta r também é uniforme [113]. Mais ainda, no caso de um verificador que escolhe honestamente um desafio uniforme, podemos simular um registo sem conhecer k : escolhemos $c, r \in_R \mathbb{F}_l$ e pomos $A = rG - cK$. O resultado tem a mesma distribuição que uma execução real. Esta é a intuição precisa de que a conversa não revela informação adicional sobre k além da que já estava na chave pública K . Há, porém, uma condição decisiva: o provedor não pode reutilizar α . Se o mesmo compromisso A receber desafios distintos $c \neq c'$ e produzir respostas válidas $r = \alpha + ck$ e $r' = \alpha + c'k$, qualquer observador extrai a chave:

¹¹

$$k = (r - r')(c - c')^{-1} \bmod l.$$

¹¹ Podemos imaginar que alguém copia o estado de um computador depois de este escolher α e apresenta um desafio diferente a cada cópia. Nas provas de segurança, esta imagem está próxima da técnica de “rebobinar” o provedor.

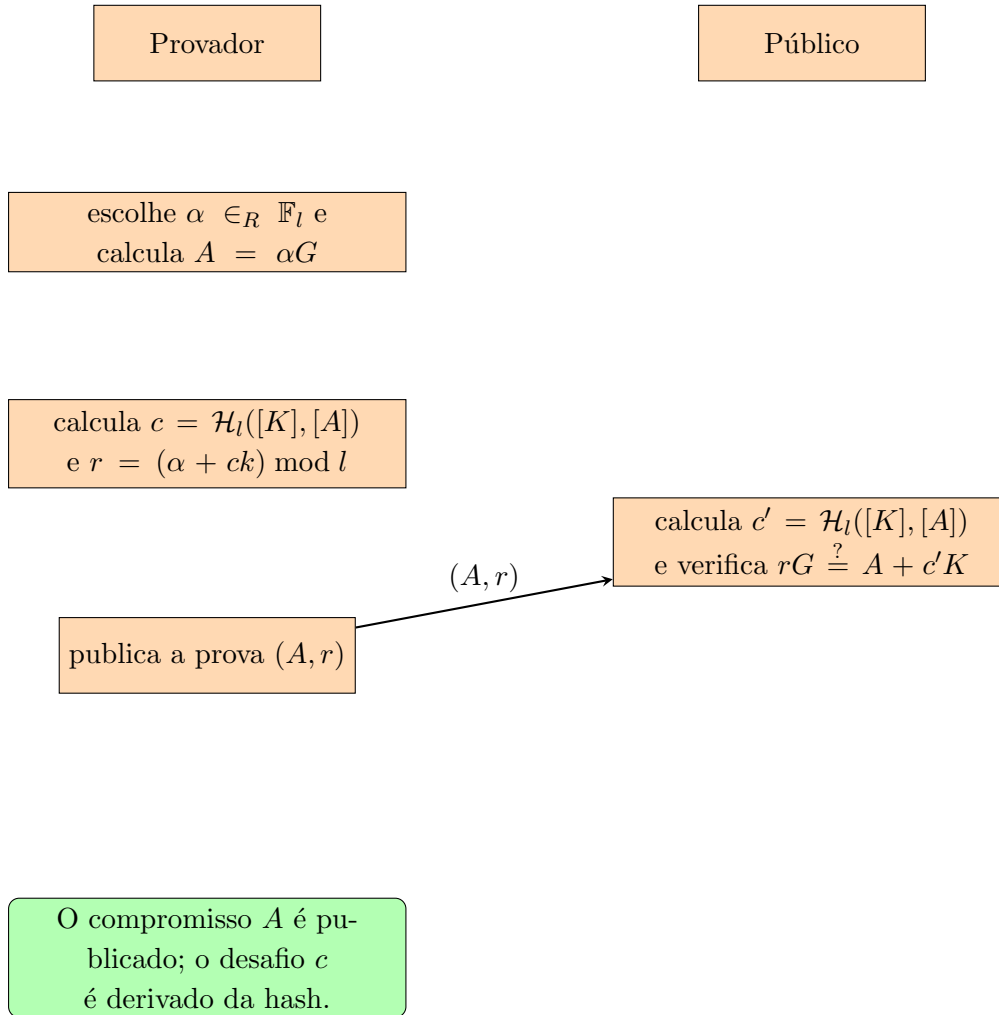
A inversa existe porque l é primo e $c - c' \not\equiv 0 \pmod{l}$. Esta possibilidade de extracção explica em que sentido uma resposta válida demonstra conhecimento, salvo a probabilidade negligível de adivinhar o desafio.

O atraso do desafio é essencial. Se conhecesse c antes de fixar A , um impostor poderia escolher r ao acaso e definir $A = rG - cK$; a verificação passaria sem que ele conhecesse k . A transformada de Fiat–Shamir [55] substitui o verificador por uma função hash que só pode ser avaliada depois de A estar determinado. No modelo do oráculo aleatório, isso torna a prova não interactiva e publicamente verificável [80]. ¹² ¹³ ¹⁴

¹² Em criptografia, um oráculo é uma abstracção que responde a consultas segundo uma regra especificada. Um oráculo aleatório conserva uma resposta uniforme e independente para cada entrada nova [2].

¹³ Uma função hash real é determinística. É a análise no modelo do oráculo aleatório que idealiza cada saída nova como uniforme; a conversão dessa saída num escalar deve ainda evitar ou limitar o viés de redução.

¹⁴ O desafio deve ficar ligado à declaração que se prova. Inclui-se a chave pública K na hash — o chamado prefixo de chave — e, se G não for fixo para o protocolo, inclui-se também o gerador. Voltaremos a esta ideia nas secções 3.4 e 9.2.3.

Prova não interactiva

Já não há uma conversa: o provador calcula o desafio localmente e publica uma única prova. Qualquer pessoa pode repetir a hash e a verificação. A correcção continua a ser a mesma,

$$rG = (\alpha + ck)G = A + cK.$$

É importante distinguir o registo interactivo (A, c, r) da prova não interactiva publicada (A, r) : nesta última, c é reconstruído de forma determinística a partir de K e A .

Os requisitos computacionais de uma prova incluem o espaço necessário para a guardar e o trabalho de verificação. Neste esquema, a prova contém um ponto de curva e um escalar; a declaração contém ainda a chave pública, outro ponto. A verificação exige uma hash e multiplicações escalares da curva.

src/ringct/
rctOps.cpp

2.3.5 Assinar mensagens

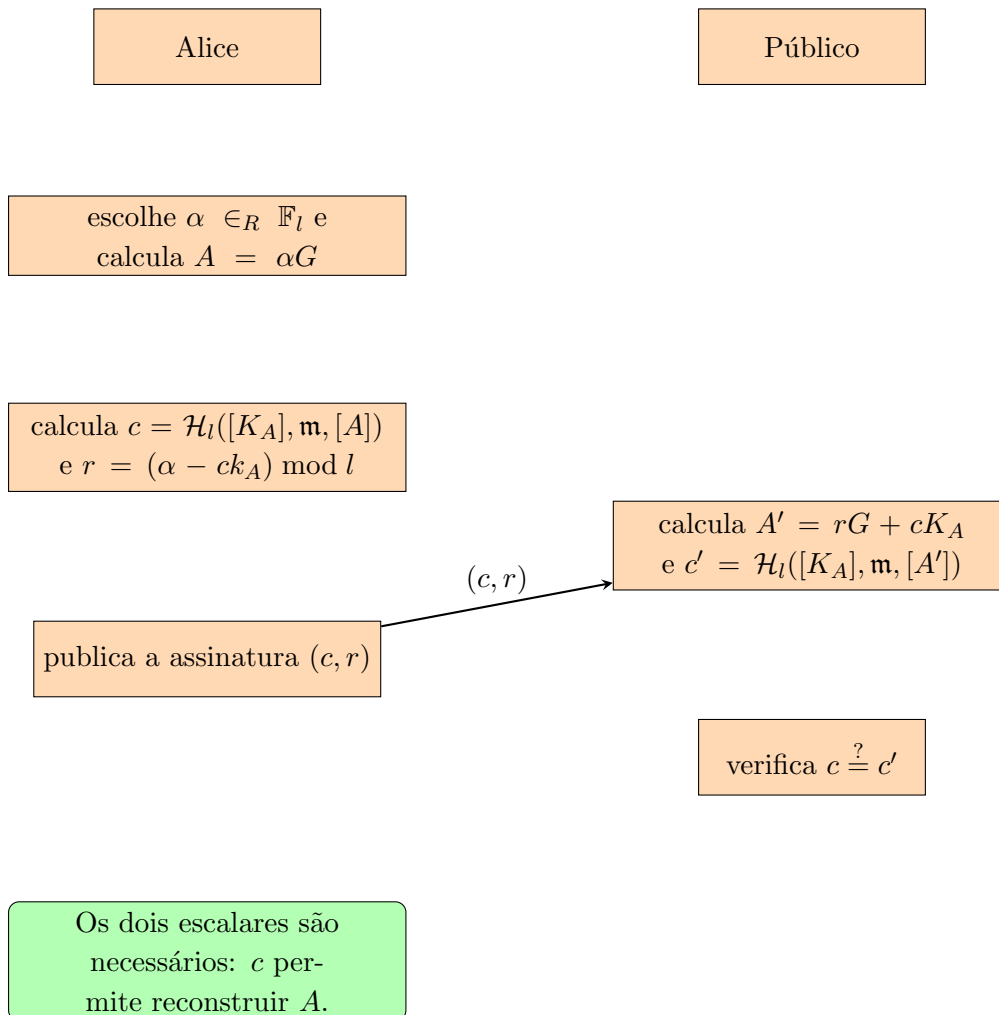
Um esquema de assinatura recebe uma mensagem de comprimento arbitrário e incorpora-a numa hash de comprimento fixo. Representaremos por $\mathbf{m}$ a sequência de bytes que o esquema recebe.

Algumas variantes definem uma modalidade de pré-hash, na qual essa entrada já é a hash da mensagem; essa escolha faz parte do protocolo e não deve ficar implícita.

As assinaturas permitem ao destinatário verificar que uma mensagem foi autorizada pelo titular de uma chave privada e que o conteúdo assinado não foi alterado. ECDSA é um esquema comum; veja-se [65], ANSI X9.62 e [62].

O esquema seguinte é uma formulação da prova de Schnorr transformada, com uma convenção de sinais que permite publicar dois escalares (c, r) em vez do ponto A . Pensar em assinaturas desta forma preparar-nos-á para as assinaturas em anel do capítulo seguinte.

Alice tem o par (k_A, K_A) . Para cada mensagem, escolhe um escalar secreto α novo e imprevisível; reutilizá-lo exporia k_A , exactamente como na prova interactiva.



Se $c = c'$, a assinatura é válida. A chave pública e a mensagem entram na hash, e as codificações devem ser inequívocas; um protocolo real inclui ainda um separador de domínio para não reutilizar o mesmo desafio noutro contexto.

Funciona porque

O verificador reconstrói exactamente o compromisso de Alice:

$$\begin{aligned} r &= \alpha - ck_A, \\ A' &= rG + cK_A \\ &= (\alpha - ck_A)G + cK_A \\ &= \alpha G - cK_A + cK_A \\ &= \alpha G = A. \end{aligned}$$

Logo,

$$c' = \mathcal{H}_l([K_A], \mathbf{m}, [A']) = \mathcal{H}_l([K_A], \mathbf{m}, [A]) = c.$$

“dabra pokus!”

Portanto, uma assinatura produzida por Alice passa a verificação. Esta álgebra demonstra a correcção, não basta por si só para provar a infalsificabilidade; essa propriedade depende da dificuldade do PLD, da segurança da hash, da escolha correcta de α e do modelo de segurança do esquema.

Requisitos:

Neste esquema, a assinatura guarda dois escalares. Para a verificar, são ainda necessários a mensagem, a chave pública de curva e os parâmetros do protocolo.

2.4 Curva Edwards25519

Monero usa a curva de Edwards torcida *Edwards25519*, que é birracionalmente equivalente à curva de Montgomery *Curve25519*.¹⁵ As construções *Curve25519*, *Edwards25519* e *Ed25519* foram publicadas por Bernstein e colaboradores [34, 35, 36]. Convém não confundir os nomes: *Edwards25519* é a curva; *Ed25519* é uma instância do esquema de assinatura EdDSA sobre essa curva, que estudaremos mais abaixo.¹⁶

A curva é definida sobre o corpo primo $\mathbb{F}_q$, com $q = 2^{255} - 19$, pela equação

$$-x^2 + y^2 = 1 - \frac{121665}{121666}x^2y^2.$$

A fracção representa uma divisão em $\mathbb{F}_q$. Assim, na notação geral da secção 2.3.1, $a = -1$ e $d = -121665/121666 \bmod q$.

Os parâmetros foram escolhidos por um processo público e explicável. Essa propriedade, por vezes chamada rigidez, reduz a liberdade que um criador teria para procurar secretamente uma curva com uma fraqueza rara.¹⁷

src/crypto/
crypto_ops_
builder/
ref10Comm-
entedComb-
ined/
ge.h

¹⁵ Sem entrarmos nos pormenores, uma equivalência birracional relaciona pontos das duas curvas por funções racionais, com as excepções que essas funções necessariamente admitem.

¹⁶ Bernstein desenvolveu também a cifra ChaCha [33, 91], usada pela implementação Monero para cifrar certos dados sensíveis da carteira.

¹⁷ Uma curva pode não ter fraquezas públicas conhecidas e, ainda assim, ter sido seleccionada entre muitas candidatas por uma razão que o seu criador não revelou. Exigir uma derivação reproduzível dos parâmetros limita esse espaço de escolha. *Edwards25519* é classificada como uma curva de geração totalmente explicada [111].

src/wallet/
ringdb.cpp

O cuidado não é apenas teórico. O gerador Dual_EC_DRBG, que chegou a ser normalizado pelo NIST¹⁸, ficou associado a uma potencial porta de trás baseada nos seus parâmetros elípticos [61]. A influência da NSA sobre esse processo de normalização tornou-se um exemplo dos riscos de concentrar escolhas criptográficas pouco transparentes [17]. Isto não implica que todos os standards do NIST sejam defeituosos; explica por que valorizamos parâmetros reproduzíveis e diversidade de análise. O projecto Edwards25519 foi também apresentado com atenção à ausência de restrições por patentes (veja-se [71]) e a operações eficientes [36]. O grupo de pontos de Edwards25519 tem ordem $N = hl$, com cofactor $h = 8 = 2^3$ e ordem prima

$l = 7237005577332262213973186563042994240857116359379907606001950938285454250989$.

O inteiro l tem comprimento binário de 253 bits, pois $2^{252} < l < 2^{253}$; isso não significa que um escalar uniforme contenha 253 bits completos de entropia. A sua entropia é $\log_2 l$, muito próxima de 252 bits. Escalares e chaves privadas são habitualmente guardados em recipientes de 32 bytes, mas o tamanho da representação, o comprimento binário do módulo e a entropia são três grandezas distintas. A ordem total $8l$, por sua vez, é um inteiro de 256 bits e tem 77 algarismos decimais.

```
src/crypto/
crypto_ops_
builder/
src/ringct/
rctOps.h
curve-
Order()
```

2.4.1 Representação binária

O representante canónico de um elemento de $\mathbb{F}_{2^{255}-19}$ está entre 0 e $q-1$, pelo que cabe em 255 bits. A codificação usada por Edwards25519 ocupa 32 bytes em ordem *little-endian*; antes da compressão de pontos, o bit 255 desse recipiente está livre. Duas coordenadas completas ocupariam, portanto, 64 bytes.

Um escalar canónico em $\mathbb{F}_l$ também é guardado em 32 bytes, mas obedece ao limite diferente $0 \leq s < l$. Não devemos confundir uma sequência arbitrária de 32 bytes, um elemento do corpo de coordenadas, um escalar reduzido e uma codificação canónica: têm o mesmo tamanho exterior, não o mesmo domínio.

2.4.2 Compressão de ponto elíptico

Um ponto de Edwards25519 pode ser codificado nos mesmos 32 bytes que uma única coordenada. A ideia geral de compressão de pontos é anterior a esta curva [85]; a codificação de Edwards25519 é descrita em [35, 67].

Com $a = -1$, a equação da curva pode ser reorganizada como

$$x^2 = \frac{y^2 - 1}{dy^2 + 1}, \quad d = -\frac{121665}{121666} \bmod q.$$

Para um y válido, recuperar x consiste em extrair uma raiz quadrada no corpo. Em regra há duas possibilidades, x e $-x$; quando $x = 0$, elas coincidem. Como q é ímpar, os representantes canónicos de $x \neq 0$ e $-x$ têm paridades opostas: por exemplo, módulo 5, os valores 3 e $-3 \bmod 5 = 2$ são um ímpar e um par. Um único bit escolhe, portanto, a raiz pretendida. Esse bit cabe na posição 255, que estava livre na codificação de y . Eis o procedimento conceptual para um ponto (x, y) :

¹⁸ National Institute of Standards and Technology, <https://www.nist.gov/>

Codificar Codifica-se y canonicamente em 32 bytes *little-endian* e coloca-se no bit 255 a paridade do representante canônico de x : zero se x é par, um se é ímpar. Chamemos y' ao resultado.

Descodificar

1. Copie-se o bit 255 de y' para b , limpe-se esse bit e interpretem-se os restantes como y .
Rejeita-se a codificação se $y \geq q$, pois não seria canônica.
2. Calcule-se $u = (y^2 - 1) \bmod q$ e $v = (dy^2 + 1) \bmod q$, de modo que $x^2 = u/v$ no corpo.
3. Calcule-se¹⁹

$$z = uv^3(uv^7)^{(q-5)/8} \bmod q.$$

- (a) Se $yz^2 \equiv u \pmod{q}$, ponha-se $x' = z$.
- (b) Se $yz^2 \equiv -u \pmod{q}$, ponha-se $x' = z 2^{(q-1)/4} \bmod q$.
- (c) Se nenhuma congruência se verificar, y' não codifica um ponto e deve ser rejeitado.
4. Se a paridade de x' diferir de b , ponha-se $x = (-x') \bmod q$; caso contrário, $x = x'$.
Rejeita-se ainda o caso $x = 0$ e $b = 1$, a codificação não canônica de “zero negativo”.
5. O resultado é o ponto descomprimido (x, y) .

O gerador convencional do subgrupo de ordem l é $G = (x, 4/5)$, com a raiz par de x [35]. Por isso, a sua codificação usa $y = 4/5 \bmod q$ e bit de sinal zero.

2.4.3 Algoritmo de assinatura EdDSA

EdDSA designa uma família de assinaturas de estilo Schnorr. *Ed25519* é a instância que usa Edwards25519; não é o nome da curva. O artigo original apresentou-a como uma alternativa eficiente a ECDSA e relatou mais de 100 000 assinaturas por segundo no equipamento então usado [35]. A especificação completa encontra-se na RFC 8032 [68]. Ed25519 deriva o valor efêmero deterministicamente de um segredo e da mensagem, em vez de pedir ao sistema um novo número aleatório por assinatura. Isto evita uma classe importante de falhas de geradores aleatórios, embora não elimine a necessidade de proteger o segredo, a implementação e os cálculos contra falhas e canais laterais. O desenho procura, entre outros objectivos, evitar acessos a posições de memória dependentes de dados secretos e reduzir ataques temporais à cache [35].

A chave privada externa é uma semente s de 32 bytes. Ed25519 calcula uma hash SHA-512 dessa semente. Os primeiros 32 bytes são podados: limpam-se os três bits menos significativos e o bit mais significativo, e fixa-se o segundo bit mais significativo. Interpretado em *little-endian*, o resultado é o inteiro secreto a ; os 32 bytes restantes formam um prefixo secreto p . A chave pública é $A = aG$, transmitida na sua codificação comprimida $[A]$.

Estas grandezas voltam a ilustrar a diferença entre tamanho e entropia. Uma semente uniforme de 32 bytes tem 256 bits de entropia; o inteiro podado a tem comprimento de 255 bits, mas apenas 251 posições variáveis, e não é uma codificação canônica de um escalar menor que l . Já os escalares da assinatura são reduzidos módulo l e codificados em 32 bytes.

¹⁹ Como $q = 2^{255} - 19 \equiv 5 \pmod{8}$, os expoentes $(q-5)/8$ e $(q-1)/4$ são inteiros.

src/crypto/
crypto_ops_
builder/
ref10Comm-
entedComb-
ined/
ge_to-
bytes.c

ge_from-
bytes.c

Assinatura

Para a variante pura Ed25519, seja $\mathbf{m}$ a mensagem exacta. A variante Ed25519ph, que recebe uma pré-hash, é um protocolo distinto e tem de ser seleccionada explicitamente [68]. Escrevendo $\mathcal{H}_l$ para SHA-512 seguida da conversão e redução módulo l :

1. Calcule-se o escalar efémero $\rho = \mathcal{H}_l(p, \mathbf{m})$ e o ponto $R = \rho G$.
2. Calcule-se o desafio $e = \mathcal{H}_l([R], [A], \mathbf{m})$.
3. Calcule-se $S = (\rho + ea) \bmod l$.
4. A assinatura é $([R], [S])$, um ponto comprimido e um escalar canónico.

Verificação

A verificação procede da seguinte forma:

1. Descodifiquem-se $[A]$ e $[R]$ como pontos e $[S]$ como escalar. Rejeitem-se codificações inválidas ou não canónicas e valores $S \geq l$.
2. Calcule-se $e' = \mathcal{H}_l([R], [A], \mathbf{m})$.
3. Aceite-se apenas se a equação de verificação da RFC 8032 se cumprir:

$$8SG \stackrel{?}{=} 8R + 8e'A.$$

O factor $8 = 2^3$ é o cofactor de Edwards25519 [35, 68]. Multiplicar um ponto arbitrário por 8 elimina a sua componente de pequena ordem; não demonstra que o ponto original já pertencia ao subgrupo primo. Em particular, a equação cofactorizada compara os dois lados depois de apagar possíveis componentes de torção. Uma verificação explícita $lA \stackrel{?}{=} 0$, acompanhada da rejeição da identidade, testa se a chave pública pertence ao subgrupo de ordem l . Não é sinónima de multiplicar toda a equação pelo cofactor. Protocolos e bibliotecas podem escolher regras de validação mais estritas, mas signatário e verificador têm de concordar exactamente nas mesmas regras. Esta distinção reaparecerá na secção 3.4.

Funciona porque

Para uma assinatura honesta, $A = aG$ e $R = \rho G$. Logo,

$$\begin{aligned} 8SG &= 8(\rho + ea)G \\ &= 8\rho G + 8e(aG) \\ &= 8R + 8eA. \end{aligned}$$

O verificador recompõe o mesmo e a partir da mensagem, da chave e de $[R]$, pelo que a igualdade se verifica.

Requisitos:

A assinatura Ed25519 guarda um ponto comprimido de 32 bytes e um escalar de 32 bytes: 64 bytes no total. A chave pública comprimida ocupa mais 32 bytes, mas normalmente é fornecida separadamente da assinatura. A semente privada nunca é transmitida.

2.5 Operador binário XOR

O operador binário XOR será útil nas secções 4.4 e 5.3. Recebe dois bits e devolve 1 se exactamente um deles for 1 [21]. Eis a tabela de verdade:

A	B	$A \oplus B$
1	1	0
1	0	1
0	1	1
0	0	0

Bit a bit, XOR é a adição módulo 2. Por exemplo,

$$\{1, 1\} \oplus \{1, 0\} = \{(1 + 1) \bmod 2, (1 + 0) \bmod 2\} = \{0, 1\}.$$

Da tabela seguem três identidades simples:

$$A \oplus 0 = A, \quad A \oplus A = 0, \quad (A \oplus B) \oplus B = A.$$

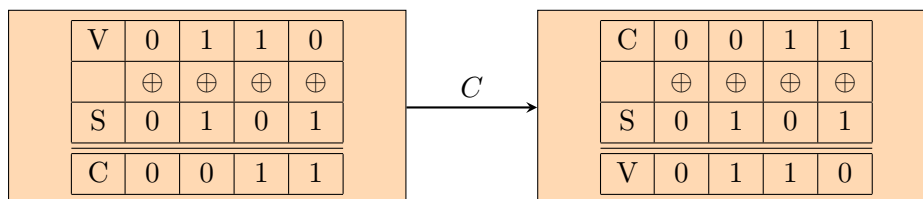
Suponhamos que Alice quer cifrar um vector privado V de n bits. Ela escolhe uma chave S também de n bits e envia a Bob

$$C = V \oplus S.$$

Bob, que conhece S , recupera a mensagem porque

$$C \oplus S = (V \oplus S) \oplus S = V.$$

Para cada valor fixo de C , há 2^n pares (V, S) que o produzem: a cada candidato V corresponde exactamente $S = C \oplus V$.²⁰



Isto só dá segredo perfeito, no sentido da teoria da informação, sob hipóteses fortes: S tem de ser secreta, uniforme em $\{0, 1\}^n$, independente de V e usada uma única vez. Nessas condições,

²⁰ Outra aplicação de XOR é trocar dois registos sem um terceiro: $\{A, B\} \rightarrow \{A \oplus B, B\} \rightarrow \{A \oplus B, A\} \rightarrow \{B, A\}$.

observar C não altera a probabilidade de qualquer mensagem. Se a mesma chave for reutilizada, então $C_1 \oplus C_2 = V_1 \oplus V_2$, o que revela uma relação entre as mensagens. Uma chave enviesada, previsível ou mais curta também destrói a conclusão.

Quando S é derivada de Diffie–Hellman ou de uma palavra-passe, a sua segurança é apenas computacional e depende da função de derivação e das hipóteses subjacentes. Além disso, XOR sozinho não autentica o texto cifrado: uma construção real usa normalmente uma cifra autenticada, como já antecipámos na secção [2.3.3](#).

CAPÍTULO 3

Assinaturas tipo Schnorr avançadas

Entre as ferramentas do capítulo anterior, construímos uma prova de Schnorr a partir de uma só relação $K = kG$. Podemos agora ligar várias relações ao mesmo desafio, esconder a posição da relação verdadeira num anel e, por fim, tornar as assinaturas ligáveis. SAG e bLSAG servirão de antepassados conceptuais; MLSAG explica a evolução histórica de RingCT; CLSAG é a construção usada pelas transacções activas na versão 16 do protocolo. Esta ordem permite-nos ver o mecanismo a crescer sem confundir história com consenso actual.

Em todo o capítulo trabalhamos no subgrupo de ordem prima l e escrevemos $\mathbb{Z}_l$ para os escalares. As listas passadas a uma função hash têm uma codificação canónica e não ambígua: incluem a ordem e o número dos seus elementos. $\mathcal{H}_n$ designa uma função hash para escalares e $\mathcal{H}_p$, quando aparecer, uma função hash para pontos. São operações diferentes.

3.1 Igualdade do logaritmo discreto em várias bases

Suponhamos que Alice publica $K_1 = kJ_1$ e $K_2 = kJ_2$. O primeiro ponto pode ser uma chave pública kG , enquanto o segundo pode ser o segredo Diffie–Hellman kR da secção 2.3.3. Alice quer provar duas coisas ao mesmo tempo: conhece o logaritmo discreto e esse logaritmo é o mesmo nas duas bases. Uma resposta Schnorr comum às duas relações faz precisamente esse trabalho.

Declaração e testemunha

Seja $d \geq 1$. A declaração pública, escrita como listas ordenadas, é

$$\mathcal{J} = (J_1, \dots, J_d), \quad \mathcal{K} = (K_1, \dots, K_d).$$

A testemunha privada é um único $k \in \mathbb{Z}_l$ tal que

$$K_i = kJ_i \quad \text{para todo } i \in \{1, \dots, d\}.$$

Cada J_i e K_i deve ter uma codificação válida, pertencer ao subgrupo de ordem l e não ser a identidade. Sem estas condições, uma base nula poderia tornar uma das relações vazia de conteúdo.

A transformação Fiat–Shamir inclui ainda um rótulo de domínio e um contexto x . Numa assinatura, x contém a mensagem M ; numa prova destinada a outro protocolo, contém a sessão e os objectos a que a prova deve ficar presa.¹

Assumimos que $\mathcal{H}_n$ transforma essa codificação em $\mathbb{Z}_l$. Não se exige que cada entrada tenha uma imagem diferente; uma função hash tem colisões em princípio. A análise usa-a como oráculo aleatório e a segurança computacional exige que seja impraticável encontrar colisões ou prever desafios.²

Prova não interactiva

1. Escolhe-se $\alpha \in_R \mathbb{Z}_l$ e calculam-se os compromissos $A_i = \alpha J_i$, para $i = 1, \dots, d$.

2. Calcula-se o desafio

$$c = \mathcal{H}_n(\text{DLEQ}, x, \mathcal{J}, \mathcal{K}, [A_1], \dots, [A_d]).$$

3. Calcula-se a resposta

$$r = (\alpha - ck) \bmod l.$$

4. Publica-se a prova (c, r) . Os compromissos não precisam de ser publicados separadamente, porque o verificador os reconstrói.

Verificação

O verificador rejeita primeiro pontos ou escalares mal codificados, pontos fora do subgrupo, identidades e dimensões incoerentes. Depois calcula

$$A'_i = rJ_i + cK_i, \quad i = 1, \dots, d,$$

$$c' = \mathcal{H}_n(\text{DLEQ}, x, \mathcal{J}, \mathcal{K}, [A'_1], \dots, [A'_d])$$

e aceita apenas se $c' = c$.

Funciona porque

Para uma prova honesta, cada reconstrução devolve o compromisso original:

$$\begin{aligned} A'_i &= rJ_i + cK_i \\ &= (\alpha - ck)J_i + c(kJ_i) \\ &= \alpha J_i = A_i. \end{aligned}$$

¹ Uma prova deste género torna-se uma assinatura quando a mensagem M é incluída no contexto do desafio.

² No código Monero, a operação correspondente calcula Keccak-256 e reduz os 256 bits módulo l . O escalar zero pertence legitimamente a $\mathbb{Z}_l$; a segurança não depende de excluir esse resultado, cuja ocorrência é de probabilidade desprezável.

Logo o verificador recompõe exactamente a mesma entrada de $\mathcal{H}_n$, pelo que

$$\mathcal{H}_n(\text{DLEQ}, x, \mathcal{J}, \mathcal{K}, [A_1], \dots, [A_d]) = \mathcal{H}_n(\text{DLEQ}, x, \mathcal{J}, \mathcal{K}, [A'_1], \dots, [A'_d]),$$

$$c = c'.$$

Com $d = 1$, recuperamos a prova de Schnorr da secção 2.3.4. Para $d > 1$, usar o mesmo α , o mesmo desafio e a mesma resposta liga as relações. No modelo do oráculo aleatório, duas transcrições aceitáveis com os mesmos compromissos e desafios distintos permitem extrair k ; esta é a razão formal para falar em prova de conhecimento. A conclusão depende da dificuldade do logaritmo discreto no subgrupo e da validação dos pontos, não apenas da igualdade $c = c'$.

O valor α nunca pode ser reutilizado com outro desafio. De $r = \alpha - ck$ e $r' = \alpha - c'k$, um observador obtém

$$k = (r - r')(c' - c)^{-1} \pmod{l}.$$

Também por isso, qualquer geração determinística do valor efêmero deve ficar presa a todo o contexto e a todas as chaves públicas.

3.2 Uma prova com múltiplas chaves privadas

A construção anterior reutilizava uma testemunha em várias bases. Podemos fazer o complemento: provar simultaneamente várias relações, cada qual com a sua testemunha, mas reuni-las num só desafio. É uma composição «E» de provas de Schnorr: a prova só é válida quando todas as relações o são.

Declaração e testemunhas

Para $d \geq 1$, sejam

$$\mathcal{J} = (J_1, \dots, J_d), \quad \mathcal{K} = (K_1, \dots, K_d).$$

A declaração pública afirma que existem testemunhas $(k_1, \dots, k_d) \in \mathbb{Z}_l^d$ tais que

$$K_i = k_i J_i \quad \text{para todo } i \in \{1, \dots, d\}.$$

As bases podem repetir-se, e podem mesmo ser todas iguais a G .³ Tal como antes, o verificador exige listas com a mesma dimensão, codificações canónicas, pontos não nulos no subgrupo de ordem l e escalares canónicos. O contexto público x inclui a mensagem ou a finalidade da prova.

Prova não interactiva

1. Escolhem-se valores efêmeros independentes $\alpha_i \in_R \mathbb{Z}_l$ e calculam-se

$$A_i = \alpha_i J_i, \quad i = 1, \dots, d.$$

2. Calcula-se o único desafio

$$c = \mathcal{H}_n(\text{Schnorr-AND}, x, \mathcal{J}, \mathcal{K}, [A_1], \dots, [A_d]).$$

³Bases repetidas seriam redundantes numa prova de igualdade de logaritmos, mas não aqui: por exemplo, $K_1 = k_1 G$ e $K_2 = k_2 G$ são duas relações distintas. A construção não afirma que os k_i sejam diferentes.

3. Calcula-se uma resposta por testemunha:

$$r_i = (\alpha_i - ck_i) \bmod l, \quad i = 1, \dots, d.$$

4. Publica-se

$$(c, r_1, \dots, r_d).$$

Verificação

Depois de validar a declaração e a prova, o verificador reconstrói

$$\begin{aligned} A'_i &= r_i J_i + c K_i, \quad i = 1, \dots, d, \\ c' &= \mathcal{H}_n(\text{Schnorr-AND}, x, \mathcal{J}, \mathcal{K}, [A'_1], \dots, [A'_d]) \end{aligned}$$

e aceita apenas se $c' = c$.

Funciona porque

Para cada índice,

$$\begin{aligned} r_i J_i + c K_i &= (\alpha_i - ck_i) J_i + c(k_i J_i) \\ &= \alpha_i J_i = A_i. \end{aligned}$$

Assim, todos os argumentos reconstruídos são iguais aos argumentos usados pelo provador. Abreviando por $\mathcal{T} = (\text{Schnorr-AND}, x, \mathcal{J}, \mathcal{K})$ o prefixo comum da transcrição, obtemos

$$\begin{aligned} c' &= \mathcal{H}_n(\mathcal{T}, [A'_1], \dots, [A'_d]) \\ &= \mathcal{H}_n(\mathcal{T}, [A_1], \dots, [A_d]) = c. \end{aligned}$$

O desafio partilhado poupa $d - 1$ escalares em comparação com d provas Fiat–Shamir independentes; não junta as testemunhas numa só. No modelo do oráculo aleatório, duas transcrições com os mesmos compromissos e desafios distintos permitem extrair cada

$$k_i = (r_i - r'_i)(c' - c)^{-1} \pmod{l}.$$

Por conseguinte, a alegação de conhecimento de todas as chaves depende da dificuldade do logaritmo discreto e de a transcrição ligar o domínio, o contexto, todas as bases, todas as chaves e todos os compromissos.

Cada α_i deve ser imprevisível e não pode reaparecer sob outro desafio: reutilizar apenas um deles basta para revelar o respectivo k_i . Índices trocados, listas truncadas ou uma codificação ambígua mudariam a declaração; por isso, dimensões e ordem são parte do que se autentica.

3.3 Assinaturas espontâneas anónimas de grupo

As assinaturas de grupo de Chaum e van Heyst recorriam a uma entidade de confiança para formar o grupo e, em certas variantes, para resolver disputas [45]. Uma assinatura em anel segue outra filosofia: o signatário escolhe espontaneamente um conjunto de chaves públicas e não precisa da autorização nem da colaboração dos respectivos donos. A ideia nasceu no trabalho de

Rivest, Shamir e Tauman [110]; Liu, Wei e Wong desenvolveram depois a variante ligável LSAG [76]. Começaremos por SAG, sem ligação, porque o seu anel de desafios será a engrenagem das construções seguintes.

Declaração, testemunha e índice

Seja M a mensagem e seja

$$\mathcal{R} = (K_1, \dots, K_n), \quad n \geq 1,$$

um anel ordenado de chaves públicas distintas. O signatário conhece um índice secreto $\pi \in \{1, \dots, n\}$ e a testemunha $k_\pi \in \mathbb{Z}_l$ que satisfaz

$$K_\pi = k_\pi G.$$

A declaração é uma disjunção: «conheço a chave privada de *algum* membro de $\mathcal{R}$ ». A posição π não faz parte da assinatura. Todas as chaves devem ter codificações canónicas, ser pontos não nulos do subgrupo de ordem l , e a ordem e o comprimento do anel ficam presos à transcrição.

Para simplificar os índices, convencionamos que $c_{n+1} = c_1$. A função $\mathcal{H}_n$ recebe um rótulo de domínio, o anel inteiro, a mensagem e o compromisso da ronda. O chamado *prefixo de chave* é precisamente esta inclusão explícita de $\mathcal{R}$ no desafio; impede que a mesma transcrição seja destacada de um anel e apresentada como pertencendo a outro.

Assinatura

1. Escolhe-se $\alpha \in_R \mathbb{Z}_l$. Para cada $i \neq \pi$, escolhe-se ainda uma resposta simulada $r_i \in_R \mathbb{Z}_l$.

2. Inicia-se a ronda posterior ao signatário:

$$c_{\pi+1} = \mathcal{H}_n(\text{SAG}, \mathcal{R}, M, [\alpha G]).$$

3. Percorrem-se, por ordem cíclica, $i = \pi + 1, \pi + 2, \dots, n, 1, \dots, \pi - 1$. Em cada posição calcula-se

$$\begin{aligned} L_i &= r_i G + c_i K_i, \\ c_{i+1} &= \mathcal{H}_n(\text{SAG}, \mathcal{R}, M, [L_i]). \end{aligned}$$

4. Fecha-se o anel com a única resposta que não foi simulada:

$$r_\pi = (\alpha - c_\pi k_\pi) \bmod l.$$

A assinatura é

$$\sigma = (c_1, r_1, \dots, r_n).$$

O anel e a mensagem têm de acompanhar a verificação, mas podem estar noutra campo do protocolo; não é necessário repeti-los dentro desta lista de escalares.

Verificação

O verificador confirma $n \geq 1$, o número exacto de respostas e a canonicidade de $c_1, r_1, \dots, r_n$, além de validar todas as chaves do anel. Depois põe $\hat{c}_1 = c_1$ e, para $i = 1, \dots, n$, calcula

$$\begin{aligned}\hat{L}_i &= r_i G + \hat{c}_i K_i, \\ \hat{c}_{i+1} &= \mathcal{H}_n(\text{SAG}, \mathcal{R}, M, [\hat{L}_i]).\end{aligned}$$

No último passo, $\hat{c}_{n+1}$ é o desafio reconstruído para a primeira posição. Aceita-se se e só se

$$\hat{c}_{n+1} = c_1.$$

Comparar o desafio final com o inicial, e não com o desafio da ronda anterior, é o que testa o fecho circular.

Funciona porque

Nas posições simuladas, o verificador repete literalmente os cálculos do signatário. Na posição verdadeira,

$$\begin{aligned}\hat{L}_\pi &= r_\pi G + c_\pi K_\pi \\ &= (\alpha - c_\pi k_\pi)G + c_\pi (k_\pi G) \\ &= \alpha G.\end{aligned}$$

Assim, também essa ronda devolve o compromisso inicial e a cadeia inteira fecha.

Vejamos o antigo exemplo de três membros, agora com os índices bem à vista. Se $\pi = 2$, o signatário escolhe α, r_1, r_3 e calcula

$$\begin{aligned}c_3 &= \mathcal{H}_n(\text{SAG}, \mathcal{R}, M, [\alpha G]), \\ c_1 &= \mathcal{H}_n(\text{SAG}, \mathcal{R}, M, [r_3 G + c_3 K_3]), \\ c_2 &= \mathcal{H}_n(\text{SAG}, \mathcal{R}, M, [r_1 G + c_1 K_1]), \\ r_2 &= (\alpha - c_2 k_2) \bmod l.\end{aligned}$$

Como

$$r_2 G + c_2 K_2 = (\alpha - c_2 k_2)G + c_2 (k_2 G) = \alpha G,$$

o verificador obtém sucessivamente

$$\begin{aligned}\hat{c}_2 &= \mathcal{H}_n(\text{SAG}, \mathcal{R}, M, [r_1 G + c_1 K_1]), \\ \hat{c}_3 &= \mathcal{H}_n(\text{SAG}, \mathcal{R}, M, [r_2 G + \hat{c}_2 K_2]), \\ \hat{c}_1 &= \mathcal{H}_n(\text{SAG}, \mathcal{R}, M, [r_3 G + \hat{c}_3 K_3]) = c_1.\end{aligned}$$

O que a assinatura garante

Sob a dificuldade do logaritmo discreto e no modelo do oráculo aleatório, a cadeia prova conhecimento de uma das chaves privadas sem revelar qual. A distribuição das respostas simuladas deve ser indistinguível da resposta verdadeira; valores efêmeros enviesados, previsíveis ou reutilizados podem denunciar π e, no caso de reutilização, revelar k_π . A infalsificabilidade exige ainda que mensagem, domínio, anel completo e compromissos sejam ligados ao hash e que chaves malformadas ou de pequena ordem sejam rejeitadas.

O tamanho n é apenas o conjunto de anonimidade criptográfico máximo. Se os membros forem escolhidos de maneira reconhecível ou informação externa excluir alguns deles, a ambiguidade efectiva será menor. Para $n = 1$, SAG reduz-se essencialmente a Schnorr e não oferece anonimidade. Finalmente, SAG não é ligável: duas assinaturas válidas não revelam, por si só, se usaram a mesma chave. A imagem de chave da próxima secção acrescentará essa propriedade.

3.4 Assinaturas em anel ligáveis de Adam Back

SAG esconde qual membro assinou, mas permite repetir a mesma chave sem deixar um marcador público. Uma assinatura em anel *ligável* acrescenta esse marcador: a imagem de chave. O objectivo não é revelar o signatário; é permitir que qualquer pessoa reconheça uma segunda utilização da mesma chave de uso único.

LSAG [76] ligava assinaturas relativas ao mesmo anel ordenado. A variante que estudaremos, habitualmente chamada bLSAG, torna a imagem independente dos chamarizes escolhidos. A adaptação foi exposta em [99] a partir de uma observação de Adam Back [32] sobre a assinatura CryptoNote [124], por sua vez relacionada com o trabalho de Fujisaki e Suzuki [59]. É um antepassado conceptual: não é a assinatura usada pelas transacções Monero activas.

Propriedades pretendidas

Ambiguidade do signatário Uma assinatura válida demonstra que alguém conhece a chave privada de um membro do anel, sem identificar a posição, salvo com probabilidade desprezável.

⁴

Ligação Duas assinaturas válidas feitas com a mesma chave pública de uso único produzem a mesma imagem de chave, mesmo que as mensagens e os restantes membros dos anéis sejam diferentes. Reutilizar uma chave apenas como chamariz não cria uma ligação.

Infalsificabilidade Sem conhecer a chave privada de algum membro, um adversário não deve conseguir produzir uma assinatura válida, mesmo depois de escolher mensagens e anéis e observar assinaturas legítimas.

Estas propriedades são afirmações criptográficas no modelo do oráculo aleatório, sob as hipóteses indicadas adiante. Uma imagem repetida não prova, sozinha, propriedade civil, intenção nem validade de um gasto: é preciso validar a assinatura e todas as restantes regras do protocolo.

⁴ O anel dá apenas um limite superior para o conjunto de anonimidade. Chamarizes escolhidos de forma reconhecível, informação temporal ou análise de outras transacções podem excluir membros; aumentar n não transforma automaticamente todos os membros em candidatos igualmente plausíveis.

Hash para ponto e imagem de chave

Além de $\mathcal{H}_n$, que devolve escalares, precisamos de

$$\mathcal{H}_p : \{0, 1\}^* \longrightarrow \mathbb{G},$$

onde $\mathbb{G}$ é o subgrupo de ordem prima l . O resultado deve ser um ponto não nulo e a sua relação de logaritmo discreto com G não deve ser conhecida. Não bastaria calcular $\mathcal{H}_n(K)G$: como esse multiplicador é público, qualquer observador obteria $\mathcal{H}_n(K)K$ sem conhecer a chave privada.⁵

Para o membro verdadeiro $K_\pi = k_\pi G$, define-se

$$I = k_\pi \mathcal{H}_p(K_\pi).$$

I é a imagem de chave. Sendo determinística para a relação (K_π, k_π) , reaparece quando essa chave de uso único volta a assinar; como a base depende de K_π , chaves de uso único diferentes da mesma carteira não ficam linearmente ligadas.

Declaração, testemunha e assinatura

Sejam a mensagem M , o anel ordenado $\mathcal{R} = (K_1, \dots, K_n)$, com $n \geq 1$, o índice secreto $\pi \in \{1, \dots, n\}$ e a testemunha k_π tais que $K_\pi = k_\pi G$. Convencionamos novamente $c_{n+1} = c_1$.

1. Calcula-se a imagem $I = k_\pi \mathcal{H}_p(K_\pi)$.
2. Escolhem-se $\alpha \in_R \mathbb{Z}_l$ e $r_i \in_R \mathbb{Z}_l$ para todo $i \neq \pi$.
3. Inicia-se a ronda seguinte ao signatário:

$$c_{\pi+1} = \mathcal{H}_n(\text{bLSAG}, \mathcal{R}, I, M, [\alpha G], [\alpha \mathcal{H}_p(K_\pi)]).$$

4. Para $i = \pi + 1, \dots, n, 1, \dots, \pi - 1$, calculam-se

$$L_i = r_i G + c_i K_i,$$

$$R_i = r_i \mathcal{H}_p(K_i) + c_i I,$$

$$c_{i+1} = \mathcal{H}_n(\text{bLSAG}, \mathcal{R}, I, M, [L_i], [R_i]).$$

5. Fecha-se o anel com

$$r_\pi = (\alpha - c_\pi k_\pi) \bmod l.$$

A assinatura é

$$\sigma = (I, c_1, r_1, \dots, r_n).$$

Incluimos $\mathcal{R}$ e I no prefixo de cada desafio. A apresentação original de bLSAG omitia o anel nesse hash; o prefixo explícito é a prática prudente para assinaturas deste género e evita depender de uma interpretação implícita da transcrição [70].

⁵ A família Monero historicamente mapeia a codificação da chave para um ponto e limpa o cofactor; veja-se a descrição em [98] e a origem citada em [122]. Hash para ponto e hash para escalar não são operações permutáveis.

Verificação

O verificador confirma as dimensões, valida todos os escalares e chaves públicas e exige

$$I \neq 0 \quad \text{e} \quad lI = 0.$$

Depois põe $\hat{c}_1 = c_1$ e, para $i = 1, \dots, n$, calcula

$$\hat{L}_i = r_i G + \hat{c}_i K_i,$$

$$\hat{R}_i = r_i \mathcal{H}_p(K_i) + \hat{c}_i I,$$

$$\hat{c}_{i+1} = \mathcal{H}_n(\text{bLSAG}, \mathcal{R}, I, M, [\hat{L}_i], [\hat{R}_i]).$$

Aceita apenas se

$$\hat{c}_{n+1} = c_1.$$

A condição $lI = 0$, acompanhada da rejeição da identidade, prova que a imagem pertence ao subgrupo de ordem l . Isto não é o mesmo que multiplicar simplesmente a equação de verificação pelo cofactor. Se se aceitasse $I + T$, com T um ponto de pequena ordem t , um atacante poderia explorar a componente de torção e procurar desafios com resíduos convenientes módulo t , criando variantes que escapassem à ligação. Para um desafio divisível por t , por exemplo,

$$\begin{aligned} c(I + T) &= cI + cT \\ &= cI. \end{aligned}$$

A verificação de subgrupo elimina todas essas variantes. Esta falha existiu nos primeiros anos do Monero e foi corrigida antes de ser explorada, nas regras da versão 5 [58].

```
src/cryptonote_
core/cryptonote_
core.cpp
check_tx_inputs_
keyimages_domain()
```

Funciona porque

Na posição verdadeira, as duas reconstruções devolvem os compromissos de α :

$$\begin{aligned} \hat{L}_\pi &= r_\pi G + c_\pi K_\pi \\ &= (\alpha - c_\pi k_\pi)G + c_\pi (k_\pi G) = \alpha G, \\ \hat{R}_\pi &= r_\pi \mathcal{H}_p(K_\pi) + c_\pi I \\ &= (\alpha - c_\pi k_\pi) \mathcal{H}_p(K_\pi) + c_\pi k_\pi \mathcal{H}_p(K_\pi) \\ &= \alpha \mathcal{H}_p(K_\pi). \end{aligned}$$

As outras rondas foram simuladas exactamente como serão verificadas; portanto a cadeia fecha no c_1 publicado.

Ligação e hipóteses de segurança

Dadas duas assinaturas válidas

$$\sigma = (I, c_1, r_1, \dots, r_n),$$

$$\sigma' = (I', c'_1, r'_1, \dots, r'_{n'}),$$

o teste de ligação é simplesmente $I = I'$. Para uma mesma chave de uso único, ambas calculam

$$I = k_\pi \mathcal{H}_p(K_\pi),$$

logo a igualdade é inevitável. No sentido inverso, concluir que imagens iguais correspondem à mesma relação depende de a função hash para ponto se comportar como um oráculo aleatório no subgrupo, de serem impraticáveis colisões e logaritmos discretos e de todos os pontos terem sido validados.

Há uma hipótese distinta por trás da ambiguidade. Para testar se o candidato $K_i = k_i G$ corresponde à imagem $I = k_i \mathcal{H}_p(K_i)$, um observador teria de decidir se

$$(G, K_i, \mathcal{H}_p(K_i), I)$$

é um quádruplo Diffie–Hellman. A dificuldade deste problema decisional impede o teste. O PCDH pede antes *calcular* I , e o PLD pede recuperar k_i . Um algoritmo para o PLD resolveria também os dois problemas Diffie–Hellman, mas a mera dificuldade conhecida do PLD não demonstra, em geral, a dificuldade decisional ou computacional.

Mesmo quando duas assinaturas estão ligadas, a imagem não revela qual membro assinou. Se os dois anéis tiverem apenas uma chave em comum, porém, a intersecção denuncia essa chave; é uma perda de anonimidade causada pelo contexto dos anéis, não uma quebra da equação. A infalsificabilidade exige, além do logaritmo discreto, prefixos completos, separação de domínio e resistência do Fiat–Shamir no modelo do oráculo aleatório. Chaves públicas malformadas, pontos de torção, respostas não canónicas e cumprimentos incoerentes têm de ser rejeitados antes do fecho da cadeia.

Como em Schnorr e SAG, reutilizar α sob desafios diferentes revela a testemunha. Uma fonte de aleatoriedade defeituosa pode também distinguir a posição verdadeira das respostas simuladas. A assinatura guarda $n + 1$ escalares e uma imagem de chave, além do anel público.

3.5 Assinaturas em anel ligáveis com múltiplas camadas

MLSAG generaliza bLSAG a uma matriz de chaves [99]. Foi uma peça central das primeiras transacções RingCT, pelo que continua a ser necessária para compreender dados históricos e a evolução do protocolo. Não é, contudo, a assinatura criada nem admitida para novas transacções na versão 16: CLSAG foi admitida na versão 13 e tornou-se obrigatória a partir da versão 14 na revisão Monero aqui consultada.

Matriz, declaração e testemunhas

Seja M a mensagem. Fixemos a orientação antes de escrever índices. Há $n \geq 2$ colunas, uma por membro do anel, e $m \geq 1$ linhas, uma por relação que o signatário deve provar. Escrevemos

$$\mathcal{R} = (K_{i,j})_{\substack{1 \leq i \leq n \\ 1 \leq j \leq m}}.$$

O primeiro índice i escolhe a *coluna*; o segundo j , a *linha*. A declaração pública é que existe uma coluna secreta $\pi \in \{1, \dots, n\}$ para a qual o signatário conhece todas as testemunhas

$$(k_{\pi,1}, \dots, k_{\pi,m}) \in \mathbb{Z}_l^m, \quad K_{\pi,j} = k_{\pi,j} G.$$

Portanto, o conjunto de anonimidade tem no máximo n colunas, não nm chaves independentes. Todas as relações verdadeiras partilham o mesmo índice π ; se uma linha denunciar a coluna, denuncia-as a todas.

Na versão integralmente ligável que apresentamos, cada linha tem uma imagem

$$I_j = k_{\pi,j} \mathcal{H}_p(K_{\pi,j}), \quad j = 1, \dots, m.$$

Uma variante permite tornar ligáveis apenas $q \leq m$ linhas: essas linhas têm termos $R_{i,j}$ e imagens I_j ; as restantes têm apenas termos $L_{i,j}$. A fórmula abaixo corresponde a $q = m$, que torna visível a estrutura sem esconder casos num índice adicional.

O verificador exige uma matriz rectangular $n \times m$, pontos canónicos, não nulos e no subgrupo de ordem l , imagens também não nulas no mesmo subgrupo e exactamente nm respostas. Convencionamos $c_{n+1} = c_1$ e abreviamos o prefixo completo por

$$\Phi = (\text{MLSAG}, n, m, \mathcal{R}, I_1, \dots, I_m, M).$$

Esta codificação liga o domínio, as dimensões, cada posição da matriz, as imagens de chave e a mensagem.

Assinatura

1. Calculam-se as imagens

$$I_j = k_{\pi,j} \mathcal{H}_p(K_{\pi,j}), \quad j = 1, \dots, m.$$

2. Escolhem-se valores independentes $\alpha_j \in_R \mathbb{Z}_l$, para todas as linhas, e respostas $r_{i,j} \in_R \mathbb{Z}_l$, para todas as posições com $i \neq \pi$.

3. Inicia-se a ronda seguinte à coluna signatária:

$$c_{\pi+1} = \mathcal{H}_n(\Phi, [\alpha_1 G], [\alpha_1 \mathcal{H}_p(K_{\pi,1})], \dots, [\alpha_m G], [\alpha_m \mathcal{H}_p(K_{\pi,m})]).$$

4. Para as colunas $i = \pi + 1, \dots, n, 1, \dots, \pi - 1$, calculam-se, em cada linha,

$$L_{i,j} = r_{i,j} G + c_i K_{i,j},$$

$$R_{i,j} = r_{i,j} \mathcal{H}_p(K_{i,j}) + c_i I_j, \quad j = 1, \dots, m,$$

e encadeia-se

$$c_{i+1} = \mathcal{H}_n(\Phi, [L_{i,1}], [R_{i,1}], \dots, [L_{i,m}], [R_{i,m}]).$$

5. Na coluna verdadeira, definem-se

$$r_{\pi,j} = (\alpha_j - c_{\pi} k_{\pi,j}) \bmod l, \quad j = 1, \dots, m.$$

A assinatura é

$$\sigma = (I_1, \dots, I_m, c_1, r_{1,1}, \dots, r_{1,m}, \dots, r_{n,1}, \dots, r_{n,m}).$$

Verificação

Após as validações de pontos, escalares e dimensões, põe-se $\widehat{c}_1 = c_1$. Para cada coluna $i = 1, \dots, n$ e linha $j = 1, \dots, m$, calculam-se

$$\begin{aligned}\widehat{L}_{i,j} &= r_{i,j}G + \widehat{c}_i K_{i,j}, \\ \widehat{R}_{i,j} &= r_{i,j}\mathcal{H}_p(K_{i,j}) + \widehat{c}_i I_j.\end{aligned}$$

O desafio seguinte é

$$\widehat{c}_{i+1} = \mathcal{H}_n(\Phi, [\widehat{L}_{i,1}], [\widehat{R}_{i,1}], \dots, [\widehat{L}_{i,m}], [\widehat{R}_{i,m}]).$$

Aceita-se apenas se o desafio depois da última coluna fechar a cadeia:

$$\widehat{c}_{n+1} = c_1.$$

Funciona porque

Nas colunas simuladas, os valores reconstruídos são os que o signatário usou. Na coluna π , para cada j ,

$$\begin{aligned}\widehat{L}_{\pi,j} &= r_{\pi,j}G + c_{\pi}K_{\pi,j} \\ &= (\alpha_j - c_{\pi}k_{\pi,j})G + c_{\pi}(k_{\pi,j}G) = \alpha_j G, \\ \widehat{R}_{\pi,j} &= r_{\pi,j}\mathcal{H}_p(K_{\pi,j}) + c_{\pi}I_j \\ &= (\alpha_j - c_{\pi}k_{\pi,j})\mathcal{H}_p(K_{\pi,j}) + c_{\pi}k_{\pi,j}\mathcal{H}_p(K_{\pi,j}) \\ &= \alpha_j \mathcal{H}_p(K_{\pi,j}).\end{aligned}$$

O verificador recompõe, linha a linha e na mesma ordem, o compromisso que iniciou $c_{\pi+1}$. Uma só resposta errada basta para alterar esse desafio e impedir o fecho.

Ligação, anonimidade e infalsificabilidade

Se alguma testemunha $k_{\pi,j}$ for reutilizada para a mesma chave pública de uso único, a imagem I_j reaparece e liga as assinaturas. Uma chave que surja apenas como chamariz não produz imagem e não cria ligação. Tal como em bLSAG, anéis cuja intersecção seja pequena podem transformar uma ligação numa forte pista sobre π ; a criptografia não corrige um conjunto de anonimidade mal formado.

A infalsificabilidade exige conhecimento de *todas* as testemunhas da coluna escolhida, sob a dificuldade do PLD e no modelo do oráculo aleatório. A associação de uma imagem I_j ao candidato $K_{i,j}$ forma uma instância decisional de Diffie–Hellman $(G, K_{i,j}, \mathcal{H}_p(K_{i,j}), I_j)$; por isso, a ambiguidade requer que esse teste decisional seja difícil. Esta hipótese não deve ser confundida com o PCDH, que pede calcular o quarto ponto, nem com o PLD, que pede recuperar o escalar. Em grupos gerais, uma destas dificuldades não prova automaticamente as outras.

Imagens e chaves fora do subgrupo, identidades, matrizes não rectangulares, ordens de linhas divergentes, desafios ou respostas não canónicos e prefixos incompletos invalidam o argumento de segurança. Reutilizar α_j sob desafios diferentes revela $k_{\pi,j}$, mesmo que os outros valores efémeros sejam novos.

Requisitos de espaço

Na variante com todas as linhas ligáveis, guardam-se $mn + 1$ escalares e m imagens de chave; a matriz contém mn chaves públicas. A assinatura cresce, portanto, com o produto das linhas pelas colunas. CLSAG conservará um único ciclo de n respostas ao agregar as relações de cada coluna.

3.6 Assinaturas em anel ligáveis concisas

CLSAG, de *concise linkable spontaneous anonymous group signature*, conserva a coluna secreta de MLSAG mas agrega as relações dessa coluna antes de percorrer o anel [60]. Em vez de publicar uma resposta por linha e por coluna, publica uma resposta por coluna. É esta a assinatura em anel usada pelas transacções activas na versão 16 do Monero.⁶

Chaves primárias e auxiliares

Seja M a mensagem. Voltamos à matriz

$$\mathcal{R} = (K_{i,j})_{\substack{1 \leq i \leq n, \\ 1 \leq j \leq m}},$$

com $n \geq 1$ colunas e $m \geq 1$ linhas. A linha $j = 1$ contém as chaves *primárias*; as linhas $j > 1$, as chaves *auxiliares*. O signatário conhece uma coluna secreta π e todas as testemunhas

$$K_{\pi,j} = k_{\pi,j}G, \quad j = 1, \dots, m.$$

A ligação deve depender apenas da chave primária $k_{\pi,1}$; as relações auxiliares têm de ser provadas, mas não devem criar marcadores de gasto independentes.

Para cada coluna, define-se uma única base de imagem

$$H_i = \mathcal{H}_p(K_{i,1}).$$

As imagens da coluna verdadeira são

$$I_j = k_{\pi,j}H_{\pi}, \quad j = 1, \dots, m.$$

I_1 é a imagem primária e será usada no teste de ligação; $I_2, \dots, I_m$ são imagens auxiliares necessárias à agregação.

Todos os escalares, pontos, dimensões e codificações são validados como nas secções anteriores. Em particular, a imagem primária deve ser não nula e pertencer ao subgrupo de ordem l . O caso $n = 1$ é algebricamente válido, mas não oferece anonimidade; nas transacções v16 o anel tem exactamente 16 membros.

Pesos de agregação

Não podemos somar as linhas com coeficientes fixos. Um atacante poderia escolher uma chave ou imagem auxiliar que cancelasse outra e obter um agregado legítimo a partir de componentes

⁶ No código consultado, CLSAG é admitida desde a versão 13 do protocolo e obrigatória para novas transacções RingCT desde a versão 14. MLSAG continua necessária para interpretar formatos históricos, mas não é válida para uma nova transacção v16.

ilegítimos. Por isso, CLSAG deriva um peso independente para cada linha:

$$\mu_j = \mathcal{H}_n(T_j, m, n, \mathcal{R}, I_1, \dots, I_m), \quad j = 1, \dots, m.$$

Os rótulos T_j separam os domínios e toda a declaração, incluindo as imagens, fica presa a cada peso [60]. Definem-se então

$$\begin{aligned} W_i &= \sum_{j=1}^m \mu_j K_{i,j}, \\ \widetilde{W} &= \sum_{j=1}^m \mu_j I_j, \\ w_\pi &= \sum_{j=1}^m \mu_j k_{\pi,j}. \end{aligned}$$

Na coluna verdadeira,

$$W_\pi = w_\pi G, \quad \widetilde{W} = w_\pi H_\pi.$$

Assim, muitas relações transformam-se numa só relação bLSAG, sem perder a vinculação às componentes.

Assinatura

Seja

$$\Psi = (T_c, m, n, \mathcal{R}, I_1, \dots, I_m, M)$$

o prefixo das rondas, com um domínio T_c distinto dos domínios de agregação. Convencionamos $c_{n+1} = c_1$.

1. Calculam-se H_i , as imagens I_j , os pesos μ_j , as chaves agregadas W_i , a imagem agregada $\widetilde{W}$ e a testemunha agregada w_π .
2. Escolhem-se $\alpha \in_R \mathbb{Z}_l$ e $r_i \in_R \mathbb{Z}_l$ para todo $i \neq \pi$.
3. Inicia-se a ronda seguinte à coluna signatária:

$$c_{\pi+1} = \mathcal{H}_n(\Psi, [\alpha G], [\alpha H_\pi]).$$

4. Para $i = \pi + 1, \dots, n, 1, \dots, \pi - 1$, calculam-se

$$\begin{aligned} L_i &= r_i G + c_i W_i, \\ R_i &= r_i H_i + c_i \widetilde{W}, \\ c_{i+1} &= \mathcal{H}_n(\Psi, [L_i], [R_i]). \end{aligned}$$

5. Fecha-se a cadeia com

$$r_\pi = (\alpha - c_\pi w_\pi) \bmod l.$$

A assinatura conceptual é

$$\sigma = (I_1, \dots, I_m, c_1, r_1, \dots, r_n).$$

Verificação

O verificador recompõe os mesmos pesos, W_i e $\widetilde{W}$. Depois põe $\widehat{c}_1 = c_1$ e, para $i = 1, \dots, n$, calcula

$$\begin{aligned}\widehat{L}_i &= r_i G + \widehat{c}_i W_i, \\ \widehat{R}_i &= r_i H_i + \widehat{c}_i \widetilde{W}, \\ \widehat{c}_{i+1} &= \mathcal{H}_n(\Psi, [\widehat{L}_i], [\widehat{R}_i]).\end{aligned}$$

Aceita apenas se

$$\widehat{c}_{n+1} = c_1.$$

O desafio que se compara é, uma vez mais, o valor obtido *depois* da última coluna; trocar c_n , c_{n+1} e c_1 seria um erro de fronteira capaz de destruir o teste circular.

Funciona porque

Na coluna verdadeira, a resposta agregada reconstrói ambos os compromissos:

$$\begin{aligned}\widehat{L}_\pi &= r_\pi G + c_\pi W_\pi \\ &= (\alpha - c_\pi w_\pi)G + c_\pi(w_\pi G) = \alpha G, \\ \widehat{R}_\pi &= r_\pi H_\pi + c_\pi \widetilde{W} \\ &= (\alpha - c_\pi w_\pi)H_\pi + c_\pi(w_\pi H_\pi) = \alpha H_\pi.\end{aligned}$$

As rondas simuladas coincidem por construção; portanto o desafio volta ao c_1 publicado.

Os pesos são parte da correcção, não apenas uma compressão. Como cada μ_j depende de todas as chaves e imagens, escolher uma componente para cancelar outra altera os próprios coeficientes. No modelo do oráculo aleatório, encontrar o ponto fixo necessário para esse ataque tem probabilidade desprezável.

Especialização usada pelo Monero

Nas transacções Monero há duas linhas. A primeira contém chaves públicas de uso único

$$P_i = p_i G.$$

A segunda contém diferenças entre o compromisso candidato C_i^{mz} e um compromisso de compensação comum C_{off} :

$$C_i = C_i^{\text{mz}} - C_{\text{off}} = z_i G.$$

Na coluna verdadeira, o signatário conhece $p = p_\pi$ e $z = z_\pi$. Com $H_i = \mathcal{H}_p(P_i)$, calcula

$$I = p H_\pi,$$

$$D = z H_\pi.$$

I é a imagem primária. A assinatura serializa a imagem auxiliar como $\overline{D} = 8^{-1}D$; o verificador recupera $D = 8\overline{D}$. Esta multiplicação pelo cofactor coloca a imagem auxiliar no subgrupo primo e o verificador rejeita $D = 0$. A imagem I , que determina a ligação, é rejeitada se for nula ou se $II \neq 0$.

```
src/ringct/
rctSigs.cpp
CLSAG.Gen()
verRctCLSAG
Simple()
```

Na implementação consultada, os pesos são exactamente

$$\mu_P = \mathcal{H}_n(\text{CLSAG_agg_0}, P, C^{\text{nz}}, I, \overline{D}, C_{\text{off}}),$$

$$\mu_C = \mathcal{H}_n(\text{CLSAG_agg_1}, P, C^{\text{nz}}, I, \overline{D}, C_{\text{off}}).$$

As duas relações agregadas são

$$W_i = \mu_P P_i + \mu_C (C_i^{\text{nz}} - C_{\text{off}}),$$

$$\widetilde{W} = \mu_P I + \mu_C D,$$

e a testemunha verdadeira é

$$w_\pi = \mu_P p + \mu_C z.$$

O hash de cada ronda usa o domínio `CLSAG_round` e liga, pela ordem serializada, todas as P_i , todos os C_i^{nz} , C_{off} , a mensagem da transacção e os dois compromissos L_i, R_i . I e $\overline{D}$ ficam ligados às rondas através dos pesos μ_P, μ_C . Em notação compacta:

$$L_i = r_i G + c_i \{ \mu_P P_i + \mu_C (C_i^{\text{nz}} - C_{\text{off}}) \},$$

$$R_i = r_i H_i + c_i (\mu_P I + \mu_C D).$$

A assinatura serializada contém $r_1, \dots, r_n$, c_1 e $\overline{D}$. I já aparece como imagem da entrada da transacção e é repostado na estrutura antes da verificação; não é duplicado no corpo CLSAG. O verificador da revisão consultada rejeita anéis vazios, números errados de respostas, c_1 ou respostas não canónicos, $I = 0$, $8\overline{D} = 0$ e codificações de pontos inválidas; a validação da transacção exige ainda $lI = 0$. Cada desafio de ronda reconstruído tem também de ser não nulo.

Ligação e hipóteses de segurança

O teste de ligação usa somente

$$I = k_{\pi,1} \mathcal{H}_p(K_{\pi,1}).$$

Se a chave primária de uso único assinar de novo, I reaparece. As imagens auxiliares não entram nesse teste: existem para provar a consistência da agregação, e tratá-las como marcadores poderia criar ligações que o protocolo não pretende.

A infalsificabilidade e o conhecimento da coluna dependem da dificuldade do PLD, da segurança Fiat–Shamir no modelo do oráculo aleatório e de os pesos impedirem cancelamentos de chaves. A impossibilidade de calcular uma imagem a partir de G, K, H sem a testemunha é uma hipótese do tipo PCDH; esconder a qual chave K_i corresponde a (H_i, I) exige a dificuldade do problema decisional de Diffie–Hellman. Resolver o PLD permitiria resolver os dois problemas Diffie–Hellman; o inverso, e a equivalência entre as respectivas dificuldades, não se seguem sem uma redução para o grupo usado.

A unicidade útil de I exige ainda uma função hash para ponto resistente a colisões, bases no subgrupo correcto e rejeição de identidade e torção. Uma imagem repetida só estabelece ligação entre assinaturas válidas; não prova, isoladamente, que uma saída pertencia a alguém nem que todas as regras de valor de uma transacção foram cumpridas.

Reutilizar α em dois desafios revela a testemunha agregada w_π . Se a reutilização ocorrer sob conjuntos de pesos independentes, várias equações lineares podem ainda revelar as testemunhas

individuais. Respostas previsíveis ou não uniformes podem denunciar π . Por isso, valor efêmero, mensagem, matriz, imagens, dimensões, domínios e todos os prefixos devem ficar ligados à mesma execução.

Requisitos de espaço

Na forma geral, CLSAG guarda $n+1$ escalares e m imagens, usando nm chaves públicas. A redução de $mn + 1$ para $n + 1$ escalares é a vantagem concisa em relação a MLSAG. Na especialização Monero, a imagem primária acompanha a entrada e a assinatura serializa apenas a imagem auxiliar, além desses $n + 1$ escalares.

CAPÍTULO 4

Endereços

Um endereço de Monero não é um cofre escrito na lista de blocos. É uma instrução pública que permite a um remetente criar uma nova saída para o destinatário. A lista de blocos guarda essa saída sob uma chave pública de utilização única; quem conhecer a chave privada correspondente poderá depois usá-la como entrada de uma nova transacção. A mesma saída não pode ser gasta duas vezes.

Por isso, quando dizemos que um endereço tem um saldo, abreviamos uma ideia mais precisa: uma carteira encontrou saídas que consegue detectar para esse endereço, determinou os respectivos montantes e ainda não as marcou como gastas. A conservação dos valores e a ocultação dos montantes serão tratadas nos capítulos 5 e 6; aqui isolaremos o encaminhamento e a detecção das saídas.

Salvo indicação em contrário, todas as observações sobre a implementação neste capítulo referem-se à fotografia do código Monero identificada no resumo inicial. A distinção será importante: algumas formas estão impostas pelo consenso da versão 16, ao passo que outras são convenções das carteiras para que remetentes e destinatários se entendam.

4.1 Chaves

Bob escolhe dois escalares privados não nulos do corpo escalar, $k_B^s, k_B^v \in \mathbb{Z}_l^*$, e publica os pontos

$$K_B^s = k_B^s G, \quad K_B^v = k_B^v G.$$

Como na Secção 2.3.2, G gera o subgrupo de ordem prima l . Chamaremos a k_B^s *chave privada de gasto* e a k_B^v *chave privada de vista*; os expoentes s e v recordar-nos-ão essas duas funções. O endereço principal é o par público (K_B^s, K_B^v) , nunca o par privado.

A separação permite entregar k_B^v , juntamente com o endereço público, a uma máquina de leitura ou a um auditor sem entregar k_B^s . Uma tal *carteira só de vista* consegue detectar saídas que lhe foram dirigidas, reconhecer o sub-endereço usado, decodificar os montantes RingCT e ler um ID de pagamento curto destinado a Bob. Não consegue calcular a chave privada de utilização única, criar a respectiva imagem de chave, autorizar um gasto nem assinar uma transacção. Sem imagens de chave importadas de uma carteira completa, também não consegue decidir com segurança se uma saída detectada já foi gasta. Ver não é possuir; muito menos é gastar.

As carteiras determinísticas da implementação consultada geram primeiro k^s e derivam normalmente $k^v = \mathcal{H}_n(k^s)$. É uma convenção de recuperação da carteira, não uma equação exigida pelo consenso; também é possível importar dois segredos independentes. Em qualquer caso, os segredos devem ser escalares canónicos e os pontos públicos esperados pertencem ao subgrupo gerado por G .

Codificação de um endereço principal

Para comunicar o par sem uma salada de bytes, Monero usa uma variante própria de Base58 [25, 4]. Na rede principal, a entrada binária de um endereço principal é

$$\text{varint}(18) \parallel K^s \parallel K^v,$$

pela mesma ordem em que os dois pontos aparecem na estrutura `account_public_address`. Calcula-se Keccak-256 sobre essa entrada, juntam-se os primeiros quatro bytes do resultado como soma de verificação e codifica-se tudo em blocos Base58. O resultado tem 95 caracteres e começa habitualmente por «4». O número 18, e não esse primeiro carácter isolado, é o prefixo que identifica simultaneamente a rede e o tipo de endereço.

Ao decodificar, a implementação exige Base58 bem formada, soma de verificação correcta, prefixo da rede e comprimento apropriados, e duas codificações canónicas de pontos da curva. A soma de quatro bytes detecta enganos de cópia; não autentica Bob, não cifra o endereço e não prova que alguém conhece as chaves privadas.

Requisitos. A rotina `check_key()` confirma que cada sequência de 32 bytes representa canonicamente um ponto da curva, mas não testa por si só se o ponto é não nulo e pertence ao subgrupo de ordem l . Uma carteira robusta deve exigir estas últimas condições para chaves de endereço, além de rejeitar escalares nulos ou não canónicos. As chaves produzidas pelo gerador normal satisfazem-nas por construção.

```
src/
common/
base58. cpp
encode_
addr();
src/ crypto-
note_ basic/
crypto-
note_ basic_
impl. cpp
```

4.2 Endereços ocultos

Alice não escreve o endereço principal de Bob numa saída. Aplica-lhe antes uma troca semelhante a Diffie–Hellman e cria uma chave pública nova para cada saída [124]. O livro chamou historicamente

a estes objectos *endereços ocultos*; diremos também, com maior precisão, *chaves de saída de utilização única*. Um observador pode ver a chave de saída, mas não o endereço público de que ela nasceu.

Uma saída para um endereço principal

Isolemos a saída de índice $t = 0$ em que Alice paga 0,123 XMR a Bob. Uma transacção RingCT comum da versão 16 tem pelo menos duas saídas, mas a construção de cada uma pode ser estudada separadamente. Alice conhece (K_B^s, K_B^v) e procede assim:

1. Gera, de modo uniforme, um escalar efémero não nulo $r \in_R \mathbb{Z}_l^*$. A função usada pela implementação rejeita zero e produz uma representação escalar canónica.
2. Calcula a *chave pública de transacção* ordinária

$$R = rG$$

e a derivação partilhada

$$\mathcal{D}_B = 8rK_B^v.$$

O factor 8 é o cofactor da curva. Faz parte da rotina `generate_key_derivation()` e não deve ser omitido.

3. Liga a derivação à posição serializada da saída:

$$h_0 = \mathcal{H}_n(\mathcal{D}_B, \text{varint}(0)),$$

$$K_0^o = h_0G + K_B^s.$$

A função $\mathcal{H}_n$ reduz o hash módulo l ; o índice é codificado como inteiro de comprimento variável, não como o carácter «0».

4. Coloca K_0^o no alvo da saída e, segundo a convenção das carteiras, coloca R no campo `extra`. Na versão 16 o alvo leva ainda a etiqueta de vista que definiremos abaixo. O compromisso que oculta o montante será explicado na Secção 5.3.

Ao percorrer a lista de blocos, Bob lê R e calcula

$$\mathcal{D}'_B = 8k_B^v R.$$

As duas partes obtêm exactamente os mesmos bytes de derivação, pois

$$\mathcal{D}'_B = 8k_B^v(rG) = 8r(k_B^v G) = 8rK_B^v = \mathcal{D}_B.$$

Para o índice $t = 0$, Bob recompõe h_0 e subtrai

$$K_0^o - h_0G = K_B^s.$$

Uma carteira guarda as chaves públicas de gasto que reconhece numa tabela; a igualdade coloca esta saída na conta principal de Bob. Repetirá o teste para os índices e chaves públicas de transacção apropriados.

Funciona porque a chave privada correspondente é

$$k_0^o = h_0 + k_B^s \pmod{l},$$

$$K_0^o = (h_0 + k_B^s)G = k_0^o G.$$

```
src/crypto/
crypto.cpp
generate_
key_ deri-
vation(),
deri-
vation_
to_
scalar() e
derive_
pu_ blic_
key()
```

Alice conhece r , $\mathcal{D}_B$, h_0 e K_0^o , mas não conhece k_B^s e portanto não obtém k_0^o . Uma carteira só de vista conhece k_B^v e reconhece K_B^s , mas continua sem a parcela k_B^s . Só a carteira completa calcula k_0^o , a imagem de chave e a assinatura necessárias para gastar. A detecção é uma conclusão local da carteira, não uma prova pública de propriedade; veremos a autorização do gasto no Capítulo 6.

Cofactor, codificações e pontos malformados

Se as duas chaves públicas provêm de escalares sobre G , todos os pontos acima já estão no subgrupo de ordem l . A multiplicação por 8 tem uma função mais geral: elimina a componente de torção de qualquer ponto Edwards descodificável antes de este ser usado como derivação. Não transforma, porém, uma chave maliciosa numa chave autenticada. Um ponto de baixa ordem pode resultar na identidade depois dessa multiplicação, e uma sequência que nem sequer codifique canonicamente um ponto faz `generate_key_derivation()` falhar.

As carteiras devem, pois, rejeitar ou saltar chaves públicas de transacção malformadas e exigir chaves de endereço não nulas no subgrupo primo. Também devem evitar o caso negligível $k_t^o = 0$, regenerando o segredo efêmero se necessário. A implementação gera r uniformemente em $\mathbb{Z}_l^*$, e os seus pontos honestos têm codificação canónica.

Há aqui uma fronteira de protocolo fácil de perder. O consenso valida que a chave pública de cada saída é uma codificação de ponto admissível e, na versão 16, que o alvo tem a variante com etiqueta. Não prova a relação entre K_t^o , R e qualquer endereço, nem exige que uma chave pública de transacção semanticamente válida exista em `extra`; esses são contractos de interoperabilidade das carteiras. A rotina de consenso `check_key()` também não impõe, isoladamente, pertença ao subgrupo primo. Uma transacção deliberadamente defeituosa pode assim ser aceite pelo consenso e, ainda assim, criar uma saída que a carteira pretendida não consiga encontrar ou gastar.

Etiquetas de vista

Testar $K_t^o - h_t G$ contra uma grande tabela de chaves públicas custa mais do que comparar um byte. Para cada derivação e índice, o remetente da implementação consultada calcula

$$v_t = \text{primeiroByte}(\mathcal{H}(\text{view_tag} \parallel \mathcal{D}_B \parallel \text{varint}(t))),$$

onde $\mathcal{H}$ é Keccak-256 sem redução escalar, e inclui v_t junto de K_t^o . O sal ASCII de oito bytes `view_tag` separa este uso dos restantes hashes.

Ao examinar uma saída, Bob calcula primeiro o seu próprio v_t . Se os bytes diferirem, a carteira salta logo a derivação pública completa. Se coincidirem, a saída é apenas uma *candidata*: a carteira ainda tem de subtrair $h_t G$ e encontrar a chave de gasto resultante na sua tabela. Para saídas alheias, e sob o comportamento pseudo-aleatório do hash, o filtro elimina em média 255 de cada 256; cerca de uma em 256 passa como falso positivo e recebe o teste completo.

Na versão 15 houve um período de transição em que uma transacção podia usar alvos antigos sem etiqueta ou alvos etiquetados, desde que todas as suas saídas tivessem o mesmo tipo. Na versão 16,

```
src/crypto/
crypto.cpp
derive_
view_
tag();
src/crypto-
note.basic/
cryptonote_
format_
utils.cpp
```

congelada para esta edição, cada saída tem obrigatoriamente a variante `txout_to_tagged_key` e uma etiqueta de exactamente um byte. Saídas históricas sem etiqueta continuam a ser lidas pelo caminho de varrimento completo.

O consenso verifica a presença do byte, mas não consegue verificar se ele foi derivado do segredo partilhado. Uma etiqueta errada pode fazer uma carteira ignorar uma saída que, pela equação de K_t^o , lhe pertenceria. Logo, a etiqueta é uma optimização de varrimento, não prova de propriedade, não autorização de gasto e não mecanismo independente de privacidade. Um observador sem r nem k_B^v continua sem poder calcular $\mathcal{D}_B$; quem recebe a chave privada de vista calcula tanto a etiqueta como o teste completo. Para revelar partes escolhidas do histórico sem entregar essa chave, recorreremos ao Capítulo 8.

4.2.1 Transacções de múltiplas saídas

A transacção corrente costuma pagar ao destinatário e devolver troco ao remetente. Para transacções RingCT não mineiras, a versão 16 exige por consenso pelo menos duas saídas. A forma canónica de Bulletproof+ usada nessa versão agrega no máximo 16; por isso, uma transacção corrente tem $2 \leq p \leq 16$ saídas. As razões de valor e a prova de intervalo ficam para o capítulo seguinte.

Antes de criar as chaves, a carteira remetente baralha os destinos. O *índice de saída* $t \in \{0, \dots, p-1\}$ é a posição final no vector serializado `vout`, e é essa posição que todos os cálculos devem usar. Para uma transacção cujos destinatários têm endereços principais, um único segredo fresco $r \in_R \mathbb{Z}_l^*$ e $R = rG$ bastam. Para a saída t , destinada a (K_t^s, K_t^v) , definimos

$$\begin{aligned}\mathcal{D}_t &= 8rK_t^v = 8k_t^v R, \\ h_t &= \mathcal{H}_n(\mathcal{D}_t, \text{varint}(t)), \\ K_t^o &= h_t G + K_t^s = (h_t + k_t^s)G, \\ k_t^o &= h_t + k_t^s \pmod{l}.\end{aligned}$$

Mesmo que Alice pague duas vezes ao mesmo endereço na mesma transacção, os índices diferentes entram no hash e produzem chaves de saída distintas, salvo colisão de probabilidade negligível. A etiqueta de vista de cada saída usa a mesma $\mathcal{D}_t$ e o mesmo `varint(t)`. O destinatário recompõe ambos a partir de $8k_t^v R$; uma carteira completa soma depois k_t^s para obter a testemunha privada.

É correcto partilhar r entre estas saídas da *mesma* transacção. Não se deve reutilizá-lo noutra transacção: R repetiria imediatamente, ligando as duas construções, e repetiria também as derivações para os mesmos destinatários. Se houver sub-endereços, a escolha de R e o número de segredos efémeros mudam; veremos já esse caso na Secção 4.3.

```
src/crypto-
note_core/
cryptonote-
tx_utils.cpp
construct-
tx_with-
tx_key()
```

4.3 Sub-endereços

Um endereço principal pode originar muitos sub-endereços. Todos conduzem à mesma carteira, mas Bob pode entregar um diferente a cada cliente, factura ou serviço e, quando a saída for

detectada, saber qual deles foi usado. Ao contrário de publicar repetidamente o endereço principal, isto evita que um observador ligue essas identidades apenas por igualdade textual [96].

Os sub-endereços entraram no software associado à versão 7 do protocolo [69]. A carteira mantém uma tabela das respectivas chaves públicas de gasto: construir essa tabela requer tempo e memória proporcionais ao número de sub-endereços, mas cada saída candidata pode ser classificada por uma consulta à tabela, sem uma nova passagem completa pela lista de blocos para cada endereço.

Derivação e codificação

O índice de uma conta e de um sub-endereço é o par

$$\iota = (j, i) \in \{0, \dots, 2^{32} - 1\} \times \{0, \dots, 2^{32} - 1\}.$$

A implementação reserva $\iota = (0, 0)$ para o endereço principal. Para outro par, codifica j e i como inteiros de 32 bits em ordem *little-endian* e define

$$T_{\text{sub}} = \text{SubAddr} \parallel 0x00,$$

uma sequência de oito bytes: os sete caracteres ASCII e o byte NUL final. A partir das chaves principais de Bob calcula

$$\begin{aligned} m_\iota &= \mathcal{H}_n(T_{\text{sub}} \parallel k_B^v \parallel \text{LE32}(j) \parallel \text{LE32}(i)), \\ k_B^{s,\iota} &= k_B^s + m_\iota \pmod{l}, \\ K_B^{s,\iota} &= K_B^s + m_\iota G = k_B^{s,\iota} G, \\ K_B^{v,\iota} &= k_B^v K_B^{s,\iota}. \end{aligned}$$

O sub-endereço é o par $(K_B^{s,\iota}, K_B^{v,\iota})$.

Não se introduz aqui uma nova chave privada $k_B^{v,\iota}$ com o papel da chave de vista principal. Uma carteira completa conhece ambos os factores e sabe que $K_B^{v,\iota} = (k_B^v k_B^{s,\iota})G$. Mas não é esse produto que o algoritmo de varrimento usa: a chave de vista da conta continua a ser k_B^v , agora aplicada como escalar a $K_B^{s,\iota}$. Esta distinção é precisamente o que permite classificar todos os sub-endereços num só varrimento.

Uma carteira só de vista que possua k_B^v e K_B^s consegue recompor m_ι , $K_B^{s,\iota}$ e $K_B^{v,\iota}$, e preencher a tabela de detecção. Não consegue obter $k_B^{s,\iota} = k_B^s + m_\iota$, porque lhe falta k_B^s . Por sua vez, quem recebe apenas um sub-endereço público não aprende o endereço principal. Testar se um sub-endereço e um endereço principal partilham a mesma chave de vista exigiria distinguir a relação Diffie–Hellman $(G, K_B^v, K_B^{s,\iota}, K_B^{v,\iota})$; a privacidade assenta, portanto, nas hipóteses de dificuldade usuais do grupo, não em segredo da fórmula. Divulgar k_B^v permite fazer o teste e liga todos os sub-endereços da conta. Reutilizar literalmente o mesmo sub-endereço também o liga por igualdade.

Na rede principal, a codificação Base58 usa o prefixo numérico 42 seguido de $K_B^{s,\iota}$ e $K_B^{v,\iota}$, e a mesma soma de verificação de quatro bytes. Tem 95 caracteres e começa habitualmente por «8». Tal como no endereço principal, o carácter visível não substitui a validação do prefixo, do comprimento, da soma e dos pontos. Os pontos produzidos honestamente estão no subgrupo primo; uma carteira deve rejeitar identidades e pontos fora desse subgrupo. O caso $k_B^s + m_\iota = 0 \pmod{l}$ tem probabilidade negligível, mas uma implementação defensiva não deve emitir o endereço degenerado.

```
src/device/
device_
default.cpp
get_sub-
address_
secret_
key() e
get_sub-
address()
```

4.3.1 Enviar para um sub-endereço

Suponhamos que Alice paga a saída de índice t ao sub-endereço $(K_B^{s,\iota}, K_B^{v,\iota})$. Escolhe $r \in_R \mathbb{Z}_l^*$ e, no caso simples em que esse é o único endereço distinto entre os destinatários que não são troco, publica

$$R = rK_B^{s,\iota}$$

em vez de rG . A derivação e a chave da saída são

$$\begin{aligned}\mathcal{D}_{B,t} &= 8rK_B^{v,\iota}, \\ h_t &= \mathcal{H}_n(\mathcal{D}_{B,t}, \text{varint}(t)), \\ K_t^o &= h_tG + K_B^{s,\iota}.\end{aligned}$$

A etiqueta de vista é calculada sobre esta mesma $\mathcal{D}_{B,t}$ e o mesmo índice.

Bob usa a chave privada de vista principal:

$$\begin{aligned}8k_B^v R &= 8k_B^v (rK_B^{s,\iota}) \\ &= 8r(k_B^v K_B^{s,\iota}) = 8rK_B^{v,\iota} = \mathcal{D}_{B,t}.\end{aligned}$$

Depois do teste da etiqueta, subtrai h_tG de K_t^o . O resultado $K_B^{s,\iota}$ identifica a linha da tabela e, portanto, o índice ι . A carteira completa pode ainda formar

$$\begin{aligned}k_t^o &= h_t + k_B^{s,\iota} = h_t + k_B^s + m_\iota \pmod{l}, \\ K_t^o &= k_t^o G.\end{aligned}$$

Alice conhece r , as duas chaves públicas e h_t , mas não conhece $k_B^{s,\iota}$ e portanto não obtém k_t^o . A carteira só de vista chega à identificação, mas não a esse segredo.

Chave principal e chaves adicionais

Uma só fórmula não descreve todos os lotes de destinatários. Na revisão consultada, a carteira classifica os *endereços distintos que não são de troco* e escolhe as chaves públicas de transacção assim:

- Se todos forem endereços principais, usa uma chave principal $R = rG$, sem chaves adicionais.
- Se houver exactamente um sub-endereço distinto e nenhum endereço principal entre esses destinatários, usa $R = rK_B^{s,\iota}$, sem chaves adicionais. Várias saídas para esse mesmo sub-endereço podem partilhar a derivação; os seus índices continuam a produzir h_t distintos.
- Se misturar endereços principais e sub-endereços, ou se houver mais de um sub-endereço distinto, mantém uma chave principal $R = rG$ e cria um segredo fresco $r_t \in_R \mathbb{Z}_l^*$ para cada posição. A chave adicional alinhada com uma saída de endereço principal é $R_t = r_tG$; para uma saída de sub-endereço, é $R_t = r_tK_t^{s,\iota}$.

As chaves adicionais aparecem em **extra** num vector cuja posição corresponde ao índice final de **vout**. A posição reservada ao troco pode não ser usada para a sua derivação: como o remetente conhece a própria chave de vista, a implementação deriva o troco da chave principal. Esse pormenor não autoriza deslocar as restantes posições.

```
src/crypto/
crypto.cpp
derive_
sub-
address_
public_
key();
src/crypto-
note_core/
crypto-
note_tx_
utils.cpp
```

Numa saída que use R_t , o segredo partilhado é $8r_tK_t^v = 8k_t^vR_t$ para um endereço principal, ou $8r_tK_t^{v,\iota} = 8k_t^vR_t$ para um sub-endereço. É essa derivação, e não a da chave principal, que entra em h_t e na etiqueta de vista da saída.

Ao varrer, a carteira tenta primeiro a derivação da chave principal e, quando existe, a chave adicional da mesma posição t . Uma chave malformada não produz uma derivação válida: o caminho de varrimento afectado falha ou salta o candidato. Quando um auxiliar interno coloca a identidade como valor de recurso, isso marca a falha e não torna válida a chave original. Cada etiqueta de vista tem de ser calculada com a derivação que realmente criou a sua saída. Nem o consenso confirma que o vector está alinhado, nem prova as relações $R = rG$, $R_t = r_tG$ ou $R_t = r_tK^{s,\iota}$: estas são convenções necessárias à descoberta pelas carteiras.

O limite de Janus

O teste de recepção acima confirma que a chave de gasto recuperada pertence à tabela de Bob; não confirma publicamente que as duas metades do endereço fornecido a Alice foram usadas como um par. Suponhamos que um remetente malicioso suspeita da relação entre $(D_1, C_1) = (K_B^{s,1}, k_B^v D_1)$ e $(D_2, C_2) = (K_B^{s,2}, k_B^v D_2)$. Pode escolher r e publicar

$$R = rD_2, \quad K_t^o = \mathcal{H}_n(8rC_2, \text{varint}(t))G + D_1.$$

Bob calcula $8k_B^v R = 8rC_2$, recupera D_1 e associa a saída ao primeiro sub-endereço. Se depois revelar ao remetente que recebeu esse pagamento, a resposta confirma que D_1 e D_2 usam a mesma chave de vista: é o chamado ataque de Janus [51].

Isto não entrega ao atacante a chave privada nem lhe permite gastar a saída; é uma possibilidade de ligação provocada activamente. Evitar confirmações externas automáticas reduz o oráculo, mas uma defesa criptográfica completa exige um contracto adicional entre remetente e carteira receptora. As propostas incluem publicar material que permita à carteira completa verificar que a chave de vista e a chave de gasto usadas pelo remetente pertencem ao mesmo sub-endereço [92]. Tal ligação não é hoje uma regra de consenso, pelo que uma mitigação de carteira deve declarar exactamente o que verifica e como trata transacções antigas.

4.4 Endereços integrados

Um endereço integrado junta um endereço *principal* a um ID de pagamento curto escolhido pelo destinatário. Serve, por exemplo, para Bob atribuir um identificador de oito bytes a uma factura e reconhecer esse identificador quando Alice paga. Não cria outro conjunto de chaves e não é uma variante de sub-endereço.

Na rede principal, a entrada Base58 é

$$\text{varint}(19) \parallel K_B^s \parallel K_B^v \parallel \text{ID}_8,$$

seguida da soma de verificação Keccak de quatro bytes. O texto tem 106 caracteres e começa habitualmente por «4». A descodificação exige exactamente oito bytes de ID. O prefixo integrado

só é aceite com o par de um endereço principal; não existe uma codificação integrada de sub-endereço [8].

Estes endereços continuam suportados na revisão consultada por motivos de compatibilidade. Não devem confundir-se com os antigos IDs de pagamento autónomos de 32 bytes, escritos em claro em **extra**: a carteira corrente recusa criá-los e ignora-os em blocos a partir da versão 12, conservando apenas caminhos de leitura histórica. Um **extra** transporta no máximo um ID curto cifrado, sem indicar a que saída pertence. Por isso, a construção corrente requer uma única chave de vista de destinatário não troco a quem o associar; um pagamento agrupado para destinatários distintos não oferece uma atribuição inequívoca.

Codificar

Se r é o segredo efémero da chave pública principal $R = rG$, Alice forma para o endereço integrado de Bob

$$\mathcal{D}_B = 8rK_B^v,$$

e calcula Keccak-256 sobre os 33 bytes $\mathcal{D}_B \parallel 0x8d$. Sejam os primeiros oito bytes

$$q = \text{primeiros8}(\mathcal{H}(\mathcal{D}_B \parallel 0x8d)).$$

O valor colocado no nonce de **extra** é

$$c = \text{XOR}(\text{ID}_8, q).$$

Aqui não se aplica $\mathcal{H}_n$, não há redução módulo l e não entra qualquer índice de saída. O byte $0x8d$, de valor decimal 141, separa este hash dos hashes das chaves de saída e das etiquetas de vista.

Descodificar

Com a chave pública principal $R = rG$, Bob recompõe

$$\mathcal{D}'_B = 8k_B^v R = \mathcal{D}_B, \quad \text{ID}_8 = \text{XOR}(c, \text{primeiros8}(\mathcal{H}(\mathcal{D}'_B \parallel 0x8d))).$$

A igualdade pressupõe, como nas saídas, que R corresponde ao segredo efémero usado por Alice. A operação XOR é a da Secção 2.5.

Esta transformação apenas oculta o ID a quem não conhece a derivação. Não o autentica, não o liga por consenso a uma saída, não prova que Bob recebeu o pagamento e não impede Alice de escolher qualquer texto cifrado. A presença do campo também distingue estruturalmente a transacção. Para reduzir essa distinção, o construtor da carteira tenta inserir um ID curto nulo cifrado em certas transacções comuns sem ID explícito, quando há no máximo dois destinos e uma única chave de vista aplicável. Trata-se de comportamento de carteira, não de uma regra de consenso; ao descodificá-lo obtêm-se oito bytes nulos.

```
src/device/
device_
default.cpp
encrypt_
payment_
id()
```

4.5 Endereços de multi-assinatura

Uma carteira de multi-assinatura faz várias partes cooperarem para controlar as chaves descritas neste capítulo. Não existe, no entanto, um prefixo Base58, um tipo de endereço ou uma espécie

de saída de consenso reservados para «multi-assinatura»: o endereço partilhado contém o par público habitual e o remetente cria uma chave de saída de utilização única habitual. A diferença está em como os participantes formam e usam os segredos da carteira.

Na revisão de implementação fixada para este capítulo, o suporte da carteira é marcado como experimental, vem desactivado e a interface de linha de comandos exige activá-lo explicitamente com `set enable-multisig-experimental 1`; a interface RPC apresenta igualmente um aviso de perigo. Isto é estado do software, não uma restrição de consenso. O Capítulo 9 desenvolve o protocolo e as suas hipóteses operacionais.

CAPÍTULO 5

Montantes ocultos

O capítulo anterior separou a chave de saída de utilização única do endereço público que lhe deu origem. Falta ocultar o número que essa saída representa. Uma transacção RingCT publica um *compromisso* para cada montante: todos podem verificar relações entre compromissos, mas só quem possui a derivação partilhada apropriada recupera o montante e a máscara da saída.

Um compromisso não basta, porém, para definir dinheiro válido. Os seus escalares vivem módulo l , pelo que um valor modular enorme poderia fazer de «montante negativo». Veremos por isso duas camadas distintas: compromissos de Pedersen ocultam os montantes e preservam somas; provas de intervalo obrigam cada montante de saída a pertencer a $[0, 2^{64} - 1]$.

Salvo indicação em contrário, as referências à implementação neste capítulo dizem respeito à fotografia do código Monero identificada no resumo inicial.

5.1 Compromissos

Um esquema de compromisso permite a Alice fixar uma mensagem sem a revelar de imediato. Mais tarde, Alice apresenta uma *abertura*; Bob verifica que é a mensagem originalmente fixada. Queremos duas propriedades:

Ocultação. Antes da abertura, o compromisso não deve revelar a mensagem.

Vinculação. Depois de publicado o compromisso, Alice não deve conseguir abri-lo como duas mensagens diferentes.

Suponhamos que Alice tem de escolher «cara» ou «coroa» antes de Bob lançar uma moeda. Ela gera um sal aleatório longo s e publica

$$h = \mathcal{H}(s \parallel \text{“cara”}).$$

Depois do lançamento, revela $(s, \text{“cara”})$. Bob repete o hash e compara-o com h . O sal impede-o de ensaiar antecipadamente as duas mensagens possíveis. Neste compromisso por hash, a ocultação depende do sal e a vinculação depende, em particular, de ser impraticável encontrar colisões. Nada nesta construção faz com que compromissos somem como os valores comprometidos.

5.2 Compromissos de Pedersen

Para somar montantes ocultos, empregamos um compromisso de Pedersen [104]. Sejam G e H pontos não nulos do subgrupo de ordem prima l . O grupo é cíclico, portanto existe matematicamente um escalar γ tal que

$$H = \gamma G.$$

Aqui «geradores independentes» não quer dizer independência linear: quer dizer que ninguém conhece essa relação de logaritmo discreto. O valor fixo de H usado pelo Monero nasceu historicamente da interpretação de Keccak-256 da codificação de G como um ponto comprimido, seguida de multiplicação pelo cofactor 8. Essa receita particular não deve ser reutilizada como função hash-para-ponto genérica.

Para uma máscara $x \in \mathbb{Z}_l$ e um montante escalar $b \in \mathbb{Z}_l$, definimos

$$C(x, b) = xG + bH. \quad (5.1)$$

O par (x, b) é uma abertura do ponto C . Os escalares canónicos usados pela implementação ocupam 32 bytes em ordem *little-endian* e representam inteiros menores que l . Um montante Monero é um inteiro sem sinal de 64 bits, colocado nos oito bytes menos significativos de um escalar; os outros 24 bytes são nulos. Os pontos têm a codificação comprimida de Edwards de 32 bytes.

Ocultação. Se x for uniforme em $\mathbb{Z}_l$, então, para qualquer b , o ponto $xG + bH$ é uniforme no subgrupo. O esquema abstracto tem assim ocultação perfeita: mesmo cálculo ilimitado não escolhe uma abertura preferida entre todas as possibilidades [79, 113]. Na carteira, as máscaras de saída são derivadas de um segredo Diffie–Hellman por um hash reduzido; para um observador, a sua pseudo-aleatoriedade é uma garantia computacional, não uma fonte literal de aleatoriedade perfeita.

Vinculação. Se Alice encontrasse duas aberturas distintas $(x, b) \neq (x', b')$ para o mesmo ponto, obteria

$$(x - x')G = (b' - b)H.$$

Salvo o caso trivial em que ambas as diferenças são nulas, isso revelaria a relação entre G e H . A vinculação é portanto *computacional*, sob a dificuldade do logaritmo discreto; não é «vinculação perfeita». Quem conhecesse γ poderia escolher qualquer b' e calcular $x' = x + (b - b')\gamma$, quebrando-a.

```
src/ringct/
rctTypes.h,
H;
tests/unit_
tests/
ringct.cpp,
HPow2

src/ringct/
rctTy-
pes.cpp,
d2h();
src/ringct/
rctOps.cpp,
genC()
```

Homomorfismo aditivo. A aritmética dos escalares é módulo l , e

$$C(x, a) + C(y, b) = C(x + y, a + b), \quad (5.2)$$

$$C(x, a) - C(y, b) = C(x - y, a - b). \quad (5.3)$$

Isto deixa comparar somas ocultas. Não significa que a igualdade pública de dois compromissos revele a igualdade dos montantes: máscaras diferentes podem compensar montantes diferentes. Para provar uma relação entre montantes é preciso também ligar ou cancelar as máscaras de maneira verificável.

5.3 Montantes ocultos

Consideremos a saída t , de montante b_t , enviada a Bob. A Secção 4.2 definiu a derivação partilhada $\mathcal{D}_B$ e o escalar dependente do índice

$$h_t = \mathcal{H}_n(\mathcal{D}_B \parallel \text{varint}(t)).$$

Para um endereço principal ordinário, $\mathcal{D}_B = 8rK_B^v$ do lado de Alice e $\mathcal{D}_B = 8k_B^vR$ do lado de Bob. Subendereços, troco e chaves adicionais escolhem a chave pública de transacção apropriada como descrito no capítulo anterior; depois de obtido h_t , o tratamento do montante é o mesmo.

A carteira calcula a máscara de compromisso

$$y_t = \mathcal{H}_n(\text{commitment_mask} \parallel h_t) \quad (5.4)$$

e publica o compromisso normal

$$C_t^b = C(y_t, b_t) = y_t G + b_t H. \quad (5.5)$$

Aqui y_t é um escalar, não um ponto. A redução do hash produz a sua codificação módulo l ; a construção não rejeita explicitamente a máscara nula, cuja ocorrência tem probabilidade desprezável no modelo do hash.

O montante propriamente dito cabe em oito bytes. Seja

$$q_t = \text{primeiros8}(\mathcal{H}(\text{amount} \parallel h_t)).$$

Alice coloca no campo `ecdhInfo.amount`

$$e_t = \text{LE8}(b_t) \oplus q_t, \quad (5.6)$$

onde $\oplus$ é o XOR da Secção 2.5. Desde o formato `RCTTypeBulletproof2`, incluindo CLSAG e Bulletproof+, só estes oito bytes são serializados; a máscara não é transmitida porque ambos os lados a recompõem pela Equação (5.4). O formato anterior, mais volumoso, guardava 32 bytes de montante e uma máscara codificada; a versão 10 introduziu a forma compacta sob o nome histórico `RCTTypeBulletproof2` [30].

Bob calcula o mesmo h_t , recupera

$$\text{LE8}(b_t) = e_t \oplus q_t$$

e recompõe y_t . A carteira exige que as representações escalares obtidas sejam canónicas, que os 24 bytes superiores do montante sejam nulos e que

$$y_t G + b_t H = C_t^b.$$

Uma chave de vista permite assim detectar a saída, decodificar o montante e confirmar a abertura; não permite gastá-la. O XOR não oferece segurança de teoria da informação nem autenticação

```
src/device/
device_
default.cpp,
generate_
output_
ephemeral_
keys();
src/ringct/
rctOps.cpp,
genCommit-
mentMask()
e ecdh-
Encode()
```


autónoma: a confidencialidade depende computacionalmente do segredo partilhado e do hash, enquanto o compromisso público fornece a verificação de consistência da abertura.

5.4 Introdução a RingCT

Uma saída reúne objectos com funções diferentes. A chave de saída de utilização única liga a saída à autorização de gasto; C_t^b oculta o montante; e e_t permite à carteira certa recuperar esse montante. Nenhum destes objectos, isoladamente, oferece anonimato completo nem prova que a transacção está autorizada.

```
src/ringct/
rctSigs.cpp,
decode-
Rct();
src/ringct/
rctTypes.h,
serialize-
rctsig-
base()
```

Suponhamos que a transacção gasta m saídas anteriores de montantes $a_1, \dots, a_m$, cria p saídas de montantes $b_0, \dots, b_{p-1}$ e declara a taxa pública f . A conservação desejada é

$$\sum_{j=1}^m a_j = \sum_{t=0}^{p-1} b_t + f. \quad (5.7)$$

O verdadeiro compromisso gasto pela entrada j tem a forma

$$C_j^a = x_j G + a_j H.$$

Publicá-lo como a entrada verdadeira denunciaria o membro escolhido no anel. Por isso, o reme-
tente cria um *pseudo-compromisso de saída*

$$\tilde{C}_j^a = x'_j G + a_j H.$$

Para o membro verdadeiro,

$$C_j^a - \tilde{C}_j^a = (x_j - x'_j)G. \quad (5.8)$$

O Capítulo 6 mostrará como a CLSAG prova, sem revelar o membro, conhecimento da chave de gasto e desta diferença de máscaras. Um pseudo-compromisso não é, só por ser publicado, uma prova de propriedade ou de igualdade de montantes.

Os compromissos das novas saídas são os da Equação (5.5). Para os primeiros $m - 1$ pseudo-compromissos, a implementação escolhe máscaras independentes e uniformes em $\mathbb{Z}_l^*$. A última é determinada por

$$x'_m = \sum_{t=0}^{p-1} y_t - \sum_{j=1}^{m-1} x'_j \pmod{l}. \quad (5.9)$$

Logo, $\sum_j x'_j = \sum_t y_t$, e o verificador testa a equação de pontos

$$\boxed{\sum_{j=1}^m \tilde{C}_j^a = \sum_{t=0}^{p-1} C_t^b + fH.} \quad (5.10)$$

Ao expandi-la, o componente em G cancela por construção e sobra a Equação (5.7), interpretada módulo l . O público vê os compromissos e f , mas não as suas aberturas. Esta verificação prova o balanço dos compromissos; a CLSAG trata da autorização e da escolha anónima da entrada, e as provas de intervalo que se seguem excluem montantes de saída modulares ilegítimos.

```
src/ringct/
rctSigs.cpp,
genRct-
Simple() e
verRct-
Semantics-
Simple()
```

5.5 Provas de intervalo

A Equação (5.10) é uma igualdade no subgrupo de ordem l . Sem outra condição, ela aceita escalares que não são montantes Monero. Por exemplo, tomemos entradas de 6 e 5 unidades e saídas formalmente iguais a 21 e -10 . No corpo escalar,

$$(6 + 5) - (21 + (l - 10)) = -l \equiv 0 \pmod{l}.$$

Se as máscaras também se cancelassem, a equação de compromissos passaria, apesar de $l - 10$ não ser um montante admissível. O problema não é que a curva tenha inteiros negativos; é que a mesma classe modular admite representantes que dariam a aparência de criar valor.

Uma prova de intervalo fecha esta porta. Para cada compromisso de saída C_t^b , a afirmação exacta é

$$\exists y_t \in \mathbb{Z}_l, \quad \exists b_t \in \{0, \dots, 2^{64} - 1\} : \quad C_t^b = y_t G + b_t H. \quad (5.11)$$

O montante b_t e a máscara y_t são as testemunhas privadas; o compromisso, os geradores e a prova são públicos. Os extremos são inclusivos: há exactamente 2^{64} valores possíveis, todos representáveis nos oito bytes usados pela carteira.

Esta prova não identifica o destinatário, não prova propriedade, não autoriza o gasto, não impede uma dupla despesa e não verifica por si só o balanço da transacção. Prova apenas a relação de intervalo para os compromissos que entram no seu traslado. A validação RingCT combina-a depois com a equação de balanço e com as assinaturas das entradas.

5.6 Assinaturas em anel Borromean

As primeiras transacções RingCT provaram a Equação (5.11) com assinaturas em anel Borromean [80, 99]. Esta construção permanece no código para validar a história da lista de blocos, mas deixou de ser permitida em novas transacções depois da versão 8 do protocolo. É portanto um formato histórico, não a prova usada pela versão 16.

Vale a pena perceber a ideia, porque nela se vê directamente o intervalo. Escrevemos os 64 bits de b como

$$b = \sum_{i=0}^{63} b_i 2^i, \quad b_i \in \{0, 1\}.$$

O provador escolhe máscaras $x_i \in \mathbb{Z}_l^*$, forma

$$C_i = x_i G + b_i 2^i H \quad (5.12)$$

e obtém

$$C(x, b) = \sum_{i=0}^{63} C_i, \quad x = \sum_{i=0}^{63} x_i \pmod{l}. \quad (5.13)$$

Para cada índice, publica-se o anel de dois pontos

$$(C_i, C_i - 2^i H).$$

Se $b_i = 0$, o provador conhece x_i tal que $C_i = x_i G$. Se $b_i = 1$, conhece o mesmo x_i tal que $C_i - 2^i H = x_i G$. Uma assinatura em anel esconde qual destes dois casos foi usado; a composição Borromean liga os 64 anéis numa só assinatura.

```
src/crypto-
note_core/
blockchain.cpp,
check_tx_
outputs();
src/ringct/
rctSigs.cpp,
proveRange() e
verRange()
```

Funciona porque uma assinatura válida demonstra conhecimento do logaritmo discreto, na base G , de um ponto em cada par. Sob a vinculação do compromisso e a solidez da assinatura, isso obriga cada b_i a ser um bit. A soma pública dos C_i liga os 64 bits ao compromisso original, pelo que $0 \leq b \leq 2^{64} - 1$.

As testemunhas privadas são $(b_0, \dots, b_{63})$ e $(x_0, \dots, x_{63})$. Os dados públicos são C , os 64 pontos C_i , os pontos fixos $2^i H$ e a assinatura. O percurso legado codifica cada ponto e escalar em 32 bytes; descodifica os C_i como pontos canónicos da curva e reduz a Keccak-256 a escalares quando encadeia os desafios. Não existe nesse percurso um teste autónomo de pertença de cada C_i ao subgrupo primo, nem existe no traslado um prefixo textual de separação de domínio. São regras históricas de compatibilidade que não devemos transportar para um protocolo novo.

O custo de armazenamento explica a substituição. Para uma única saída, o objecto histórico contém 64 pontos C_i , dois vectores de 64 respostas e um desafio: $64 + 64 + 64 + 1 = 193$ objectos de 32 bytes, isto é, 6176 bytes antes de pequenos custos de serialização. Além disso, existe uma prova separada para cada saída. As Bulletproofs seguintes comprimem a relação e agregam várias saídas.

5.7 Bulletproofs

Sou mais rápido quando me mexo.

Sundance

Em português: provas de bala.

À partida nem se percebe como é que alguém chega a descobrir o algoritmo anterior, o das assinaturas Borromean. Também não se explicaram todos os vectores de ataque contra essas assinaturas, nem se, para o seu custo de comunicação, eram a forma mais compacta possível. O leitor que pergunta «como é que se chega a tal coisa?» ou «não se pode forjar isto?» ficou sem a história da descoberta.

O que se segue é ainda mais complicado. Não se promete aqui inventar a prova de novo; promete-se algo mais honesto: seguir a bala enquanto ela se mexe, passo a passo, até se perceber por que acerta no alvo. Assim se vê por que o método funciona, que peças impedem a falsificação e onde uma implementação mal feita poderia estragar a magia. Esta é a Bulletproof clássica de Bünz *et al.* [39, 126], apresentada primeiro para um único montante. A agregação e a passagem para Bulletproof+ virão depois; não se atropela esta batalha com a seguinte.

Fixe-se $n = 64$. Se v está entre 0 e $2^{64} - 1$, passa-se primeiro v para a sua representação binária:

$$\underline{a}_L \leftarrow \text{bits}(v), \quad v = \langle \underline{a}_L, \underline{2}^n \rangle, \quad \underline{2}^n = (2^0, 2^1, \dots, 2^{n-1}).$$

Depois impõem-se dois constrangimentos adicionais:

$$v = \langle \underline{a}_L, \underline{2}^n \rangle$$

$$\underline{a}_R = \underline{a}_L - \underline{1}$$

$$\underline{a}_L \circ \underline{a}_R = \underline{0}.$$

O segundo diz que cada coordenada é mesmo um bit: $a_{L,i}(a_{L,i} - 1) = 0$. Agora faz-se o seguinte:

$$\begin{aligned} v &= \langle \underline{a}_L, \underline{2}^n \rangle \\ 0 &= \langle \underline{a}_L - \underline{1} - \underline{a}_R, \underline{y}^n \rangle \\ 0 &= \langle \underline{a}_L \circ \underline{a}_R, \underline{y}^n \rangle. \end{aligned}$$

Aqui entra a primeira pequena faísca. As afirmações vectoriais e as duas igualdades escalares são equivalentes, excepto com probabilidade negligenciável de falhar. Daqui em diante abreviaremos esta ressalva por *enp*: excepto com probabilidade negligenciável. Com $\underline{y}^n = (y^0, y^1, \dots, y^{n-1})$, qualquer vector $\underline{c} = (c_0, c_1, \dots, c_{n-1})$ define o polinómio

$$F(y) = c_0 + c_1 y + c_2 y^2 + \dots + c_{n-1} y^{n-1}.$$

Se $\underline{c} \neq \underline{0}$, este polinómio não nulo tem no máximo $n - 1$ raízes em $\mathbb{Z}_l$. Como o provador já se comprometeu aos vectores antes de o traslado produzir y , só pode esperar que o desafio calhe numa dessas raízes. Para $n = 64$, são no máximo 63 valores bons entre os l valores possíveis: a probabilidade de uma relação falsa sobreviver é no máximo $63/l < 2^{-246}$. É isto que quer dizer *enp*; não é impossível por decreto, é tão improvável que se trata como impossível. Se $\underline{c} = \underline{0}$, pelo contrário, F é o polinómio zero e aceita todos os y . É precisamente essa a resposta honesta.

Continua-se a representar a mesma coisa de outra forma:

$$\begin{aligned} vz^2 &= \langle \underline{a}_L, \underline{2}^n \rangle z^2 \\ 0z &= \langle \underline{a}_L - \underline{1} - \underline{a}_R, \underline{y}^n \rangle z \\ 0 &= \langle \underline{a}_L \circ \underline{a}_R, \underline{y}^n \rangle \end{aligned}$$

Depois de muita manipulação, $\underline{a}_L$ e $\underline{a}_R$ acabam em lados opostos do mesmo produto interno:

$$z^2 v + \delta(y, z) = \langle \underline{a}_L - z\underline{1}, \underline{y}^n \circ (\underline{a}_R + z\underline{1}) + z^2 \underline{2}^n \rangle. \quad (5.14)$$

$$\delta(y, z) = (z - z^2) \langle \underline{1}, \underline{y}^n \rangle - z^3 \langle \underline{1}, \underline{2}^n \rangle.$$

Para ver cada passo até estas equações, veja-se o Apêndice D. À primeira vista, o que se conseguiu foi representar as três regras iniciais de uma forma altamente elaborada. Mas já há movimento: os 64 constrangimentos cabem agora num produto interno. O provador não pode enviar os vectores do lado direito da Equação (5.14), senão o verificador fica a saber os bits de v .

Há portanto que ofuscar $\underline{a}_L$ e $\underline{a}_R$. Por agora, x é a indeterminada formal do polinómio; só depois de T_1, T_2 entrarem no traslado se tornará no desafio concreto:

$$\begin{aligned} &(\underline{a}_L + x\underline{s}_L) - z\underline{1} \quad *^5 \\ &\underline{y}^n \circ (\underline{a}_R + x\underline{s}_R + z\underline{1}) + z^2 \underline{2}^n \quad *^6. \end{aligned}$$

Note-se que se alcançou agora um polinómio diferente. Em geral,

$$z^2v + \delta(y, z) \not\equiv \langle \underline{a}_L + x\dot{\underline{s}}_L - z\underline{1}, \underline{y}^n \circ (\underline{a}_R + x\dot{\underline{s}}_R + z\underline{1}) + z^2\underline{2}^n \rangle.$$

Define-se então o que já se tinha conseguido como $\underline{l}_0 = \underline{a}_L - \underline{1}z$

$$\underline{r}_0 = \underline{y}^n \circ (\underline{a}_R + \underline{1}z) + z^2\underline{2}^n$$

ou seja :

$$z^2v + \delta(y, z) = \langle \underline{l}_0, \underline{r}_0 \rangle = t_0$$

e os factores ofuscantes como $\underline{l}_1 = \dot{\underline{s}}_L$

$$\underline{r}_1 = \underline{y}^n \circ \dot{\underline{s}}_R$$

Forçam-se então novas igualdades: $*^2, *^3 \underline{l}(x) = \underline{l}_0 + \underline{l}_1x$

$$\underline{r}(x) = \underline{r}_0 + \underline{r}_1x$$

Finalmente: $*^4$

$$t(x) = \langle \underline{l}(x), \underline{r}(x) \rangle = t_0 + t_1x + t_2x^2$$

Obtém-se assim um polinómio cujo termo constante $t_0 = z^2v + \delta(y, z)$ está ligado ao compromisso V . O provador enviará compromissos e respostas que formam um circuito aritmético: em conjunto, eles obrigam v a satisfazer o domínio requerido sem mostrar os seus bits.

Calculam-se também os coeficientes de $t(x)$: $*^1$

$$t(x) = \langle (\underline{l}_0 + \underline{l}_1x), (\underline{r}_0 + \underline{r}_1x) \rangle = \langle \underline{l}_0, \underline{r}_0 \rangle + \langle \underline{l}_0, \underline{r}_1x \rangle + \langle \underline{l}_1x, \underline{r}_0 \rangle + \langle \underline{l}_1x, \underline{r}_1x \rangle$$

$$t_0 = \langle \underline{l}_0, \underline{r}_0 \rangle$$

$$t_1x = \langle \underline{l}_0, \underline{r}_1x \rangle + \langle \underline{l}_1x, \underline{r}_0 \rangle = \langle \underline{l}_0, \underline{r}_1 \rangle x + \langle \underline{l}_1, \underline{r}_0 \rangle x = (\langle \underline{l}_0, \underline{r}_1 \rangle + \langle \underline{l}_1, \underline{r}_0 \rangle)x$$

$$t_2x^2 = \langle \underline{l}_1, \underline{r}_1 \rangle x^2$$

$$\begin{aligned}
& G, H, \underline{G}, \underline{H} & {}^0 \perp [init] \\
& V = \gamma G + vH & {}^1 \perp_U(V) \\
& \underline{a}_L \leftarrow \text{bits}(v) \\
& \underline{a}_R = \underline{a}_L - \underline{1} \\
& \dot{\alpha} \leftarrow R() \\
& A = \langle \underline{a}_L, \underline{G} \rangle + \langle \underline{a}_R, \underline{H} \rangle + \dot{\alpha} G & {}^2 \perp_U(A) \\
& \dot{\rho} \leftarrow R() \\
& \dot{s}_L \leftarrow R() \\
& \dot{s}_R \leftarrow R() \\
& S = \langle \dot{s}_L, \underline{G} \rangle + \langle \dot{s}_R, \underline{H} \rangle + \dot{\rho} G & {}^3 \perp_U(S) \\
& y \leftarrow {}^4 \perp_C(), y^{-1} \\
& z \leftarrow {}^5 \perp_C() \\
& (t_0, t_1, t_2) * {}^1 \\
& \dot{\tau}_1 \leftarrow R() \\
& \dot{\tau}_2 \leftarrow R() \\
& T_1 = t_1 H + \dot{\tau}_1 G & {}^6 \perp_U(T_1) \\
& T_2 = t_2 H + \dot{\tau}_2 G & {}^7 \perp_U(T_2) \\
& x \leftarrow {}^8 \perp_C() \\
& \tau_x = \dot{\tau}_1 x + \dot{\tau}_2 x^2 + \gamma z^2 & {}^9 \perp_U(\tau_x) \\
& \mu = x \dot{\rho} + \dot{\alpha} & {}^{10} \perp_U(\mu) \\
& (\underline{l}(x) = \underline{l}_0 + \underline{l}_1 x) * {}^2 \\
& (\underline{r}(x) = \underline{r}_0 + \underline{r}_1 x) * {}^3 \\
& t(x) = \langle \underline{l}(x), \underline{r}(x) \rangle = (t_0 + t_1 x + t_2 x^2) * {}^4 \\
& & {}^{11} \perp_U(t(x)) \\
& x_{ip} \leftarrow {}^{12} \perp_C() \\
& U = x_{ip} H
\end{aligned}$$

$$IP \leftarrow \{\underline{G}, \underline{H}', U, \underline{l}(x), \underline{r}(x)\}$$

O símbolo $\perp$ é o traslado: a memória sequencial da conversa. O operador $\perp_U(\cdot)$ absorve uma mensagem e renova o estado; $\perp_C()$ extrai desse estado um desafio escalar. O j em ${}^j \perp$ marca o instante do traslado, para que provador e verificador voltem a colher o mesmo desafio sem o terem combinado fora da prova.

Vectoros são sublinhados: $\underline{a}$ contém escalares e $\underline{G}$ pontos da curva elíptica. Um ponto sobre uma letra, como em $\dot{\alpha}$ ou $\dot{s}_L$, marca aqui aleatoriedade fresca escolhida por $R()$. Esses segredos não são necessariamente enviados; aparecem escondidos nos compromissos e combinados nas respostas. Já $\underline{G}, \underline{H}$ são geradores públicos determinísticos, que o verificador pode obter sozinho. Nesta apresentação de uma saída, cada vector tem 64 elementos. O argumento de produto interno reduzirá essa dimensão a 1 em $\log_2 64 = 6$ dobras.

Antes das dobras, o provador constrói os dois circuitos aritméticos seguintes. O primeiro amarra o valor à Equação (5.14); o segundo amarra os vectores sem os revelar:

$$\begin{aligned}
 t(x)H &= z^2vH + \delta(y, z)H + t_1xH + t_2x^2H \\
 &\quad + \quad + \quad + \quad + \quad + \\
 \tau_x G &= \gamma z^2G + 0G + \dot{\tau}_1xG + \dot{\tau}_2x^2G \\
 \parallel &\quad \parallel \quad \parallel \quad \parallel \quad \parallel \\
 &= \underbrace{z^2V + \delta(y, z)H}_{\text{público}} + xT_1 + x^2T_2
 \end{aligned}$$

As mentes ávidas dizem logo: «Ah e tal, mas T_1 e T_2 podiam ser qualquer coisa; há ali variáveis aleatórias por todo o lado. Como é que isto diz alguma coisa sobre v ?» A resposta está no movimento. T_1 e T_2 comprometem o provador aos coeficientes t_1, t_2 antes de o traslado lançar x . Depois, uma única avaliação tem de satisfazer simultaneamente $t(x) = \langle \underline{l}(x), \underline{r}(x) \rangle$ e a equação do compromisso de valor. Os factores aleatórios escondem t_0 , e portanto v , do verificador; não dão ao provador liberdade para mudar o polinómio depois de ver x . Aqui «público» significa a estrutura formal: y, z, x são recompostos pelo verificador a partir do mesmo traslado.

$$\begin{aligned}
 \langle \underline{l}(x), \underline{G} \rangle &= \langle \underline{a}_L, \underline{G} \rangle + x \langle \underline{s}_L, \underline{G} \rangle + \langle -z\underline{1}, \underline{G} \rangle *^5 \\
 &\quad + \quad + \quad + \quad + \\
 \langle \underline{r}(x), \underline{H}' \rangle &= \langle \underline{a}_R, \underline{H} \rangle + x \langle \underline{s}_R, \underline{H} \rangle + \langle z\underline{y}^n + z^2\underline{2}^n, \underline{H}' \rangle *^6 \\
 &\quad + \quad + \quad + \quad \parallel \\
 \mu G &= \dot{\alpha} G + x \dot{\rho} G \\
 \parallel &\quad \parallel \quad \parallel \\
 &= A + xS + \underbrace{\langle z\underline{y}^n + z^2\underline{2}^n, \underline{H}' \rangle + \langle -z\underline{1}, \underline{G} \rangle}_{\text{público}}
 \end{aligned}$$

Em que

$$\underline{H}' = \underline{y}^{-n} \circ \underline{H} \iff \underline{H} = \underline{y}^n \circ \underline{H}'.$$

Esta é uma das partes mais elegantes. Quando se compromete a $\underline{a}_R$ e $\underline{s}_R$, o provador ainda não possui $\underline{y}^n$: y só nasce depois do compromisso S . A troca para $\underline{H}'$ introduz depois essas potências de forma implícita. Aquilo que parecia chegar tarde entra no circuito pela mudança dos geradores.

Imagine-se que o provador envia :

$$t(x), \tau_x, \underline{l}(x), \underline{r}(x), \mu, T_1, T_2, A, S$$

ao verificador. Mesmo recebendo $\underline{l}(x)$ e $\underline{r}(x)$, ele não conseguiria extrair t_0 , por causa dos factores ofuscantes $\underline{s}_L$ e $\underline{s}_R$. Já a seguir nem esses dois vectores terão de atravessar a rede.

5.7.1 Verificação

Se recebesse simplesmente os dois vectores de 64 elementos, o verificador já conseguiria conferir os dois circuitos. Mas não saltemos por cima da ponte: são mesmo *duas* igualdades.

Do primeiro circuito, o compromisso ao valor, vem

$$t(x)H + \tau_x G = z^2 V + \delta(y, z)H + xT_1 + x^2 T_2. \quad (5.15)$$

Do segundo circuito, o compromisso aos dois vectores, vem

$$\begin{aligned} \langle \underline{l}(x), \underline{G} \rangle + \langle \underline{r}(x), \underline{H}' \rangle + \mu G \\ = A + xS + \langle zy^n + z^2 \underline{2}^n, \underline{H}' \rangle + \langle -z\underline{1}, \underline{G} \rangle. \end{aligned} \quad (5.16)$$

A regra é escolar: se $a = b$ e $c = d$, então $a + c = b + d$. Somando os lados esquerdos das duas igualdades, e fazendo o mesmo aos lados direitos, obtemos

$$\begin{aligned} t(x)H + \tau_x G + \langle \underline{l}(x), \underline{G} \rangle + \langle \underline{r}(x), \underline{H}' \rangle + \mu G \\ = z^2 V + \delta(y, z)H + xT_1 + x^2 T_2 + A + xS \\ + \langle zy^n + z^2 \underline{2}^n, \underline{H}' \rangle + \langle -z\underline{1}, \underline{G} \rangle. \end{aligned}$$

Esta linha é simplesmente a soma das Equações (5.15) e (5.16). Mas enviar $\underline{l}(x), \underline{r}(x)$ seria carregar 128 escalares.

A sorte é que: há uma porta mais estreita.

Depois de $t(x)$ entrar no traslado, ambos os lados extraem x_{ip} e calculam $U = x_{ip}H$. A construção do polinómio dá então a seta decisiva:

$$t(x) = \langle \underline{l}(x), \underline{r}(x) \rangle \xrightarrow{\times U} \underbrace{\langle \underline{l}(x), \underline{r}(x) \rangle}_{\text{vectores}} U = \underbrace{t(x)}_{\text{escalar}} U.$$

Acrescentemos esse mesmo ponto aos dois lados da igualdade grande:

$$\begin{aligned} t(x)H + \tau_x G + \underbrace{\langle \underline{l}(x), \underline{G} \rangle + \langle \underline{r}(x), \underline{H}' \rangle + \langle \underline{l}(x), \underline{r}(x) \rangle U}_{P_k} + \mu G \\ = t(x)U + z^2 V + \delta(y, z)H + xT_1 + x^2 T_2 + A + xS \\ + \langle zy^n + z^2 \underline{2}^n, \underline{H}' \rangle + \langle -z\underline{1}, \underline{G} \rangle. \end{aligned}$$

Ponhamos $k = \log_2 64 = 6$. O argumento de produto interno dobra o termo sob a chaveta até restarem os escalares a, b :

$$\begin{aligned} P_k &= \langle \underline{l}(x), \underline{G} \rangle + \langle \underline{r}(x), \underline{H}' \rangle + \langle \underline{l}(x), \underline{r}(x) \rangle U \\ &= P_0 - \sum_{j=1}^k \left(u_j^2 L_j + u_j^{-2} R_j \right). \end{aligned} \quad (5.17)$$

Substituir a chaveta P_k por esta expressão dá finalmente

$$\begin{aligned} t(x)H + \tau_x G + P_0 - \sum_{j=1}^k \left(u_j^2 L_j + u_j^{-2} R_j \right) + \mu G \\ = t(x)U + z^2 V + \delta(y, z)H + xT_1 + x^2 T_2 + A + xS \\ + \langle zy^n + z^2 \underline{2}^n, \underline{H}' \rangle + \langle -z\underline{1}, \underline{G} \rangle. \end{aligned}$$

pow!

Nem os u_j atravessam a rede: ambos os lados os colhem do traslado. Para uma saída enviam-se $2 \cdot 6 + 9 = 21$ objectos de 32 bytes:

$$t(x), \tau_x, a, b, \mu, T_1, T_2, A, S, \quad L_1, \dots, L_6, \quad R_1, \dots, R_6.$$

São 672 bytes, antes dos pequenos prefixos de serialização, em vez dos 6176 bytes da prova Bor-romean de uma saída. É aqui que a bala começa realmente a ganhar velocidade.

Mais detalhes do provador

Aqui continua o processo do provador no protocolo do produto interno IP. Começa-se com vectores de dimensão 2^k , onde $k = 6$, e com

$$P_k = \langle \underline{l}_k(x), \underline{G}_k \rangle + \langle \underline{r}_k(x), \underline{H}'_k \rangle + \langle \underline{l}_k(x), \underline{r}_k(x) \rangle U.$$

Aqui $isto^{\rightarrow}$ significa a primeira metade do vector e $isto^{\leftarrow}$ a segunda. Enquanto $k > 0$, o provador cruza as metades:

$$\begin{aligned} L_k &= \langle \underline{l}_k^{\rightarrow}(x), \underline{G}_k^{\leftarrow} \rangle + \langle \underline{r}_k^{\leftarrow}(x), \underline{H}'_k^{\rightarrow} \rangle + \langle \underline{l}_k^{\rightarrow}(x), \underline{r}_k^{\leftarrow}(x) \rangle U, \\ R_k &= \langle \underline{l}_k^{\leftarrow}(x), \underline{G}_k^{\rightarrow} \rangle + \langle \underline{r}_k^{\rightarrow}(x), \underline{H}'_k^{\leftarrow} \rangle + \langle \underline{l}_k^{\leftarrow}(x), \underline{r}_k^{\rightarrow}(x) \rangle U. \end{aligned}$$

$$^{13} \perp_U(L_k)$$

$$^{14} \perp_U(R_k)$$

$$u_k \leftarrow^{15} \perp_C(), \quad u_k \in \mathbb{Z}_l^*.$$

Os números 13, 14 e 15 marcam apenas a primeira ronda. Na ronda seguinte o traslado continua a contar; não recomeça. Se algum desafio for zero, não existe u_k^{-1} : o provador recomeça e o verificador rejeita.

Agora dobram-se testemunhas e geradores:

$$\begin{aligned} \underline{l}_{k-1}(x) &= u_k \underline{l}_k^{\rightarrow}(x) + u_k^{-1} \underline{l}_k^{\leftarrow}(x), \\ \underline{r}_{k-1}(x) &= u_k^{-1} \underline{r}_k^{\rightarrow}(x) + u_k \underline{r}_k^{\leftarrow}(x), \\ \underline{G}_{k-1} &= u_k^{-1} \underline{G}_k^{\rightarrow} + u_k \underline{G}_k^{\leftarrow}, \\ \underline{H}'_{k-1} &= u_k \underline{H}'_k^{\rightarrow} + u_k^{-1} \underline{H}'_k^{\leftarrow}. \end{aligned}$$

Depois faz-se $k \leftarrow k - 1$. Quando $k = 0$, os dois vectores de testemunhas já são os escalares a e b .

Passamos agora à magia em volta de P_k . Para chegar ao compromisso dobrado P_{k-1} , substituem-se exactamente as quatro dobras:

$$\begin{aligned} P_{k-1} &= \langle u_k \underline{l}_k^{\rightarrow} + u_k^{-1} \underline{l}_k^{\leftarrow}, u_k^{-1} \underline{G}_k^{\rightarrow} + u_k \underline{G}_k^{\leftarrow} \rangle \\ &\quad + \langle u_k^{-1} \underline{r}_k^{\rightarrow} + u_k \underline{r}_k^{\leftarrow}, u_k \underline{H}'_k^{\rightarrow} + u_k^{-1} \underline{H}'_k^{\leftarrow} \rangle \\ &\quad + \langle u_k \underline{l}_k^{\rightarrow} + u_k^{-1} \underline{l}_k^{\leftarrow}, u_k^{-1} \underline{r}_k^{\rightarrow} + u_k \underline{r}_k^{\leftarrow} \rangle U. \end{aligned}$$

O u_k de cada termo diagonal encontra o seu inverso e desaparece. Os termos cruzados não desaparecem: ficam marcados por u_k^2 ou u_k^{-2} .

Agora ponhamo-los lado a lado, como peças de um fecho éclair:

$$\begin{aligned}
 P_{k-1} = & \\
 & \langle \underline{l}_k^{\rightarrow}(x), \underline{G}_k^{\rightarrow} \rangle + u_k^2 \langle \underline{l}_k^{\rightarrow}(x), \underline{G}_k^{\leftarrow} \rangle \\
 & + \langle \underline{l}_k^{\leftarrow}(x), \underline{G}_k^{\leftarrow} \rangle + u_k^{-2} \langle \underline{l}_k^{\leftarrow}(x), \underline{G}_k^{\rightarrow} \rangle \\
 & + \langle \underline{r}_k^{\rightarrow}(x), \underline{H}_k^{\rightarrow} \rangle + u_k^{-2} \langle \underline{r}_k^{\rightarrow}(x), \underline{H}_k^{\leftarrow} \rangle \\
 & + \langle \underline{r}_k^{\leftarrow}(x), \underline{H}_k^{\leftarrow} \rangle + u_k^2 \langle \underline{r}_k^{\leftarrow}(x), \underline{H}_k^{\rightarrow} \rangle \\
 & + \langle \underline{l}_k^{\rightarrow}(x), \underline{r}_k^{\rightarrow}(x) \rangle U + u_k^2 \langle \underline{l}_k^{\rightarrow}(x), \underline{r}_k^{\leftarrow}(x) \rangle U \\
 & + \underbrace{\langle \underline{l}_k^{\leftarrow}(x), \underline{r}_k^{\leftarrow}(x) \rangle U}_{P_k} + \underbrace{u_k^2 \langle \underline{l}_k^{\leftarrow}(x), \underline{r}_k^{\rightarrow}(x) \rangle U}_{u_k^2 L_k + u_k^{-2} R_k}
 \end{aligned}$$

click!

E como tal possibilita-se a seguinte recursão:

$$\begin{aligned}
 P_k &= P_{k-1} - (u_k^2 L_k + u_k^{-2} R_k) \\
 &\downarrow \\
 P_{k-1} &= P_{k-2} - (u_{k-1}^2 L_{k-1} + u_{k-1}^{-2} R_{k-1}) \\
 &\downarrow \\
 &\vdots \\
 &\downarrow \\
 P_1 &= P_0 - (u_1^2 L_1 + u_1^{-2} R_1).
 \end{aligned}$$

Logo,

$$P_k = P_0 - \sum_{j=1}^k (u_j^2 L_j + u_j^{-2} R_j), \quad (5.18)$$

$$P_0 = a^0 \underline{G} + b^0 \underline{H}' + abU. \quad (5.19)$$

A floresta de 64 coordenadas acabou em dois escalares, a, b , e dois pontos finais que o verificador consegue reconstruir. A Equação (5.17), prometida acima, já não é um truque por explicar.

O verificador e o fim da batalha

Os escalares finais a, b têm de ser transmitidos. Os pontos finais ${}^0\underline{G}$ e ${}^0\underline{H'}$, porém, são reconstruídos pelo verificador. É aqui que as setas mostram todo o caminho. Para não esconder a mecânica atrás de reticências, imagine-se um vector inicial com oito pontos e escreva-se $G_{q,i}$ para o elemento i de $\underline{G}_q$:

$$\begin{array}{ccc}
 \begin{bmatrix} G_{3,0} \\ G_{3,1} \\ G_{3,2} \\ G_{3,3} \\ G_{3,4} \\ G_{3,5} \\ G_{3,6} \\ G_{3,7} \end{bmatrix} & \xrightarrow{u_3} & \begin{bmatrix} G_{2,0} = u_3^{-1}G_{3,0} + u_3G_{3,4} \\ G_{2,1} = u_3^{-1}G_{3,1} + u_3G_{3,5} \\ G_{2,2} = u_3^{-1}G_{3,2} + u_3G_{3,6} \\ G_{2,3} = u_3^{-1}G_{3,3} + u_3G_{3,7} \end{bmatrix} \\
 & & \downarrow u_2 \\
 \begin{bmatrix} {}^0\underline{G} = u_1^{-1}G_{1,0} + u_1G_{1,1} \end{bmatrix} & \xleftarrow{u_1} & \begin{bmatrix} G_{1,0} = u_2^{-1}G_{2,0} + u_2G_{2,2} \\ G_{1,1} = u_2^{-1}G_{2,1} + u_2G_{2,3} \end{bmatrix}
 \end{array}$$

Cada seta corta o vector ao meio e volta a cosê-lo com um desafio e o seu inverso. Nenhuma coordenada desaparece; muda apenas o peso com que chega ao último ponto.

Façamos o jogo das setas. Cada $G_{3,i}$ começa com uma mochila vazia. Ao passar por uma seta, guarda nela o factor escrito por cima. As setas abaixo seguem a *contribuição* de cada ponto; não afirmam que, por exemplo, $G_{3,0} = G_{2,0}$, pois $G_{2,0}$ recebe também $G_{3,4}$.

$$\begin{aligned}
G_{3,0} &\xrightarrow{u_3^{-1}} G_{2,0} \xrightarrow{u_2^{-1}} G_{1,0} \xrightarrow{u_1^{-1}} \underline{G}, & \phi_0 &= u_3^{-1} u_2^{-1} u_1^{-1}, & 0 &= 000_2, \\
G_{3,1} &\xrightarrow{u_3^{-1}} G_{2,1} \xrightarrow{u_2^{-1}} G_{1,1} \xrightarrow{u_1} \underline{G}, & \phi_1 &= u_3^{-1} u_2^{-1} u_1, & 1 &= 001_2, \\
G_{3,2} &\xrightarrow{u_3^{-1}} G_{2,2} \xrightarrow{u_2} G_{1,0} \xrightarrow{u_1^{-1}} \underline{G}, & \phi_2 &= u_3^{-1} u_2 u_1^{-1}, & 2 &= 010_2, \\
G_{3,3} &\xrightarrow{u_3^{-1}} G_{2,3} \xrightarrow{u_2} G_{1,1} \xrightarrow{u_1} \underline{G}, & \phi_3 &= u_3^{-1} u_2 u_1, & 3 &= 011_2, \\
G_{3,4} &\xrightarrow{u_3} G_{2,0} \xrightarrow{u_2^{-1}} G_{1,0} \xrightarrow{u_1^{-1}} \underline{G}, & \phi_4 &= u_3 u_2^{-1} u_1^{-1}, & 4 &= 100_2, \\
G_{3,5} &\xrightarrow{u_3} G_{2,1} \xrightarrow{u_2^{-1}} G_{1,1} \xrightarrow{u_1} \underline{G}, & \phi_5 &= u_3 u_2^{-1} u_1, & 5 &= 101_2, \\
G_{3,6} &\xrightarrow{u_3} G_{2,2} \xrightarrow{u_2} G_{1,0} \xrightarrow{u_1^{-1}} \underline{G}, & \phi_6 &= u_3 u_2 u_1^{-1}, & 6 &= 110_2, \\
G_{3,7} &\xrightarrow{u_3} G_{2,3} \xrightarrow{u_2} G_{1,1} \xrightarrow{u_1} \underline{G}, & \phi_7 &= u_3 u_2 u_1, & 7 &= 111_2.
\end{aligned}$$

O padrão já se vê sem pedir licença: lendo os bits da esquerda para a direita, o primeiro escolhe o expoente de u_3 , o segundo o de u_2 e o terceiro o de u_1 ; 0 escolhe -1 , 1 escolhe $+1$. Escrevendo a regra com $m = 1$ a partir do bit menos significativo, a seta geral é

$$G_{k,i} \longrightarrow G_{k,i} \prod_{m=1}^k u_m^{b(i,m)}, \quad b(i,m) = \begin{cases} -1, & \text{se o } m\text{-ésimo bit de } i \text{ for } 0, \\ +1, & \text{se o } m\text{-ésimo bit de } i \text{ for } 1, \end{cases} \quad (5.20)$$

e o verificador forma o vector $\underline{\phi}$ juntando o conteúdo das oito mochilas:

$$\phi_i = \prod_{m=1}^k u_m^{b(i,m)}, \quad {}^0\underline{G} = \langle \underline{\phi}, \underline{G}_k \rangle.$$

Para $\underline{H}'$, cada seta troca o sinal do expoente. O espelho fica visível nesta pequena tabela:

	bit 0	bit 1	
$\underline{G}$	u_m^{-1}	u_m	$\implies \varphi_i = \phi_i^{-1}.$
$\underline{H}'$	u_m	u_m^{-1}	

Portanto,

$$\varphi_i = \prod_{m=1}^k u_m^{-b(i,m)}, \quad {}^0\underline{H}' = \langle \underline{\varphi}, \underline{H}'_k \rangle.$$

Este sinal contrário é essencial. A versão anterior escrevia os dois padrões com o mesmo sinal; essa era uma das avarias escondidas nas setas.

O verificador refaz agora toda a viagem do traslado:

$$\begin{array}{ll}
 G, H, \underline{G}, \underline{H} & {}^0 \perp [init] \\
 V & {}^1 \perp_U(V) \\
 A & {}^2 \perp_U(A) \\
 S & {}^3 \perp_U(S) \\
 y \leftarrow^4 \perp_C(), & y^{-1}, \quad z \leftarrow^5 \perp_C().
 \end{array}$$

Com esses desafios calcula

$$\delta(y, z) = (z - z^2) \langle \underline{1}, \underline{y}^n \rangle - z^3 \langle \underline{1}, \underline{z}^n \rangle.$$

Depois continua:

$$\begin{array}{ll}
 T_1 & {}^6 \perp_U(T_1) \\
 T_2 & {}^7 \perp_U(T_2) \\
 x \leftarrow^8 \perp_C() & \\
 \tau_x & {}^9 \perp_U(\tau_x) \\
 \mu & {}^{10} \perp_U(\mu) \\
 t(x) & {}^{11} \perp_U(t(x)) \\
 x_{ip} \leftarrow^{12} \perp_C(), & U = x_{ip}H.
 \end{array}$$

Finalmente, para $j = k, k-1, \dots, 1$, absorve L_j , absorve R_j e extrai u_j . Os estados 13, 14 e 15 são a primeira destas rondas; os números continuam a avançar nas cinco seguintes. Assim o verificador não recebe $y, z, x, x_{ip}, u_1, \dots, u_k$: reencontra-os.

Agora está armado para a verificação final. Defina-se

$$\Sigma_{IP} = \sum_{j=1}^k \left(u_j^2 L_j + u_j^{-2} R_j \right).$$

Não saltemos logo para a última linha: devolvamos ao fim da batalha as equações que mostram como cada peça cai no seu lugar. A primeira equação de verificação, depois de substituir os dois vectores por $P_k - t(x)U$, é

$$\begin{aligned}
 & t(x)H + \tau_x G + P_k - t(x)U + \mu G \\
 &= z^2 V + \delta(y, z)H + xT_1 + x^2 T_2 + A + xS \\
 &+ \langle z\underline{y}^n + z^2 \underline{z}^n, \underline{H}' \rangle + \langle -z\underline{1}, \underline{G} \rangle.
 \end{aligned} \tag{5.21}$$

Substituam-se agora as Equações (5.18) e (5.19). A floresta dobrada reaparece só nos seus dois escalares finais:

$$\begin{aligned}
 & t(x)H - t(x)U + (\tau_x + \mu)G + a^0 \underline{G} + b^0 \underline{H}' + abU - \Sigma_{IP} \\
 &= z^2 V + \delta(y, z)H + xT_1 + x^2 T_2 + A + xS \\
 &+ \langle z\underline{y}^n + z^2 \underline{z}^n, \underline{H}' \rangle + \langle -z\underline{1}, \underline{G} \rangle.
 \end{aligned} \tag{5.22}$$

Passando tudo para o lado direito, sem esconder nenhum sinal,

$$\begin{aligned}\mathcal{O} = & z^2V + xT_1 + x^2T_2 + A + xS - (\tau_x + \mu)G + \Sigma_{\text{IP}} \\ & + [\delta(y, z) - t(x)]H + [t(x) - ab]U \\ & + \langle zy^n + z^2\underline{2}^n, \underline{H}' \rangle + \langle -z\underline{1}, \underline{G} \rangle \\ & - a^0\underline{G} - b^0\underline{H}'.\end{aligned}\tag{5.23}$$

O verificador reconstruiu ${}^0\underline{G} = \langle \underline{\phi}, \underline{G} \rangle$ e ${}^0\underline{H}' = \langle \underline{\varphi}, \underline{H}' \rangle$. Logo, as duas últimas linhas juntam-se aos produtos internos públicos:

$$\begin{aligned}\mathcal{O} = & z^2V + xT_1 + x^2T_2 + A + xS - (\tau_x + \mu)G + \Sigma_{\text{IP}} \\ & + [\delta(y, z) - t(x)]H + [t(x) - ab]U \\ & + \langle -z\underline{1} - a\underline{\phi}, \underline{G} \rangle \\ & + \langle zy^n + z^2\underline{2}^n - b\underline{\varphi}, \underline{H}' \rangle.\end{aligned}\tag{5.24}$$

Finalmente usa-se $U = x_{ip}H$. Só agora se chega à equação compacta que a implementação pode verificar como uma soma multiescalar:

$$\begin{aligned}\mathcal{O} = & z^2V + xT_1 + x^2T_2 + A + xS - (\tau_x + \mu)G + \Sigma_{\text{IP}} \\ & + [\delta(y, z) - t(x) + (t(x) - ab)x_{ip}]H \\ & + \langle -z\underline{1} - a\underline{\phi}, \underline{G} \rangle \\ & + \langle zy^n + z^2\underline{2}^n - b\underline{\varphi}, \underline{H}' \rangle.\end{aligned}\tag{5.25}$$

E, como $\underline{H}' = \underline{y}^{-n} \circ \underline{H}$, a mesma batalha pode terminar toda escrita nos geradores originais:

$$\begin{aligned}\mathcal{O} = (0, 1) = & z^2V + xT_1 + x^2T_2 + A + xS - (\tau_x + \mu)G + \Sigma_{\text{IP}} \\ & + [\delta(y, z) - t(x) + (t(x) - ab)x_{ip}]H \\ & + \langle z\underline{1} + \underline{y}^{-n} \circ (z^2\underline{2}^n - b\underline{\varphi}), \underline{H} \rangle \\ & + \langle -z\underline{1} - a\underline{\phi}, \underline{G} \rangle.\end{aligned}\tag{5.26}$$

bang!

Se esta soma multiescalar dá a identidade, as codificações são canónicas e todos os desafios que precisam de inverso são não nulos, o verificador aceita. Sob a dificuldade do logaritmo discreto e o modelo Fiat-Shamir, uma prova forjada de que $0 \leq v \leq 2^{64} - 1$ passa apenas com probabilidade negligenciável. Os bits, a máscara γ e os vectores intermédios nunca saíram do provador. As setas mexeram-se; a prova ficou mais pequena; Sundance tinha razão.

5.8 Bulletproof+

A Bulletproof clássica já teve o seu palco, as suas setas e o seu fim da batalha. Bulletproof+ não apaga nada disso. É a batalha seguinte: prova a mesma afirmação sobre os montantes, conserva a ideia de dobrar vectores com pares L_j, R_j , mas muda a coreografia que liga o compromisso inicial ao fecho.

5.8.1 O que fica e o que muda

Fica. O compromisso de Pedersen, a decomposição em bits, os desafios de Fiat–Shamir colhidos de um traslado, a agregação de vários montantes e as $\log_2(64M)$ rondas que publicam pares L_j, R_j .

Muda. Desaparecem do objecto transmitido

$$S, T_1, T_2, \tau_x, \mu, t, a, b.$$

Em seu lugar aparecem A_1, B, r_1, s_1, d_1 , com novas equações de fecho, um traslado separado por domínio e uma equação de verificação própria. As setas continuam a dobrar vectores; já não se pode, porém, pegar na Equação (5.26) e trocar apenas os nomes.

As duas formas serializadas tornam a diferença visível:

$$\Pi_{BP} = (A, S, T_1, T_2, \tau_x, \mu, \mathbf{L}, \mathbf{R}, a, b, t),$$

$$\Pi_+ = (A, A_1, B, r_1, s_1, d_1, \mathbf{L}, \mathbf{R}).$$

Para a mesma dimensão, Bulletproof+ retira três objectos de 32 bytes. A agregação, note-se, não nasceu com o sinal +: a Bulletproof clássica já agregava montantes. O ganho estrutural de Bulletproof+ está no fecho mais curto e na verificação reorganizada.

A ideia algébrica continua a começar em bits. Para p montantes, ponhamos

$$N = 64, \quad M = 2^{\lceil \log_2 p \rceil}, \quad n = MN,$$

com $M = 1$ quando $p = 1$. Concatenam-se os bits dos p montantes num vector $\mathbf{a}_L \in \mathbb{Z}_l^n$; as $M - p$ posições de montante restantes são preenchidas implicitamente com zeros. Definimos

$$\mathbf{a}_R = \mathbf{a}_L - \mathbf{1}.$$

As duas relações

$$b_j = \left\langle \mathbf{a}_L^{(j)}, \mathbf{2}^{64} \right\rangle, \tag{5.27}$$

$$\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{0} \tag{5.28}$$

dizem, respectivamente, que cada valor é a soma dos seus bits e que cada coordenada é 0 ou 1. Aqui $\mathbf{2}^{64} = (1, 2, \dots, 2^{63})$, $\circ$ é o produto coordenada a coordenada e $\mathbf{a}_L^{(j)}$ é o bloco de 64 coordenadas do montante j , para $0 \leq j < p$.

O provador compromete-se a estes vectores sem os revelar. Cada dobra publica dois pontos, pelo que são necessárias $\log_2(64M)$ rondas em vez de 64 compromissos por saída. A verificação continua a ter trabalho proporcional a $64M$: a agregação poupa sobretudo espaço, não transforma toda a verificação num cálculo logarítmico.

5.8.2 Dos tipos históricos ao Bulletproof+

Há três nomes parecidos que convém não misturar:

RCTTypeBulletproof. A versão 8 admitiu a prova Bulletproof original. O objecto serializado contém

$$\Pi_{BP} = (A, S, T_1, T_2, \tau_x, \mu, \mathbf{L}, \mathbf{R}, a, b, t).$$

A versão 9 deixou de admitir as provas Borromean em novas transacções.

RCTTypeBulletproof2. A versão 10 admitiu este tipo e a versão 11 tornou-o obrigatório em lugar do tipo anterior. O sufixo «2» não designa Bulletproof+: a prova continua a usar exactamente a estrutura Π_{BP} e o mesmo verificador. A revisão reduziu os dados ECDH por saída para o montante cifrado de oito bytes e passou a derivar a máscara, como vimos na Secção 5.3.

RCTTypeBulletproofPlus. A versão 15 admitiu Bulletproof+; a versão 16 proibiu os tipos que ainda usam a prova Bulletproof anterior. Na rede principal, a versão 16 começou no bloco 2689608. Este é o tipo criado para novas transacções na implementação congelada.

Os leitores históricos encontrarão ainda **RCTTypeCLSAG**: esse tipo já usava CLSAG para autorizar entradas, mas conservava a estrutura de prova Π_{BP} . O código continua a analisar e a verificar todos estes formatos para ler blocos antigos. Essa compatibilidade não os torna escolhas válidas para construir uma transacção da versão 16.

```
src/crypto-
note_core/
blockchain.cpp,
check_tx-
outputs();
src/hardforks/
hardforks.cpp
```

Nas equações da secção clássica usámos os pontos matemáticos já restaurados. No objecto Monero serializado, $A, S, T_1, T_2, \mathbf{L}, \mathbf{R}$ da Bulletproof clássica eram guardados multiplicados por 8^{-1} ; o verificador volta a multiplicá-los por 8. Bulletproof+ conserva esta convenção para $A, A_1, B, \mathbf{L}, \mathbf{R}$. É uma representação de implementação, não uma seta nova na álgebra.

Para o mesmo M , a parte formada por objectos de 32 bytes ocupa

$$|\Pi_{BP}| = 32(9 + 2 \log_2(64M)), \quad (5.29)$$

$$|\Pi_+| = 32(6 + 2 \log_2(64M)). \quad (5.30)$$

Não se contam aqui os pequenos prefixos de comprimento da serialização. Bulletproof+ retira três objectos, ou 96 bytes: para $M = 2$, são 736 contra 640 bytes; para $M = 16$, 928 contra 832 bytes. A prova Borromean, recordemos, ocupava 6176 bytes *por saída*. Estes números são comparações de formato; não afirmam, sem uma medição separada, que um verificador seja tantas vezes mais rápido do que outro.

```
src/ringct/
rctTypes.h,
Bulletproof e
Bulletproof-
Plus;
src/crypto-
note_basic/
cryptonote_
format_
utils.cpp,
get_pruned_
transaction_
weight()
```

5.8.3 Bulletproof+ na versão 16

Para não sobrecarregar a notação anterior, chamemos v_j ao montante e γ_j à máscara de compromisso nesta subsecção:

$$C_j = \gamma_j G + v_j H.$$

Uma Bulletproof+ agregada prova, para todos os seus p compromissos,

$$\exists (\gamma_j, v_j)_{j=0}^{p-1} : C_j = \gamma_j G + v_j H \quad \wedge \quad 0 \leq v_j < 2^{64}. \quad (5.31)$$

As aberturas (γ_j, v_j) são as testemunhas. Os compromissos $(C_0, \dots, C_{p-1})$, os geradores, o traslado e

$$\Pi_+ = (A, A_1, B, r_1, s_1, d_1, \mathbf{L}, \mathbf{R})$$

são públicos. O vector de compromissos internos, chamado V no código, não é serializado dentro da prova: é reconstruído a partir de `outPk.mask`.

Agregação e preenchimento

Uma transacção RingCT canónica deste tipo contém exactamente uma Bulletproof+, e essa prova cobre todos os seus compromissos de saída. O formato admite $1 \leq p \leq 16$; a regra geral da versão 16 exige separadamente pelo menos duas saídas numa transacção comum. O valor interno M é a menor potência de dois não inferior a p :

p	1	2	3-4	5-8	9-16
M	1	2	4	8	16
$ \mathbf{L} = \mathbf{R} $	6	7	8	9	10

Os montantes fictícios usados para completar M são zeros implícitos; não criam saídas nem compromissos serializados.

Durante a análise da transacção, o tamanho de $\mathbf{L}, \mathbf{R}$ tem de corresponder precisamente a esse M , o número reconstruído de V_j tem de ser p , e o limite é 16. Assim, uma prova para capacidade 16 não é uma codificação canónica de apenas duas saídas.

As duas representações do compromisso

A estrutura de transacção guarda o compromisso normal C_j . O provador, porém, constrói

$$V_j = 8^{-1}C_j = (8^{-1}\gamma_j)G + (8^{-1}v_j)H, \quad (5.32)$$

onde 8^{-1} é o inverso do cofactor no corpo $\mathbb{Z}_l$. Na criação, `bulletproof_plus_PROVE()` devolve estes V_j ; `genRctSimple()` multiplica-os por 8 antes de os colocar em `outPk.mask`. Na leitura acontece o inverso: o analisador multiplica cada compromisso de `outPk` por 8^{-1} para restaurar V_j .

Os pontos $A, A_1, B, \mathbf{L}, \mathbf{R}$ são igualmente produzidos e serializados na representação dividida por 8. Antes da equação final, o verificador multiplica todos esses pontos e cada V_j por 8. Isso elimina qualquer componente de torsão e deixa os pontos usados na multiplicação multiescalar no subgrupo de ordem prima. Não se deve, pois, multiplicar `outPk.mask` por 8 mais uma vez: ele já é C_j .

Geradores e traslado

Além de G e H , a prova usa vectores públicos $\mathbf{G} = (G_0, \dots, G_{n-1})$ e $\mathbf{H} = (H_0, \dots, H_{n-1})$. A implementação deriva cada ponto dos bytes de H , da cadeia ASCII `bulletproof_plus`, da codificação `varint` do índice e de uma função hash-para-ponto com limpeza pelo cofactor. Uma derivação que produzisse a identidade seria rejeitada. A segurança pressupõe que não se conhecem relações de logaritmo discreto úteis entre estes geradores. Na notação do artigo Bulletproof+, os papéis dos geradores de valor e de máscara aparecem trocados: no código Monero, H leva o montante e G leva a máscara.

O traslado começa num ponto fixo derivado da cadeia `bulletproof_plus_transcript`. Escrevendo

```
src/crypto-
note_core/
tx_verification_
utils.cpp,
is_canonical_
bulletproof_
plus_layout();
src/ringct/
rctTypes.cpp,
n_bulletproof_
plus_amounts()
```

```
src/ringct/
rctSigs.cpp,
proveRange-
Bulletproof
Plus() e
genRct
Simple();
src/crypto-
note_basic/
cryptonote_
format_
utils.cpp,
expand_
transaction_
1()
```

$\mathcal{H}_n$ para Keccak–256 seguido de redução módulo l , a ordem é:

$$\begin{aligned} T_1 &= \mathcal{H}_n(T_0 \parallel \mathcal{H}_n(V_0 \parallel \cdots \parallel V_{p-1})), \\ y &= \mathcal{H}_n(T_1 \parallel A), \end{aligned} \quad z = \mathcal{H}_n(y), \quad (5.33)$$

$$T^{(0)} = z, \quad u_k = T^{(k+1)} = \mathcal{H}_n(T^{(k)} \parallel L_k \parallel R_k), \quad (5.34)$$

$$e = \mathcal{H}_n(T^{(\log_2 n)} \parallel A_1 \parallel B). \quad (5.35)$$

A cadeia inicial separa este protocolo de outros usos do hash. Todos os desafios são escalares reduzidos; o provador recomeça e o verificador rejeita se y, z, u_k ou e forem zero.

Mais detalhes do provador

Definamos o compromisso vectorial

$$\langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle = \sum_{i=0}^{n-1} a_i G_i + \sum_{i=0}^{n-1} b_i H_i.$$

O primeiro ponto enviado é

$$A = 8^{-1}(\langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{a}_R, \mathbf{H} \rangle + \alpha G), \quad (5.36)$$

com $\alpha \in_R \mathbb{Z}_l^*$. Depois dos desafios y, z , o código constrói a janela

$$d_{jN+i} = z^{2(j+1)} 2^i, \quad 0 \leq j < M, \quad 0 \leq i < N, \quad (5.37)$$

e os vectores

$$\begin{aligned} \mathbf{a}' &= \mathbf{a}_L - z\mathbf{1}, \\ b'_i &= a_{R,i} + z + d_i y^{n-i}. \end{aligned} \quad (5.38)$$

A máscara acumulada que liga estes vectores aos compromissos é

$$\alpha' = \alpha + y^{n+1} \sum_{j=0}^{p-1} z^{2(j+1)} \gamma_j. \quad (5.39)$$

Todas estas igualdades são em $\mathbb{Z}_l$.

O argumento de produto interno dobra $(\mathbf{a}', \mathbf{b}', \mathbf{G}, \mathbf{H})$ em cada desafio u_k . Cada ronda envia L_k, R_k , inclui duas máscaras aleatórias novas e reduz sucessivamente a dimensão até 1. Chamemos a^*, b^* aos escalares finais, G^*, H^* aos geradores finais e α^* à máscara acumulada depois de todas as dobras. Com escalares aleatórios não nulos r, s, d, η , o fecho calculado pelo código é

$$A_1 = 8^{-1}(rG^* + sH^* + dG + y(rb^* + sa^*)H), \quad (5.40)$$

$$B = 8^{-1}(\eta G + rysH), \quad (5.41)$$

$$r_1 = r + ea^*, \quad s_1 = s + eb^*, \quad (5.42)$$

$$d_1 = \eta + ed + e^2 \alpha^*. \quad (5.43)$$

Na criação pela carteira, as máscaras do compromisso são derivadas como na Equação (5.4); α , as máscaras das rondas e r, s, d, η vêm do gerador de escalares uniformes não nulos. Esta aleatoriedade é necessária para ocultar os bits e as aberturas. A ocultação de Pedersen perfeita, a propriedade de conhecimento da prova e a heurística Fiat–Shamir são afirmações distintas; a solidez e o conhecimento de Bulletproof+ são garantias computacionais sob as hipóteses do protocolo, não uma consequência da ocultação perfeita por si só.

```
src/crypto-
note_config.h,
HASH_KEY_
BULLETPROOF_
PLUS_*;
src/ringct/
bulletproofs_
plus.cc,
get_exponent()
e
transcript_
update()
```

5.8.4 Verificação

O verificador não recebe os bits nem as máscaras. Recompõe V_j e todos os desafios do traslado das Equações (5.33)–(5.35). Recompõe também as potências de y, z, u_k e os geradores dobrados. As respostas r_1, s_1, d_1 substituem os escalares finais nas Equações (5.40)–(5.43). Depois de reunir termos, toda a prova se reduz a uma multiplicação multiescalar que tem de dar a identidade $\mathcal{O}$.

A implementação pode verificar várias provas desta família numa só equação. Se $\mathcal{F}_q$ é a combinação de pontos que deveria ser $\mathcal{O}$ para a prova q , escolhe pesos independentes $w_q \in_R \mathbb{Z}_l^*$ e testa

$$\sum_q w_q \mathcal{F}_q = \mathcal{O}. \quad (5.44)$$

Os pesos impedem, excepto com probabilidade desprezável da ordem de $1/l$, que erros de provas diferentes se cancelem de propósito. Isto é *verificação em lote* de várias provas; não é a agregação de vários montantes dentro de uma prova.

No percurso de consenso congelado, `verRctSemanticsSimple()` reúne separadamente as Bulletproof+ e as Bulletproof antigas recebidas no conjunto de transacções que lhe foi passado. Um suplemento de transacções de bloco pode assim ser verificado em lote; a validação isolada de uma transacção chama o mesmo verificador com uma única prova. Não afirmamos que toda a verificação aconteça sempre «por bloco».

O verificador e o fim da batalha

Antes da equação final, há condições de aceitação que não devem desaparecer atrás da álgebra:

- r_1, s_1, d_1 têm de ser codificações escalares canónicas, isto é, menores que l . Os desafios produzidos por hash são reduzidos módulo l e têm de ser não nulos.
- A, A_1, B, V_j, L_k, R_k têm de decodificar como pontos comprimidos canónicos sobre a curva. Uma codificação não canónica ou que não representa um ponto provoca rejeição; as excepções de decodificação são convertidas em `false` pelo invólucro RingCT.
- Os pontos fornecidos pela prova são multiplicados pelo cofactor 8 antes da multiplicação multiescalar. Não há, contudo, uma regra separada que rejeite toda ocorrência da identidade: uma identidade honesta é extremamente improvável nos pontos aleatórios, mas o verificador decide-a pela estrutura e pela equação completa. Os geradores derivados são explicitamente não nulos.
- A prova tem de ser não vazia, os vectores $\mathbf{L}, \mathbf{R}$ têm o mesmo comprimento exacto e esse comprimento tem de concordar com o número de compromissos e com o limite de 16. A transacção RingCT deste tipo tem exactamente uma prova canónica.
- A equação de lote, a correspondência entre prova e `outPk`, a Equação (5.10) e as CLSAG são testes distintos; todos têm de passar.

```
src/ringct/
rctSigs.cpp,
verRct-
Semantics
Simple();
src/crypto-
note_core/
tx_verification_
utils.cpp,
ver_mixed_rct_
semantics()
```

Chegamos assim à afirmação prometida: para cada compromisso ligado ao traslado de Π_+ , o provador conhece uma abertura cujo montante está em $[0, 2^{64} - 1]$. Continuam públicos o número de saídas, os compromissos, o formato e tamanho da prova e a taxa da transacção. Continuam fora desta afirmação a identidade do destinatário, a propriedade da saída, a ausência de dupla despesa, a autorização das entradas e a validade completa da transacção. O próximo capítulo juntará essas peças sem alterar o significado limitado, mas essencial, da prova de intervalo.

```
src/ringct/  
bulletproofs_  
plus.cc,  
bulletproof_  
plus_VERIFY();  
src/ringct/  
rctOps.cpp,  
scalarmult8();  
src/crypto/  
crypto-ops.c,  
ge_frombytes_  
vartime()
```

CAPÍTULO 6

Transacções Confidenciais em Anel (RingCT)

Depois dos capítulos 4 e 5, uma transacção de um remetente anónimo para um destinatário anónimo soa mais ou menos assim:

“A minha transacção publica uma chave de transacção e cria duas saídas, cada uma com o seu endereço oculto, a sua etiqueta de visualização, o seu montante em código e o seu compromisso. Uma única prova Bulletproofs+ mostra que ambos os montantes cabem em 64 bits. Para pagar, gasto uma saída que escondo entre quinze desvios. A assinatura CLSAG mostra que conheço a chave de gasto e a abertura correcta de um dos compromissos do anel, sem dizer qual. A imagem de chave mostra que não gastei antes essa mesma saída. Finalmente, os pseudo-compromissos das entradas equilibram os compromissos das saídas e a taxa.”

É uma frase curta para um mecanismo com muitas rodas dentadas. Ainda ficam as perguntas certas. O remetente possui mesmo uma das saídas de cada anel? O montante escondido nessa saída é o montante colocado no respectivo pseudo-compromisso? Nenhuma saída cria dinheiro? E será que alguém mudou o destinatário, a taxa, os anéis ou as provas depois de a transacção ter sido assinada?

As respostas vêm de três construções que já conhecemos:

- os endereços ocultos e as imagens de chave do capítulo 4;
- os compromissos, pseudo-compromissos e provas Bulletproofs+ do capítulo 5;
- a assinatura CLSAG da secção 3.6.

Este capítulo encaixa essas peças nos bytes que a versão 16 aceita. As afirmações acerca da implementação referem-se à fotografia do código Monero identificada no resumo inicial. Nesse estado, a versão 16 activou-se na rede principal à altura 2 689 608 e é a última versão agendada. Isto é um retrato verificável dessa revisão, não uma promessa de que o protocolo nunca mais mudará.¹

6.1 O caminho activo: RCTTypeBulletproofPlus

Uma transacção ordinária criada pelo programa de referência na versão 16 tem versão de serialização 2 e tipo RingCT ‘6’, isto é, `RCTTypeBulletproofPlus`. Usa uma CLSAG por entrada, anéis de 16 membros e uma prova Bulletproofs+ agregada para todas as saídas. Tem entre duas e dezasseis saídas. A transacção de mineiro é a excepção importante: tem tipo ‘0’ (`RCTTypeNull`), montantes visíveis, nenhuma CLSAG e nenhuma prova de domínio. A rotina que constrói o bloco cria actualmente uma saída de mineiro; o consenso não contém, porém, uma regra isolada de “exactamente uma saída”: verifica a entrada de geração, o tipo, o desbloqueio, os alvos e a soma da recompensa.²

O nome merece ser lido com cuidado. *Confidencial* quer dizer que os montantes ficam escondidos em compromissos. *Em anel* quer dizer que a autorização aponta para um conjunto de saídas possíveis, não para uma saída declarada. Nenhuma destas duas propriedades esconde tudo: a taxa, o número de entradas, o número de saídas, os membros de cada anel, as imagens de chave e o campo `extra` continuam públicos.

Uma placa de arquivo: RCTTypeBulletproof2

O tipo ‘4’, `RCTTypeBulletproof2`, pertence à história do protocolo. Usava Bulletproofs e MLSAG, e era o tipo activo na versão 12 descrita pela edição anterior deste relatório. O apêndice A conserva uma transacção real desse período como espécime histórico: é ótima para aprender a ler os bytes, mas não é um molde de uma nova transacção v16. Entre os dois extremos existiu ainda o tipo ‘5’, `RCTTypeCLSAG`, que já trocava MLSAG por CLSAG mas ainda não usava Bulletproofs+. Os tipos ‘1’–‘5’ continuam necessários para validar a história da cadeia; na versão 16 não são o caminho para novas transacções ordinárias.³

¹ Veja-se `src/hardforks/hardforks.cpp` para a agenda da rede principal e `src/cryptonote_config.h` para as constantes de cada versão.

² A admissão dos tipos por versão está em `src/cryptonote_core/blockchain.cpp`; a enumeração dos tipos, em `src/ringct/rctTypes.h`; e a escolha feita pela carteira, em `src/wallet/wallet2.cpp` e `src/ringct/rctSigs.cpp`.

³ As transições estão codificadas em `src/cryptonote_core/blockchain.cpp`: CLSAG é admitida desde v13, Bulletproofs+ desde v15, e v16 deixa de admitir os Bulletproofs anteriores em novas transacções.

6.1.1 Compromissos a montantes e taxas de transacção

Um moneriano recebeu m saídas com montantes $a_1, \dots, a_m$, endereços ocultos $P_{\pi_1,1}, \dots, P_{\pi_m,m}$ e compromissos

$$C_{\pi_j,j}^a = x_j G + a_j H, \quad j \in \{1, \dots, m\}.$$

Ele conhece as chaves privadas de gasto p_j tais que $P_{\pi_j,j} = p_j G$, os montantes a_j e as máscaras x_j .

Pretende criar p saídas com montantes $b_1, \dots, b_p$ e pagar uma taxa f , todos em unidades atómicas, de modo que

$$\sum_{j=1}^m a_j = \sum_{t=1}^p b_t + f. \quad (6.1)$$

A taxa f é pública e serializada como um inteiro de comprimento variável. O mínimo de taxa que os nós costumam exigir para propagar uma transacção é política da carteira e da *mempool*, não esta equação de consenso: um bloco pode conter uma transacção de taxa inferior, desde que esta seja válida nos restantes aspectos. O valor declarado continua, claro, a entrar no equilíbrio e na recompensa do mineiro. Veja-se a secção 7.3.4.

Para cada saída nova a carteira deriva, como na secção 5.3, uma máscara determinística

$$y_t = \mathcal{H}_n(\text{"commitment_mask"} \parallel h_t)$$

e publica o compromisso

$$C_t^b = y_t G + b_t H.$$

Para cada entrada cria também um pseudo-compromisso

$$\tilde{C}_j^a = x'_j G + a_j H.$$

As primeiras $m - 1$ pseudo-máscaras x'_j são escolhidas ao acaso. A última fecha a cortina sem deixar uma frincha:

$$x'_m = \sum_{t=1}^p y_t - \sum_{j=1}^{m-1} x'_j \pmod{l}.$$

Por homomorfismo dos compromissos, a equação

$$\sum_{j=1}^m \tilde{C}_j^a \stackrel{?}{=} \sum_{t=1}^p C_t^b + f H \quad (6.2)$$

verifica ao mesmo tempo o equilíbrio dos montantes e o das máscaras, sem revelar nenhum a_j ou b_t . A própria igualdade não prova que o autor possui as entradas. É a CLSAG que prende cada pseudo-compromisso à saída real de um anel.

O programa de referência baralha as saídas antes de lhes atribuir os índices t e ordena as entradas por imagem de chave. São escolhas de construção da carteira; o consenso recebe apenas a ordem final.

6.1.2 O que nasce em cada saída

Para a saída t , o remetente escolhe a chave de transacção apropriada e calcula a derivação partilhada D_t , o escalar

$$h_t = \mathcal{H}_n(D_t \parallel \text{varint}(t))$$

e o endereço oculto do destinatário

$$P_t^b = h_t G + K_t^s.$$

Na versão 16 o alvo serializado é etiquetado: contém P_t^b e o primeiro byte de

$$\mathcal{H}(\text{"view_tag"} \parallel D_t \parallel \text{varint}(t)).$$

A etiqueta deixa a carteira rejeitar quase todas as saídas alheias antes de fazer o trabalho mais caro; não é uma prova de propriedade.

O montante em código contém apenas oito bytes:

$$b_t \oplus_8 \mathcal{H}(\text{"amount"} \parallel h_t)[0 \dots 7].$$

A máscara não viaja cifrada ao lado do montante. O destinatário volta a calcular y_t pela fórmula anterior e confirma que o compromisso publicado é $C_t^b = y_t G + b_t H$. Esta confirmação é importante: decifrar oito bytes com aspecto razoável não basta para possuir uma saída. ⁴

Uma transacção publica uma chave principal de transacção em **extra**. Quando há sub-endereços, pode publicar ainda uma lista de chaves adicionais, uma por saída, seguindo exactamente os casos da secção 4.2.1. Essas chaves permitem ao destinatário obter D_t ; não demonstram ao consenso que certo endereço foi o destinatário pretendido.

6.1.3 Uma prova Bulletproofs+ para todas as saídas

Os compromissos $C_1^b, \dots, C_p^b$ entram numa única prova Bulletproofs+ que mostra

$$b_t \in [0, 2^{64} - 1] \quad \text{para todo } t.$$

Na transacção ordinária v16, $2 \leq p \leq 16$. A prova é preenchida internamente até

$$M = 2^{\lceil \log_2 p \rceil},$$

e o verificador exige precisamente o menor M que cobre as saídas; não se pode escolher uma prova maior só para mudar o aspecto dos bytes.

O objecto serializado é

$$\Pi_{\text{BPP}+} = (A, A_1, B, r_1, s_1, d_1, \mathbf{L}, \mathbf{R}).$$

Os compromissos V_t não são serializados dentro da prova: o nó reconstrói-os a partir dos C_t^b que já estão na base RingCT. Há aqui uma convenção de cofactor que é fácil perder de vista. **outPk** guarda o compromisso normal C_t^b , enquanto a prova usa $V_t = 8^{-1} C_t^b$. Do mesmo modo, A, A_1, B, L_i, R_i viajam divididos por 8 e o verificador multiplica-os por 8. Isto é representação no fio, não uma mudança do montante comprometido. A prova e a verificação completas estão na secção 5.8. Os compromissos históricos do anel, os compromissos de saída em **outPk** e os pseudo-compromissos são serializados na representação normal; não se lhes aplica esta divisão por 8. ⁵

⁴ A derivação da saída e do *view tag* está em `src/device/device_default.cpp`; a máscara e o XOR de oito bytes, em `src/ringct/rctOps.cpp`; e a forma serializada, em `src/ringct/rctTypes.h`.

⁵ Comparem-se `src/ringct/rctSigs.cpp`, `src/ringct/rctTypes.cpp`, `src/cryptonote_basic/cryptonote_format_utils.cpp` e `src/ringct/bulletproofs_plus.cc`.

6.1.4 Um anel: uma entrada real e quinze desvios

Para cada entrada j , a transacção contém 16 índices de saídas históricas. Depois de converter os *offsets* relativos em índices absolutos, o nó obtém o anel público

$$\mathcal{R}_j = \{(P_{i,j}, C_{i,j}^a)\}_{i=0}^{15}. \quad (6.3)$$

Um desses pares, no índice secreto π_j , é a saída que o remetente possui; os outros quinze são desvios. Os índices absolutos aparecem por ordem crescente e só o primeiro é guardado em absoluto. Por exemplo, $\{7, 11, 15, 20\}$ torna-se $\{7, 4, 4, 5\}$. A repetição de um índice dentro do mesmo anel produz um *offset* relativo zero e é rejeitada.

Na versão 16 todas as entradas RingCT têm exactamente quinze desvios, isto é, dezasseis membros, e todas têm o mesmo tamanho de anel. Isto é consenso. A escolha de *quais* saídas ocupam os quinze lugares é comportamento da carteira. O programa de referência insere a saída real e amostra candidatos históricos com a sua `gamma_picker`: parte de uma distribuição gamma de forma 19,28 e escala 1/1,61, corrige a janela de maturação, estima um índice pela densidade recente de saídas e escolhe ao acaso uma saída dentro do bloco encontrado.⁶

Quem verifica conhece todos os pares de $\mathcal{R}_j$, mas não recebe esses pontos repetidos dentro da assinatura. Recebe os *offsets* e vai buscá-los à cadeia. Antes de verificar a CLSAG confirma que as referências existem, têm tipos admissíveis, já maturaram e estão desbloqueadas.

6.1.5 Assinatura

O pseudo-compromisso da entrada j é subtraído a cada compromisso do anel:

$$Q_{i,j} = C_{i,j}^a - \tilde{C}_j^a.$$

Para os desvios $i \neq \pi_j$, $Q_{i,j}$ é apenas um ponto para o qual o signatário não conhece necessariamente logaritmo em base G . Na coluna real, os montantes cancelam:

$$\begin{aligned} Q_{\pi_j,j} &= (x_j G + a_j H) - (x'_j G + a_j H) \\ &= (x_j - x'_j) G = z_j G. \end{aligned} \quad (6.4)$$

Assim, para assinar a entrada j , o remetente conhece dois testemunhos na mesma coluna secreta:

$$\begin{aligned} p_j &= \log_G(P_{\pi_j,j}), \\ z_j &= \log_G(Q_{\pi_j,j}) = x_j - x'_j \pmod{l}. \end{aligned}$$

A CLSAG comprime estas duas relações numa só assinatura, sem revelar π_j . As imagens ligáveis são

$$\begin{aligned} I_j &= p_j \mathcal{H}_p(P_{\pi_j,j}), \\ D_j &= z_j \mathcal{H}_p(P_{\pi_j,j}). \end{aligned}$$

I_j é guardada na entrada, antes da parte RingCT. A assinatura CLSAG serializa, para um anel v16, dezasseis respostas $s_{0,j}, \dots, s_{15,j}$, o desafio inicial $c_{1,j}$ e

$$\bar{D}_j = 8^{-1} D_j.$$

⁶ Veja-se `src/wallet/wallet2.cpp`, em particular `gamma_picker::pick()` e `wallet2::get_outs()`. Esta distribuição é uma heurística de privacidade da carteira congelada, não uma regra de validade. Também não se deve chamar aos desvios “saídas não gastas”: a cadeia não marca uma saída RingCT como gasta, apenas regista imagens de chave.

Não serializa I_j uma segunda vez. Os pesos de agregação, o ciclo de desafios e as equações completas estão na secção 3.6; repeti-los aqui só vestiria a mesma assinatura com outro casaco.

Em suma, a *declaração pública* de uma CLSAG desta transacção contém $\mathbf{m}$, os dezasseis pares $(P_{i,j}, C_{i,j}^a)$, $\tilde{C}_j^a$, I_j e $\bar{D}_j$. Os *testemunhos privados* são o índice π_j , a chave de utilização única p_j e a diferença de máscaras $z_j = x_j - x'_j$. Os *hashes* de agregação incluem ambas as filas, as imagens e o pseudo-compromisso; o ciclo de desafios usa os dois testemunhos na mesma coluna. É esta coincidência de índice, e não uma vaga prova de que “alguém conhece alguma coisa”, que prende a chave de gasto à diferença correcta de compromissos.

Para m entradas, a assinatura total é

$$\tau = \{\sigma_1(\mathbf{m}), \dots, \sigma_m(\mathbf{m})\}.$$

Cada entrada tem o seu próprio anel e o seu próprio índice real. Todas as CLSAG assinam a mesma mensagem $\mathbf{m}$, mas cada uma prende também o seu pseudo-compromisso à sua assinatura. A transacção só passa se todas passarem.

6.1.6 A mensagem que as CLSAG assinam

“Assina-se toda a transacção” é uma boa intuição, mas não é uma descrição de bytes suficientemente exacta. Na revisão congelada, a mensagem anterior às CLSAG é

$$\mathbf{m} = \mathcal{H}(h_0 \parallel h_1 \parallel h_2), \quad (6.5)$$

onde:

h_0 é o *hash* do prefixo serializado: versão, `unlock_time`, entradas (montantes visíveis zero, *offsets* e imagens de chave), saídas (montantes visíveis zero, tipo de alvo, endereços ocultos e etiquetas de visualização) e os bytes exactos de `extra`;

h_1 é o *hash* da base RingCT serializada: tipo ‘6’, taxa, os oito bytes cifrados de cada montante e os compromissos normais de saída em `outPk`;

h_2 é o *hash* da prova Bulletproofs+ pela ordem $A, A_1, B, r_1, s_1, d_1, L_0, \dots, L_q, R_0, \dots, R_q$.

Os corpos CLSAG ficam necessariamente de fora, pois ainda estão a ser calculados. Há uma subtilidade menos óbvia: os pseudo-compromissos também não entram em h_0, h_1, h_2 . Em vez disso, a transcrição de cada CLSAG inclui explicitamente o seu próprio $\tilde{C}_j^a$, juntamente com $\mathbf{m}$, os $P_{i,j}$, os $C_{i,j}^a$, I_j e D_j . Logo, mudar um pseudo-compromisso invalida a assinatura correspondente; não é, porém, verdade literal que exista um único *hash* de todos os bytes excepto as assinaturas.⁷

Esta disposição prende os destinatários, os anéis, as imagens de chave, a taxa, `extra`, os montantes em código, os compromissos de saída, a prova de domínio e cada pseudo-compromisso à autorização do dono. Uma alteração deixa de fechar pelo menos um ciclo CLSAG.

⁷ A construção está em `src/ringct/rctSigs.cpp`, funções `get_pre_mlsag_hash()`, `genRctSimple()` e `CLSAG_Gen()`; o *hash* final dos três termos está em `src/device/device_default.cpp`.

Cada camada responde apenas à sua pergunta. Bulletproofs+ não prova propriedade, destinatário, ausência de gasto duplo nem validade completa da transacção. CLSAG não prova que os novos montantes estão no domínio. A equação de equilíbrio não autoriza uma entrada. A transacção só é válida quando estas respostas, a serialização e as regras exteriores de consenso passam em conjunto.

6.1.7 Verificação

A implementação divide a verificação RingCT em duas partes com nomes um pouco enganadores:

1. a verificação *semântica* confirma a forma dos vectores, a correspondência entre saídas e montantes em código, a existência de uma única prova Bulletproofs+ com a capacidade exacta, a equação (6.2) e a prova de domínio;
2. a verificação *não semântica* reconstrói os anéis a partir da cadeia e verifica uma CLSAG por entrada contra o seu pseudo-compromisso.

Em redor destas duas partes, o núcleo verifica ainda a versão e o tipo, os alvos das saídas, o número e a igualdade dos tamanhos dos anéis, os *offsets*, a existência e maturidade das referências, as imagens de chave e o limite de peso. “Semântica” não quer dizer aqui “tudo o que a transacção significa”; é o nome histórico da metade que pode ser verificada sem procurar os membros dos anéis.

Os escalares de resposta e desafio CLSAG têm de estar em forma canónica. Os pontos têm de decodificar, I_j não pode ser a identidade e tem de satisfazer $lI_j = 0$; depois de recuperar $D_j = 8\bar{D}_j$, a verificação rejeita também a identidade. Bulletproofs+ verifica os escalares canónicos, a forma dos vectores e as suas próprias equações depois das multiplicações por 8. É por isso demasiado forte dizer simplesmente que “todos os pontos da transacção são testados um a um no subgrupo primo”: a implementação usa testes e limpezas de cofactor diferentes em fronteiras diferentes.

Os testes locais tornam estas fronteiras visíveis. Os testes de núcleo para Bulletproofs+ rejeitam provas em falta, provas duplicadas e uma prova sobre o compromisso errado; o teste de maleabilidade CLSAG altera $\bar{D}$ por um ponto de torção e espera rejeição; os testes de serialização confirmam que V_t e I_j são reconstruídos, não repetidos no corpo das provas.⁸

6.1.8 Evitar o gasto duplo

A imagem de chave $I_j = p_j \mathcal{H}_p(P_{\pi_j, j})$ é determinística para a chave privada de utilização única, mas não revela qual dos dezasseis $P_{i, j}$ lhe corresponde. O núcleo rejeita imagens repetidas dentro da mesma transacção, entre transacções do mesmo bloco e contra o conjunto de imagens já gastas

⁸ Veja-se `tests/core_tests/bulletproof_plus.cpp`, `tests/core_tests/rct2.cpp`, `tests/unit_tests/bulletproofs_plus.cpp` e `tests/unit_tests/serialization.cpp`.

na cadeia. Sob a segurança da CLSAG e da função de *hash* para ponto, uma segunda assinatura com a mesma chave produz a mesma I_j : o marcador denuncia a repetição sem desmascarar o membro real.

A igualdade de uma imagem de chave não diz quanto foi gasto, para onde foi, nem qual membro do anel era verdadeiro. Diz apenas que duas tentativas não podem ambas entrar na história canónica. Antes disso, a *mempool* também evita aceitar conflitos que conhece; a regra decisiva é a validação do bloco.

6.1.9 Maturação e tempo de desbloqueio

Todas as saídas referidas por um anel v16, verdadeira e desvios, têm de ter pelo menos dez blocos. Uma saída criada no bloco de altura h pode, por esta regra, aparecer pela primeira vez num anel de uma transacção incluída à altura $h + 10$. Não é apenas o comportamento por omissão da carteira: desde v12 o núcleo impõe-o ao conjunto inteiro de referências.

O prefixo contém ainda um único `unlock_time` para todas as saídas da transacção. Valores inferiores a 500 000 000 são interpretados como altura; valores a partir daí, como tempo Unix. Na versão 16, o teste temporal usa o tempo ajustado derivado dos blocos anteriores e uma tolerância de 120 segundos. A carteira de referência constrói transacções ordinárias com `unlock_time=0`, e os nós não propagam normalmente uma transacção ordinária com outro valor; isto é política de *mempool*. O consenso conserva a funcionalidade e verifica também o `unlock_time` original de cada membro do anel.

Uma transacção de mineiro criada à altura h tem obrigatoriamente `unlock_time = h + 60`. A sua saída só pode portanto ser referida a partir dessa altura; o bloqueio de 60 domina a maturação ordinária de 10.⁹

6.1.10 o segredo público γ

Chamemos “segredo público” ao segredo que toda a gente sabe que existe e ninguém deve saber qual é. O ponto H é público e, por viver no subgrupo gerado por G , existe matematicamente um escalar γ tal que

$$H = \gamma G.$$

O que se exige é que ninguém conheça esse escalar. A construção de H por *hash* para ponto destina-se precisamente a não deixar uma pessoa com esta porta das traseiras. γ não é, portanto, um segredo publicamente conhecido: é o logaritmo discreto desconhecido por detrás de uma relação pública. Daqui em diante, toda a aritmética escalar desta conversa é módulo l .

⁹ Veja-se `is_tx_spendtime_unlocked()` e a verificação das referências em `src/cryptonote_core/blockchain.cpp`, a política em `src/cryptonote_core/tx_pool.cpp` e a construção da transacção de mineiro em `src/cryptonote_core/cryptonote_tx_utils.cpp`.

Suponha-se, contudo, que o Óscar conhece γ . Compra ou recebe 0,1 xmr numa saída já gastável que possui. Conhece a chave de gasto, o montante b , a máscara y e o compromisso histórico

$$C^a = yG + bH.$$

Não precisa de publicar um novo ponto C_{mau}^a . Escolhe qualquer montante de 64 bits e quer uma *segunda abertura do mesmo ponto*; para continuar a história, toma $b_{\text{mau}} = 100$ xmr. Define

$$y_{\text{mau}} = y + (b - b_{\text{mau}})\gamma \pmod{l}. \quad (6.6)$$

Então

$$\begin{aligned} y_{\text{mau}}G + b_{\text{mau}}H &= (y + (b - b_{\text{mau}})\gamma)G + b_{\text{mau}}\gamma G \\ &= yG + b\gamma G \\ &= C^a. \end{aligned}$$

O ponto na cadeia não mexeu um milímetro; foi a história que o Óscar conta sobre ele que engordou.

Agora podemos seguir a burla até ao fim, em vez de parar à porta. Ao gastar a saída, o Óscar trata C^a como compromisso a b_{mau} . Escolhe uma pseudo-máscara x' e publica

$$\tilde{C}^a = x'G + b_{\text{mau}}H.$$

Na coluna real do anel,

$$C^a - \tilde{C}^a = (y_{\text{mau}} - x')G,$$

portanto conhece o segundo testemunho CLSAG $z = y_{\text{mau}} - x'$. Continua a precisar de p , a chave de gasto da saída: conhecer γ não rouba moedas alheias por si só. Mas permite reabrir as próprias moedas com montantes inventados.

O Óscar cria, por exemplo, duas saídas cuja soma, mais a taxa, é b_{mau} . Escolhe as máscaras de saída, fecha (6.2), produz uma Bulletproofs+ válida para esses dois novos montantes de 64 bits e assina cada entrada. Tudo passa: a CLSAG conhece p e z , o equilíbrio usa a abertura falsa e a prova de domínio só pergunta se os *novos* montantes de saída estão no domínio. A prova de domínio que acompanhou C^a quando este nasceu demonstrava que existia uma abertura antiga em 64 bits; não demonstrava que essa abertura era única, e o gasto não volta a provar o domínio das entradas.

Sem γ , o mesmo truque pediria

$$y_{\text{mau}} = \log_G(C^a - b_{\text{mau}}H),$$

e agora, “ah e tal”, queremos calcular um logaritmo discreto. É aqui que o PLD fecha a porta.

Duas aberturas, um segredo

O ponto essencial é que não conseguimos calcular γ olhando apenas para C^a . Precisamos de conhecer duas descrições completas do mesmo ponto:

$$C^a = yG + bH,$$

$$C^a = y_{\text{mau}}G + b_{\text{mau}}H.$$

Ou seja, precisamos dos quatro números

$$(y, b) \quad \text{e} \quad (y_{\text{mau}}, b_{\text{mau}}).$$

Como as duas expressões dão exactamente o mesmo compromisso, subtraímos uma da outra:

$$\begin{aligned} 0 &= (y - y_{\text{mau}})G + (b - b_{\text{mau}})H \\ &= (y - y_{\text{mau}} + (b - b_{\text{mau}})\gamma)G. \end{aligned}$$

Na segunda linha substituímos simplesmente $H = \gamma G$. Como G tem ordem prima l , o escalar entre parênteses tem de ser zero módulo l . Podemos então isolar γ :

$$\gamma = (y_{\text{mau}} - y)(b - b_{\text{mau}})^{-1} \pmod{l}. \quad (6.7)$$

O inverso existe: dois montantes distintos de 64 bits têm diferença não nula módulo o primo l . Encontrar uma segunda abertura não trivial quebra, pois, a vinculação computacional e revela exactamente $\gamma = \log_G H$, a relação de logaritmo discreto entre os geradores.

Um exemplo pequenino

Imaginemos, apenas para aprender, um grupo de ordem 11 no qual a relação secreta é

$$H = 4G.$$

Neste brinquedo, portanto, $\gamma = 4$. Uma primeira abertura é

$$y = 2, \quad b = 3.$$

O compromisso correspondente é

$$\begin{aligned} C^a &= 2G + 3H \\ &= 2G + 3(4G) \\ &= 14G = 3G \quad (\text{pois } 14 \equiv 3 \pmod{11}). \end{aligned}$$

Suponhamos que alguém encontra outra abertura do mesmo C^a :

$$y_{\text{mau}} = 5, \quad b_{\text{mau}} = 5.$$

Com efeito,

$$\begin{aligned} 5G + 5H &= 5G + 5(4G) \\ &= 25G = 3G \quad (\text{pois } 25 \equiv 3 \pmod{11}). \end{aligned}$$

As duas aberturas dão o mesmo ponto $C^a = 3G$. Agora usamos os quatro números na equação (6.7):

$$\gamma = (5 - 2)(3 - 5)^{-1} \pmod{11}.$$

Temos $3 - 5 = -2 \equiv 9 \pmod{11}$. O inverso de 9 módulo 11 é 5, porque $9 \cdot 5 = 45 \equiv 1 \pmod{11}$. Assim,

$$\gamma = 3 \cdot 5 = 15 \equiv 4 \pmod{11}.$$

Recuperámos precisamente o segredo com que construímos o brinquedo.

Vemos agora as duas direcções sem saltos:

- se o Óscar conhece γ , consegue fabricar uma segunda abertura;
- se alguém consegue fabricar uma segunda abertura diferente e conhecemos as duas aberturas, conseguimos extrair γ .

Uma máquina que recebesse a primeira abertura e encontrasse facilmente a segunda dar-nos-ia, pela equação (6.7), um algoritmo para calcular $\log_G H$. Portanto, encontrar uma segunda abertura

não é uma porta lateral mais barata para evitar o PLD; se o fosse, essa mesma porta resolveria o PLD. γ não sai de C^a sozinho: sai da comparação entre duas aberturas distintas do mesmo C^a .

Os 64 bits dos montantes também não encolhem o problema para 64 bits. Depois de escolher um candidato b_{mau} , o Óscar consegue calcular o ponto

$$P = C^a - b_{\text{mau}}H,$$

mas continua a precisar do escalar y_{mau} tal que $P = y_{\text{mau}}G$. Como G gera o subgrupo, qualquer montante candidato produz um ponto que tem algum logaritmo; o compromisso não acende uma luz para dizer se esse montante é o histórico. O obstáculo é calcular o logaritmo correspondente, um PLD no grupo inteiro, cujo tamanho é l , não uma busca no intervalo dos montantes.

Há, claro, quem se esfole a tentar arrombar esta porta. Os melhores ataques *genéricos* clássicos conhecidos, como o método rho de Pollard, precisam de uma ordem de grandeza de

$$\sqrt{l}$$

operações de grupo: encontram uma colisão depois de explorar aproximadamente a raiz quadrada do espaço, pelo mesmo efeito do paradoxo dos aniversários. Aqui l está um pouco acima de 2^{252} , logo $\sqrt{l}$ está perto de 2^{126} , cerca de $8,5 \times 10^{37}$ operações. Mesmo uma máquina fantasiosa que executasse 10^{18} operações de grupo por segundo levaria cerca de $2,7 \times 10^{12}$ anos. Uma operação de grupo real é, de resto, muito mais trabalhosa do que uma instrução elementar de processador.

“Genérico” quer dizer que o ataque usa apenas as operações do grupo e não uma fraqueza especial da curva, da geração de H ou da implementação. A estimativa 2^{126} não é uma prova de que nunca surgirá matemática melhor; é o nível de segurança clássico que resulta dos melhores métodos conhecidos. Um atalho genuíno não faria apenas o Óscar rico: abalaria todas as construções que dependem do mesmo PLD. É por isso que as pessoas tentam — e é também por isso que não lhes basta tentar com um computador um bocadinho maior.

O segredo de γ é, pois, a hipótese de *vinculação* dos compromissos e da oferta monetária. Não é a hipótese de ocultação: com uma máscara uniforme, $yG + bH$ esconde perfeitamente b mesmo de quem conhecesse γ . E os muitos logaritmos que aparecem ao variar b_{mau} não são segredos independentes; são transformações afins da mesma porta das traseiras γ . Se ela se abre, pode-se “gamar”; se fica fechada, cada transformação continua a parecer um PLD novo.

Finalmente, três gammas que partilham a letra não partilham a função. Este $\gamma = \log_G H$ é a relação desconhecida entre geradores. A distribuição gamma da `gamma_picker` escolhe idades de desvios. E os símbolos γ_j usados por vezes na literatura de Bulletproofs para máscaras de compromissos são apenas escalares escolhidos pelo provador. Confundi-los é dar a mesma alcunha ao cofre, ao carteiro e à cortina.

Provas de domínio (saídas)

Uma prova Bulletproofs+ agregada, com M a menor potência de dois tal que $M \geq p$, requer

$$32(6 + 2 \log_2(64M)) \text{ bytes}$$

para os seus elementos de 32 bytes, além dos prefixos de comprimento dos vectores. Para $p = 2$, são 640 bytes; para $p = 16$, 832 bytes.

6.2 Resumo conceptual

Para resumir, apresenta-se aqui o conteúdo principal de uma transacção v16, organizado para clareza conceptual. O apêndice A mostra bytes reais de uma transacção mais antiga; as diferenças estão assinaladas no próprio apêndice.

- Tipo: ‘6’ é `RCTTypeBulletproofPlus`; ‘0’ é `RCTTypeNull` para a transacção de mineiro.
- Entradas: para cada entrada j :
 - montante visível zero;
 - dezasseis *offsets* relativos que localizam o anel;
 - imagem de chave I_j ;
 - pseudo-compromisso $\tilde{C}_j^a$, na parte podável;
 - CLSAG com dezasseis $s_{i,j}$, $c_{1,j}$ e $\bar{D}_j$, também na parte podável.
- Saídas: para cada saída t :
 - montante visível zero;
 - alvo etiquetado com o endereço oculto P_t^b e um *view tag* de um byte;
 - compromisso normal $C_t^b = y_t G + b_t H$ em `outPk`;
 - oito bytes b_t cifrados por XOR em `ecdhInfo`.
- Base RingCT: tipo, taxa pública, `ecdhInfo` e `outPk`. Esta parte não é podável.
- Parte podável: uma prova Bulletproofs+, seguida das CLSAG e dos pseudo-compromissos.
- extra: bytes opacos no prefixo, onde as carteiras colocam normalmente a chave pública principal de transacção, eventuais chaves adicionais e, quando aplicável, um *nonce*. O consenso assina os bytes exactos, mas não prova que estes obedecem às relações secretas pretendidas pela carteira.

Finalmente, uma transacção exemplo com uma entrada e duas saídas soa assim:

“A transacção publica as chaves necessárias para os destinatários reconhecerem as saídas. Gasto uma das dezasseis saídas do meu anel. A CLSAG prova que conheço, na mesma posição escondida, a chave privada do endereço oculto e a diferença de máscaras entre o compromisso histórico e o pseudo-compromisso. A imagem de chave ainda não apareceu, portanto esta saída não foi aceite antes noutro gasto. Crio duas saídas com endereços ocultos, etiquetas de visualização, compromissos e montantes em código. Uma Bulletproofs+ prova que ambos os montantes estão no domínio de 64 bits. A taxa f é pública, e $\tilde{C}^a = C_1^b + C_2^b + fH$. A mensagem CLSAG prende o prefixo, a base RingCT e a prova, e a transcrição da própria CLSAG prende ainda o pseudo-compromisso. Não vos digo qual entrada era minha nem quanto transferi; dou-vos exactamente as provas necessárias para não ter de mo dizer.”

6.2.1 Requisitos de espaço

Um anel v16 não repete no *blob* os 16 endereços ocultos e os 16 compromissos: os *offsets* apontam para esses dados na cadeia. Por entrada, os elementos fixos directamente ligados ao gasto são:

$$\underbrace{16 \cdot 32 + 32 + 32}_{\text{CLSAG}} + \underbrace{32}_{I_j} + \underbrace{32}_{\tilde{C}_j^a} = 640 \text{ bytes},$$

antes dos dezasseis *varints* dos *offsets* e dos pequenos prefixos de serialização. A CLSAG propriamente dita ocupa 576 bytes: 512 nas respostas, 32 em c_1 e 32 em $\bar{D}$.

Para as saídas, cada t acrescenta pelo menos os 32 bytes do endereço oculto, um byte de *view tag*, 32 bytes do compromisso e oito bytes do montante em código, mais as marcas e cumprimentos da serialização. As chaves de transacção vivem em **extra**. A prova Bulletproofs+ custa

$$32(6 + 2 \log_2(64M)), \quad M = 2^{\lceil \log_2 p \rceil},$$

mais dois cumprimentos de vector.

Peso, não apenas bytes

Para uma transacção actual, o peso é o tamanho serializado mais uma penalização de “recuperação” da economia obtida pela agregação de Bulletproofs+. Se $M \leq 2$, a penalização é zero e peso e bytes coincidem. Para $M > 2$, a revisão congelada calcula

$$w = |\text{blob}| + \frac{4}{5} [320M - 32(6 + 2 \log_2(64M))],$$

com divisão inteira no último termo. Na versão 16 uma transacção não pode exceder

$$\frac{300\,000}{2} - 600 = 149\,400$$

unidades de peso. A reserva de 600 deixa espaço para a transacção de mineiro; a relação com o peso dinâmico do bloco é explicada na secção 7.3.2.¹⁰

¹⁰ Veja-se `get_transaction_weight()` e `get_transaction_weight_clawback()` em `src/cryptonote_basic/cryptonote_format_utils.cpp`, e o limite em `src/cryptonote_core/tx_verification_utils.cpp`.

O campo **extra**: conteúdo, limite e fronteira

extra é um vector de bytes do prefixo. As convenções conhecidas começam cada campo por uma etiqueta: por exemplo, `0x01` para a chave pública principal, `0x02` para um *nonce* com comprimento e `0x04` para chaves públicas adicionais; `0x00` representa enchimento. Portanto, não é verdade que as etiquetas tenham “todos os bits a um”.

A carteira de referência ordena campos que consegue analisar e recusa criar **extra** com mais de 1060 bytes. Um nó recusa normalmente propagar pela *mempool* uma transacção que exceda esse valor. O limite de 1060 é política, não uma regra de consenso do bloco: no consenso, **extra** continua limitado indirectamente pelo tamanho e pelo peso de toda a transacção.

Também aqui “não é verificado” precisa de tradução. O vector e o seu comprimento têm de ser serializáveis, e os bytes exactos entram em h_0 , logo não podem ser alterados depois da assinatura. Mas a validação de um bloco não exige que esses bytes formem todos campos reconhecidos, que haja uma só chave de cada espécie, nem que uma chave de transacção corresponda a um segredo usado pelo remetente. Essas são convenções que permitem às carteiras encontrar e decifrar saídas. Bytes mal formados podem tornar uma saída invisível ou inutilizável sem, por essa razão isolada, tornar inválido o bloco.¹¹

É uma fronteira característica de Monero: o consenso protege a integridade dos bytes e as equações monetárias; a carteira dá significado operacional a alguns desses bytes. Confundir as duas coisas faz o protocolo parecer ao mesmo tempo mais rígido e menos seguro do que realmente é.

¹¹ As etiquetas e formatos estão em `src/cryptonote_basic/tx_extra.h`; a análise e ordenação, em `src/cryptonote_basic/cryptonote_format_utils.cpp`; os limites de construção e propagação, em `cryptonote_tx_utils.cpp` e `tx_pool.cpp`.

CAPÍTULO 7

A Blockchain

Em português: a lista de blocos.

Nota de leitura. Neste capítulo, “regra activa” significa uma regra da rede principal no estado técnico identificado no resumo inicial. As afirmações sobre a implementação foram conferidas na mesma fotografia do código-fonte de Monero [88]; as notas marginais indicam os ficheiros e funções relevantes. Comportamentos da carteira ou do nó, regras históricas e código inactivo serão assinalados como tais. Esta nota vale para todo o capítulo.

A experiência humana ganhou uma nova dimensão através da Internet. Podemos corresponder com pessoas em todos os cantos do planeta e temos ao alcance uma quantidade inimaginável de informação. A troca de bens e serviços é fundamental para uma sociedade próspera e pacífica [87]; no reino digital, podemos contribuir valor para o mundo inteiro.

Os meios de troca (dinheiros) são essenciais: dão-nos um ponto de referência para uma imensa diversidade de bens económicos que, de outra forma, seriam impossíveis de comparar, e permitem interacções mutuamente benéficas entre pessoas que nada têm em comum [87]. Ao longo da história houve muitos tipos de dinheiro, de conchas a papel e ouro. Foram trocados à mão; agora, o dinheiro também pode ser trocado electronicamente.

No modelo actual, de longe o mais adoptado, as transacções electrónicas são controladas por instituições financeiras. Estas entidades terceiras guardam o dinheiro e recebem a confiança necessária para o transferir. Têm de mediar disputas; os pagamentos são reversíveis; e podem ser censuradas ou controladas por organizações poderosas [89].

Para aliviar estas inconveniências, surgiram moedas digitais descentralizadas.

1

7.1 Moedas digitais

Desenvolver uma moeda digital não é trivial. Imaginemos três tipos: pessoal, centralizada e distribuída. Uma moeda digital é apenas uma colecção de mensagens; os “montantes” guardados nelas são interpretados como quantidades monetárias.

No **modelo de correio electrónico**, qualquer pessoa pode criar moedas: por exemplo, escrever “eu tenho 5 moedas” e enviar a mensagem a todos os que têm um endereço. A quantidade total não é limitada e é possível gastar as mesmas moedas mais do que uma vez (gasto duplo).

No **modelo de jogo de vídeo**, as moedas são guardadas numa base de dados centralizada. Os utilizadores confiam na honestidade do guardião. Observadores externos não conseguem verificar a quantidade total; o guardião pode alterar as regras a qualquer altura ou ser censurado por terceiros poderosos.

7.1.1 Versão de eventos comuns e distribuídos

No dinheiro digital “comum”, muitos computadores possuem o registo de cada transacção. Quando um deles recebe uma nova transacção, transmite-a à rede, que a aceita se ela seguir regras predefinidas.

Os utilizadores só beneficiam das moedas se outros as aceitarem em troca. Para maximizar a sua utilidade, existe uma tendência natural para adoptar um conjunto de regras comuns sem autoridade central.

2

Regra n.º 1: Dinheiro só pode ser criado em cenários bem definidos.

Regra n.º 2: As transacções gastam dinheiro que já existe.

Regra n.º 3: Cada transacção só ocorre uma vez.

Regra n.º 4: Só o proprietário do montante o pode gastar.

Regra n.º 5: As saídas de transacção representam o dinheiro gasto.

Regra n.º 6: As transacções seguem a sintaxe correcta.

¹ Este capítulo inclui mais detalhes de implementação do que os anteriores, pois a natureza da lista de blocos depende muito da sua estrutura específica.

² Na ciência política diz-se a isto um contrato social.

As regras 2–6 estão contidas no esquema do capítulo 6, que acrescenta fungibilidade e privacidade: signatário ambíguo, destinatário anónimo e montantes ocultos a terceiros. A primeira regra será explicada mais adiante. Como as transacções usam criptografia, trata-se de uma *criptomoeda*.

Imaginemos que dois computadores distantes recebem, quase ao mesmo tempo, transacções distintas que gastam a mesma saída. Pouco depois, ambos conhecem as duas: como decidem qual é válida? O histórico divide-se em duas cópias que, à primeira vista, seguem as mesmas regras — uma bifurcação (*fork*).

Parece claro que a transacção anterior deve ser canónica. É mais fácil dizê-lo do que fazê-lo. Como veremos, obter consenso sobre o histórico de transacções é a *raison d'être* da blockchain, isto é, da lista de blocos.

7.1.2 Lista de blocos simples

Primeiro é necessário que todos os computadores, doravante *nós*, concordem com a ordem das transacções.

Digamos que uma criptomoeda começa com uma declaração de génese: “A moeda exemplo começa!”

Esta mensagem é, neste caso, um “bloco”, e o seu ID é:

$$BH_G = \mathcal{H}(\text{“A moeda exemplo começa!”})$$

Cada vez que um nó recebe transacções, calcula os seus IDs TH ; cada bloco recebe também um ID BH . Como cada bloco aponta para o anterior, uma lista de blocos é um vector unidimensional cujo primeiro elemento é o bloco de génese:

$$\begin{aligned} BH_1 &= \mathcal{H}(BH_G, TH_1, TH_2, \dots) \\ BH_2 &= \mathcal{H}(BH_1, TH_3, TH_4, \dots) \end{aligned}$$

Desta forma há uma ordem clara de eventos, do bloco actual até à génese. Com os seus ramos alternativos, a estrutura é um grafo acíclico dirigido; o ramo canónico de Monero é a sua variante unidimensional. As arestas são setas unidireccionais e nenhum caminho regressa ao nó de onde partiu [5].

Os mineiros incluem data e hora nos blocos. Se a maioria for razoavelmente honesta, a lista dá também uma aproximação da altura temporal de cada transacção; veremos depois os limites impostos ao carimbo. Se dois blocos referem o mesmo antecessor e se propagam ao mesmo tempo, metade da rede pode receber um primeiro e a outra metade o outro. Nasce um ramo. Não é uma catástrofe; resolveremos a bifurcação daqui a pouco.

7.2 Dificuldade

Se os nós pudessem publicar blocos quando quisessem, a rede poderia divergir em listas igualmente legítimas. E, se um bloco demora 30 segundos a propagar-se, o que aconteceria se surgissem blocos a cada 31 segundos? Ou a cada 10? É possível controlar a velocidade a que a rede cria blocos. Se o tempo de geração for muito maior do que o tempo necessário para o bloco anterior atingir a maioria dos nós, a rede tenderá a permanecer unida.

7.2.1 Mineração de um bloco

O resultado de uma função hash criptográfica parece uniformemente distribuído e independente do argumento de entrada. Isto significa que, perante entradas novas, cada resultado possível parece igualmente provável. Além disso, cada tentativa exige algum tempo de cálculo. Imagine-se uma função hash $\mathcal{H}_i(x)$ que tem como resultado um número de 1 a 100:

$$\mathcal{H}_i(x) \in_R^D \{1, \dots, 100\}$$

3

Para um dado x , $\mathcal{H}_i(x)$ devolve sempre o mesmo número pseudo-aleatório de $\{1, \dots, 100\}$. Imaginemos que demora um minuto a calculá-lo. Seja agora uma mensagem $\mathbf{m}$. O objectivo é encontrar um inteiro n tal que $\mathcal{H}_i(\mathbf{m}, n)$ não exceda o alvo $t = 5$:

$$\mathcal{H}_i(\mathbf{m}, n) \in \{1, \dots, 5\}$$

Como só 1/20 dos resultados cai no alvo, esperamos cerca de 20 tentativas para encontrar um n que funcione: aproximadamente 20 minutos de cálculo. Procurar esse argumento, ou *nonce*, é *mineração*; publicar a mensagem com n é uma *prova de trabalho*. Qualquer pessoa a verifica calculando uma única vez $\mathcal{H}_i(\mathbf{m}, n)$.

Seja uma função hash usada para provas de trabalho:

$$\mathcal{H}_{PoW} \in_R^D \{0, \dots, l\}.$$

Aqui, l é o resultado máximo. Dada uma mensagem $\mathbf{m}$, um argumento n e um alvo t , podemos aproximar o número esperado de tentativas por

$$d = l/t.$$

Chamamos *dificuldade* a d .

Se $\mathcal{H}_{PoW}(\mathbf{m}, n)d \leq l$, então $\mathcal{H}_{PoW}(\mathbf{m}, n) \leq t$ e n é aceitável.⁴ Alvos menores aumentam a dificuldade e o número esperado de tentativas. Verificar continua a exigir uma avaliação da função de prova de trabalho, independentemente de quão afortunado foi o mineiro.

src/crypto-
note_basic/
difficulty.cpp
check_hash()

7.2.2 Velocidade de mineração

Assuma-se que todos os nós estão a minerar ao mesmo tempo, mas param quando recebem um novo bloco. Começam imediatamente outro que referencia o último aceite.

³ Usa-se $\in_R^D$ para dizer que o resultado se comporta como uma escolha uniforme no conjunto indicado.

⁴ Monero guarda e calcula a dificuldade; testa directamente o produto, sem precisar de materializar t .

Recolhamos b blocos recentes, indexados por $u \in \{1, \dots, b\}$, cada um com dificuldade d_u . Por enquanto, admitamos carimbos temporais honestos. O intervalo entre o mais antigo e o mais recente é

$$\text{Tempo total} = TS_b - TS_1.$$

O trabalho esperado para os blocos no intervalo é ⁵

$$\text{Dificuldade total} = \sum_{u=1}^b d_u.$$

Se a potência da rede não mudou muito durante a amostra, podemos aproximá-la por ⁶

$$\text{velocidade de hash} \approx \text{dificuldade total} / \text{Tempo total}$$

Para obter, em média, um bloco por tempo alvo, escolhe-se

$$\text{nova dificuldade} = \text{velocidade de hash} \cdot \text{Tempo alvo}.$$

Arredonda-se para cima.

Nada garante que o próximo bloco exija exactamente esse número de tentativas. Ao longo de muitos blocos e reajustamentos, porém, a dificuldade acompanha a potência real e os blocos tendem a surgir no intervalo alvo. ⁷

7.2.3 Consenso: maior dificuldade cumulativa

Agora é possível resolver conflitos entre garfos da rede.

Cada bloco tem uma dificuldade, e cada ramo tem a soma das dificuldades dos seus blocos: a *dificuldade cumulativa*. Esta soma é metadado calculado e guardado pelo nó; não é um campo serializado no bloco. A regra de consenso activa escolhe o ramo com a maior dificuldade cumulativa, isto é, aquele que representa mais trabalho. Um ramo alternativo só substitui o principal quando a sua soma é *estritamente* maior. Se houver igualdade, o nó conserva o ramo principal que já adoptara; não há uma segunda regra que prefira o ramo com mais blocos.

Uma alteração das regras de consenso produz uma bifurcação rígida (*hard fork*): software que aplica regras incompatíveis deixa de aceitar os mesmos blocos. Na rede principal, a história de versões chegou a v16:

⁵ O carimbo é escolhido pelo mineiro e não prova o instante exacto em que começou ou terminou. Esta hipótese serve apenas para chegar à intuição; o algoritmo activo apara carimbos extremos.

⁶ Dois mineiros podem experimentar o mesmo *nonce*, mas as suas transacções de mineiro contêm normalmente chaves públicas distintas; os objectos de hash são portanto diferentes, excepto com probabilidade negligenciável. Não estão, na realidade, a repetir a mesma tentativa.

⁷ Se a potência de hash subir continuamente, as dificuldades calculadas com trabalho passado ficarão ligeiramente atrasadas e o intervalo médio poderá ser um pouco inferior ao alvo. A emissão real também depende das penalizações de peso exploradas na secção 7.3.3.

```
src/crypto-
note_core/
block-
chain.cpp
handle_
alter-
native_
block()
```

versão	altura de activação	mudança principal
v1	0	génese, 18 de Abril de 2014
v2	1009827	anéis ≥ 3 , alvo de 120 s, zona de 60 kB
v3	1141317	decomposição das saídas da transacção de mineiro
v4	1220516	transacções RingCT permitidas
v5	1288616	zona mínima de 300 kB e novas taxas
v6	1400000	RingCT obrigatório e anéis ≥ 5
v7	1546000	variante de CryptoNight, anéis ≥ 7
v8	1685555	Bulletproofs permitidas, anel fixo de 11
v9	1686275	Bulletproofs obrigatórias
v10	1788000	CryptoNight-R e peso de longo termo
v11	1788720	formato RingCT antigo proibido
v12	1978433	RandomX e novas regras da transacção de mineiro
v13	2210000	CLSAG permitido e recompensa exacta
v14	2210720	MLSAG antigo proibido
v15	2688888	anel 16; Bulletproof+ e etiquetas de vista permitidas
v16	2689608	Bulletproof+ e etiquetas de vista obrigatórias

As datas de activação foram, pela mesma ordem: v2 em 22 de Março de 2016; v3 em 21 de Setembro de 2016; v4 em 5 de Janeiro de 2017; v5 em 15 de Abril de 2017; v6 em 16 de Setembro de 2017; v7 em 6 de Abril de 2018; v8 e v9 em 18 e 19 de Outubro de 2018; v10 e v11 em 9 e 10 de Março de 2019; v12 em 30 de Novembro de 2019; v13 e v14 em 17 e 18 de Outubro de 2020; e v15 e v16 em 13 e 14 de Agosto de 2022.

```
src/hardforks/
hardforks.cpp
mainnet_hard_
forks[]
```

Esta tabela é histórica até v16. O conjunto activo acumula as mudanças até à última linha, excepto quando uma regra posterior substituiu outra mais antiga. O campo de versão menor conserva a função de voto do mecanismo de bifurcação; em v16 tem de ser pelo menos igual à versão activa.

Para que um atacante convença nós honestos a alterar a história de transacções, tem de construir um ramo válido cuja dificuldade cumulativa ultrapasse a do ramo principal, enquanto este continua a crescer. Com menos de metade da potência, a sorte ainda pode permitir uma reorganização curta, mas a probabilidade de recuperar uma distância crescente decai rapidamente. Manter uma vantagem em regime estável exige uma fracção maioritária da potência de mineração [89].

7.2.4 Minerar em Monero

Para garantir que os garfos da lista de blocos não geram conflitos com o cálculo da dificuldade, não se usam os blocos mais recentes para este cálculo. Por exemplo, se existem 29 blocos na lista, o conjunto tem $b = 10$ e o atraso é $l = 5$, usam-se os blocos 15–24 para calcular a dificuldade do bloco n.º 30.

Mineiros desonestos podem manipular carimbos temporais. Para limitar esse efeito, ordenam-se apenas os carimbos e cortam-se os valores de cada extremo. Fica uma janela efectiva $w = b - 2o$. No exemplo, com $o = 3$, conservaríamos os quatro carimbos centrais.

As dificuldades cumulativas mantêm a ordem do ramo. É uma assimetria deliberada: os extremos temporais são aparados sem reordenar o trabalho.

Usando índices i e j nos extremos da janela conservada, define-se

$$\begin{aligned} \text{Tempo total} &= TS_j - TS_i, \\ \text{Dificuldade total} &= D_j^{\text{cum}} - D_i^{\text{cum}}, \\ d_{\text{seguinte}} &= \left\lceil \frac{\text{Dificuldade total} \cdot 120}{\text{Tempo total}} \right\rceil. \end{aligned}$$

Na regra activa de Monero, o alvo é 120 segundos, $l = 15$, $b = 720$ e $o = 60$. O nó recolhe até $b + l = 735$ blocos anteriores, ignora os 15 mais recentes quando a janela está cheia e mede o intervalo entre 600 carimbos centrais. Como vimos, não reordena com eles o trabalho cumulativo. Este continua a ser o algoritmo de dificuldade activo em v16; não é uma descrição histórica. Tanto a dificuldade como a sua soma cumulativa são inteiros sem sinal de 128 bits; o cálculo intermédio usa 256 bits e rejeita um resultado que não caiba novamente em 128. ^{8 9}

As dificuldades não são serializadas nos blocos. Quem verifica a lista tem de as recalculas a partir dos carimbos temporais e das dificuldades cumulativas já verificadas. Antes de haver 735 blocos disponíveis, a função reduz gradualmente o corte; o bloco de génese não entra na amostra e a dificuldade mínima inicial é 1. A formulação pormenorizada que se segue é histórica apenas quanto à fase inicial da rede, não quanto ao algoritmo:

Regra n.º 1: o bloco de génese é ignorado; com uma amostra de zero ou um elemento, $d = 1$.

Regra n.º 2: até 600 elementos, usa-se toda a amostra.

Regra n.º 3: entre 601 e 720, o corte cresce simetricamente; se o excedente for ímpar, corta-se mais um valor no extremo inferior.

Regra n.º 4: com mais de 720 elementos, conservam-se os primeiros 720, produzindo o atraso de 15 blocos quando há 735 disponíveis.

Prova de trabalho em Monero

Monero usou historicamente CryptoNight, uma função concebida para ser relativamente ineficiente em GPU, FPGA e ASIC quando comparada com SHA-256 [115]. Seguiram-se variantes em v7, CryptoNight V2 em v8 e CryptoNight-R em v10 [26, 10, 11]. A regra activa desde v12 é **RandomX** [63, 14]. Aqui basta saber que produz o hash de prova de trabalho de 256 bits verificado contra a dificuldade; a sua construção fica para o tratamento próprio da etapa seguinte.

⁸ Em Março de 2016 (v2), Monero mudou de um para dois minutos entre blocos [13].

⁹ O algoritmo talvez seja subóptimo quando comparado com propostas mais recentes [132]; tem, contudo, resistência útil à mineração egoísta [46].

```
src/crypto-
note_core/
block-
chain.cpp
get_diff-
iculty_for_
next_
block()
```

```
src/crypto-
note_basic/
difficulty.cpp
next_
difficulty()
```

7.3 Quantidade monetária

Há dois mecanismos básicos para criar dinheiro numa criptomoeda baseada numa lista de blocos. Primeiro, os criadores podem conjurar moedas na génese e distribuí-las num *airdrop*, ou reservar para si um grande montante numa pré-mineração [19]. Segundo, a moeda pode ser distribuída automaticamente como recompensa por minerar blocos. No modelo de Bitcoin, a quantidade é limitada e as recompensas acabam por chegar a zero.

Monero deriva de Bytecoin, que teve uma pré-mineração substancial seguida de recompensas de bloco [12]. Monero não teve pré-mineração. A recompensa base declinou até 0,6 XMR e entrou depois numa cauda constante. A quantidade absoluta cresce indefinidamente, mas a inflação percentual tende para zero; a subvenção continua, entretanto, a incentivar os mineiros.

7.3.1 Recompensa de bloco

Antes de procurar o argumento, o mineiro constrói uma transacção de mineiro. Ela tem uma única entrada especial, que declara a altura, e pelo menos uma saída quando há recompensa a reclamar.

¹⁰

Na regra de consenso activa, a soma dos montantes claros das saídas é *exactamente* a subvenção do bloco, depois da eventual penalização de peso, mais todas as taxas das transacções normais. Cada nó verifica essa igualdade. Assim, a emissão monetária de base e as taxas são directamente auditáveis, apesar de os montantes das transacções normais serem ocultos. Historicamente, v2–v12 permitiam ao mineiro reclamar menos do que o máximo; v13 tornou a igualdade obrigatória.

Para além de distribuir dinheiro, a recompensa incentiva a mineração. Sem recompensas nem taxas, por que haveria alguém de minerar? Talvez por altruísmo ou curiosidade. Com pouca potência honesta, porém, torna-se mais barato reunir uma maioria e tentar reescrever blocos recentes.

¹¹ É também por isso que a recompensa base de Monero não chega a zero. A competição leva os mineiros a acrescentar potência até o custo marginal superar o rendimento marginal esperado. Se o valor protegido aumenta, há espaço económico para mais potência e torna-se mais caro dominar a maioria.

Deslocamento de bits

O deslocamento de bits calcula a recompensa base (que a penalização da secção 7.3.3 ainda pode reduzir). Seja $A = 13$, ou $[1101]$ em binário. Deslocar duas posições para a direita, $A \gg 2$, daria

¹⁰ As regras activas não exigem uma única saída. O construtor de modelos do nó limita normalmente a transacção de mineiro a uma saída; isto é comportamento implementado, não consenso. O peso desta transacção conta para o peso total do bloco.

¹¹ Num modelo simplificado de potências constantes, se o atacante tem $v > v_h$, um histórico com x dias demora aproximadamente $y = xv_h/(v - v_h)$ dias a ultrapassar.

$[0011], 01 = 3,25$ se guardássemos a fracção. O computador deixa os bits “cair da mesa” e conserva $[0011] = 3$. Em geral, $A \gg n = \lfloor A/2^n \rfloor$.

Calcular a recompensa base de bloco em Monero

Seja M o contador interno de moeda já emitida e $L = 2^{64} - 1$ o alvo aritmético da emissão principal. L não é um limite à quantidade que pode existir para sempre: depois da emissão principal, o contador é saturado em L e a cauda continua. No início, a recompensa base era

$$B = (L - M) \gg 20.$$

Com $M = 0$, em formato decimal,

$$L = 18\,446\,744\,073\,709\,551\,615$$

$$B_0 = (L - 0) \gg 20 = 17\,592\,186\,044\,415.$$

Estes números estão em unidades atómicas, a menor divisão de Monero. Claramente são números ridículos para levar no bolso: L tem vinte algarismos. Dividir por 10^{12} leva-nos às unidades usuais de XMR:

$$\frac{L}{10^{12}} = 18\,446\,744,073\,709\,551\,615$$

$$B_0 = \frac{(L - 0) \gg 20}{10^{12}} = 17,592\,186\,044\,415.$$

A primeira recompensa, atribuída ao pseudónimo `thankful_for_today` no bloco de génese [118], foi portanto cerca de 17,6 XMR (apêndice C)! Na lista, os montantes continuam em unidades atómicas. À medida que os blocos eram minerados, M aumentava e a recompensa diminuía. Desde a génese até v2, o alvo era um minuto por bloco; em Março de 2016 passou a dois minutos [13]. Para conservar o ritmo de emissão, a recompensa base de bloco foi duplicada.¹² Isto só significa que, depois da mudança, se faz $(L - M) \gg 19$ em vez de $(L - M) \gg 20$. A curva da emissão principal passou a ser:

$$B = \frac{(L - M) \gg 19}{10^{12}}.$$

Esta última expressão já não basta para calcular a regra activa. Em unidades atómicas, a recompensa base sem penalização é

$$B_0 = \max \{ (L - M) \gg 19, 600\,000\,000\,000 \}.$$

O segundo termo é 0,6 XMR. A emissão de cauda já está activa; não é uma promessa para o futuro.

7.3.2 Peso dinâmico de bloco

Seria bom incluir imediatamente cada nova transacção. Mas que aconteceria se alguém submetesse milhões delas por malícia? A lista de blocos tornar-se-ia rapidamente enorme. Um limite fixo por bloco trava o crescimento, mas, quando aumenta o volume honesto, os remetentes competem pelo pouco espaço oferecendo taxas maiores. Os pagamentos pequenos podem tornar-se proibitivos. Monero evita os dois extremos — tamanho rígido ou crescimento sem limite — com peso dinâmico e uma penalização económica.

¹² Para uma comparação entre as emissões de Bitcoin e Monero, veja-se [18].

```
src/crypto-
note_basic/
cryptonote_
basic_
impl.cpp
get_block_
reward()
```

Tamanho vs. peso

Desde que as Bulletproofs foram permitidas (v8), o tamanho serializado e o peso deixaram de ser sempre iguais. O algoritmo atribui a uma transacção sem Bulletproof nem Bulletproof+ exactamente o número de bytes do seu objecto serializado; num bloco v16, este caso abrange a transacção de mineiro `RCTTypeNull`, enquanto uma transacção normal tem de usar Bulletproof+. Para esta última, acrescenta-se uma recuperação (*clawback*) que cobra parte da economia obtida ao agregar muitas saídas.

Se p é a capacidade preenchida pela prova, arredondada a uma potência de dois até ao máximo de 16 saídas, então, para $p > 2$, a prova Bulletproof+ ocupa

$$S_{BP+}(p) = 32(6 + 2(\log_2 p + 6))$$

bytes, e o acréscimo de peso é

$$C(p) = \left\lfloor \frac{4}{5}(320p - S_{BP+}(p)) \right\rfloor.$$

Para $p \leq 2$, $C(p) = 0$. Logo,

$$peso(tx) = bytes(tx) + C(p).$$

O leitor reconhece a pequena malícia pedagógica: a prova cresce logaritmicamente, mas verificá-la não fica grátis. O peso põe parte desse custo de volta na balança. A fórmula mais antiga, com a constante 9 das Bulletproofs originais, pertence a v8–v15; em v15 coexistiu transitoriamente com a constante 6 de Bulletproof+. V16 só permite esta última.

O peso de um bloco é a soma dos pesos da transacção de mineiro e de todas as transacções normais. O cabeçalho e o vector de identificadores não são somados separadamente. Peso é uma grandeza de consenso; tamanho é uma propriedade da serialização.

O peso de bloco a longo termo

Se os blocos dinâmicos pudessem crescer depressa sem memória, a lista de blocos poderia tornar-se rapidamente enorme [74]. A v10 introduziu por isso um peso de longo termo. Depois de aceitar um bloco, o nó calcula o valor que guardará para esse bloco e as medianas que regerão o *bloco seguinte*; estes números são metadados derivados, não campos do bloco.

Chamemos W ao peso normal do bloco acabado de aceitar e M_L à mediana efectiva anterior de longo termo. Na regra activa desde v15,

$$W_L = \min \left\{ \max \left\{ W, \frac{10}{17}M_L \right\}, \frac{17}{10}M_L \right\},$$

$$M_L = \max \{300\,000, \text{mediana}(W_L \text{ dos últimos } 100\,000 \text{ blocos})\}.$$

As divisões são inteiras no código. A versão histórica v10–v14 só limitava por cima, a $1,4M_L$; não se deve usar essa regra para blocos v16. ¹³

São necessários cerca de 50 000 blocos, aproximadamente 69 dias ao alvo de dois minutos, para que uma mudança persistente atravessasse a mediana. Mesmo então, cada nova amostra fica entre $M_L/1,7$ e $1,7M_L$: o crescimento e a contracção de longo prazo ficam ambos amarrados.

¹³ A zona mínima foi 20 kB em v1, 60 kB em v2 e é 300 kB desde v5 [13, 1].

```
src/crypto-
note.basic/
cryptonote_
format_
utils.cpp
get_trans-
action_
weight()
```

```
src/crypto-
note.core/
block-
chain.cpp
get_next_
long_term_
block_
weight()
```

As medianas de curto e longo prazo

O volume de transacções pode mudar dramaticamente num curto período, especialmente à volta das férias [119]. Chamemos M_S à mediana dos pesos normais dos 100 blocos *anteriores*. A mediana efectiva que determina a penalização do bloco seguinte é

$$M_N = \min \{ \max \{ M_L, M_S \}, 50M_L \}.$$

Daí resulta a regra de consenso activa

$$W_{\max} = 2M_N.$$

Um bloco com $W > 2M_N$ é inválido. O limite pode reagir depressa até $100M_L$, mas o ponto de apoio M_L só se move lentamente. A mediana “cumulativa” das edições antigas corresponde aqui a M_N ; chamar-lhe simplesmente mediana de 100 blocos esconde metade do mecanismo. ¹⁴

```
src/crypto-
note_core/
block-
chain.cpp
update_next_
cumulative_
weight_limit()
```

7.3.3 Recompensa de bloco com multa

Para minerar blocos maiores do que M_N , os mineiros pagam uma penalização na forma de uma subvenção reduzida. Existem portanto duas zonas: até M_N , a zona sem penalização; de M_N até $2M_N$, a zona penalizada.

Se o peso W excede M_N , dada a recompensa base B_0 , a penalização idealizada em aritmética real é

$$P = B_0 \left(\frac{W}{M_N} - 1 \right)^2.$$

A regra de consenso faz as multiplicações com precisão alargada e as divisões inteiras. A recompensa de bloco é portanto:

¹⁵

$$B(W) = \left\lfloor B_0 \frac{W(2M_N - W)}{M_N^2} \right\rfloor, \quad M_N < W \leq 2M_N.$$

Para $W \leq M_N$, $B(W) = B_0$. Em $W = 2M_N$, a subvenção é zero; para $W > 2M_N$, o bloco é inválido. As taxas não são penalizadas: somam-se depois a $B(W)$ na transacção de mineiro.

```
src/crypto-
note_basic/
cryptonote_
basic_
impl.cpp
get_block_
reward()
```

O quadrado torna a penalização suave no princípio e feroz perto do limite. Um peso 10% acima de M_N perde aproximadamente 1% da subvenção; 50% acima perde 25%; 90% acima perde 81% [18]. A viagem pedagógica continua a mesma, mas agora sabemos exactamente qual é a mediana à qual esses exemplos se referem.

É de esperar que os mineiros entrem na zona penalizada quando as taxas adicionais compensarem a perda marginal de subvenção. Isto é economia do modelo de bloco, não uma condição de validade.

¹⁴ Antes de v12, a penalização usava variantes da mediana de curto prazo; v12 tornou activa a mediana efectiva. V15 alterou os limites de longo termo para a escala 1,7.

¹⁵ Antes da activação das transacções confidenciais em v4, os montantes eram comunicados em texto claro e decompostos, por exemplo $1244 \rightarrow 1000 + 200 + 40 + 4$. Em v2–v3, o construtor da transacção de mineiro arredondava ainda a recompensa para baixo, retirando os algarismos menos significativos segundo `BASE_REWARD_CLAMP_THRESHOLD`. O resto não desaparecia: ficava disponível para recompensas posteriores.

7.3.4 Taxas dinâmicas: política, não consenso

Actores maliciosos podem inundar a rede com transacções e fazer crescer a lista de blocos. Para desencorajar esse comportamento, os nós aplicam uma taxa mínima por unidade de *peso* ao admitir uma transacção na memória de transacções (*txpool*) e ao propagá-la.

Isto não é uma regra de consenso. Um bloco pode conter uma transacção sem taxa, ou com taxa inferior ao mínimo local, se satisfizer todas as restantes regras. Ao processar transacções vindas de um bloco, o nó ignora o teste de taxa da *txpool*. É por isso que carteiras e nós podem actualizar o algoritmo sem uma bifurcação da rede. Historicamente, a taxa era 0,01 XMR/KiB em v1 e 0,002 XMR/KiB em v3 [72]. V4 introduziu uma taxa dinâmica por KiB; v8 passou a cobrar por byte; v15 activou no software a escala de taxas de 2021. O fio condutor mantém-se: relacionar o preço do espaço com a recompensa e com o espaço que a rede consegue oferecer [42, 41, 40].¹⁶

```
src/crypto-
note_core/
block-
chain.cpp
check_fee()
```

```
src/crypto-
note_core/
tx_pool.cpp
add_tx()
```

¹⁷

Intuição histórica: a penalização marginal

Para não perdermos a ideia que deu origem ao mecanismo, regressemos à transacção de referência de peso 3000 e à zona mínima de 300 000. Uma taxa natural é aquela que compensa a penalização marginal causada ao acrescentar a transacção [41, 66]. Esta é a derivação histórica do algoritmo anterior a v15; já a seguir chegaremos à fórmula que o nó v16 executa.

Primeiro, a taxa T que iguala a penalização marginal de acrescentar uma transacção de peso P_T a um bloco de peso P_B é

$$T = B_0 \left[\left(\frac{P_B + P_T}{M_N} - 1 \right)^2 - \left(\frac{P_B}{M_N} - 1 \right)^2 \right].$$

Define-se o factor de peso do bloco:

$$F_b = \frac{P_B}{M_N} - 1$$

e o factor de peso de uma transacção:

$$F_t = \frac{P_T}{M_N}$$

De forma simples:

$$T = B_0(2F_b F_t + F_t^2).$$

No ponto de referência, $P_B = M_N = 300\,000$ e $P_T = 3000$. Logo $F_b = 0$ e

$$\begin{aligned} T_{\text{ref}} &= B_0(2 \cdot 0 \cdot F_t + F_t^2) \\ &= B_0 \left(\frac{3000}{300\,000} \right)^2. \end{aligned}$$

¹⁶ Os conceitos apresentados nesta secção devem muito a Francisco Cabañas, também conhecido por “ArticMine”, arquitecto dos sistemas de bloco e de taxa dinâmicos [42, 41, 40].

¹⁷ A unidade KiB (kibibyte, 1 KiB = 1024 bytes) é diferente de kB (kilobyte, 1 kB = 1000 bytes).

Esses 3000 representam 1% da zona. Para uma mediana geral M , escolhemos uma transacção de referência P_T^* que ocupe a mesma fracção:

$$\frac{P_T^*}{M} = \frac{3000}{300\,000}.$$

Escalamos depois linearmente para uma transacção real de peso P_T :

$$\begin{aligned} T_{\text{hist}} &= B_0 \left(\frac{P_T^*}{M} \right) \left(\frac{3000}{300\,000} \right) \frac{P_T}{P_T^*} \\ &= B_0 \frac{P_T}{M} \frac{3000}{300\,000}. \end{aligned}$$

Dividindo pelo peso, obtemos a antiga taxa padrão por unidade de peso:

$$\begin{aligned} f_{\text{standard}}^{\text{hist}} &= \frac{B_0}{M} \frac{3000}{300\,000}, \\ f_{\text{min}}^{\text{hist}} &= \frac{1}{5} f_{\text{standard}}^{\text{hist}} = \frac{0,002B_0}{M}. \end{aligned}$$

O último factor $1/5$ reflectia a ideia de cobrar menos abaixo da mediana [66]. É uma bela intuição marginal, mas a potência de M mudou na política activa.

A regra activa da txpool

Acontece que usar M_N directamente para a taxa permitiria um ataque de *spam*: elevar M_S reduziria o preço precisamente quando a rede estivesse mais ocupada. Por isso o nó usa a menor entre a mediana de penalização e a mediana efectiva de longo termo. Em v16, como $M_N \geq M_L$, a mediana de taxa é simplesmente M_L . Se B_0 é a recompensa base sem penalização e w o peso da transacção, o preço local por unidade de peso é calculado, com aritmética inteira, em torno de

$$f_{\text{pool}} \simeq \max \left\{ 1, 0,95 \frac{B_0 \cdot 3000}{M_L^2} \right\} \text{ unidades atómicas por unidade de peso.}$$

O mínimo pedido é wf_{pool} , arredondado para cima ao múltiplo seguinte de 10 000 unidades atómicas (oito casas decimais de XMR). Para absorver diferenças de arredondamento, a txpool aceita até 2% abaixo desse total.

```
src/crypto-
note_core/
block-
chain.cpp
get_dynamic_
base_fee()
```

18

$$M_{\text{taxa}} = \min\{M_N, M_L\} = M_L.$$

O quadrado no denominador é a ponte para a derivação anterior: se a capacidade de longo termo duplica, o custo por unidade de peso cai quatro vezes. Se a recompensa base diminui, o preço cai com ela. Assim a política não se desliga da economia que limita o crescimento dos blocos.

```
src/crypto-
note_core/
block-
chain.cpp
check_fee()
```

A taxa da txpool é portanto uma porta de entrada local, não um imposto inscrito no bloco. Um operador pode modificar a sua política; não pode, por isso, tornar inválido para os outros nós um bloco que obedece ao consenso.

¹⁸ A fórmula anterior desta secção, proporcional a $B_0/M \cdot 0,002$, e os exemplos com recompensa de 2 XMR descreviam a política anterior a v15. A regra activa depende de B_0/M_L^2 e a cauda fixa B_0 em pelo menos 0,6 XMR.

Carteira e modelo de bloco

Como Cabañas disse na sua apresentação perspicaz sobre este tópico [41], “as taxas dizem ao mineiro” até onde os remetentes estão dispostos a comprar espaço na zona penalizada. O construtor de modelos do nó ordena a txpool por taxa por unidade de peso, da maior para a menor (e pela antiguidade em caso de igualdade). Reserva 600 unidades de peso para a transacção de mineiro e experimenta acrescentar cada transacção válida e pronta, mas rejeita-a se ultrapassar o espaço restante até $2M_N$ ou se a nova soma

$$B(W_{\text{novo}}) + \sum \text{taxas}$$

ficar abaixo do melhor rendimento já obtido. Esta selecção é comportamento do mineiro; o consenso verifica apenas o bloco final, incluindo o peso real da transacção de mineiro. ¹⁹

A carteira consulta ao nó quatro estimativas por unidade de peso: `unimportant`, `normal`, `elevated` e `priority`. Para as calcular, o nó projecta dez blocos de folga, obtém medianas projectadas e devolve quatro patamares da escala de 2021, arredondados para cima a dois algarismos significativos. Os antigos multiplicadores 1, 5, 25 e 1000 continuam em ramos históricos de compatibilidade do código, mas não descrevem a política activa v16. A carteira multiplica o patamar escolhido pelo peso estimado e volta a verificar o peso real ao construir a transacção.

```
src/crypto-
note_core/
tx_pool.cpp
fill_block_
template()
```

Uma consequência importante dos pesos dinâmicos é o laço de realimentação: transacções que competem com taxas maiores comprem mais espaço, mas esse espaço faz crescer lentamente M_L e reduz a taxa por unidade de peso. O mecanismo liga a oferta de espaço, a subvenção e o preço de propagação, sem fingir que a política da carteira é consenso. Este laço de realimentação é uma boa defesa contra a ameaça do “mineiro egoísta” [54].

```
src/crypto-
note_core/
block-
chain.cpp
get_dynamic_
base_fee_
estimate_2021_
scaling()
```

7.3.5 Cauda de emissão

Suponha-se uma criptomoeda com quantidade monetária fixa e peso de bloco dinâmico. Depois de algum tempo, as recompensas declinam para zero. Sem uma subvenção para penalizar o crescimento, os mineiros tenderiam a acrescentar qualquer transacção com taxa positiva.

O peso dos blocos estabilizaria à volta do volume submetido à rede e os remetentes procurariam a taxa mínima. Na nossa fórmula de política, uma recompensa que chegasse a zero arrastaria a taxa calculada para o piso de uma unidade atómica por unidade de peso.

Isto introduziria uma situação instável: com pouco incentivo para minerar, a potência de hash poderia cair. A dificuldade manteria aproximadamente o tempo entre blocos, mas o custo de atacar a rede diminuiria. Taxas por si só também podem agravar incentivos de mineração egoísta [43, 54]. ²⁰

¹⁹ A intuição marginal permanece: uma transacção entra na zona penalizada quando a sua taxa compensa a subvenção que o mineiro perde ao acrescentar-lhe o peso. Não há, porém, uma regra de consenso que obrigue o mineiro a maximizar esse rendimento.

²⁰ O caso de uma quantidade monetária limitada e um tamanho de bloco fixo, como em Bitcoin, também é considerado como instável [43].

Monero evita esse precipício: a recompensa base nunca desce abaixo de 0,6 XMR por bloco, isto é, 0,3 XMR por minuto ao alvo de dois minutos. A curva principal encontrou esse piso quando

$$0,6 > ((L - M) >> 19)/10^{12}$$

$$M > L - 0,6 \cdot 2^{19} \cdot 10^{12}$$

$$M/10^{12} > L/10^{12} - 0,6 \cdot 2^{19}$$

$$M/10^{12} > 18\,132\,171,273709551615$$

A lista de blocos de Monero entrou então na chamada *cauda de emissão*, em Maio de 2022 [16]. A regra activa conserva a recompensa base de 0,6 XMR para cada novo bloco. A emissão absoluta prossegue; a inflação percentual decresce à medida que a quantidade já emitida aumenta. ²¹

```
src/crypto-
note.basic/
cryptonote.
basic_
impl.cpp
get_block_
reward()
```

7.3.6 Transacção de mineiro: RCTTypeNull

O mineiro reclama a subvenção e as taxas através da transacção de mineiro (também chamada *coinbase transaction*). Parece uma transacção normal à distância, mas as regras de consenso activas são muito específicas:

- tem versão 2 e exactamente uma entrada `txin_gen`, cuja altura é a altura do bloco;
- usa `RCTTypeNull`: não contém CLSAG, Bulletproof+ nem pseudo-saídas;
- o tempo de desbloqueio é exactamente $h + 60$;
- em v16 todas as saídas são `txout_to_tagged_key`, com etiqueta de vista, chave pública válida e montante claro;
- a soma dos montantes é exactamente $B(W) + \sum \text{taxas}$.

```
src/crypto-
note.core/
cryptonote_
tx_utils.cpp
construct_
miner_tx()
```

²²

O construtor do nó cria normalmente uma única saída para o endereço do mineiro, inclui a chave pública da transacção no campo `extra` e pode incluir aí um argumento adicional fornecido pela reserva de mineração. Esta é uma escolha de modelo; o consenso permite várias saídas, desde que todas as condições acima e o peso máximo do bloco sejam satisfeitos.

²³

Desde que RingCT foi implementado em Janeiro de 2017 (v4) [15], o nó que guarda uma saída de mineiro v2 fabrica também o compromisso de máscara nula

$$C = 0G + aH = aH.$$

²¹ L continua a ser o maior contador de emissão representável em 64 bits; a implementação satura o contador em L durante a cauda para evitar um subfluxo em $L - M$. Não se deve interpretar isso como o fim da emissão. Cada montante individual continua limitado à gama de 64 bits demonstrada pelas provas de intervalo.

²² V12 tornou histórica a possibilidade de usar uma transacção de mineiro v1 ou de incluir assinaturas RingCT: passou a exigir versão 2 e `RCTTypeNull` [7]. V13 tornou exacta a soma reclamada.

²³ Se esta transacção aparece no bloco 10, a altura de desbloqueio é 70 e a saída pode ser usada numa transacção incluída no bloco 70 ou posterior. Esta maturidade de 60 blocos é distinta da idade mínima normal de 10 blocos para uma saída ser usada como membro real de uma entrada, activa desde v12 [20].

```
src/block-
chain.db/
blockchain.
db.cpp
add_trans-
action()
```

```
src/crypto-
note_core/
block-
chain.cpp
is_tx_
spendtime_
unlocked()
```

Esse compromisso não está escondido na transacção de mineiro; é metadado derivado do montante claro. Permite, contudo, que a saída apareça mais tarde num anel CLSAG juntamente com saídas RingCT normais. A antiga expressão $G + aH$ estava errada: “máscara identidade” no comentário do código significa o escalar zero, não o escalar um.

7.4 A estrutura da lista de blocos

O estilo da lista de blocos de Monero é simples.

Começa com o bloco de génese, neste caso essencialmente uma transacção de mineiro que dispersa a primeira recompensa (apêndice C). Cada bloco seguinte contém no cabeçalho o ID do bloco anterior.

Para calcular o ID, não se aplica a função hash à serialização completa do bloco. Primeiro constrói-se o *objecto de hash*: a serialização binária do cabeçalho, seguida pela raiz de Merkle dos IDs de transacção e pelo número total de transacções, incluindo a de mineiro. Depois aplica-se a função hash rápida de CryptoNote (Keccak):

$$ID_B = \mathcal{H}_n(\text{serializar}(\text{cabeçalho}) \parallel R_{\text{Merkle}} \parallel \text{varint}(1 + \#\text{txs normais})).$$

24

Para produzir um novo bloco, o mineiro altera o *nonce* de 4 bytes e, quando precisa de mais domínio, altera também a reserva no **extra** da transacção de mineiro. O mesmo objecto de hash alimenta a prova de trabalho, mas não a mesma função: em blocos v16 usa-se RandomX para obter h_{PoW} ; para o ID usa-se $\mathcal{H}_n$. Interpretando h_{PoW} como inteiro de 256 bits em ordem *little-endian*, o bloco é válido quando

$$h_{\text{PoW}} \cdot d \leq 2^{256} - 1.$$

Enquanto o produto for maior, continua-se a procurar. A verificação faz uma avaliação do hash de prova de trabalho e uma multiplicação de precisão suficiente; a mineração faz tantas tentativas quantas a sorte exigir.

7.4.1 ID de transacção

As transacções activas têm versão 2. O seu ID não é simplesmente a hash de um único objecto monolítico: separa deliberadamente os dados que um nó podado tem de conservar daqueles que pode apagar.

Calculam-se três hashes de 32 bytes:

- h_p : a hash do prefixo serializado: versão, desbloqueio, entradas (incluindo *offsets* e imagens de chave), saídas (chaves e etiquetas de vista) e **extra**;

²⁴ Há uma excepção histórica de consenso para o bloco 202612, cujo ID é tratado especialmente. Não altera a construção dos blocos v16.

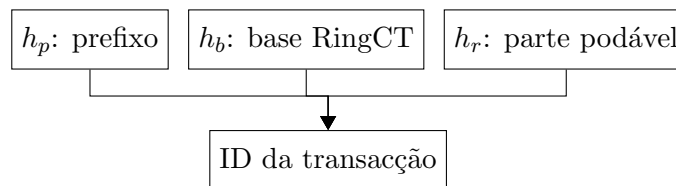
```
src/crypto-
note_core/
cryptonote-
tx_utils.cpp
generate_
genesis_
src/cryp-
to-
note_basic/
cryptonote-
format_
utils.cpp
get_block_
hashing_
blob()
src/crypto-
note_basic/
cryptonote-
format_
utils.cpp
calculate_
block_
hash()
```

- h_b : a hash da parte RingCT de base, não podável: tipo (`RCTTypeBulletproofPlus` nas transacções normais activas, ou `RCTTypeNull` na de mineiro), taxa, compromissos de saída e informação de montante;
- h_r : a hash da parte RingCT podável, que nas transacções activas normais inclui Bulletproof+, CLSAG e pseudo-saídas. Para `RCTTypeNull`, h_r é a hash nula de 32 bytes.

Finalmente,

$$ID_{tx} = \mathcal{H}_n(h_p \| h_b \| h_r).$$

Neste diagrama de árvore a seta preta indica uma hash de entradas.



```
src/crypto-
note.basic/
cryptonote_
format_
utils.cpp
calculate_
transa-
ction_
hash()
```

Em vez de entradas que gastam saídas anteriores, a transacção de mineiro contém a altura do bloco actual. Assim, mesmo que o endereço e o montante se repetissem, a altura mudaria o prefixo e tornaria o ID distinto. Como `RCTTypeNull` não tem parte podável, usa-se $h_r = 0^{256}$; não é a hash de uma lista imaginária de assinaturas vazias.

7.4.2 Árvore de Merkle

Uma raiz de Merkle [83] organiza os IDs das transacções do bloco numa árvore binária de hashes. Ela compromete o mineiro com a lista e com a ordem exactas dos IDs sem colocar a raiz como campo separado no bloco: qualquer alteração de uma transacção muda o seu ID, sobe pela árvore e muda o ID do bloco.

Monero usa a árvore CryptoNote, não a regra de Bitcoin de duplicar sempre a última folha. Para um número que não seja potência de dois, algumas folhas entram directamente no nível intermédio e as restantes são emparelhadas. A primeira folha é sempre o ID da transacção de mineiro; seguem-se, pela ordem serializada no bloco, os IDs das transacções normais.

Um exemplo com uma transacção de mineiro e quatro transacções normais aparece na Figura 7.1.

```
src/crypto/
tree-hash.c
tree_hash()
```

²⁵ Um erro no código-fonte da raiz de Merkle deu origem a um ataque sério, embora aparentemente não crítico, em 4 de Setembro de 2014 [77].

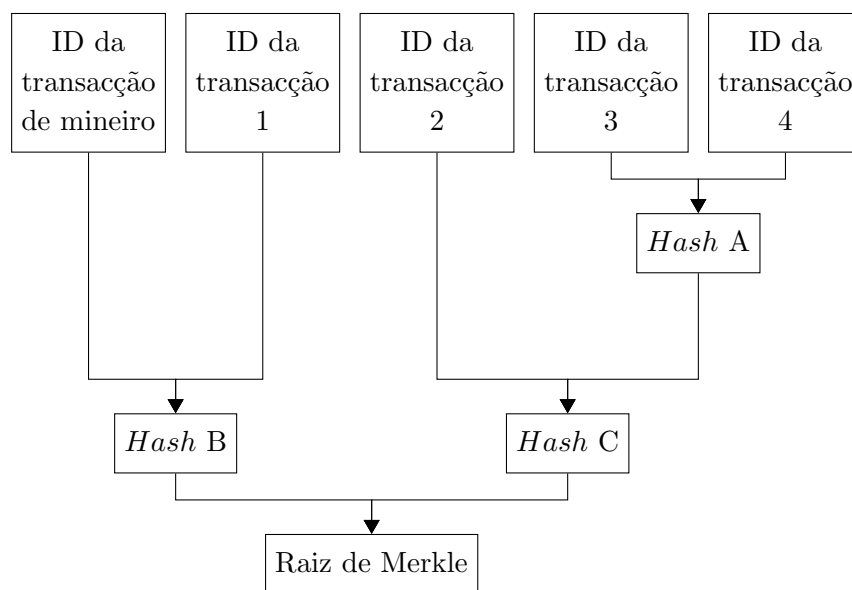


Figura 7.1: árvore de Merkle

Uma raiz de Merkle é, pois, uma referência compacta a todas as transações incluídas. Mas há um pormenor: a poda real de Monero não corta ramos desta árvore. É a decomposição $h_p || h_b || h_r$ do ID da transacção que permite retirar dados e ainda reconstruir o mesmo ID.

Poda da base de dados

A poda é comportamento local do nó, não regra de consenso. Ao podar uma base de dados que construiu normalmente, o nó verificou primeiro os blocos e as transações completos. Depois conserva sempre o prefixo, a parte RingCT de base, h_r e os índices necessários à validação; pode eliminar a parte RingCT podável de transações v2 antigas. Transações v1 não são podadas por este mecanismo.

Desde a versão 0.14.1 do programa, os blocos antigos são repartidos em oito faixas determinísticas, cada uma formada por troços de 4096 blocos. Um nó podado conserva os dados podáveis completos da sua faixa e apaga-os nas outras sete; conserva ainda, por inteiro, a ponta mais recente de 5500 blocos [50].

Escolher faixas diferentes ajuda a rede a distribuir os dados, mas não há uma garantia de consenso de que “um em cada oito nós” guarde cada transacção. Um nó sem poda conserva tudo; um nó podado deixa de poder servir aos pares os dados antigos que apagou. Por omissão, a sincronização continua a pedir dados completos. A opção explícita `--sync-pruned-blocks` permite, apenas na zona coberta pelos hashes de blocos compilados no programa, receber de outro nó os dados já podados; nesse modo não é possível reverificar localmente as provas e assinaturas ausentes. Esta é uma troca de sincronização do nó, não uma regra nova para os blocos.

```
src/crypto-
note-protocol/
cryptonote_
protocol_
handler.inl
should_ask_for_
pruned_data()
```

```
src/common/
pruning.cpp
has_unpruned_
block()
```

7.4.3 Blocos

Um bloco serializado é um cabeçalho, a transacção de mineiro completa e um vector de IDs de transacções normais. Os corpos destas últimas são obtidos da txpool ou fornecidos separadamente pelo par; não aparecem repetidos dentro do objecto `block`. Os nossos leitores podem encontrar um exemplo real no apêndice B.

- cabeçalho de bloco:
 - **versão maior**: versão das regras que construíram o bloco; na rede principal actual é 16;
 - **versão menor**: voto do mecanismo de bifurcação; a regra activa exige um valor pelo menos 16;
 - **carimbo temporal**: inteiro UTC escolhido pelo mineiro;
 - **ID do bloco anterior**: 32 bytes que encadeiam a lista;
 - **nonce**: inteiro sem sinal de 4 bytes usado na prova de trabalho.
- transacção de mineiro: objecto completo que cria a subvenção e reclama as taxas;
- IDs de transacção: vector ordenado de referências de 32 bytes às transacções normais.

Na serialização binária, as três primeiras grandezas do cabeçalho usam inteiros variáveis; o ID e o *nonce* têm tamanho fixo:

- versão maior e versão menor: inteiros de 8 bits serializados como *varints*, normalmente um byte cada;
- carimbo temporal: inteiro de 64 bits serializado como *varint*;
- ID do bloco anterior: 32 bytes;
- *nonce*: 4 bytes. O domínio pode ser estendido alterando a reserva do campo **extra** da transacção de mineiro [73];
- transacção de mineiro completa, com prefixo e base RingCT nula. A base de dados deriva ainda, para cada saída v2, o compromisso $C = aH$;
- contagem do vector como *varint* e IDs de transacção de 32 bytes cada. O desserializador impõe ainda um limite defensivo de 2^{28} IDs, muito acima do que o limite de peso permitiria.

O que um nó valida

Receber bytes bem formados ainda não é receber um bloco. Para o acrescentar ao ramo principal, um nó v16 aplica condições de consenso e uma condição de admissão dependente do seu relógio:

- que o ID anterior é a ponta actual e que as versões maior e menor satisfazem a programação da rede principal;

- como consenso, havendo 60 blocos anteriores, que o carimbo temporal não é menor do que a mediana dos seus carimbos; como admissão local, que não está mais de duas horas no futuro segundo o relógio do nó;
- que a dificuldade seguinte foi recalculada correctamente e que o hash RandomX satisfaz $h_{\text{PoW}}d \leq 2^{256} - 1$;
- que cada corpo de transacção referido produz o ID anunciado, tem no máximo 1 000 000 bytes, satisfaz as regras v16 de entradas, saídas, imagens de chave, CLSAG e Bulletproof+, e não duplica gastos;
- que a raiz de Merkle, o número de transacções e o ID do bloco se recompõem; que o peso total não excede $2M_N$; e que a transacção de mineiro satisfaz altura, maturidade, tipo de saída e recompensa exacta.

Se o ID anterior não é a ponta, o bloco pode iniciar ou prolongar um ramo alternativo. O nó valida-o pelas regras da altura desse ramo e só reorganiza a lista quando a dificuldade cumulativa alternativa se torna estritamente maior. A txpool, a escolha das taxas e a poda da base de dados não mudam nenhum destes testes de consenso.

Parte II

Extensões

Provas de conhecimento de transacções

Monero é uma cripto-moeda, e como qualquer moeda os seus usos são complexos. Desde contabilidade empresarial, á presença na bolsa e arbitragem legal, diferentes entidades estão interessadas nas transacções efectuadas. Como é possível determinar que dinheiro recebido veio de uma pessoa específica? Ou como se prova que uma certa transacção foi efectuada para alguém, apesar de acusações contrárias? Remetentes e destinatários no registro público de Monero são ambiguos. Visto que os montantes em Monero estão escondidos do público, como é possível provar a posse de um montante, sem que as chaves privadas sejam comprometidas?

São considerados diferentes tipos de asserções sobre transacções, algumas das quais estão implementadas em Monero e disponíveis com ferramentas da carteira. Apresentam-se também estruturas para auditar o saldo de uma pessoa ou organização, o que não requer vaziar informação sobre transacções futuras.

8.1 Provas de transacção em Monero

As provas de transacção em Monero estão no processo de serem renovadas [94]. As provas actualmente implementadas são todas ‘versão 1’, e não incluem a separação de domínios. Aqui só irão ser descritas as provas mais avançadas, porque estão implementadas, poderão ser implementadas [95], ou provas hipotéticas que servem só de teoria (secções 8.1.5 [121], e 8.2.1).

8.1.1 Provas de transacção com múltiplas bases

Existem alguns detalhes a ter em conta ao prosseguir. A maioria das provas de transacção em Monero envolvem provas com múltiplas bases (secção 3.1). Sempre que relevante o separador de domínio é :

$$T_{txprf2} = \mathcal{H}_n(\text{"TXPROOF_V2"}).$$

A mensagem que é assinada é usualmente (salvo indicação em contrário) :

$$\mathbf{m} = \mathcal{H}_n(\mathbf{tx_hash}, \mathbf{message}).$$

Em que `tx_hash` é a ID de transacção relevante () e `message` é uma mensagem opcional que os provedores ou entidades terceiras podem fornecer para dar a certeza que a prova foi de facto feita e não roubada ¹ .

As provas estão codificadas em base-58, um esquema de código de binário para texto, primeiro introduzido para Bitcoin [25]. Verificar estas provas envolve sempre primeiro decodificar da base-58 de volta a binário. Note-se que os verificadores também precisam de ter acesso á lista de blocos, para que através da ID de transacção possam extrair informação como endereços ocultos. ²

A estrutura de prefixo de chave em provas está algo torta, em parte por causa de desenvolvimentos acumulados que ainda não reorganizaram isto. Desafios para provas, base-2 ‘versão 2’, são construídas com este formato, em que se ‘chave 1 base’ é G então a sua posição no desafio é preenchida com 32 bytes de zero.

$$c = \mathcal{H}_n(\mathbf{m}, \text{chave pública 2, prova parte 1, prova parte 2, } T_{txprf2}, \\ \text{chave pública 1, chave base 2, chave base 1}).$$

8.1.2 Provar a criação de uma entrada de transacção (SpendProofV1)

Seja que um remetente fez uma transacção, e agora precisa de o provar. Claramente, ao refazer a assinatura á entrada de transacção numa outra mensagem, qualquer verificador teria de concluir que o remetente também fez a assinatura original. Ao refazer *todas* as assinaturas para cada entrada de uma transacção significa que o remetente tem de ter feito a transacção inteira.

Alguém que fez uma assinatura a uma entrada, não implica que tenha assinado todas as entradas (capítulo 11).

3

Uma assim chamada ‘SpendProof’ (do inglês : *prova de gasto*) contém novas assinaturas para cada entrada de transacção. Importante é que a prova de gasto re-utiliza os membros de anel originais para evitar a identificação do verdadeiro signatário através da intersecção de aneis.

```
src/wallet/
wallet2.cpp
get_spend_
proof()
```

¹ Bem como na secção 3.6, funções hash deviam ter um separador de domínio, através de um prefixo com tags. As provas de transacção actualmente implementadas em Monero não têm nenhuma separação de domínio. Portanto todos os tags neste capítulo são características ainda *não* implementadas.

² Os ID’s de transacção são usualmente comunicados separadamente das provas.

³ Como veremos no capítulo 11, alguém ter assinado uma entrada não implica que tenha assinado todas as entradas.

Provas de gasto são implementadas em Monero, o provador concatena o prefixo “SpendProofV1” com a lista de assinaturas. Note-se que este prefixo está em texto claro e não encriptado visto que o seu propósito é de ser lido por uma pessoa.

Prova de gasto

Provas de gasto inesperadamente não utilizam MLSAG, mas o esquema original de assinaturas em anel que foi utilizado no primeiro protocolo de transacção (pre-RingCT) [124].

1. Calcule a imagem de chave $\tilde{K} = k_{\pi}^o \mathcal{H}_p(K_{\pi}^o)$.
2. Gera-se o número aleatório $\alpha \in_R \mathbb{Z}_l$ e $c_i, r_i \in_R \mathbb{Z}_l$ para cada $i \in \{1, 2, \dots, n\}$ mas excluindo $i = \pi$.

3. Computa-se

$$c_{tot} = \mathcal{H}_n(\mathbf{m}, [r_1 G + c_1 K_1^o], [r_1 \mathcal{H}_p(K_1^o) + c_1 \tilde{K}], \dots, [\alpha G], [\alpha \mathcal{H}_p(K_{\pi}^o)], \dots, \text{etc.})$$

4. Define-se o desafio verdadeiro

$$c_{\pi} = c_{tot} - \sum_{i=1, i \neq \pi}^n c_i$$

5. Define-se $r_{\pi} = \alpha - c_{\pi} * k_{\pi}^o \pmod{l}$.

A assinatura é :

$$\sigma = (c_1, r_1, c_2, r_2, \dots, c_n, r_n).$$

Verificação

Para verificar uma prova de gasto de uma dada transacção, o verificador confirma que todas as assinaturas em anel são válidas usando informação encontrada na transacção original em questão (imagens de chave, offsets de saída etc)

src/wallet/
wallet2.cpp
check_spe-
nd_proof()

1. Compute-se

$$c_{tot} = \mathcal{H}_n(\mathbf{m}, [r_1 G + c_1 K_1^o], [r_1 \mathcal{H}_p(K_1^o) + c_1 \tilde{K}], \dots, [r_n G + c_n K_n^o], [r_n \mathcal{H}_p(K_n^o) + c_n \tilde{K}])$$

2. Verifica-se que

$$c_{tot} \stackrel{?}{=} \sum_{i=1}^n c_i$$

Funciona porque

Note-se como este esquema é o mesmo como o bLSAG (Secção 3.4) quando só existe um membro de anel. Para adicionar um membro desvio, em vez de pôr o desafio $c_{\pi+1}$ dentro de uma nova hash de desafio, o membro é adicionado para a hash original. Como a equação seguinte :

$$c_s = c_{tot} - \sum_{i=1, i \neq s}^n c_i$$

é verdade para cada index s , um verificador não terá forma de identificar o desafio verdadeiro. Para mais, sem o conhecimento de k_π^o o provador não teria conseguido definir r_π .

8.1.3 Provar a criação de uma saída de transacção (OutProofV2)

Agora suponha-se que um remetente enviou dinheiro a alguém (uma saída) e quer prová-lo. Saídas de transacção contêm principalmente 3 componentes : o endereço do destinatário, o montante enviado e a chave privada de transacção. Os montantes estão em código, portanto só é necessário o endereço e a chave privada de transacção. Se a chave privada de transacção é perdida será impossível fazer uma prova de saída. ⁴

A tarefa aqui é de mostrar que o endereço oculto foi feito pelo endereço do destinatário, e deixar que verificadores reconstruam o compromisso de saída. Isto é feito começando pelo segredo partilhado entre o remetente e o destinatário :

$$rK^v.$$

Prova-se a criação do mesmo, e que corresponde á chave pública de transacção e ao endereço do destinatário. Isto é conseguido ao fazer uma assinatura com duas bases, nomeadamente G e K^v . Os verificadores usam o segredo partilhado para verificar o recipiente (secção 4.2), decodificar o montante (secção 5.3), e reconstruir o compromisso de saída (secção 5.3). São apresentados aqui detalhes para endereços normais bem como para sub-endereços.

```
src/wallet/
wallet2.cpp
check_tx_
proof()
```

Prova de saída (OutProof)

Para gerar uma prova de saída dirigida a um endereço

$$(K^s, K^v)$$

ou a um sub-endereço

$$(K^{s,i}, K^{v,i})$$

```
src/crypto/
crypto.cpp
generate_
tx_proof()
```

com uma chave privada de transacção r em que o segredo partilhado entre o remetente e o destinatário é rK^v . Note-se que uma chave pública de transacção guardada nos dados de transacção é rG para um endereço normal ou então $rK^{s,i}$ para um sub-endereço.

1. Gera-se um número aleatório $\alpha \in_R \mathbb{Z}_l$, e calcula-se

⁴ Uma ‘prova de saída’ (OutProof) que mostra que uma saída de transacção, *saí* do provador. Uma ‘prova de entrada’ (secção 8.1.4) mostra que uma saída de transacção, *entra* para o endereço do provador.

- (a) *Endereço normal*: αG e αK^v
- (b) *Sub-endereço*: $\alpha K^{s,i}$ e $\alpha K^{v,i}$

2. Calcula-se o desafio

- (a) *Endereço normal*:⁵

$$c = \mathcal{H}_n(\mathbf{m}, [rK^v], [\alpha G], [\alpha K^v], [T_{txprf2}], [rG], [K^v], [0])$$

- (b) *Sub-endereço*:

$$c = \mathcal{H}_n(\mathbf{m}, [rK^{v,i}], [\alpha K^{s,i}], [\alpha K^{v,i}], [T_{txprf2}], [rK^{s,i}], [K^{v,i}], [K^{s,i}])$$

3. Define-se a resposta⁶ $r^{resp} = \alpha - c * r$.

4. A assinatura é : $\sigma^{outproof} = (c, r^{resp})$.

Um provador gera várias *OutProofs*, e envia estas para o verificador. Ele concatena o string `src/wallet/` prefixo “OutProofV2” com a lista de provas, em que cada item (codificado na base-58) consiste no `wallet2.cpp` segredo partilhado entre o remetente e o destinatário rK^v (ou $rK^{v,i}$ para um sub-endereço), e o `get_tx_` correspondente $\sigma^{outproof}$. Assume-se que o verificador sabe o endereço respectivo para cada prova. `proof()`

Verificação

1. Calcula-se o desafio

- (a) *Endereço normal*:

$$c' = \mathcal{H}_n(\mathbf{m}, [rK^v], [r^{resp}G + c * rG], [r^{resp}K^v + c * rK^v], [T_{txprf2}], [rG], [K^v], [0])$$

- (b) *Sub-endereço*:

$$c' = \mathcal{H}_n(\mathbf{m}, [rK^{v,i}], [r^{resp}K^{s,i} + c * rK^{s,i}], [r^{resp}K^{v,i} + c * rK^{v,i}], [T_{txprf2}], [rK^{s,i}], [K^{v,i}], [K^{s,i}])$$

2. Se $c = c'$ então o provador sabe r , e rK^v é um segredo partilhado legítimo entre rG e K^v (enp).

3. O verificador devia verificar que o dado endereço do destinatário pode ser usado para criar um endereço oculto a partir da transacção relevante (trata-se da mesma computação para endereços normais bem como para sub-endereços)

$$K^s \stackrel{?}{=} K_t^o - \mathcal{H}_n(rK^v, t)$$

4. Eles também deviam decodificar o montante b_t , calcular a máscara y_t , e tentar reconstruir o compromisso de saída correspondente.⁷

$$C_t^b \stackrel{?}{=} y_t G + b_t H$$

⁵ Aqui o valor ‘0’ são 32 bytes a zero.

⁶ Due to the limited number of available symbols, we unfortunately used r for both responses and the transaction private key. Superscript ‘resp’ for ‘response’ will be used to differentiate the two when necessary.

⁷ Uma prova de saída válida não significa necessariamente que o destinatário considerado, é o destinatário verdadeiro. Um provador malicioso podia gerar uma chave de ver aleatória K'^v , calcular $K'^s = K^o - \mathcal{H}_n(rK'^v, t) * G$, e oferecer (K'^s, K'^v) como o recipiente nominal. Ao recalculer o compromisso de saída, os verificadores podem ter

8.1.4 Provar a posse de uma saída (InProofV2)

Uma prova de saída (*OutProof*) mostra que o provador enviou uma saída para um endereço, enquanto que uma prova de entrada (*InProof*) mostra que uma saída foi recebida por um certo endereço. É essencialmente o outro ‘lado’ do segredo partilhado entre o remetente e o destinatário, rK^v . Desta vez o provador prova conhecimento de k^v em K^v , e que em combinação com a chave pública rG , aparece o segredo partilhado, $k^v * rG$.

Uma vez que o verificador tem rK^v , ele verifica se o endereço oculto correspondente é possuído pelo endereço do provador através de :

$$K^o - \mathcal{H}_n(k^v * rG, t) * G \stackrel{?}{=} K^s.$$

Ao fazer uma prova de entrada para cada chave pública de transacção na lista de blocos, o provador revela todas as saídas que possui. Dar a chave de ver directamente ao verificador têm o mesmo efeito, mas com a diferença de que o verificador seria também capaz de identificar a posse de saídas criadas no futuro. Com as provas de entrada o provador retém o controlo das suas chaves privadas, ao custo do tempo que leva a provar e verificar se cada saída lhe pertence ou não.

A prova de entrada (InProof)

Uma prova de entrada, é construída da mesma forma que uma prova de saída, excepto que as chaves base agora são :

$$\mathcal{J} = \{G, rG\},$$

as chaves públicas são :

$$\mathcal{K} = \{K^v, rK^v\},$$

e a chave signatária é : k^v (em vez de ser r). Mostra-se só o passo de verificação para clarificar o que isto significa. Note-se que a ordem do prefixo de chave muda, rG e K^v trocam de posições para coincidir com a função que têm. É possível enviar uma multitude de provas de entrada ao verificador, estas são respectivas a várias saídas possuídas pelo mesmo endereço. Estas provas tem o prefixo “InProofV2”, e cada uma delas (codificada na base-58) contém o segredo partilhado entre o destinatário e o remetente rK^v (ou $rK^{v,i}$), e a prova de entrada $\sigma^{inproof}$.

src/wallet/
wallet2.cpp
get_tx_
proof()

Verificação

1. Calcula-se o desafio

- (a) *Endereço normal:*

$$c' = \mathcal{H}_n(\mathbf{m}, [rK^v], [r^{resp}G + c * K^v], [r^{resp} * rG + c * k^v * rG], [T_{txprf2}], [K^v], [rG], [0])$$

src/crypto/
crypto.cpp
check_tx_
proof()

mais confiança que o endereço de destinatário em questão é legítimo. Contudo, um provador e um destinatário podiam colaborar para codificar o compromisso de saída usando K^v , enquanto que o endereço oculto usa (K^s, K^v) . Desde que o destinatário teria de saber a chave privada k^v (assumindo que o montante na saída ainda é suposto de ser gasto), é questionável o qual a utilidade para esse tipo de decepção. Porque que o destinatário não usaria simplesmente (K'^s, K'^v) (ou outro endereço) para a saída inteira? Desde que a computação de C_t^b está relacionada com o recipiente, considera-se o processo de verificação *OutProof* adequado. Por outras palavras, neste processo, o provador não pode enganar os verificadores sem uma coordenação com o destinatário.

(b) *Sub-endereço*:

$$c' = \mathcal{H}_n(\mathbf{m}, [rK^{v,i}], [r^{resp}K^{s,i} + c * K^{v,i}], [r^{resp} * rK^{s,i} + c * k^v * rK^{s,i}], [T_{txprf2}], [K^{v,i}], [rK^{s,i}], [K^{s,i}])$$

2. Se $c = c'$ então o provador sabe que k^v e $k^v * rG$ é um segredo partilhado entre K^v e rG (enp).

Provar a posse de uma saída através do endereço oculto

Enquanto que uma prova de entrada mostra que um endereço oculto foi construído por um endereço específico, isso não quer dizer que o provador possa *gastar* essa saída. Só quem pode gastar uma saída realmente a possui.

Provar a posse de uma saída, uma vez que a prova de entrada está completa, é tão simples como assinar uma mensagem com a *chave de gasto*.

8

8.1.5 Prova de não-gasto de uma saída numa transacção

Para provar que uma saída está gasta ou não gasta pode-se recriar a imagem de chave com uma prova de múltiplas bases em :

$$\mathcal{J} = \{G, \mathcal{H}_p(K^o)\}$$

e :

$$\mathcal{K} = \{K^o, \tilde{K}\}.$$

Enquanto que isto funciona, os verificadores têm de descobrir a *imagem de chave*, o que também revela quando uma saída não gasta irá ser gasta no futuro.

Acontece que pode-se provar que uma saída não foi gasta numa transacção específica sem revelar a imagem chave. Para além disso pode-se provar que a saída actualmente não está gasta, ao estender esta prova de não-gasto [121] a ‘todas as transacções em que esta saída foi incluída como membro de anel’.

9

Mais especificamente, as provas de não-gasto revelam que uma dada imagem de chave de uma transacção na lista de blocos corresponde ou não a um endereço oculto específico do seu anel correspondente.

⁸ A possibilidade de oferecer tal assinatura directamente não existe em Monero. Mas como veremos isto está incluído na prova de reserva *ReserveProofs* (secção 8.1.6).

⁹ Provas de não-gasto não estão implementadas em Monero

Construir uma prova de não-gasto (UnspentProof)

O verificador de uma prova de não-gasto tem de saber rK^v , o segredo partilhado entre remetente e destinatário de uma dada saída com o endereço oculto K^o e a chave pública de transacção rG . Ele sabe a chave de ver k^v , que lhe permitiu calcular :

$$k^v * rG$$

e verificar :

$$K^o - \mathcal{H}_n(k^v * rG, t) * G \stackrel{?}{=} K^s$$

e desta forma ele sabe que a saída em questão pertence ao provador (secção 4.2).

Ou o provador forneceu rK^v . É aqui que as provas de entradas são usadas, pois com uma prova de entrada o verificador pode estar assegurado que rK^v veio da chave de ver do provador. E que corresponde com uma saída lhe pertencente sem que a chave privada de ver tenha sido fornecida.

Antes de mais, um verificador irá descobrir a imagem de chave a ser testada $\tilde{K}_?$, e verifica que o anel correspondente inclui um endereço oculto K^o , que possui uma saída pertencente ao provador. Ele depois calcula a imagem parcial de *gasto* $\tilde{K}_?^s$.

$$\tilde{K}_?^s = \tilde{K}_? - \mathcal{H}_n(rK^v, t) * \mathcal{H}_p(K^o)$$

Se a imagem de chave em questão foi criada com K^o então o ponto resultante irá ser :

$$\tilde{K}_?^s = k^s * \mathcal{H}_p(K^o).$$

O provador cria duas provas de múltipla-base (secção 3.1). O seu endereço, que possui a saída em questão é (K^v, K^s) or $(K^{v,i}, K^{s,i})$. Provas de não-gasto são feitas da mesma forma para endereços normais como para sub-endereços. A chave de gasto do sub-endereço é necessária, ou seja :

$$k^{s,i} = k^s + \mathcal{H}_n(k^v, i) \quad 4.3$$

Juntamente com as provas $\sigma_3^{unspent}$ e $\sigma_2^{unspent}$, o provador também fornece as chaves públicas $k^s * K^s$, $k^s * \tilde{K}_?^s$ e $k^s * k^s * \mathcal{H}_p(K^o)$.

Verificação

1. Confirmar que as provas $\sigma_3^{\text{não-gasto}}$ e $\sigma_2^{\text{não-gasto}}$ são legítimas.
2. Ter a certeza de que a mesma chave pública $k^s * K^s$ foi utilizada em ambas as provas.
3. Verificar se $k^s * \tilde{K}_?^s$ e $k^s * k^s * \mathcal{H}_p(K^o)$ são iguais. Se são então a saída está gasta, e se não então a saída não está gasta (enp).

Funciona porque

Este meio previne que o verificador descubra $k^s * \mathcal{H}_p(K^o)$ para uma saída não gasta, que ele poderia utilizar em combinação com rK^v para calcular a imagem de chave verdadeira. Enquanto que ele está confiante que a imagem de chave testada não corresponde a essa saída.

A prova $\sigma_2^{unspent}$ pode ser reutilizada para qualquer número de provas de não-gasto que envolvam a mesma saída. Se essa saída foi de facto gasta então

8.1.6 Provar que um endereço tem um saldo mínimo não gasto (ReserveProofV2)

Apesar da fuga de privacidade, ao revelar a imagem de chave de uma saída quando esta ainda não foi gasta, isto ainda é um método algo útil e foi implementado em Monero [117] antes das provas de não-gasto terem sido inventadas [121]. A prova de ‘reserva’ é utilizada para provar que um endereço possui um montante mínimo ao criar imagens de chave para algumas saídas de transacção. Mais especificamente, dado um saldo mínimo, o provador encontra suficientes saídas não gastas que o cubra, e demonstra a posse destas com provas de entrada. Depois cria imagens de chave para estas saídas, e prova que são legítimas, para com essas saídas com provas de duas bases (com um formato de prefixo de chave diferente). Depois prova o conhecimento das chaves privadas de gasto usadas, com assinaturas normais tipo Schnorr (pode haver mais de uma se algumas saídas são possuídas por sub-endereços distintos). Um verificador verifica que as imagens de chave não apareceram na lista de blocos, e assim essas saídas têm de ser não gastas.

A prova de reserva

Todas as sub-provas dentro de uma prova de reserva *ReserveProof* assinam uma mensagem diferente do que os outros tipos de provas (e.g. OutProofs, InProofs, or SpendProofs). Desta vez é :

$$\mathbf{m} = \mathcal{H}_n(\text{message}, \text{address}, \tilde{K}_1^o, \dots, \tilde{K}_n^o),$$

em que **address** é o endereço normal (K^s, K^v), de forma codificada (veja-se [4]). E as imagens de chave correspondem a saídas não gastas a serem incluídas na prova.

1. Cada saída tem uma prova de entrada, o que mostra que o endereço do provador (ou um dos seus sub-endereços) possui essa saída.
2. Cada imagem de chave de uma saída é assinada com uma prova de duas bases, em que o desafio está formatado da seguinte forma :

$$c = \mathcal{H}_n(\mathbf{m}, [rG + c * K^o], [r\mathcal{H}_p(K^o) + c * \tilde{K}])$$

3. Cada endereço (e sub-endereço) que possui pelo menos uma saída tem uma assinatura normal tipo Schnorr (secção 2.3.5), e o desafio (que é igual para endereços normais e sub-endereços) é :

$$c = \mathcal{H}_n(\mathbf{m}, K^{s,i}, [rG + c * K^{s,i}]),$$

```
src/crypto/
crypto.cpp
generate_
signature()
```

Para enviar uma prova de reserva a alguém, o provador concatena o string de prefixo “ReserveProofV2” com duas listas codificadas em base-58 (e.g. “ReserveProofV2, list 1, list 2”). Cada item na lista 1 está relacionada a uma saída específica e contém o seu hash de transacção (secção 7.4.1), o index de saída dentro dessa transacção (secção 4.2.1), o segredo partilhado relevante rK^v , a sua imagem de chave, a sua prova de entrada ($\sigma^{inproof}$), e a sua prova de imagem de chave. A lista 2 contém items que são os endereços que possuem essas saídas juntamente com as suas assinaturas tipo Schnorr.

```
src/wallet/
wallet2.cpp
get_rese-
rve-proof()
```


Verificação

1. Verifica-se se as imagens de chave das provas de reserva não apareceram na lista de blocos.
2. Verifica-se a prova de entrada para cada saída, e que um dos endereços dados possui uma saída.
3. Verificam-se as assinaturas (de duas bases), das imagens de chave.
4. Usam-se os segredos partilhados entre remetente e destinatário para decodificar os montantes nas saídas (secção 5.3).
5. Verifica-se a assinatura de cada endereço.

```
src/wallet/  
wallet2.cpp  
check_rese-  
rve_proof()
```

Se tudo é legítimo, então o provador tem de possuir pelo menos o montante total contido nas saídas da prova de reserva (enp). ¹⁰

8.2 Estrutura de auditoria

Nos estados unidos da america a maioria das empresas são sujeitas a auditorias financeiras [120]. Isto inclui a declaração de rendimentos, de cash flow e de saldo. Os primeiros dois envolvem em larga parte a contabilidade interna de uma empresa enquanto que o saldo envolve cada transacção, que afecta quanto dinheiro é que uma empresa actualmente possui. Cripto-moedas são como cash, portanto qualquer auditoria ao cash-flow de um utilizador de uma cripto-moeda, está relacionada com as transacções na lista de blocos.

A primeira tarefa ao auditar uma pessoa, é de identificar todas as saídas que esta possui (gastas e não gastas). Isto pode ser feito com provas de entrada usando todos os endereços em causa. Uma grande empresa pode ter uma multitude de sub-endereços, especialmente vendedores que operem em mercados online (ver capítulo 10). Criar provas de entrada para todas as transacções para cada sub-endereço pode resultar em requisitos computacionais e de espaço enormes para ambos o provador e o verificador.

Em vez disso, pode-se fazer uma prova de entrada só para o endereço normal do provador para todas as transacções. O auditor usa o segredo partilhado entre o remetente e o destinatário para verificar se alguma saída é possuída pelo endereço principal do provador ou pelos sub-endereços respectivos. Note-se que a *chave de ver* é suficiente para identificar todas as saídas possuídas pelos sub-endereços de um endereço.

Para garantir que um provador não está a tentar enganar um auditor, ao esconder um endereço normal para alguns dos seus sub-endereços, ele tem de provar também que todos os sub-endereços dados correspondem com um endereço normal.

¹⁰ Uma prova de reserva, enquanto que demonstra a posse de fundos, não inclui a prova de que dados sub-endereços pertencem a um endereço normal.

8.2.1 Provar que um sub-endereço corresponde a um endereço

É possível mostrar com este tipo de prova que a chave de ver de um endereço normal pode ser utilizada para identificar saídas possuídas por um dado sub-endereço ¹¹.

Prova de sub-endereço

As provas de sub-endereço podem ser feitas á semelhança de provas de saída ou de entrada. Aqui as chaves base são :

$$\mathcal{J} = \{G, K^{s,i}\}$$

as chaves públicas são :

$$\mathcal{K} = \{K^v, K^{v,i}\}$$

e a chave que assina é k^v .

Verificação

Um verificador sabe o endereço do provador (K^v, K^s) , o sub-endereço $(K^{v,i}, K^{s,i})$ e tem a prova de sub-endereço $\sigma^{subproof} = (c, r)$.

1. Calcula-se o desafio

$$c' = \mathcal{H}_n(\mathbf{m}, [K^{v,i}], [rG + c * K^v], [rK^{s,i} + c * K^{v,i}], [T_{txprf2}], [K^v], [K^{s,i}], [0])$$

2. Se $c = c'$ então o provador sabe k^v para K^v , e $K^{s,i}$ em combinação com essa chave de ver faz $K^{v,i}$ (enp).

8.2.2 a estrutura de auditoria

Agora é possível aprender o máximo possível sobre a história de transacção de uma pessoa. ¹²

1. O provador junta uma lista de todas as suas contas, em que cada conta consiste de um endereço normal e vários sub-endereços. Ele faz uma prova de sub-endereços a todos os seus sub-endereços. Depois faz uma assinatura com a chave de gasto de cada endereço e sub-endereço, demonstrando que possui o direito de gastar de todas as saídas pertencentes a esses endereços.

¹¹ Este tipo de prova ainda não foi implementado em Monero.

¹² Esta estrutura de auditoria não está completamente disponível em Monero. Provas de sub-endereço e provas de não-gasto não estão implementadas. Provas de entrada não estão preparadas para a optimização relacionada com sub-endereços. E não existe uma verdadeira estrutura para facilmente obter e organizar todas as informações para os provadores e verificadores.

2. O provador gera, para cada um dos seus endereços normais, provas de entrada para cada transacção na lista de blocos. Isto revela ao auditor todas as saídas possuídas pelo endereço do provador. Pois é possível verificar todos os endereços ocultos com o segredo partilhado entre o remetente e o destinatário. Há a certeza que as saídas pertencentes a sub-endereços irão ser identificadas por causa das provas de sub-endereço.¹³
3. O provador gera, para cada uma das saídas que possui, provas de não-gasto para todas as entradas de transacção em que estas saídas sejam membros de anel. Agora o auditor saberá o saldo do provador e pode passar a investigar saídas gastas.¹⁴
4. *Opcional:* O provador gera, para cada transacção em que ele gastou uma saída, uma prova de saída para mostrar ao auditor o destinatário e o montante. Este passo só é possível para transacções em que o provador salvaguardou as chaves privadas de transacção.

Note-se que um provador não tem maneira de mostrar as origens de fundos directamente. O seu único recurso é requisitar um conjunto de provas de pessoas que lhe enviam dinheiro.

1. Para uma transacção que envia dinheiro para o provador, o seu autor faz uma prova de gasto o que demonstra que o dinheiro foi de facto enviado.
2. Quem envia dinheiro ao provador também faz uma assinatura com uma chave pública identificável, por exemplo a chave de gasto do endereço normal. A prova de gasto e esta assinatura assina uma mensagem que contém essa chave pública identificável. Assim garante-se que a prova de gasto não foi roubada ou de facto feita por outra pessoa.

¹³ Este passo também pode ser completado com as chaves privadas de ver, o que implica certas fugas de privacidade.

¹⁴ Alternativamente, ele podia fazer provas de reserva para todas as saídas que possui. Mas revelar as imagens de chave das saídas não gastas, implica fugas de privacidade.

CAPÍTULO 9

Multi-assinaturas

Transacções de cripto-moedas não são reversíveis. Se alguém rouba as chaves privadas ou tem sucesso numa fraude, o dinheiro perdido pode nunca ser recuperado. Dividir o poder signatário entre pessoas pode enfraquecer o potencial de algo correr mal.

Por exemplo alguém deposita dinheiro numa conta com uma empresa de segurança que vai monitorar actividade suspeita relacionada com essa conta. As transacções só podem ser assinadas se ambos os partidos cooperam. Se alguém rouba as chaves, esta empresa de segurança pode ser alarmada, e como tal esta pára de assinar essas transacções. Isto é usualmente um serviço de garantia.

1

Cripto-moedas utilizam uma técnica de ‘multi-assinatura’ para alcançar assinaturas colaborativas chamada ‘M-de-N multisig’. Em M-de-N, N pessoas cooperam para fazer uma chave junta, e só M ($M \leq N$) destas precisam de assinar com esta chave. Primeiro será introduzido o básico de ‘N-de-N multisig’, depois ‘N-de-N multisig’ em Monero, depois a generalização para ‘M-de-N multisig’ e por fim será explicado como pôr chaves de multi-assinatura dentro de outras chaves de multi-assinatura.

Neste capítulo apresenta-se como as Multi-assinaturas *deviam* ser feitas, baseado nas recomendações em [102], e várias observações sobre implementações eficientes. Tenta-se mostrar em notas de rodapé quando a implementação actual desvia daquilo que é descrito.

2

¹ Multi-assinaturas têm uma diversidade de aplicações, desde contas de corporações, subscrições de jornais até aos mercados online.

² Actualmente existem 3 tipos de implementações de multi-assinaturas. A primeira é um processo muito básico e

9.1 Comunicação entre co-signatários

Construir chaves juntas e transacções juntas requiere comunicar informação secreta entre pessoas que podem estar em qualquer parte do globo. Para manter essa informação segura de observadores, co-signatários precisam de encriptar as mensagens que são enviadas mutuamente.

Uma troca tipo diffie hellman com curvas elípticas é uma maneira muito simples de encriptar mensagens com pontos de curva elíptica. Isto já foi mencionado na secção 5.3, em que montantes de saída são comunicados ao destinatário através do segredo partilhado rK^v :

$$montante_t = b_t \oplus_8 \mathcal{H}_n(\text{"montante"}, \mathcal{H}_n(rK_B^v, t))$$

Isto pode facilmente ser extendido a qualquer mensagem. Primeiro a mensagem é codificada como uma série de bits, depois particiona-se isso em partes iguais dependendo do comprimento de $\mathcal{H}_n$. Gera-se um número aleatório $r \in \mathbb{Z}_l$ e faz-se uma troca tipo Diffie-Hellman em todas as partes usando a chave pública do destinatário K . As partes, agora encriptadas, são enviadas ao destinatário com a chave pública rG . Este descripta a mensagem com o segredo partilhado krG . Remetentes de mensagens deviam também criar uma assinatura na mensagem encriptada (ou só a hash da mensagem encriptada), tal que destinatários possam confirmar que as mensagens não foram alteradas (uma assinatura só é verificável com a mensagem correcta m).

Leitores curiosos podem olhar para este resumo conceptual : [3], ou ver uma descrição técnica do esquema de encriptação popular AES aqui : [22]. Dr. Bernstein desenvolveu um esquema de encriptação conhecido como ChaCha [33, 91], que a primeira implementação de Monero utiliza para encriptar certas informações sensíveis relacionada com as carteiras dos utilizadores (tal como as imagens de chave de saídas de transacção).

src/wallet/
wallet2.cpp
export_
src/wallet/
ringdb.cpp

9.2 Agregação de chaves para endereços

9.2.1 Abordagem ingénua

Seja que N pessoas querem criar um endereço de multi-assinatura, que é :

$$(K^{v,grp}, K^{s,grp}).$$

Montantes podem ser enviados a esse endereço como para cada endereço normal, mas para gastar esses fundos, todas as N pessoas tem de trabalhar juntamente para assinar transacções.

Desde que todos os N participantes deviam poder ver os montantes recebidos pelo endereço de grupo, a chave de ver é dada a cada participante :

$$k^{v,grp}.$$

manual que usa a *cli* (interface da linha de comando)[105]. Segundo existe o excelente MMS (sistema de mensagens de multi-assinatura, do inglês : *Multisig Messaging System*), que é altamente automatizado através da *cli* [106, 107]. Terceiro existe a carteira comercialmente disponível que se chama ‘Exa Wallet’, cuja código fonte inicial está em : <https://github.com/exantech>. Todas estas implementações baseiam-se no mesmo código fonte, da equipa núcleo de Monero, ou seja praticamente estas são todas a mesma.

Para que cada participante tenha o mesmo poder, a chave de ver pode ser a soma de componentes que todos os participantes enviam uns aos outros de forma segura. Para o participante $e \in \{1, \dots, N\}$, a chave de ver é a componente base :

$$k_e^{v,base} \in_R \mathbb{Z}_l.$$

Todos os participantes podem calcular a chave de ver privada do grupo :

$$k^{v,grp} = \sum_{e=1}^N k_e^{v,base}.$$

De forma similar, a chave de gasto do grupo :

$$K^{s,grp} = k^{s,grp} G,$$

poderia ser a soma de componentes base de chaves privadas de gasto. Contudo, se alguém sabe todas essas componentes, então sabe a chave privada total de gasto. E como tal pode assinar transacções. Não seria uma multi-assinatura, só uma assinatura normal.

Em vez disso, obtem-se o mesmo efeito se a chave de gasto do grupo é a soma de chaves públicas de gasto. Seja que os participantes têm chaves base de gasto públicas $K_e^{s,base}$ que eles enviam uns aos outros de forma segura. Então cada um calcula :

$$K^{s,grp} = \sum_e K_e^{s,base}.$$

O que é o mesmo que :

$$K^{s,grp} = \left(\sum_e k_e^{s,base} \right) * G.$$

```
src/multi-
sig/multi-
sig.cpp
generate_
multisig_
view_sec-
ret_key()
src/multi-
sig/multi-
sig.cpp
generate_
multisig_
NN()
```

9.2.2 Desvantagens da abordagem ingénua

Usar a soma de chaves públicas de gasto é intuitivo, mas leva a uma serie de questões.

Teste de agregação a uma chave

Um adversário exterior que conheça todas as chaves base públicas de gasto $K_e^{s,base}$, pode testar um dado endereço público (K^v, K^s) para a agregação de chave ao calcular :

$$K^{s,grp} = \sum_e K_e^{s,base},$$

e verificar se :

$$K^s \stackrel{?}{=} K^{s,grp}.$$

Isto une-se a um requisito mais genérico que chaves agregadas sejam indistinguíveis de chaves normais, tal que observadores externos não possam ganhar conhecimento das actividades dos monerianos dependendo do tipo de endereço publicado. ³

³ Se pelo menos existe um participante honesto, que utiliza componentes seleccionados de forma aleatória seguindo uma distribuição aleatória, então as chaves agregadas por uma soma simples são indistinguíveis [113] de chaves normais.

Pode-se evitar isto ao criar novas chaves base de gasto para cada endereço de multi-assinatura. O que é fácil mas pode ser inconveniente. A segunda opção é mascarar chaves velhas. Procede-se da seguinte forma :

dado um participante e com um par de chaves públicas :

$$(K_e^v, K_e^s),$$

com chaves privadas :

$$(k_e^v, k_e^s)$$

e máscaras aleatórias μ_e^v, μ_e^s . Sejam os componentes de chave base privada para o endereço de grupo :

$$\begin{aligned} k_e^{v,base} &= \mathcal{H}_n(k_e^v, \mu_e^v) \\ k_e^{s,base} &= \mathcal{H}_n(k_e^s, \mu_e^s) \end{aligned}$$

⁴ Se os participantes não querem que observadores reúnam as novas chaves e as testem para agregação de chave, eles teriam de comunicar os seus novos componentes de chave uns aos outros de forma segura. ⁵ Se testes de agregação de chave não são uma preocupação, então os componentes de chave base pública :

$$(K_e^{v,base}, K_e^{s,base})$$

podem ser publicados como endereços normais. Qualquer entidade terceira podia então a partir desses endereços individuais, calcular e enviar montantes para o endereço de grupo. Sem interacção com os destinatários do grupo [81].

Cancelamento de chave

Se a chave de gasto do grupo é uma soma de chaves públicas, um participante desonesto que descobre os componentes de chaves base de gasto, dos outros participantes, pode cancelar estes componentes.

Por exemplo, seja que a alice e o bob querem fazer um endereço de grupo. A alice bem intencionada, diz ao bob os seus componentes de chave :

$$(k_A^{v,base}, K_A^{s,base}).$$

O bob calcula os seus componentes privadamente :

$$(k_B^{v,base}, K_B^{s,base}),$$

mas não diz algo a alice. Em vez disso ele calcula :

$$K_B^{ts,base} = K_B^{s,base} - K_A^{s,base},$$

e diz a alice :

$$(k_B^{v,base}, K_B^{ts,base}).$$

⁴ As máscaras aleatórias são facilmente derivadas de um password. Por exemplo, $\mu^s = \mathcal{H}_n(password)$ e $\mu^v = \mathcal{H}_n(\mu^s)$. Ou então como é feito em Monero, as chaves de ver e de gasto são mascaradas com uma string e.g. $\mu^s, \mu^v = \text{"Multisig"}$. Isto implica que Monero só suporta uma chave de base de multi-assinatura para cada endereço. Na realidade tornar uma carteira normal para uma carteira de multi-assinatura faz com que o moneriano perca acesso á carteira normal [105]. É necessário criar uma nova carteira para voltar a aceder os fundos desse endereço.

⁵ Como veremos na secção 9.6, a agregação de chave não funciona em multi-assinaturas em que $M < N$, devido á presença de segredos partilhados.

O endereço de grupo é :

$$\begin{aligned}
 K^{v,grp} &= (k_A^{v,base} + k_B^{v,base})G \\
 &= k^{v,grp}G \\
 K^{s,grp} &= K_A^{s,base} + K_B^{s,base} \\
 &= K_A^{s,base} + (K_B^{s,base} - K_A^{s,base}) \\
 &= K_B^{s,base}.
 \end{aligned}$$

Isto resulta num endereço de grupo :

$$(k^{v,grp}G, K_B^{s,base}).$$

Em que a alice sabe a chave de ver privada do grupo, e o bob sabe essa e também a chave privada de gasto !

O bob pode assinar as transacções por ele próprio, enganando a alice, que acredita que os montantes enviados para esse endereço só podem ser gastos, com a permissão dela.

Isto poderia ser resolvido ao requerer que cada participante, antes de agregar chaves, faça uma assinatura que prova que este sabe a chave privada correspondente ao componente da chave de gasto [100]. Isto é inconveniente e vulnerável a erros de implementação. Felizmente existe uma sólida alternativa. ⁶

9.2.3 Agregação de chave robusta

Para facilmente resistir ao cancelamento de chave faz-se uma pequena alteração á agregação das chaves de gasto (deixando a agregação das chaves de ver igual). Seja que o conjunto de N signatários tenham componentes de chave base de gasto :

$$\mathbb{S}^{base} = \{K_1^{s,base}, \dots, K_N^{s,base}\},$$

que estão ordenados seguindo uma convenção qualquer, de mínimo para máximo, ou de forma lexicográfica.

A robusta chave de gasto agregada é :

$$K^{s,grp} = \sum_e \mathcal{H}_n(T_{agg}, \mathbb{S}^{base}, K_e^{s,base}) K_e^{s,base}$$

Se o bob tenta cancelar a chave de gasto da alice, ele fica preso com um problema difícil :

$$\begin{aligned}
 K^{s,grp} &= \mathcal{H}_n(T_{agg}, \mathbb{S}, K_A^s) K_A^s + \mathcal{H}_n(T_{agg}, \mathbb{S}, K_B'^s) K_B'^s \\
 &= \mathcal{H}_n(T_{agg}, \mathbb{S}, K_A^s) K_A^s + \mathcal{H}_n(T_{agg}, \mathbb{S}, K_B'^s) K_B^s - \mathcal{H}_n(T_{agg}, \mathbb{S}, K_B'^s) K_A^s \\
 &= [\mathcal{H}_n(T_{agg}, \mathbb{S}, K_A^s) - \mathcal{H}_n(T_{agg}, \mathbb{S}, K_B'^s)] K_A^s + \mathcal{H}_n(T_{agg}, \mathbb{S}, K_B'^s) K_B^s
 \end{aligned}$$

Da mesma forma que com a abordagem ingénua, qualquer terceiro que saiba $\mathbb{S}^{base}$ e as chaves públicas correspondentes, pode calcular o endereço de grupo.

Como os participantes não precisam de provar que sabem as chaves privadas de gasto, e também

⁶ A primeira iteração (ainda a mesma actualmente) de multi-assinaturas, disponibilizada em abril de 2018 [57] (com a integração de M-de-N em outubro de 2018 [10]) Monero's current (and first) iteration of multisig, made available in April 2018 [57] (with M-of-N integration following in October 2018 [10]), used this naive key aggregation, and required users sign their spend key components.

não precisam de interagir de todo antes de assinar as transacções, esta agregação de chave robusta segue o modelo *simples de chave-pública*. O único requisito neste modelo é que cada participante e potencial signatário tenha uma chave pública [81].

7

Funções **premerge** e **merge**

Mais formalmente e para ser mais claro, diz-se que existe uma operação **premerge**, que tem como argumento um conjunto de chaves base $\mathbb{S}^{base}$, e tem como resultado um conjunto de chaves agregadas $\mathbb{K}^{agg}$ de igual tamanho. Em que $\mathbb{K}^{agg}[e]$ é o n -ésimo elemento do conjunto :

$$\mathbb{K}^{agg}[e] = \mathcal{H}_n(T_{agg}, \mathbb{S}^{base}, K_e^{s,base}) K_e^{s,base}$$

As chaves privadas de agregação k_e^{agg} , são usadas em assinaturas de grupo.⁸ Existe uma outra operação **merge**, que usa as chaves de agregação de **premerge** e constroi a chave signatária de grupo (de gasto):

$$K^{grp} = \sum_e \mathbb{K}^{agg}[e]$$

Estas funções são generalizadas para (N-1)-de-N e M-de-N na secção 9.6.2, e também para multi-assinaturas inclusivas na secção 9.7.2.

9.3 Assinaturas tipo Schnorr com limite

É preciso um certo número de signatários para que uma multi-assinatura funcione, portanto diz-se que existe um ‘limite’ de signatários abaixo do qual a assinatura não pode ser produzida. Uma multi-assinatura com N participantes que requer que todos assinem, tem um limite de N, e chama-se *multi-assinatura N-de-N*.

Todos os esquemas de assinatura neste documento, baseiam-se na prova de conhecimento genérica de *Maurer* (com conhecimento nulo [78]). Portanto mostra-se a forma essencial de uma assinatura com limite tipo Schnorr (secção 2.3.5) [100].

Assinatura

Sejam N pessoas em que cada uma tem uma chave pública no conjunto $\mathbb{K}^{agg}$. Cada pessoa $e \in \{1, \dots, N\}$, também sabe a chave privada k_e^{agg} .

A chave de grupo pública N-de-N que irá ser usada para assinar mensagens é K^{grp} .

Seja que todos os participantes querem assinar uma mensagem **m**. Uma assinatura tipo Schnorr básica é feita com colaboração da seguinte forma :

⁷ A agregação de chave só satisfaz o modelo de chave-pública para multi-assinaturas N-de-N e 1-de-N.

⁸ A agregação robusta de chave ainda não foi implementada em Monero, mas como os participantes guardam e usam a chave privada k_e^{agg} (para a agregação de chave ingénua : $k_e^{agg} = k_e^{base}$), actualizar Monero para usar a agregação robusta de chave só irá alterar o processo de *premerge*.

1. Cada participante $e \in \{1, \dots, N\}$ faz o seguinte :
 - (a) escolhe um componente aleatório $\alpha_e \in_R \mathbb{Z}_l$,
 - (b) calcula $\alpha_e G$
 - (c) compromete-se através de $C_e^\alpha = \mathcal{H}_n(T_{com}, \alpha_e G)$,
 - (d) e envia C_e^α aos outros participantes de forma segura.
2. uma vez que todos os compromissos C_e^α foram colecionados, cada participante envia o seu $\alpha_e G$ aos outros participantes de forma segura.
 Cada participante verifica que :

$$C_e^\alpha \stackrel{?}{=} \mathcal{H}_n(T_{com}, \alpha_e G),$$
 para cada um dos outros participantes.
3. Cada participante calcula

$$\alpha G = \sum_e \alpha_e G$$
4. Cada participante $e \in \{1, \dots, N\}$ faz o seguinte : ⁹
 - (a) calcula o desafio $c = \mathcal{H}_n(\mathbf{m}, [\alpha G])$,
 - (b) define a componente resposta $r_e = \alpha_e - c * k_e^{agg} \pmod{l}$,
 - (c) e envia r_e aos outros participantes de forma segura.
5. Cada participante calcula

$$r = \sum_e r_e$$
6. Qualquer participante pode publicar a assinatura :

$$\sigma(\mathbf{m}) = (c, r).$$

Verificação

Dado K^{grp} , $\mathbf{m}$, e $\sigma(\mathbf{m}) = (c, r)$:

1. Calcula-se o desafio $c' = \mathcal{H}_n(\mathbf{m}, [rG + c * K^{grp}])$.
2. Se $c = c'$, então a assinatura é legítima (enp).

O sobrescrito *grp* foi incluído para a claridade, mas na realidade o verificador não tem maneira alguma de saber que K^{grp} é uma chave junta. Só se ele sabe os componentes base ou de agregação.

⁹ É importante não re-utilizar α_e para diferentes desafios c . Isto significa que recomeçar um processo de multi-assinatura em que as respostas já foram enviadas, um recomeço implica um iniciar de novo com novos valores α_e .

Funciona porque

A resposta r é central nesta assinatura. O participante e sabe dois segredos em r_e (α_e and k_e^{agg}), assim a sua chave privada k_e^{agg} está segura em termos da teoria da informação, dos outros participantes (assumindo que ele nunca re-usa α_e). Para mais, os verificadores usam a chave de grupo pública K^{grp} , assim todos os componentes de chave são necessários para construir assinaturas.

$$\begin{aligned}
 rG &= \left(\sum_e r_e \right) G \\
 &= \left(\sum_e (\alpha_e - c * k_e^{agg}) \right) G \\
 &= \left(\sum_e \alpha_e \right) G - c * \left(\sum_e k_e^{agg} \right) G \\
 &= \alpha G - c * K^{grp} \\
 \alpha G &= rG + c * K^{grp} \\
 \mathcal{H}_n(m, [\alpha G]) &= \mathcal{H}_n(m, [rG + c * K^{grp}]) \\
 c &= c'
 \end{aligned}$$

Passo adicional de comprometer e revelar

O leitor pode estar a questionar-se de onde veio o segundo passo. Sem comprometer e revelar [102], um co-signatário malicioso podia aprender todos os $\alpha_e G$ antes do desafio ser calculado. Isto permite-lhe controlar o desafio produzido a um certo grau, ao modificar o seu próprio $\alpha_e G$ antes de o enviar para fora.

Ele pode usar as componentes de resposta coleccionados de múltiplas assinaturas controladas para derivar outras chaves privadas k_e^{agg} em tempo sub-exponencial[48]. O que é uma ameaça séria de segurança.

Esta ameaça baseia-se na generalização [125] (veja também [127] para uma explicação mais intuitiva) do problema do dia de anos [129]¹⁰.

9.4 MLSTAG assinaturas confidenciais em anel de Monero

Transacções confidenciais em anéis com limite, adicionam alguma complexidade porque chaves signatárias MLSTAG (MLSAG com o T de *threshold* do inglês : limite, barreira) são endereços ocultos e compromissos a zero para os montantes de entrada.

Um endereço oculto que dá posse á t -ésima saída a quem tiver o endereço público (K_t^s, K_t^v) funciona da seguinte forma :

$$\begin{aligned}
 K_t^o &= \mathcal{H}_n(rK_t^v, t)G + K_t^s = (\mathcal{H}_n(rK_t^v, t) + k_t^s)G \\
 k_t^o &= \mathcal{H}_n(rK_t^v, t) + k_t^s
 \end{aligned}$$

¹⁰ Comprometer e revelar não é usado na implementação actual de multi-assinaturas em Monero, no entanto está a ser considerado para um lançamento futuro [93].

Altera-se a notação para saídas recebidas por um endereço de grupo $(K_t^{v,grp}, K_t^{s,grp})$:

$$\begin{aligned} K_t^{o,grp} &= \mathcal{H}_n(rK_t^{v,grp}, t)G + K_t^{s,grp} \\ k_t^{o,grp} &= \mathcal{H}_n(rK_t^{v,grp}, t) + k_t^{s,grp} \end{aligned}$$

Qualquer pessoa que tenha $k_t^{v,grp}$ e $K_t^{s,grp}$ pode descobrir $K_t^{o,grp}$, e pode decodificar o termo de Diffie-Hellman para o montante de saída e reconstruir o compromisso de máscara correspondente (secção 5.3).

Isto também significa que sub-endereços de multi-assinatura são possíveis (secção 4.3). Transacções de multi-assinatura que utilizem fundos provenientes de um sub-endereço requerem algumas modificações aos seguintes algoritmos. Sub-endereços de multi-assinatura são suportados em Monero.

9.4.1 RCTTypeBulletproof2 com multi-assinaturas N-de-N

A maior parte de uma transacção de multi-assinatura pode ser completada por quem a iniciou. Só as assinaturas de MLSTAG requerem colaboração. Um iniciador deve fazer estes passos para preparar uma transacção RCTTypeBulletproof2 (lembrar secção 40):

1. Gera-se uma chave privada de transacção $r \in_R \mathbb{Z}_l$ (secção 4.2) e calcula-se a correspondente chave pública rG (ou múltiplas chaves destas se o destinatário for um sub-endereço; secção 4.3).
2. Escolhem-se as entradas a serem gastas ($j \in \{1, \dots, m\}$ saídas possuídas com endereços ocultos $K_j^{o,grp}$ e montantes $a_1, \dots, a_m$), e os destinatários que recebem os fundos ($t \in \{0, \dots, p-1\}$ novas saídas com montantes $b_0, \dots, b_{p-1}$ e endereços ocultos K_t^o). Isto inclui a taxa do mineiro f e o seu compromisso fH . Escolhe-se também o conjunto de membros desvio do anel.
3. Codifica-se cada montante de saída $montante_t$ (secção 5.3), e calcula-se os compromissos de saída C_t^b .
4. Selecciona-se para cada entrada $j \in \{1, \dots, m-1\}$, componentes máscara de pseudo compromissos de saída :

$$x'_j \in_R \mathbb{Z}_l.$$

Calcula-se a m -ésima máscara como na secção 5.4 :

$$x'_m = \sum_t y_t - \sum_{j=1}^{m-1} x'_j$$

Calcule o compromisso da pseudo saída $C_j'^a$.

5. É feita a prova de domínio agregada tipo *Bulletproof* para todas as saídas. Lembrar secção 5.5.
6. Preparação para as assinaturas MLSTAG, ao gerar para os compromissos a zero, componentes semente :

$$\alpha_j^z \in_R \mathbb{Z}_l,$$

e calcular $\alpha_j^z G$. Não há necessidade de comprometer e revelar desde que estes compromissos a zero são conhecidos por todos os signatários.

Depois o iniciador envia toda esta informação aos outros participantes de forma segura. Agora o grupo de signatários está pronto a construir assinaturas nas entradas com as suas chaves privadas $k_e^{s,agg}$, e os compromissos a zero :

$$C_{\pi,j}^a - C_{\pi,j}^{t_a} = z_j G$$

MLSTAG RingCT

Primeiro são construídas as imagens de chave de grupo para todas as entradas $j \in \{1, \dots, m\}$ com endereços ocultos $K_{\pi,j}^{o,grp}$ ¹¹.

1. Para cada entrada j cada participante e faz o seguinte :

(a) calcula a imagem de chave parcial :

$$\tilde{K}_{j,e}^o = k_e^{s,agg} \mathcal{H}_p(K_{\pi,j}^{o,grp}),$$

(b) e envia $\tilde{K}_{j,e}^o$, aos outros participantes de forma segura.

2. Cada participante, usa u_j como o index de saída na transacção em que $K_{\pi,j}^{o,grp}$ foi enviado ao endereço de multi-assinatura, e calcula : ¹²

$$\tilde{K}_j^{o,grp} = \mathcal{H}_n(k^{v,grp} r G, u_j) \mathcal{H}_p(K_{\pi,j}^{o,grp}) + \sum_e \tilde{K}_{j,e}^o$$

Depois é construída uma assinatura MLSTAG para cada entrada j .

1. Cada participante e faz o seguinte :

(a) gera componentes semente :

$$\alpha_{j,e} \in_R \mathbb{Z}_l$$

componentes semente $\alpha_{j,e} \in_R \mathbb{Z}_l$ e calcula $\alpha_{j,e} G$, e $\alpha_{j,e} \mathcal{H}_p(K_{\pi,j}^{o,grp})$,

(b) gera para $i \in \{1, \dots, v+1\}$ excepto $i = \pi$, componentes aleatórios $r_{i,j,e}$ e $r_{i,j,e}^z$,

(c) calcula o compromisso

$$C_{j,e}^\alpha = \mathcal{H}_n(T_{com}, \alpha_{j,e} G, \alpha_{j,e} \mathcal{H}_p(K_{\pi,j}^{o,grp}), r_{1,j,e}, \dots, r_{v+1,j,e}, r_{1,j,e}^z, \dots, r_{v+1,j,e}^z)$$

¹¹ Se $K_{\pi,j}^{o,grp}$ é construído de um sub-endereço de multi-assinatura indexado por i , então (secção 4.3) a chave privada é a composta :

$$k_{\pi,j}^{o,grp} = \mathcal{H}_n(k^{v,grp} r_{u_j} K^{s,grp,i}, u_j) + \sum_e k_e^{s,agg} + \mathcal{H}_n(k^{v,grp}, i)$$

¹² Se o endereço oculto corresponde a um sub-endereço de multi-assinatura indexado por i , adiciona-se :

$$\tilde{K}_j^{o,grp} = \dots + \mathcal{H}_n(k^{v,grp}, i) \mathcal{H}_p(K_{\pi,j}^{o,grp})$$

src/wallet/
wallet2.cpp
get_multi-
sig_kLRki()

src/crypto-
note_basic/
cryptonote_
format_
utils.cpp
generate_key_
image_helper_
precomp()

- (d) e envia $C_{j,e}^\alpha$ aos outros participantes de forma segura.
2. Depois de receber todos os $C_{j,e}^\alpha$ dos outros participantes, envie todos os $\alpha_{j,e}G$, $\alpha_{j,e}\mathcal{H}_p(K_{\pi,j}^{o,grp})$ e $r_{i,j,e}$ e $r_{i,j,e}^z$, e verifique que cada compromisso original de cada participante é válido.

3. Cada participante calcula

$$\begin{aligned}\alpha_j G &= \sum_e \alpha_{j,e} G \\ \alpha_j \mathcal{H}_p(K_{\pi,j}^{o,grp}) &= \sum_e \alpha_{j,e} \mathcal{H}_p(K_{\pi,j}^{o,grp}) \\ r_{i,j} &= \sum_e r_{i,j,e} \\ r_{i,j}^z &= \sum_e r_{i,j,e}^z\end{aligned}$$

4. Cada participante constroí o ciclo de assinatura (veja-se a secção 3.5).
5. Para fechar a assinatura, cada participante e faz o seguinte:

- (a) define $r_{\pi,j,e} = \alpha_{j,e} - c_\pi k_e^{s,agg} \pmod{l}$,
- (b) e envia $r_{\pi,j,e}$ aos outros participantes de forma segura.

6. Qualquer participante pode calcular (lembre-se que $\alpha_{j,e}^z$ foi criado pelo iniciador) ¹³

$$\begin{aligned}r_{\pi,j} &= \sum_e r_{\pi,j,e} - c_\pi * \mathcal{H}_n(k^{v,grp} rG, u_j) \\ r_{\pi,j}^z &= \alpha_{j,e}^z - c_\pi z_j \pmod{l}\end{aligned}$$

```
src/ringct/
rctSigs.cpp
signMulti-
sig()
```

A assinatura para a entrada j é $\sigma_j(\mathbf{m}) = (c_1, r_{1,j}, r_{1,j}^z, \dots, r_{v+1,j}, r_{v+1,j}^z)$ com $\tilde{K}_j^{o,grp}$.

Desde que em Monero a mensagem $\mathbf{m}$ e o desafio c_π dependem em todas as outras partes da transacção, cada participante que tenta enganar os seus co-signatários, ao enviar o valor errado, em qualquer ponto do processo inteiro, irá causar que a assinatura falhe. A resposta $r_{\pi,j}$ só é válida para a mensagem $\mathbf{m}$ e para o desafio c_π para o qual foi definido.

9.4.2 Comunicação simplificada

Em Monero, são necessários muitos passos para construir uma transacção de multi-assinatura. É possível simplificar e reorganizar alguns deles, tal que as interações entre signatários acontecam em duas partes com um total de 5 rondas.

¹³ Se o endereço oculto $K_{\pi,j}^{o,grp}$ corresponde a um sub-endereço de multi-assinatura indexado por i , inclui-se :

$$r_{\pi,j} = \dots - c_\pi * \mathcal{H}_n(k^{v,grp}, i)$$

```
src/crypto-
note_basic/
cryptonote-
format_
utils.cpp
generate_key_
image_helper_
precomp()
```

1. *Agregação de chave para um endereço público de multi-assinatura*: Qualquer pessoa com um conjunto de endereços públicos pode executar **premerge**, e depois **merge** o que resulta num endereço N-de-N. Mas nenhum participante irá saber a chave de ver de grupo excepto se eles aprendam todos os componentes, portanto o grupo começa por enviar k_e^v e $K_e^{s,base}$ uns aos outros de forma segura. . Qualquer participante pode executar **premerge** e **merge** e publicar $(K^{v,grp}, K^{s,grp})$, o que permite ao grupo receber fundos através do endereço de grupo. A agregação M-de-N requer mais passos, que nós descrevemos na secção 9.6.

```
src/wallet/
wallet2.cpp
pack_multi-
signature_
keys()
```

2. Transacções:

- (a) Algum participante ou sub-coalição (o iniciador) decide escrever uma transacção. Eles então escolhem m entradas com endereços ocultos $K_j^{o,grp}$ e compromissos a montantes C_j^a , m conjuntos de v endereços ocultos adicionais e compromissos para serem usados como membros desvio de anel. Os participantes escolhem também p destinatários com endereços públicos (K_t^s, K_t^v) e montantes b_t para lhes enviar, decidir uma taxa de transacção f , e gerar uma chave privada de transacção r , ¹⁴ gerar máscaras de pseudo compromisso de saída x'_j com $j \neq m$, construir o termo ECDH $montante_t$ para cada saída, produzir uma prova de domínio agregada, e gerar aberturas de assinatura α_j^z para todos os compromissos a zero das entradas e escalares aleatórios $r_{i,j}$ e $r_{i,j}^z$ com $i \neq \pi_j$. generate pseudo output commitment masks x'_j with $j \neq m$, construct the ECDH term $amount_t$ for each output, produce an aggregate range proof, and generate signature openers α_j^z for all inputs' commitments to zero and random scalars $r_{i,j}$ and $r_{i,j}^z$ with $i \neq \pi_j$. ¹⁵ Os participantes também preparam a contribuição para a próxima ronda de comunicação.

```
src/wallet/
wallet2.cpp
transfer_
selected_
rct()
```

O iniciador envia toda esta informação aos outros participantes de forma segura. ¹⁶ Os outros participantes podem sinalizar consentimento ao enviar a parte deles da próxima ronda de comunicação, ou negociar mudanças.

```
src/wallet/
wallet2.cpp
save_multi-
sig-tx()
```

- (b) Cada participante escolhe os seus componentes de abertura para as assinaturas MLSTAG, compromete-se às mesmas, calcula as imagens de chave parciais, e envia esses compromissos e imagens parciais aos outros participantes de forma segura.

Assinatura(s) MLSTAG : A imagem de chave $\tilde{K}_{j,e}^o$, a entropia de assinatura $\alpha_{j,e}G$, e $\alpha_{j,e}\mathcal{H}_p(K_{\pi,j}^{o,grp})$. Imagens de chave parciais não precisam de estar nos dados comprometidos, desde que esses dados não podem ser usados para extrair as chaves privadas dos signatários. As imagens de chave parciais também são úteis para observar quais

¹⁴ Ou chaves privadas de transacção r_t ao enviar para pelo menos um sub-endereço.

¹⁵ Note-se que o processo signatário é simplificado ao deixar que o iniciador gere escalares aleatórios $r_{i,j}$ e $r_{i,j}^z$, em vez de que cada co-signatário gere componentes que eventualmente sejam somados juntos.

¹⁶ Ele não precisa de enviar os montantes de saída b_t directamente, pois estes podem ser calculados a partir de $amount_t$. Em Monero faz-se a abordagem razoável de criar uma transacção parcial, que contém informação seleccionada pelo iniciador, isto envia-se para outros co-signatários juntamente com uma lista de informação relevante; como chaves privadas de transacção, endereços de destino, entradas verdadeiras, etc.

- saídas foram gastas, portanto para uma arquitectura modular estas devem ser tratadas individualmente.
- (c) Depois de receber todos os compromissos de assinatura, cada participante envia a informação comprometida aos outros participantes de forma segura.
 - (d) Cada participante fecha a sua parte das assinaturas MLSTAG, e envia todos os $r_{\pi_j, j, e}$ aos outros participantes de forma segura. ¹⁷

Assumindo que o processo correu bem, todos os participantes podem acabar de escrever a transacção e envia-la á rede. As transacções feitas por uma coaligação de multi-assinatura são indistinguíveis de transacções normais, feitas por só um signatário.

9.5 Recalcular imagens de chaves

Se alguém perde os seus dados e quer calcular o saldo de um dos seus endereços (montantes recebidos menos montantes gastos), então precisa de sacar dados da lista de blocos. As chaves de ver só são úteis para ler quais os montantes recebidos, assim um moneriano precisa de calcular imagens de chave para todas as saídas que possui, para ver se essas foram gastas, ao comparar com imagens de chave guardadas na lista de blocos. Desde que os membros de um endereço de grupo nao podem calcular as imagens de chave individualmente, estes requerem a ajuda do outros membros.

Calcular imagens de chave de uma simples soma de componentes pode falhar se participantes deshonestos reportam chaves falsas. ¹⁸ Dada uma saída recebida com o endereço oculto $K^{o,grp}$, o grupo pode produzir uma simples prova tipo Schnorr, para que a imagem de chave $\tilde{K}^{o,grp}$ é legítima sem que componentes de chaves privadas sejam revelados, nem a necessidade de confiarem uns nos outros.

Prova

1. Cada participante e faz o seguinte :
 - (a) calcula $\tilde{K}_e^o = k_e^{s,agg} \mathcal{H}_p(K^{o,grp})$,
 - (b) gera a componente semente $\alpha_e \in_R \mathbb{Z}_l$ e calcula $\alpha_e G$ e $\alpha_e \mathcal{H}_p(K^{o,grp})$,
 - (c) compromete-se aos dados com : $C_e^\alpha = \mathcal{H}_n(T_{com}, \alpha_e G, \alpha_e \mathcal{H}_p(K^{o,grp}))$,
 - (d) e envia C_e^α e $\tilde{K}_e^o$ aos outros participantes de forma segura.
2. Depois de receber todos os C_e^α , cada participante envia a informação comprometida e verifica os compromissos dos outros participantes.

¹⁷ É imperativo que cada tentativa de assinar por um certo signatário, use um $\alpha_{j,e}$ único, para evitar a fuga da chave privada a outros signatários (secção 2.3.4) [102]. As carteiras deviam por princípio forçar isto ao apagarem sempre $\alpha_{j,e}$, cada vez que uma resposta que use isso seja enviada para fora da carteira.

¹⁸ Actualmente em Monero, usa-se uma soma simples .

3. Cada participante pode calcular : ¹⁹

$$\tilde{K}^{o,grp} = \mathcal{H}_n(k^{v,grp}rG, u)\mathcal{H}_p(K^{o,grp}) + \sum_e \tilde{K}_e^o$$

$$\alpha G = \sum_e \alpha_e G$$

$$\alpha \mathcal{H}_p(K^{o,grp}) = \sum_e \alpha_e \mathcal{H}_p(K^{o,grp})$$

4. Cada participante calcula o desafio ²⁰

$$c = \mathcal{H}_n([\alpha G], [\alpha \mathcal{H}_p(K^{o,grp})])$$

5. Cada participante faz o seguinte :

- (a) define-se $r_e = \alpha_e - c * k_e^{s,agg} \pmod{l}$,
- (b) e envia-se r_e aos outros participantes de forma segura.

6. Cada participante pode calcular : ²¹

$$r^{resp} = \sum_e r_e - c * \mathcal{H}_n(k^{v,grp}rG, u)$$

A prova é (c, r^{resp}) com $\tilde{K}^{o,grp}$.

Verificação

1. Verifica $l\tilde{K}^{o,grp} \stackrel{?}{=} 0$.

2. Calcula

$$c' = \mathcal{H}_n([r^{resp}G + c * K^{o,grp}], [r^{resp}\mathcal{H}_p(K^{o,grp}) + c * \tilde{K}^{o,grp}]).$$

3. Se $c = c'$ então a imagem chave $\tilde{K}^{o,grp}$ corresponde ao endereço oculto $K^{o,grp}$ (enp).

9.6 M < N

No início deste capítulo foram discutidos serviços de garantia, que usam uma multi-assinatura 2-de-2, para separar o poder signatário entre um moneriano e uma empresa de segurança. Esse

¹⁹ Se o endereço oculto corresponde a um sub-endereço de multi-assinatura indexado por i , adicione-se :

$$\tilde{K}^{o,grp} = \dots + \mathcal{H}_n(k^{v,grp}, i)\mathcal{H}_p(K^{o,grp})$$

²⁰ Esta prova devia incluir separação de domínio e prefixo de chave, o que é omitido por brevidade.

²¹ Se o endereço oculto $K^{o,grp}$ corresponde a um sub-endereço de multi-assinatura indexado por i , inclui-se :

$$r^{resp} = \dots - c * \mathcal{H}_n(k^{v,grp}, i)$$

sistema não é ideal, pois se a empresa de segurança for comprometida, ou se recusa cooperar, então os montantes podem ficar bloqueados.

Isto tem solução, usa-se um endereço de multi-assinatura 2-de-3, em que há 3 participantes mas só são necessários dois signatários para efectuar uma transacção. Um serviço de garantia guarda uma chave e os outros participantes guardam as outras.

Os participantes podem guardar uma chave num lugar seguro, e usar a outra chave para o uso quotidiano. Se o serviço de garantia falha, o participante pode usar a chave do quotidiano e a chave segura para tirar os montantes do endereço de multi-assinatura.

Multi-assinaturas com limites abaixo de N , têm vários usos.

9.6.1 Agregação de chave 1-de- N

Seja que um grupo de pessoas querem fazer uma chave de multi-assinatura K^{grp} , com a qual todos podem assinar. A solução é trivial: deixa-se que cada participante saiba a chave privada k^{grp} . Existem 3 formas de fazer isto.

1. Um participante ou uma sub-coalição selecciona uma chave e envia esta a todos os outros participantes de forma segura.
2. Todos os participantes calculam componentes de chaves privadas, e enviam estas de forma segura a todos os outros participantes. Em que a soma é uma chave junta. ²²
3. Os participantes estendem multi-assinaturas de M -de- N a 1-de- N . Isto pode ser útil se um adversário tem acesso às comunicações de grupo.

Neste caso, para Monero, cada participante saberia as chaves privadas :

$$(k^{v,grp,1 \times N}, k^{s,grp,1 \times N}).$$

Antes desta secção todas as chaves de grupo eram N -de- N , mas agora usa-se o sobrescrito $1 \times N$ para denotar chaves relacionadas com multi-assinaturas 1-de- N .

9.6.2 Agregação de chave $(N-1)$ -de- N

Numa chave de grupo $(N-1)$ -de- N , como 2-de-3 ou 4-de-5, cada conjunto de $(N-1)$ participantes pode efectuar uma transacção. Isto é conseguido com segredos partilhados do tipo Diffie-Hellman. Seja que existem participantes $e \in \{1, \dots, N\}$, cada um com uma chave base pública K_e^{base} .

Cada participante e calcula para $w \in \{1, \dots, N\}$ excepto $w \neq e$.

$$k_{e,w}^{sh,(N-1) \times N} = \mathcal{H}_n(k_e^{base} K_w^{base})$$

²² Note-se que aqui o cancelamento de chave, não é muito relevante visto que cada participante conhece a chave privada inteira.

Depois cada participante calcula todos os :

$$K_{e,w}^{sh,(N-1) \times N} = k_{e,w}^{sh,(N-1) \times N} G,$$

e envia isso aos outros participantes de forma segura. Agora utiliza-se o sobrescrito *sh* para denotar chaves partilhadas por um sub-grupo dos participantes.

Cada participante irá ter (N-1) componentes de chaves privadas partilhadas, correspondentes a cada um dos outros participantes. O que perfaz para todos os participantes, um total de $N^*(N-1)$ componentes de chaves privadas partilhadas.

Cada chave é partilhada por dois parceiros que executam a troca Diffie-Hellman, portanto só existem $[N^*(N-1)]/2$ chaves únicas. Estas chaves únicas compõem o conjunto :

$$\mathbb{S}^{(N-1) \times N}.$$

src/multi-
sig/multi-
sig.cpp
generate_
multisig_
N1_N()

Generalizar premerge and merge

É aqui que renovamos a definição de **premerge** da secção 9.2.3. O seu argumento irá ser o conjunto $\mathbb{S}^{M \times N}$, em que M é o limite do conjunto de chaves. Quando $M = N$:

$$\mathbb{S}^{N \times N} = \mathbb{S}^{base},$$

e quando $M < N$ it contains shared keys. O resultado é :

$$\mathbb{K}^{agg, M \times N}.$$

Os $[N^*(N-1)]/2$ componentes de chave em $\mathbb{K}^{agg,(N-1) \times N}$ podem ser enviados para **merge**, resultando em $K^{grp,(N-1) \times N}$. Todos os componentes de chave privada podem ser construídos com só (N-1) participantes, desde que cada participante partilha um segredo partilhado tipo Diffie-Hellman com o n-ésimo participante.

Um exemplo 2-de-3

Seja que existem 3 pessoas com chaves públicas $\{K_1^{base}, K_2^{base}, K_3^{base}\}$, para as quais cada uma sabe uma chave privada, e estas querem fazer uma chave de multi-assinatura 2-de-3. Depois da troca tipo Diffie-Hellman, e de enviarem uns aos outros as chaves públicas, cada um dos participantes sabe o seguinte :

1. Participante 1: $k_{1,2}^{sh,2 \times 3}, k_{1,3}^{sh,2 \times 3}, K_{2,3}^{sh,2 \times 3}$
2. Participante 2: $k_{2,1}^{sh,2 \times 3}, k_{2,3}^{sh,2 \times 3}, K_{1,3}^{sh,2 \times 3}$
3. Participante 3: $k_{3,1}^{sh,2 \times 3}, k_{3,2}^{sh,2 \times 3}, K_{1,2}^{sh,2 \times 3}$

Em que $k_{1,2}^{sh,2 \times 3} = k_{2,1}^{sh,2 \times 3}$, e assim por diante. O conjunto $\mathbb{S}^{2 \times 3} = \{K_{1,2}^{sh,2 \times 3}, K_{1,3}^{sh,2 \times 3}, K_{2,3}^{sh,2 \times 3}\}$.

Executar **premerge** e **merge** gera a chave de grupo : ²³

²³ Desde que a chave junta é composta de segredos partilhados, um observador que saiba só as chaves base originais não seria capaz de as agregar (secção 9.2.2), e de identificar os membros da chave junta.

$$\begin{aligned}
K^{grp,2 \times 3} = & \mathcal{H}_n(T_{agg}, \mathbb{S}^{2 \times 3}, K_{1,2}^{sh,2 \times 3}) K_{1,2}^{sh,2 \times 3} + \\
& \mathcal{H}_n(T_{agg}, \mathbb{S}^{2 \times 3}, K_{1,3}^{sh,2 \times 3}) K_{1,3}^{sh,2 \times 3} + \\
& \mathcal{H}_n(T_{agg}, \mathbb{S}^{2 \times 3}, K_{2,3}^{sh,2 \times 3}) K_{2,3}^{sh,2 \times 3}
\end{aligned}$$

Seja que o participante 1 e 2 querem assinar a mensagem $\mathbf{m}$. Iremos usar uma assinatura básica tipo Schnorr para demonstrar.

1. Cada participante $e \in \{1, 2\}$ faz o seguinte :

- (a) escolhe um componente aleatório $\alpha_e \in_R \mathbb{Z}_l$,
- (b) calcula $\alpha_e G$,
- (c) compromete-se com $C_e^\alpha = \mathcal{H}_n(T_{com}, \alpha_e G)$,
- (d) e envia C_e^α aos outros participantes de forma segura.

2. Depois de receber C_e^α , cada participante envia $\alpha_e G$ e verifica os outros compromissos.

3. cada participante calcula

$$\begin{aligned}
\alpha G &= \sum_e \alpha_e G \\
c &= \mathcal{H}_n(\mathbf{m}, [\alpha G])
\end{aligned}$$

4. Participante 1 faz o seguinte :

- (a) calcula $r_1 = \alpha_1 - c * [k_{1,3}^{agg,2 \times 3} + k_{1,2}^{agg,2 \times 3}]$,
- (b) e envia r_1 ao participante 2 de forma segura.

5. Participante 2 faz o seguinte :

- (a) calcula $r_2 = \alpha_2 - c * k_{2,3}^{agg,2 \times 3}$,
- (b) e envia r_2 ao participante 1 de forma segura.

6. Cada participante calcula

$$r = \sum_e r_e$$

7. Cada participante pode publicar a assinatura $\sigma(\mathbf{m}) = (c, r)$.

A única mudança com multi-assinaturas de limite sub-N, é como ‘fechar o círculo’ ao definir $r_{\pi,e}$ (no caso de assinaturas em anel, com index secreto π). Cada participante tem de incluir o segredo partilhado correspondente á ‘pessoa desaparecida’, mas desde que todos os outros segredos partilhados são duplicados, existe um truque.

Dado o conjunto $\mathbb{S}^{base}$ de chaves originais de todos os participantes, só a *primeira pessoa* - ordenada por index em $\mathbb{S}^{base}$ - com uma cópia de um segredo partilhado uses it to calculate his $r_{\pi,e}$.^{24 25}

No exemplo prévio, o participante 1 calcula

$$r_1 = \alpha_1 - c * [k_{1,3}^{agg,2x3} + k_{1,2}^{agg,2x3}]$$

enquanto o participante 2 só calcula :

$$r_2 = \alpha_2 - c * k_{2,3}^{agg,2x3}$$

Em Monero o mesmo princípio aplica-se a calcular a imagem de chave de grupo em transacções de multi-assinatura com limites sub-N.

9.6.3 Agregação de chave M-de-N

Em (N-1)-de-N cada segredo partilhado entre duas chaves públicas, tal como K_1^{base} e K_2^{base} contêm duas chaves privadas $k_1^{base} k_2^{base} G$.

É um segredo porque só o participante 1 pode calcular $k_1^{base} K_2^{base}$, e só o participante 2 pode calcular $k_2^{base} K_1^{base}$. E se existe uma terceira pessoa com K_3^{base} , existem segredos partilhados

$$k_1^{base} k_2^{base} G, k_1^{base} k_3^{base} G, k_2^{base} k_3^{base} G.$$

E se os participantes enviam essas chaves públicas uns aos outros? Cada um contribui uma chave privada a duas chaves públicas. Agora diga-se que os participantes fazem um novo segredo partilhado com essa terceira chave pública.

O participante 1 calcula o segredo partilhado :

$$k_1^{base} * (k_2^{base} k_3^{base} G).$$

O participante 2 calcula o segredo partilhado :

$$k_2^{base} * (k_1^{base} k_3^{base} G).$$

O participante 3 calcula o segredo partilhado :

$$k_3^{base} * (k_1^{base} k_2^{base} G).$$

Agora cada um dos participantes sabe :

$$k_1^{base} k_2^{base} k_3^{base} G.$$

O que é um segredo partilhado entre os 3 participantes (desde que nenhum deles publique o segredo).

src/multi-
sig/multi-
sig.cpp
generate_
multisig-
deriv-
ations()

²⁴ Na prática isto significa que um iniciador devia determinar qual o sub-conjunto de M signatários irá assinar uma dada mensagem. Se ele descobre que nenhum signatário quer assinar, ou seja ($O \geq M$), ele pode orquestrar múltiplas tentativas simultâneas de assinar para cada sub-conjunto de tamanho M dentro de O para aumentar a probabilidade que uma funcione. Parece que Monero utiliza esta abordagem. Também acontece que (um ponto esotérico) a lista *original* de saídas destino, criadas pelo iniciador são misturadas de forma aleatória, e essa lista é então usada por cada tentativa de assinar, e todos os outros co-signatários (isto está relacionado com o **flag** obscuro **shuffle.outs**).

²⁵ Actualmente Monero usa um método signatário de *round-robin*, em que o iniciador assina com todas as suas chaves privadas, esta transacção parcialmente assinada é então enviada a outro signatário que assina com todas as suas chaves privadas (que ainda não foram utilizadas para assinar), que por sua vez envia esta transacção parcialmente assinada ao próximo signatário, e por assim diante, até ao signatário final. Este então pode enviar a transacção agora totalmente assinada á rede, ou a outro signatário para que este a envie á rede.

src/wallet/
wallet2.cpp
sign_multi-
sig.tx()

Este grupo podia usar :

$$k^{sh,1x3} = \mathcal{H}_n(k_1^{base} k_2^{base} k_3^{base} G),$$

como uma chave privada partilhada. Publica-se :

$$K^{grp,1x3} = \mathcal{H}_n(T_{agg}, \mathbb{S}^{1x3}, K^{sh,1x3}) K^{sh,1x3},$$

como um endereço de multi-assinatura 1-de-3.

Numa multi-assinatura 3-de-3 cada participante tem um segredo.

Numa multi-assinatura 2-de-3 cada sub-conjunto de 2 participantes partilha um segredo.

Para generalizar para M-de-N : cada possível sub-conjunto de (N-M+1) participantes, partilha um segredo [100]. Se (N-M) pessoas

algoritmo M-de-N

Sejam participantes $e \in \{1, \dots, N\}$ com chaves privadas iniciais $k_1^{base}, \dots, k_N^{base}$, que querem produzir uma chave M-de-N junta ($M \leq N$; $M \geq 1$ and $N \geq 2$). Utiliza-se então um algoritmo interactivo.

Utiliza-se $\mathbb{S}_s$ para denotar todas as chaves públicas *únicas* na iteração $s \in \{0, \dots, (N - M)\}$.

O conjunto final $\mathbb{S}_{N-M}$ está ordenado convencionalmente com números ascendentes ou lexicograficamente). Esta notação é uma conveniência, e $\mathbb{S}_s$ é o mesmo que $\mathbb{S}^{(N-s) \times N}$ das secções anteriores.

src/wallet/
wallet2.cpp
exchange_
multisig_
keys()

O conjunto de chaves públicas que cada participante gera na iteração s do algoritmo é denotado por :

$$\mathbb{S}_{s,e}^K.$$

No início :

$$\mathbb{S}_{0,e}^K = \{K_e^{base}\}.$$

O conjunto :

$$\mathbb{S}_e^k.$$

Irá no fim, conter as chaves privadas de agregação de cada participante.

1. Cada participante e envia o conjunto original de chaves públicas :

$$\mathbb{S}_{0,e}^K = \{K_e^{base}\},$$

aos outros participantes de forma segura.

2. cada participante constrói $\mathbb{S}_0$ ao coleccionar todos os $\mathbb{S}_{0,e}^K$ e remover duplicados.

3. Na iteração $s \in \{1, \dots, (N - M)\}$ (excepto $M = N$)

(a) Cada participante e faz o seguinte :

- i. Para cada elemento g_{s-1} de

$$\mathbb{S}_{s-1} \notin \mathbb{S}_{s-1,e}^K,$$

calcula-se um novo segredo partilhado :

$$k_e^{base} * \mathbb{S}_{s-1}[g_{s-1}]$$

- ii. Todos os novos segredos partilhados são postos em :

$$\mathbb{S}_{s,e}^K.$$

- iii. Se $s = (N - M)$,

calcula-se a chave privada partilhada para cada elemento :

$$x \text{ em } \mathbb{S}_{N-M,e}^K \cdot \mathbb{S}_e^k[x] = \mathcal{H}_n(\mathbb{S}_{N-M,e}^K[x])$$

e sobre escrever a chave pública ao pôr :

$$\mathbb{S}_{N-M,e}^K[x] = \mathbb{S}_e^k[x] * G.$$

- iv. Envia-se aos outros participantes :

$$\mathbb{S}_{s,e}^K.$$

(b) Cada participante constroi $\mathbb{S}_s$ ao coleccionar todos os $\mathbb{S}_{s,e}^K$, removendo duplicados. ²⁶

4. Cada participante ordena $\mathbb{S}_{N-M}$ segundo a convenção.

5. a função **premerge** tem como argumento $\mathbb{S}_{(N-M)}$, e cada chave de agregação é para $g \in \{1, \dots, (\text{size of } \mathbb{S}_{N-M})\}$:

$$\mathbb{K}^{agg, \text{MxN}}[g] = \mathcal{H}_n(T_{agg}, \mathbb{S}_{(N-M)}, \mathbb{S}_{(N-M)}[g]) * \mathbb{S}_{(N-M)}[g]$$

6. A função **merge** tem como argumento $\mathbb{K}^{agg, \text{MxN}}$, e a chave de grupo é :

$$K^{grp, \text{MxN}} = \sum_{g=1}^{\text{size of } \mathbb{S}_{N-M}} \mathbb{K}^{agg, \text{MxN}}[g]$$

7. Cada participante e substitui cada elemento x em $\mathbb{S}_e^k$ com a sua chave privada de agregação :

$$\mathbb{S}_e^k[x] = \mathcal{H}_n(T_{agg}, \mathbb{S}_{(N-M)}, \mathbb{S}_e^k[x]G) * \mathbb{S}_e^k[x]$$

9.7 Famílias de chaves

Até aqui foram consideradas agregações de chave entre um simples grupo de signatários. Por exemplo, a alice o bob e a carol cada um contribui componentes de chave a um endereço de multi-assinatura 2-de-3.

Agora imagine-se que a alice quer assinar *todas* as transacções desse endereço, e não quer que o bob e a carol assinem sem ela. Por outras palavras, (alice + bob) ou (alice + carol) são pares aceitáveis, mas não (bob + carol).

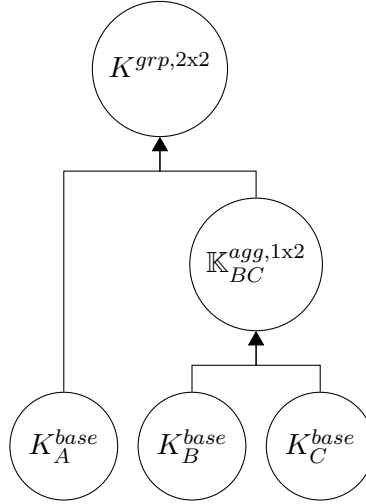
Isto pode ser conseguido com duas camadas de agregação de chave. Primeiro uma agregação de multi-assinatura 1-de-2 $\mathbb{K}_{BC}^{agg, 1 \times 2}$ entre o bob e a carol. Depois uma chave de grupo 2-de-2 $K^{grp, 2 \times 2}$ entre a alice e $\mathbb{K}_{BC}^{agg, 1 \times 2}$. Basicamente um endereço de multi-assinatura (2-de-([1-de-1] e [1-de-2])).

Isto implica que estruturas de acesso a direitos signatários podem ficar inacabados.

²⁶ Os participantes deviam saber quais participantes têm quais chaves no último passo ($s = N - M$), para facilitar as assinaturas colaborativas, em que só a primeira pessoa em $\mathbb{S}_0$ com uma certa chave privada é que assina. Veja-se secção 9.6.2.

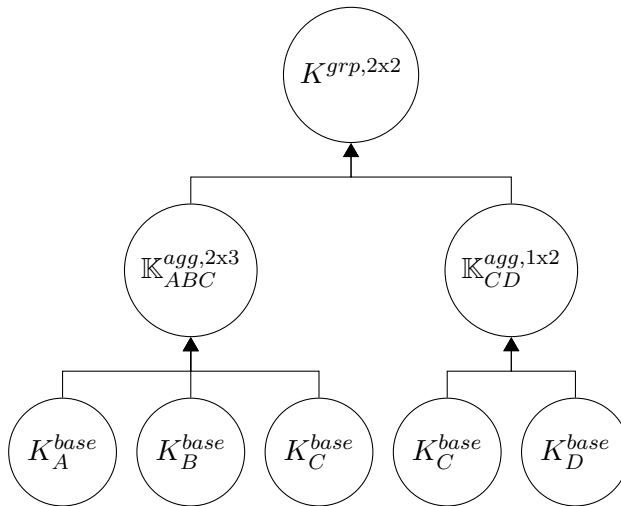
9.7.1 Árvores de família

O endereço de multi-assinatura (2-de-([1-de-1] e [1-de-2])) pode ser representado no seguinte diagrama :



As chaves K_A^{base} , K_B^{base} , K_C^{base} são consideradas *raízes ancestrais*, enquanto que $K_{BC}^{agg,1x2}$ é *filho* dos pais K_B^{base} e K_C^{base} . Pais podem ter mais de um filho, mas para a clareza conceptual nós consideramos cada cópia de um pai como distinta. Isto significa que podem haver múltiplas raízes ancestrais que são a mesma chave.

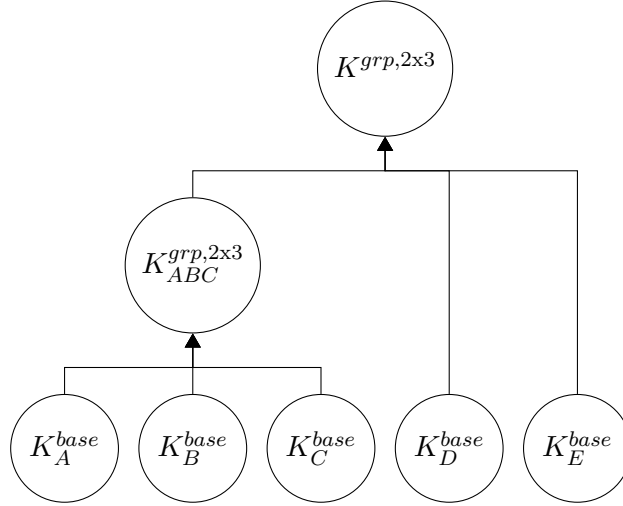
Por exemplo, neste 2-de-3 ou 1-de-2 junto a um 2-de-2, a chave da carol K_C^{base} é usada duas vezes e representada duas vezes :



Conjuntos distintos $\mathbb{S}$ são definidos para cada sub-coalição de multi-assinatura. Existem três conjuntos de premerge no exemplo anterior :

9.7.2 Chaves de multi-assinatura inclusivas

Seja a seguinte família de chaves



Se juntar-mos as chaves em $\mathbb{S}_{ABC}^{2x3}$ correspondentes aos primeiros 2-de-3, irá acontecer uma situação no próximo nível. Escolha-se só um segredo partilhado, entre $K_{ABC}^{grp,2x3}$ e K_D^{base} :

$$k_{ABC,D} = \mathcal{H}_n(k_{ABC}^{grp,2x3} K_D^{base})$$

Agora, duas pessoas de ABC podem facilmente contribuir componentes de agregação de chave, tal que a sub-coalição possa calcular :

$$k_{ABC}^{grp,2x3} K_D^{base} = \sum k_{ABC}^{agg,2x3} K_D^{base}$$

O problema é que qualquer participante de ABC pode calcular :

$$k_{ABC,D}^{sh,2x3} = \mathcal{H}_n(k_{ABC}^{grp,2x3} K_D^{base})!$$

Se todos os participantes de uma camada de multi-assinatura mais baixa, sabem todas as chaves privadas de uma camada de multi-assinatura mais elevada, então o esquema é escusado e é como se a camada de baixo fosse 1-de-N.

Isto é resolvido ao não juntar completamente as chaves até á chave final.

Em vez disso faz-se **premerge** para todas as chaves das camadas mais baixas.

Solução para a inclusão

Para usar $\mathbb{K}^{agg,M \times N}$ numa nova multi-assinatura, esta é passada entre os participantes, com uma alteração. Operações que envolvam $\mathbb{K}^{agg,M \times N}$ usam cada uma das suas chaves membro, em vez de uma chave junta de grupo. Por exemplo, a ‘chave’ pública de um segredo partilhado entre $\mathbb{K}_x^{agg,2x3}$ e K_A^{base} iria ser :

$$\mathbb{K}_{x,A}^{sh,1x2} = \{[\mathcal{H}_n(k_A^{base} \mathbb{K}_x^{agg,2x3}[1]) * G], [\mathcal{H}_n(k_A^{base} \mathbb{K}_x^{agg,2x3}[2]) * G], \dots\}$$

Desta forma todos os membros de :

$$\mathbb{K}_x^{agg, 2 \times 3},$$

só sabem os segredos partilhados correspondentes às suas chaves privadas da camada abaixo de multi-assinatura 2-de-3. Uma operação entre um conjunto de chaves de tamanho 2 ${}^2\mathbb{K}_A$ e um conjunto de chaves de tamanho 3 ${}^3\mathbb{K}_B$ iria produzir um conjunto de chaves de tamanho 6 ${}^6\mathbb{K}_{AB}$. Podemos generalizar todas as chaves numa família de chaves como conjuntos de chaves, em que chaves individuais são denotadas ${}^1\mathbb{K}$.

Elementos num conjunto de chaves são ordenados segundo uma convenção (por exemplo lexicográfica), e conjuntos que contenham conjuntos de chave (e.g. conjuntos $\mathbb{S}$), são ordenados pelo primeiro elemento em cada conjunto de chave. Deixa-se que os conjuntos de chave se propaguem pela estrutura de família, em que cada grupo de multi-assinatura inclusiva envia o seu conjunto de agregação **premerge** como a ‘chave base’ para a próxima camada.²⁷ Até que aparece o conjunto de agregação do último filho, em que finalmente é utilizado **merge**. Monerianos devem guardar as suas chaves privadas base, as chaves privadas de agregação para todas as camadas de uma estrutura de família de multi-assinatura, e as chaves públicas para todas as camadas. Ao fazer isto é mais fácil criar novas estruturas, fazer **merge** de multi-assinaturas inclusivas, e colaborar com outros signatários para reconstruir uma estrutura de família caso haja um problema.

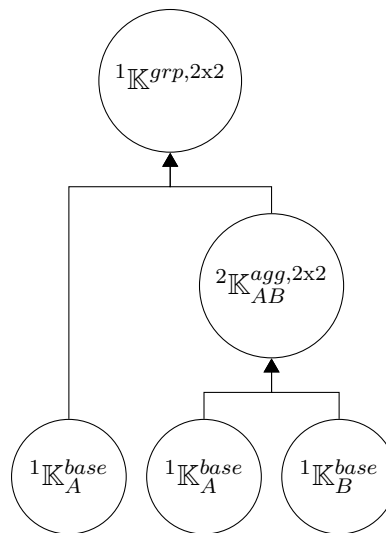
9.7.3 Implicações

Cada sub-coalição que contribui para a chave final precisa de contribuir componentes para uma transacção de Monero (tal como os valores iniciais αG), e como tal cada sub-sub-coalição precisa de contribuir para a sub-coalição filho.

Isto significa que cada raiz ancestral, mesmo quando existem múltiplas cópias da mesma chave na estrutura da família, tem de contribuir uma componente raiz ao filho do próximo nível. E cada filho por sua vez tem de contribuir uma componente ao seu filho. E isto para cada camada. São usadas somas simples a cada nível.

Por exemplo, seja esta família :

²⁷ Note-se que **premerge** precisa de ser feito às saídas de *todas* as multi-assinaturas inclusivas, mesmo quando uma multi-assinatura N-de-N está incluída em outra N-de-N, porque o conjunto $\mathbb{S}$ muda.



Seja que é necessário calcular um valor de grupo x para uma assinatura. Raízes ancestrais contribuem :

$$x_{A,1}, x_{A,2}, x_B.$$

O total é :

$$x = x_{A,1} + x_{A,2} + x_B.$$

Não existem actualmente implementações de multi-assinaturas inclusivas em Monero.

Mercados garantidos por terceiros

Na maioria das vezes, as compras online acontecem sem problemas. O comprador envia dinheiro ao vendedor, e o produto esperado aparece em casa. Se o produto tem um defeito, ou se o comprador muda de ideias, o produto pode ser devolvido, e o comprador é reembolsado. É difícil confiar numa pessoa ou organização desconhecida, e muitos compradores podem-se sentir seguros sabendo que as companhias de cartões de crédito irão possivelmente reverter as transacções [27].

As transacções de cripto-moedas não são reversíveis, e existe um recurso legal limitado para o comprador ou vendedor quando algo corre mal. Especialmente com Monero, porque as transacções não são analisadas facilmente por entidades terceiras [44].

Um aspecto central a fazer compras online são mercados garantidos por terceiros, em que são usadas multi-assinaturas 2-de-3. Assim entidades terceiras são capazes de mediar disputas. Ao confiar nessas entidades, vendedores e compradores anónimos conseguem interagir sem formalidades.

Em Monero as interacções com multi-assinaturas são complexas. Neste capítulo é descrito um mercado online eficiente garantido por terceiros ^{1,2}.

¹ René Brunner “rbrunner7”, um contribuidor de Monero que criou o MMS [107, 106], investigou integrar as multi-assinaturas em Monero no mercado digital baseado em cripto-moedas chamado *OpenBazaar* <https://openbazaar.org/>. Os conceitos aí apresentados são inspirados pelas dificuldades que René encontrou [108] (his ‘earlier analysis’).

² A implementação actual de multi-assinaturas em Monero, já tem a sequência de passos na criação de transacção, desejada para mercados online garantidos por terceiros. O que é bom, mas note-se que multi-assinaturas precisam de actualizações de segurança, antes de serem utilizadas em grande escala. [93].

10.1 Características essenciais

Existem vários requisitos básicos para as interações entre vendedores e compradores em mercados online. Estes pontos são da investigação de René acerca de OpenBazaar [108].

- *venda offline*: Um comprador devia poder aceder á loja online, enquanto o vendedor está offline, e fazer uma compra. É claro que o vendedor tem de ir online para receber a compra e finalizar a venda. Ou seja enviar o produto ao comprador.
- *Compras com pagamentos ordenados*: Os endereços do vendedor são únicos para cada venda, de forma a ligar cada venda com o seu pagamento.
- *Compras de alta confiança*: Um comprador pode, se confia no vendedor, pagar por um produto mesmo antes deste estar disponível ou ser entregue.
 - *pagamento online directo*: Depois de confirmar que o vendedor está online, e que o produto listado está disponível, o comprador envia o dinheiro numa só transacção a um endereço do vendedor, e este depois sinaliza que a compra está a ser efectuada.
 - *Pagamento offline*: Se o vendedor está offline, o comprador cria e envia um montante para um endereço multi-assinatura 1-de-2, com dinheiro suficiente para finalizar a compra. Quando o vendedor volta a estar online, ele pode tirar o dinheiro do endereço de multi-assinatura, e executar a venda. Se o vendedor nunca mais volta a estar online (ou depois de um certo tempo de espera), ou se o comprador muda de ideias antes dele voltar a estar online, o comprador pode tirar o dinheiro desse endereço de multi-assinatura de volta para a sua carteira pessoal.
- *Compra através de um moderador*: Um endereço de multi-assinatura 2-de-3 é construído entre o comprador o vendedor e o moderador. O moderador é aceite pelo comprador e pelo vendedor. O comprador envia dinheiro para este endereço antes do vendedor enviar o produto. Uma vez que o produto tenha sido entregue ao comprador, os dois partidos cooperam para que os fundos sejam libertados. Se o comprador e o vendedor não chegam a um acordo, estes podem delegar o julgamento ao moderador.

10.1.1 Sequência de passos na compra

Todas as compras deviam caber dentro do mesmo conjunto de passos, assumindo que todos os partidos agem com a diligência devida. Um moderador antes de servir dois partidos, espera um número de passos já executados, por exemplo : "antes de envolverem um moderador, um dos partidos já pediu um reembolso ?"

10.2 Multi-assinatura Monero

Uma interacção de multi-assinatura 2-de-3 pode ser feita quase completamente ao longo da sequência de passos na compra sem que os participantes notem. Existe um passo extra a fazer pelo vendedor

no fim. Em que ele tem de assinar e submeter a transacção final para receber o pagamento. O que é comparável a retirar o dinheiro todo da caixa registadora.³

10.2.1 Os básicos de uma interacção de multi-assinatura

Todas as interacções de multi-assinatura 2-de-3 contêm o mesmo conjunto de rondas de comunicação, o que envolve a construção do endereço e depois da construção da transacção. De forma a cumprir com a sequência de passos usa-se um outro processo de construção do que aquele dado no capítulo de multi-assinatura 9.4.2 e 9.6.2.

4

1. Construir o endereço

- (a) Todos os participantes têm de primeiro conhecer as chaves base dos outros participantes, o que irá ser usado para gerar segredos partilhados.

As chaves públicas desses segredos partilhados :

$$K^{sh} = \mathcal{H}_n(k_A^{base} * k_B^{base} G)G.$$

são enviados aos outros participantes.

- (b) Depois de um participante ter aprendido todas as chaves públicas de segredo partilhado, ele pode executar a função **premerge** e depois **merge**, o que gera a chave de gasto do endereço de grupo. As chaves agregadas privadas de gasto, irão ser usadas para assinar transacções.

Uma hash da chave privada do segredo partilhado :

$$k^{sh} = \mathcal{H}_n(k_A^{base} * k_B^{base} G),$$

irá ser usada como a chave de ver :

$$k^v = \mathcal{H}_n(k^{sh}).$$

2. *Construção da Transacção* : Seja que o endereço já possui pelo menos uma saída, e as imagens de chave são desconhecidas.

Existem dois signatários, o iniciador e o cosignatário.

- (a) *Iniciar uma transacção*: O iniciador decide começar uma transacção. Ele gera valores de abertura (e.g. αG) para todas as saídas possuídas, e compromete-se a estas saídas. Ele depois cria imagens de chave parciais para essas saídas possuídas, e assina essas imagens com uma prova de legitimidade (secção 9.5).

Ele depois envia essa informação, juntamente com o seu endereço para receber montantes, ao cosignatário.

- (b) *Fazer uma transacção parcial*: O cosignatário verifica que toda a informação na transacção inicial é válida. Ele depois decide os montantes e as saídas destinatárias, bem

³ Estender isto para além de (N-1)-de-N é provavelmente infazível sem adicionar mais passos, devido às rondas adicionais necessárias para construir um endereço de multi-assinatura com uma barreira (M) mais baixa.

⁴ Este processo é actualmente bastante similar á organização de transacções de multi-assinaturas em Monero.

como as entradas e os membros desvio de anel, e a taxa de transacção.

Ele gera uma chave privada de transacção, cria endereços ocultos, compromissos de saída, montantes ocultos, máscaras de compromisso de pseudo saídas, e valores iniciais para os compromissos a zero.

Para provar que os montantes estão dentro do domínio, ele constroi a prova de domínio tipo Bulletproof para cada saída.

Ele também gera valores de abertura para a sua assinatura sem compromissos, escalares aleatórios para a assinatura MLSAG, imagens de chave parciais para saídas possuídas, e provas de legitimidade para essas imagens parciais. Tudo isto é enviado ao iniciador.

- (c) *Assinatura parcial do iniciador* : O iniciador verifica que a informação da transacção parcial é válida, e é conforme as suas expectativas (os montantes e os recipientes são o que devem ser). O iniciador completa as assinaturas MLSAG e assina-as com as suas chaves privadas, depois envia esta transacção parcialmente assinada ao cosignatário juntamente com os valores de abertura revelados.
- (d) *Finalizar a transacção*: O cosignatário finaliza assinar a transacção, e submite isso á rede.

Assinaturas de compromisso único

Ao contrário daquilo que é recomendado no capítulo de multi-assinatura, só um compromisso é dado por cada transacção parcial (pelo iniciador da transacção). E este compromisso é revelado depois do cosignatário enviar para fora esse valor de abertura explicitamente. O propósito de comprometer a valores de abertura (e.g. αG) é de prevenir que um cosignatário malicioso possa usar o seu próprio valor de abertura para alterar o desafio que é produzido. Isto permitiria ao cosignatário malicioso de descobrir as chaves de agregação dos outros cosignatários (secção 9.3). É necessário somente um valor de abertura desconhecido, quando o actor malicioso gera o seu valor de abertura, para que lhe seja impossível controlar o desafio. ^{5 6}

7

⁵ Isto é também o porque do iniciador só revelar os seus valores de abertura depois de toda a informação de transacção ter sido determinada, assim nenhum signatário consegue alterar a mensagem MLSAG e influenciar o desafio.

⁶ Assinar com só um compromisso, pode ser generalizado a assinar com (M-1) compromissos em que só o autor da transacção parcial é que não compromete-e-revela, e os restantes co-signatários só revelam quando a transacção está inteiramente determinada. Por exemplo, suponha-se que existe um endereço 3-de-3 com os co-signatários (A,B,C), que irão tentar assinar com só um compromisso. Os signatários B e C estão numa coalição maliciosa contra A, enquanto que C é o iniciador e B é o autor parcial da transacção. C inicia com um compromisso, e depois A oferece o seu valor de abertura (sem compromisso). Quando B constroi a transacção parcial, ele pode conspirar com C para controlar o desafio da assinatura de forma a expor a chave privada de A. Note-se também que assinar com (M-1) compromissos é um conceito original apresentado primeiro aqui. Ou seja não foi pesquisado nem investigado em outro lado por relatórios abstractos, nem existe implementação disto em código. Como tal, isto poderá vir a ser totalmente erróneo.

⁷ Uma forma de pensar sobre isto é de considerar o significado e propósito de um ‘compromisso’ (secção 5.1). Uma vez que a alice se compromete ao valor A, ela fica presa a esse compromisso, e não pode ter vantagens de informações do evento B (causado por bob) que acontece mais tarde. Para mais, se A ainda não foi revelado, então

Simplificar desta forma remove uma ronda de comunicação, o que tem consequências importantes para a experiência de interação entre o comprador e o vendedor.

10.2.2 Experiência com garantia para o utilizador

Aqui está uma compra online que envolve uma multi-assinatura 2-de-3, apresentada em detalhe e passo a passo. As interações seguintes são entre o comprador, o vendedor e o moderador.

Existem quatro optimizações instaladas.

moderadores pre-seleccionados

Ao escolher moderadores, vendedores podem criar um segredo partilhado com cada um deles, e para cada produto e publicar essa chave pública com a informação do produto.⁸ Desta forma os compradores podem construir um endereço junto de multi-assinatura num só passo, assim que decidam comprar algo. Os compradores podem optar entre vários moderadores, o que lhes permite escolher aquele em que mais confiam.

Os compradores podem também, se não confiam nos moderadores aceites pelo vendedor, escolher e cooperar com um outro moderador online para fazer um endereço de multi-assinatura com a chave base do produto em causa. Depois de receber a ordem de compra, o vendedor pode aceitar esse novo moderador ou rejeitar a venda.⁹

Espera-se que a necessidade mútua que compradores e vendedores têm para utilizarem bons moderadores, gere ao longo do tempo uma hierarquia de moderadores organizados pela qualidade e justiça dos seus serviços. Moderadores com uma reputação menor irão receber menos dinheiro e servir para menos transacções e também de montantes mais baixos.¹⁰

B não é influenciado por A. A alice e o bob têm a certeza de que A e B são independentes. Uma Assinatura de só um compromisso satisfaz esse standard, e é equivalente a assinaturas com todos os compromissos. Se o compromisso c é uma função unidireccional de valores de abertura $\alpha_A G$ e $\alpha_B G$ (e.g. $c = \mathcal{H}_n(\alpha_A G, \alpha_B G)$), então se inicialmente é comprometido a $\alpha_A G$, e $\alpha_B G$ é revelado depois de $C(\alpha_A G)$ aparecer. E $\alpha_A G$ é revelado depois de $\alpha_B G$ aparecer, então $\alpha_B G$ e $\alpha_A G$ são independentes. Como tal c , irá ser aleatório da perspectiva da alice e do bob, excepto se ambos colaborarem, e (enp).

⁸ Estes endereços de multi-assinatura ainda são resistentes a testes de agregação de chave, porque os segredos partilhados do comprador são desconhecidos aos observadores.

⁹ Para uma experiência mais fluída, um serviço de garantia poderia estar ‘sempre online’, e em vez de ser usado um moderador pre-seleccionado, todos os endereços de multi-assinatura 2-de-3 são activamente construídos com esse serviço, quando uma ordem de compra é feita. Uma outra opção é utilizar multi-assinaturas inclusivas (secção 9.7), em que o moderador pre-seleccionado é actualmente um grupo de multi-assinatura 1-de-N. Desta forma sempre que há uma disputa, qualquer moderador desse grupo, que está disponível nesse momento pode intervir. Isto provavelmente requer um esforço substancial para implementar.

¹⁰ Não é claro qual será o melhor método de financiamento para os moderadores. Talvez eles recebem uma taxa fixa ou uma percentagem de cada transacção que moderam, ou então cada vez que são seleccionados como moderadores (e se a taxa não está presente na transacção original parcial, o moderador recusa a disputa). Ou então os utilizadores destes mercados com garantia estabelecem contratos com os moderadores.

Subendereços e ID's de produto

Vendedores criam uma nova chave base para cada ID de um produto, e essas chaves são usadas para construir endereços de multi-assinatura 2-de-3.¹¹ Quando um vendedor serve uma ordem de compra, este cria um subendereço para receber os fundos, o que pode ser usado para ligar ordens de compra aos pagamentos recebidos. O requisito para 'pagamentos baseados na ordem de compra' é satisfeito de forma eficaz, também porque fundos dirigidos a sub-endereços diferentes são trivialmente acessíveis da mesma carteira (secção 4.3).

Transacções parciais antecipadas

Transacções de multi-assinatura levam várias rondas, portanto estas são feitas assim que possível. Para a conveniência do utilizador, transacções parciais que só são usadas raramente (e.g. reembolsos) são feitas de forma antecipada. E assim estas estão disponíveis de imediato para assinar, quando for preciso.

Acesso condicional do moderador

Para a eficiência e privacidade, os moderadores só precisam de acesso aos detalhes de um negócio quando há a necessidade de resolver disputas. Para alcançar isto, a chave privada de ver de multi-assinatura, é uma hash da chave privada do segredo partilhado :

$$k_{purchase-order}^{v,grp} = \mathcal{H}_n(T_{mv}, k_{AB}^{sh,2x3}).$$

Em que T_{mv} é a chave de ver que separa domínios no mercado, e A e B correspondem com o vendedor e comprador, e

$$k_{AB}^{sh,2x3} = \mathcal{H}_n(k_A^{base} * k_B^{base} G).$$

Extende-se o que pode *ver* à transcrição de comunicação entre comprador e vendedor. Por outras palavras, a chave que encripta essa comunicação é :

$$k_{ordem-de-compra}^{ce} = \mathcal{H}_n(T_{me}, k_{ordem-de-compra}^{v,grp}),$$

e T_{me} é a chave que encripta o separador de domínio do mercado.¹²¹³ Os moderadores ganham acesso à comunicação entre o comprador e o vendedor, e a habilidade de autorizar pagamentos, só quando um dos partidos lhes liberta a chave de ver.¹⁴

¹¹ Esta chave base também é utilizada para comprar com multi-assinaturas 1-de-2. É importante não expor a chave privada de gasto no canal de comunicação, portanto usar um segredo partilhado entre o comprador e o vendedor, faz sentido.

¹² Separar a chave de ver e a chave de encriptação permite oferecer direitos de ver ao histórico de comunicações sem dar direitos de ver ao histórico de transacções do endereço de multi-assinaturas.

¹³ Este método também é utilizado para endereços de multi-assinatura 1-de-2.

¹⁴ É importante que os moderadores verifiquem que o histórico de comunicação que eles recebem não foi falsificado. Uma forma seria que cada co-signatário incluísse uma hash assinada da mensagem com o histórico, sempre que este é enviado para fora. Os moderadores podem olhar para esta comunicação, com as hashes dos históricos para identificar discrepâncias. Também ajudaria aos co-signatários identificar mensagens que falharam na transmissão, e alternativamente criar a evidência que certos co-signatários receberam de facto certas mensagens.

Para mais, vendedores podem verificar que o anfitrião do mercado online (que pode também ser o único moderador disponível, dependendo da forma como isto é implementado) não é MITM (‘homem no meio’) das suas comunicações com os clientes. Os vendedores verificam que as chaves base que eles publicam para cada produto é igual aquele que é mostrado publicamente no mercado. Como a chave base do comprador, que é usada para criar o endereço de multi-assinatura, também é parte da chave de encriptação, um anfitrião malicioso não consegue orquestrar um ataque MITM (enp).

CAPÍTULO 11

Transacções juntas (TxTangle)

Existem algumas heurísticas para construir grafos de transacções, que são inevitáveis e que resultam da natureza de certos cenários e entidades. Em particular, o comportamento de mineiros, piscinas (do inglês : pools; secção 5.1 de [84]), mercados online e bolsas de câmbio apresentam padrões abertos a análise apesar do protocolo privado de Monero.

Descreve-se aqui *TxTangle*, análogo ao *CoinJoin* de Bitcoin um método para confundir essas heurísticas.

1

Em essência, várias transacções são juntas para formar uma transacção, tal que os padrões de cada participante, derretam juntamente.

Para alcançar tal obfuscação, tem de ser praticamente impossível para que observadores usem essa informação contida numa transacção junta para associar entradas e saídas a participantes individuais ou até de saber quantos participantes estão envolvidos.

2

Para além disso, mesmo os próprios participantes não deviam estar a par desse número. Nem de associar entradas e saídas a outros participantes (excepto em certos cenários).³ Finalmente

¹ Este capítulo constitui a proposta para um protocolo de transacções juntas. Isto até á data ainda não foi implementado. Uma proposta prévia, chamada *MoJoin* e criada pelo co-autor, investigador do laboratório de pesquisas de Monero (MRL), de pseudónimo Sarang Noether, requeria uma entidade terceira de confiança para funcionar. Como tal *MoJoin*, não foi mais desenvolvido, e não está implementado.

² Como em Bitcoin os montantes são claramente visíveis, é muitas vezes possível agrupar saídas e entradas de transacções com base na soma dos montantes [31].

³ Poluir transacções juntas de forma maliciosa é um potencial ataque a este método, e que foi primeiro identificado para CoinJoin. [82]

devia ser possível de construir transacções juntas sem ter de depender numa autoridade central [53]. Felizmente todos esses requisitos são alcançados em Monero.

11.1 Construir transacções juntas

Numa transacção normal, entradas e saídas são construídas usando a prova de que os montantes se igualam. Da secção 6.1.1, a soma de pseudo compromissos de saída é igual á soma de compromissos de saída (mais o compromisso da taxa).

$$\sum_j C_j^{ta} - (\sum_t C_t^b + fH) = 0$$

Uma transacção junta trivial podia pegar todo o conteúdo de múltiplas transacções, e juntá-las. As mensagens MLSAG assinariam todos os dados das sub-transacções, e os montantes iriam-se igualar ($0 + 0 + 0 = 0$).

⁴ Porém, grupos de entradas e saídas seriam identificados trivialmente, porque seria possível testar se conjuntos de entradas e saídas têm montantes que se igualam. ⁵

É possível dar a volta a isto ao calcular segredos partilhados entre pares de participantes, ao adicionar estes offsets ás máscaras de pseudo compromissos de saída (Section 5.4). Em cada par, um participante adiciona o segredo partilhado a um dos seus pseudo compromissos de saída, e um outro participante subtrai isso de um dos *seus* pseudo compromissos. Na sua soma, os segredos partilhados cancelam-se mutuamente.

E desde que cada par de participantes tem um segredo partilhado, o montante só aparece depois de todos os compromissos serem combinados. ⁶

11.1.1 Canal de comunicação em grupo

O número máximo de participantes para um *TxTangle* é o número de entradas ou de saídas, dependendo de qual é menor. Isto é modelado através de cada participante real pretender ser uma pessoa diferente para cada saída que ele envia. Isto é feito para estabelecer um canal de comunicação em grupo com outros potenciais participantes, sem que seja revelado quantos participantes existem.

Imagine-se n ($2 \leq n \leq 16$, apesar de que pelo menos 3 é recomendado) pessoas supostamente não relacionadas juntam-se a um *chatroom*. Agendado para abrir no tempo t_0 e fechar no tempo t_1 . Só 16 pessoas podem estar no *chatroom* ao mesmo tempo. E existem condições como prioridades de taxa, taxa base por cada byte e tipos de transacção aceites. Ao tempo t_1 todos os membros

⁴ Desde que provas de domínio do tipo *Bulletproof* são agregadas para uma só (secção 5.5), os participantes teriam de colaborar entre si até um certo grau, mesmo no caso trivial.

⁵ Os participantes poderiam tentar dividir a taxa de uma forma esperta para confundir os observadores, mas isto iria falhar á face da *força bruta*, pois as taxas não são assim tão grandes (por volta de 32 bits ou menos).

⁶ O *offset* é feito dos pseudo compromissos de saída, em vez dos compromissos de saída, pois as máscaras dos compromissos de saída são construídas a partir do endereço do destinatário (secção 5.3).

signalam que querem proceder ao publicarem uma chave pública, e o *chatroom* é convertido para um canal de comunicação de grupo ao construir um segredo partilhado entre todos os membros desvio.^{7 8} Este segredo partilhado é usado para encriptar conteúdos de mensagens, enquanto que membros desvio assinam mensagens relacionadas a entradas com assinaturas SAG (secção 3.3). Portanto não é claro quem é que enviou uma dada mensagem.

As mensagens relacionadas com as saídas usam uma assinatura tipo bLSAG (secção 3.4) no conjunto das chaves públicas dos membros desvio tal que as saídas não estão associadas aos membros desvio.⁹

11.1.2 Rondas de mensagens para construir uma transacção junta

Depois do canal estar pronto, transacções de *TxTangle* podem ser construídas em cinco rondas de comunicação, em que a próxima ronda só pode começar se a ronda prévia foi finalizada. Em cada ronda as mensagens são publicadas dentro de um certo intervalo de tempo limite, e o tempo de publicação de cada uma é aleatório. Isto serve para prevenir que grupos de mensagens revelem conjuntos de saída/entrada.

1. Cada membro desvio gera privadamente um escalar aleatório para cada saída, e assina esta com uma assinatura tipo bLSAG. Uma lista ordenada destes escalares serve para determinar indexes de saída. O menor escalar recebe o index $t = 0$ (secção 4.2.1).

¹⁰ Essas assinaturas são publicadas, e também assinaturas SAG que assinam o número da versão de entradas de transacção. Depois desta ronda os participantes podem calcular o peso de transacção baseado no número de entradas e saídas, e estimar precisamente a taxa necessária.^{11 12 13}

2. Cada membro desvio utiliza a lista de chaves públicas para construir o segredo partilhado com cada outro membro. Isto é feito para fazer um offset dos pseudo compromissos de saída. É decidido quem adiciona ou subtrai baseado na chave pública menor entre cada par.

⁷ Actualmente uma transacção pode ter no máximo 16 saídas.

⁸ O método de multi-assinatura da secção 9.6.3 é uma forma, que estende M-de-N até 1-de-N.

⁹ Cada conjunto separado de assinaturas TxTangle bLSAG, devia usar as mesmas imagens de chave, desde que tudo relacionado com uma dada saída está ligado.

¹⁰ A selecção do index de saída devia ser igual a outras implementações que constroem transacções, para evitar que software diferente seja identificado. Utiliza-se efectivamente uma abordagem aleatória para alinhar com a implementação núcleo, que também mistura as saídas de forma aleatória.

¹¹ Se acontece que só podem haver dois participantes reais, baseado na comparação do número de entradas e saídas total, com o número de entradas/saídas do próprio participante, o TxTangle pode ser abandonado. Recomenda-se que cada participante tenha pelo menos duas entradas e duas saídas, no caso de actores maliciosos que não abandonam TxTangles mesmo quando reconhecem que só existem dois participantes. Esta recomendação está aberta ao debate, pois usar mais entradas e saídas não é heurísticamente neutral.

¹² Transacções de TxTangle não deviam ter informação adicional no campo *extra* (e.g. nenhum ID de pagamento, a não ser que seja um TxTangle com só duas saídas, que deve ter pelo menos um ID de pagamento desvio encriptado)

¹³ A estimação da taxa devia ser baseada numa abordagem standard, assim cada participante calcula a mesma coisa. De outra forma as saídas podem ser agrupadas com base no método do cálculo da taxa. Este standard da taxa devia ser implementado fora de TxTangle, para promover que transacções de TxTangle se pareçam iguais a transacções normais.

Cada membro desvio tem de pagar $1/n$ da taxa estimada (usando divisão inteira). O membro desvio com o index de saída menor, tem a responsabilidade de pagar o resto da divisão inteira (o que deve ser um montante infinitesimal, mas tem de ser posto em conta para evitar que a transacção seja reconhecida como junta).

Eles geram privadamente chaves públicas de transacção para cada uma das suas saídas (por enquanto não se envia isto a outros membros), e constroem os compromissos de saída, montantes codificados, a parte A de provas parciais para serem utilizadas para a prova agregada de domínio *Bulletproof*, e assinar isto tudo com bLSAGs (um compromisso, um montante encriptado, uma prova parcial por mensagem bLSAG, e a imagem de chave liga estes dados á lista original de escalares aleatórios que foi utilizado para especificar índices de saída). Pseudo compromissos de saída são gerados normalmente (secção 5.4), depois o offset com o segredo partilhado, e assinado com SAGs. Depois de bLSAGs e SAGs serem publicados, e assumindo que os participantes estimaram o total em taxas da mesma forma, eles podem agora verificar que os montantes em geral se igualam.

14 15

3. Se os montantes se igualam propriamente, começa uma ronda para construir as provas de domínio agregadas, o que prova que todas os montantes de saída estão no domínio.

Cada membro desvio usa as provas parciais e os compromissos de saída da ronda parte A, e calcula privadamente o desafio agregado A. Depois constroem a prova parcial parte B que depois é enviada para o canal usando uma assinatura tipo bLSAG.

4. Os participantes começam a preencher a mensagem para ser assinada com assinaturas do tipo MLSAG. Dois tipos de mensagens são publicados em ordem aleatória dentro do intervalo de comunicação.

Cada offset de entrada que pertence a um anel, e as imagens de chave são assinadas com uma assinatura tipo SAG, e associadas com o pseudo compromisso de saída correcto.

Cada endereço oculto de uma saída, chave pública de transacção, e a prova parcial parte C são assinados com uma assinatura do tipo bLSAG. A prova parcial parte C é calculada com base nas provas parciais da parte B, e com o desafio agregado B. Pode também existir uma componente aleatória que é uma chave pública de transacção, esta serve para mitigar o "Janus falso".

5. Os participantes usam as provas parciais para completar a prova de domínio agregada tipo *Bulletproof*. E cada um aplica uma técnica de produto interno logarítmico para que a compressão e a prova final seja incluída nos dados de transacção. Uma vez que todas as informações a serem assinadas por assinaturas do tipo MLSAG estão presentes. Cada participante completa as assinaturas MLSAG ás entradas e envia isso (com uma assinatura SAG

¹⁴ Visto que os pontos de CE são comprimidos (secção 2.4.2), as chaves são interpretadas como inteiros de 32 bytes. É convenção que o dono da chave menor adiciona, e o dono da chave maior subtrai.

¹⁵ Não se publicam os montantes das taxas separadas pagas, no caso em que um participante se enganou no cálculo da taxa. O que pode revelar um grupo de saídas, devido aos montantes estranhos que sobressaem. Se os montantes não...?

para cada) para o canal de comunicação.

Cada participante pode submeter a transacção quando quiser.

Chaves públicas de transacção e a mitigação de Janus

Se cada participante de um *TxTangle* sabe a chave privada de transacção r (Section 4.2), então qualquer um deles pode testar os endereços ocultos de saída, contra uma lista de endereços conhecidos. Por causa disto é necessário construir transacções de *TxTangle* como se existisse um recipiente com um sub-endereço (secção 4.3), e também diferentes chaves públicas de transacção para cada saída.

Para complementar uma implementação possível de uma mitigação do ataque de Janus a sub-endereços, em que uma adicional chave pública de transacção ‘base’ é incluída no campo extra [92], transacções do tipo *TxTangle* também devem ter uma falsa chave pública composta por uma soma de chaves aleatórias geradas por cada membro desvio. Muitos participantes de uma transacção *TxTangle* que enviam dinheiro para um sub-endereço irão ter provavelmente duas saídas. Uma das quais diverte o troco de volta para o participante. Isto significa que cada participante pode activar uma mitigação de Janus ao tornarem a chave pública de transacção do troco também a chave ‘base’ para o recipiente do sub-endereço.

¹⁶ ¹⁷ O recipiente do sub-endereço poderá realizar que a transacção é do tipo *TxTangle*, e que a chave ‘base’ provavelmente corresponde á saída de transacção do troco do participante. ¹⁸

11.1.3 Fraquezas

Actores maliciosos têm duas opções para derrotar o propósito de transacções do tipo *TxTangle*, que é de esconder conjuntos de saídas/entradas de transacção de analistas ou adversários. As transacções podem ser poluídas, tal que o conjunto de participantes é menor ou até não existente [82]. Eles podem também tentar fazer com que tentativas de transacção *TxTangle* falhem. E usar as tentativas subsequentes pelos mesmos participantes para estimar conjuntos de saídas/entradas.

O primeiro caso não é fácil de mitigar, especialmente no caso descentralizado em que nenhum participante tem uma reputação. Uma possível aplicação de *TxTangle* é a de piscinas colaborativas,

¹⁶ O cancelamento de chave (secção 9.2.2) não devia ser um problema, desde que é só uma chave falsa e devia idealmente ser indexada de forma aleatória, entre a lista de chaves públicas de transacção.

¹⁷ Se alguém envia para o seu próprio sub-endereço, não existe a necessidade para uma mitigação de Janus. As carteiras que estão activadas para a mitigação de Janus deviam reconhecer que o montante gasto numa transacção *TxTangle*, é igual ao montante recebido pelo próprio sub-endereço. Para que o utilizador não seja notificado, de forma errónea, de um problema.

¹⁸ Isto assume que as chaves públicas de transacção são 1:1 com as saídas, como aparenta ser o caso actualmente. Se fosse standard para as chaves públicas de transacção estarem numa ordem aleatória ou estrita dentro do campo *extra*, então as transacções *TxTangle* e não *TxTangle* seriam largamente indistinguíveis para os destinatários com sub-endereços. Existem casos específicos em que os participantes de *TxTangle* são incapazes de incluir uma chave ‘base’ (e.g. quando todas as saídas são para sub-endereços), ou quando se trata claramente de uma transacção não *TxTangle*, pois o destinatário do sub-endereço recebe a maioria ou todas as saídas. Note-se que transacções *TxTangle* geralmente têm muito mais saídas do que uma transacção típica, esta observação pode ser usada para diferenciar transacções normais de *TxTangles*.

que podem esconder a qual piscina pertencem os mineiros. Tais piscinas saberiam os conjuntos de entradas/saídas, mas desde que o propósito é ajudar os mineiros a elas ligados, existe uma motivação de os ajudar e de manter esta informação secreta. Para mais, tais transacções *TxTangle* não permitiriam maus actores, assumindo que as piscinas são honestas.

O segundo caso pode ser evitado ao só tentar participar em transacções *TxTangle* poucas vezes antes de fazer uma pausa. E sempre que novas tentativas são feitas utilizar os elementos constituintes da transacção, regenerados de forma aleatória. Estes elementos são : as chaves públicas de transacção, máscaras de pseudo compromissos, escalares da prova de domínio, e escalares ML-SAG. Em particular, o conjunto de membros desvio de anel para cada entrada devia permanecer o mesmo para prevenir comparações cruzadas que revelem a verdadeira entrada. Se possível, diferentes entradas verdadeiras deveriam ser utilizadas para tentativas de *TxTangle* diferentes. Desde que esta fraqueza é inevitável, torna a próxima secção mais atrativa.

11.2 Servidor TxTangle

TxTangle descentralizado tem algumas questões abertas. Como é que as rondas são iniciadas e enforçadas? Como é que são criados *chatrooms*, para que os participantes se encontrem mutuamente? A maneira mais simples é através de um servidor de *TxTangle*, que gera e mantém esses *chatrooms*.

Um servidor desses parece desafiar o objectivo da participação obfuscada, desde que cada individuo tem de se ligar, e enviar mensagens que podem ser usadas para correlar entradas e saídas. Pode-se usar uma rede do tipo I2P para fazer com que cada mensagem recebida pelo servidor aparente provenir de um individuo único.

¹⁹

11.2.1 Comunicação básica com um servidor sobre I2P, e outras características

Com I2P, utilizadores fazem assim chamados ‘túneis’ que passam mensagens encriptadas por outros clientes antes de chegarem á destinação. Daquilo que se percebe, estes túneis podem transportar múltiplas mensagens antes de serem destruídos e recriados. É essencial para este uso, que haja um controlo cuidado sobre a abertura e o fecho dos túneis, bem como quais mensagens saem do mesmo túnel. ²⁰

1. *Inscrever-se para TxTangles*: Na nossa proposta original com n direcções (secção 11.1.1) os participantes gradualmente adicionam os seus membros desvio a quartos de TxTangle

¹⁹ O projecto invisível da internet (<https://geti2p.net/en/>).

²⁰ Em I2P existem túneis de ‘entrada’ e de ‘saída’ (see <https://geti2p.net/en/docs/how/tunnel-routing>). Tudo o que é recebido através de um túnel de entrada aparenta ser da mesma fonte mesmo que existam múltiplas fontes. Portanto á superfície parece que utilizadores de TxTangle não precisam de criar túneis distintos para cada um dos seus usos. Contudo se o anfitrião TxTangle faz com que ele próprio seja o ponto de entrada para o seu próprio tunel de entrada, então ele ganha o acesso directo aos túneis de saída dos participantes de TxTangle.

disponíveis antes destes estarem agendados para fechar. Contudo se um volume suficientemente grande de utilizadores tenta fazer um TxTangle ao mesmo tempo, é provável de haver muitas falhas. Os utilizadores tentam de forma aleatória por todas as suas saídas no mesmo ‘quarto’ de TxTangle, mas assim os quartos enchem-se demasiadamente rápido, e como tal os utilizadores teriam de abortar o TxTangle.

Pode-se fazer uma optimização ao dizer ao anfitrião quantas saídas estão em causa (e.g. dando-lhe uma lista das chaves públicas dos membros desvio), e deixar que ele junte os participantes para o TxTangle. Desde que os participantes retêm o protocolo de mensagens SAG e bLSAG, o anfitrião não será capaz de identificar conjuntos de saídas na transacção final. Tudo o que ele sabe é o número de participantes, e quantas saídas cada um tinha. Para mais, neste cenário os observadores não conseguem monitorizar quartos de TxTangle abertos para inferir qualquer informação sobre os participantes, uma melhoria importante na privacidade. Note-se que o poder do anfitrião de polluir os TxTangles não é significamente diferente do modelo sem anfitrião, portanto esta mudança é neutral para esse vector de ataque.

2. *Método de comunicação:* Como o anfitrião já actua como o lugar de transporte das mensagens, é simples que seja ele a gerir a comunicação de TxTangle. Durante cada ronda o anfitrião colecciona mensagens dos membros desvio (estas comunicações ocorrem sobre intervalos de tempo aleatórios). E no fim de uma ronda existe uma curta phase de distribuição em que ele envia os dados coleccionados a cada participante, a isto dá-se um certo tempo de espera antes da próxima ronda para que se garanta que as mensagens foram recebidas e que houve tempo suficiente para as processar.
3. *Túneis e grupos de saídas/entradas:* Quando um TxTangle é iniciado, os participantes devem desassociar as identidades dos membros desvio das saídas verdadeiras. Isto significa criar novos túneis para mensagens assinadas tipo bLSAG. Cada túnel destes so pode transmitir mensagens relativamente a uma certa saída (a informação sobre uma saída vem sempre da mesma fonte). Os participantes devem também criar novos túneis para mensagens assinadas tipo SAG, respectivas a certas entradas.
4. *Ameaça de um ataque MITM pelo anfitrião:* O anfitrião pode enganar um participante ao pretender ser um outro participante, visto que ele controla o envio da lista de membros desvio para construir bLSAGs e SAGs. Por outras palavras a lista que ele envia ao participante A, pode conter os membros desvio do participante A, e todo o resto são os seus próprios. As mensagens recebidas pelo participante B são assinadas outra vez com a lista de A antes de ser re-transmitida a A. Como todas as mensagens assinadas por A pertencem a A, o anfitrião teria visibilidade directa para os agrupamentos de entradas/saídas de A.

É possível prevenir que o anfitrião aja como MITM a partir de interações honestas entre os participantes, ao modificar como as chaves públicas de transacção são feitas. Os participantes enviam uns aos outros as chaves públicas de transacção (com uma assinatura bLSAG), depois, bem como na agregação robusta de chave da secção 9.2.3, as chaves que de facto são incluídas nos dados de transacção (e que são utilizados para fazer máscaras de compromissos de saída,

etc.) são prefixos com uma hash da lista dos membros desvio. Por outras palavras, a t -ésima chave pública de transacção é :

$$\mathcal{H}_n(T_{agg}, S_{mock}, r_t G) * r_t G$$

Pôr a lista de membros desvio dentro da transacção faz com que seja muito difícil completar TxTangles sem que haja uma comunicação directa entre todos os actuais participantes. ²¹

11.2.2 Um anfitrião como um serviço

É importante para a sustentabilidade e para a melhoria continua que um serviço de TxTangle opere com lucro. ²² Em vez de comprometer as identidades dos utilizadores com um modelo de conta, o anfitrião pode participar em cada TxTangle com uma só saída, e requerer que os participantes financiem essa saída. Quando os participantes acedem ao serviço/‘eepsite’ do anfitrião, são informados dessa taxa actual. Esta taxa do anfitrião deve ser paga por cada saída.

Participantes seriam responsáveis por pagar fracções da taxa de mineiro e a taxa do anfitrião. Desta vez, a chave pública mais pequena do membro desvio (excluindo a chave do anfitrião) pagaria o resto de ambas as taxas. ²³ Desde que o anfitrião não tem nenhuma entrada, ele também não tem nenhum pseudo compromisso de saída para cancelar a sua máscara do compromisso de saída. Em vez disso ele cria segredos partilhados com os outros membros desvio, depois separa a sua máscara de compromisso em partes de tamanho aleatório e divide estas partes pelos segredos partilhados. Ele publica uma lista desses escalares, assina com a sua chave tal que os participantes saibam que se trata do anfitrião. Esta lista, ao aparecer assinála o começo da ronda n° 1 da secção 11.1.2 (e.g. o fim da ronda ‘0’). Membros desvio irão multiplicar o seu escalar do anfitrião pelo segredo partilhado apropriado, e adicionam isso á sua máscara de pseudo compromisso. Desta forma, mesmo a saída do anfitrião não pode ser identificada por qualquer participante na transacção final sem que haja uma coalição total contra ele.

Para simplificar as calculações da taxa, o anfitrião pode distribuir a taxa total a ser usada na transacção no fim da ronda n° 1, pois ele irá saber qual é o peso da transacção relativamente cedo. Os participantes podem verificar que esse montante é semelhante ao montante esperado, e pagar uma fracção do mesmo.

Se os participantes colaboram para enganar, e não querem pagar o preço do anfitrião, então o anfitrião pode terminar o TxTangle na ronda n° 3. Ou então se as mensagens que aparecem no canal não deviam estar ali ou são inválidas.

²¹ Se a mitigação de Janus for implementada, esta defesa ao ataque MITM devia em vez disso ser feita com a chave base falsa de Janus. Cada membro desvio oferece uma chave aleatória $r_{desvio}G$, depois a actual chave base é :

$$\sum_{desvio} \mathcal{H}_n(T_{agg}, S_{desvio}, r_{desvio}G) * r_{desvio}G$$

²² Enquanto que um serviço de TxTangle opere com lucro, o próprio código pode ser de código aberto. Isto é importante para auditar o software de carteira que interage com um tal serviço.

²³ É necessário utilizar aqui as chaves do membros desvio, pois o anfitrião não paga taxa, e o seu index de saída é desconhecido.

No fim da ronda 5 o anfitrião completa a transacção e submete isso á rede para verificação, como parte do seu serviço. Ele inclui a hash da transacção na mensagem que é distribuída no fim.

11.3 Usar um administrador á confiança

Existem aspectos negativos para o TxTangle descentralizado. Requer que os participantes se encontrem uns aos outros, e que comuniquem activamente dentro de um tempo planeado. Isto é também difícil de implementar em código.

Usar um administrador central, que é responsável para coleccionar informação de transacção de cada participante e obfuscar os grupos de entrada/saída, simplifica o processo. O custo é ter uma entidade terceira á confiança, que sabe (no mínimo) esses grupos de entrada/saída. ²⁴

11.3.1 Processo baseado num administrador

O administrador revela que ele está disponível para gerir TxTangles, e colecciona inscrições de participantes potenciais (que consistem do número de entradas [com os seus tipos] e saídas). Ele pode se quiser participar com o seu próprio conjunto de saídas/entradas.

Depois de quase 16 saídas terem sido juntadas num grupo (têm de existir pelo menos dois ou mais participantes, e nenhum participante deve ter todas as entradas ou saídas), o administrador começa a primeira de 5 rondas. Em cada ronda ele acumula informação de cada participante, faz algumas decisões, e envia mensagens que significam o começo de uma nova ronda.

1. Para começar, o administrador gera, para cada par de participantes, um escalar aleatório, e decide quem nesse par recebe a versão negativa desse escalar. Ele usa o número e tipo de entradas e saídas, para estimar a taxa total necessária. Ele faz a soma dos escalares dos participantes, e de forma privada envia para cada um essa soma, juntamente com a taxa que eles devem pagar e os índices das suas saídas (escolhidas de forma aleatória). Estas mensagens constituem um sinal para os participantes que um TxTangle está a começar.
2. Cada participante constrói a sua sub-transacção como se fosse uma transacção normal, com chaves públicas de transacção para as suas saídas (com uma mitigação de Janus se necessário), calcular endereços ocultos de saída, codificar montantes de saída, fazer pseudo compromissos de saída que igualam os compromissos de saída mais a taxa para os mineiros, fazer uma lista de offsets de membros de anel para o uso em assinaturas MLSAG juntamente com as imagens de chave respectivas, e adicionar a um dos seus pseudo compromissos de saída, o escalar enviado pelo administrador (multiplicado por G). Eles criam a parte A de provas parciais para as suas saídas, e enviam toda esta informação ao administrador. O administrador verifica que os montantes de entrada e saída se igualam, e depois envia a lista completa da parte A de provas parciais a cada participante.

²⁴ Esta secção é inspirada pelo protocolo *MoJoin*.

3. Cada participante calcula o desafio agregado A, e gera a parte B de provas parciais que depois são enviadas ao administrador. Este por sua vez colecciona as provas parciais e distribui estas por todos os participantes.
4. Cada participante calcula o desafio B, e gera a parte C de provas parciais que eles enviam ao administrador. O administrador por sua vez colecciona estas provas e aplica-lhes a técnica do produto interno logarítmico para as comprimir para a prova final. Assumindo que a prova se verifica como devia, ele gera uma chave pública ‘base’ de transacção falsa tipo Janus, e envia a mensagem para ser assinada em MLSAGs para cada participante.
5. Cada participante completa a sua assinatura MLSAG e envia isso ao administrador. Uma vez que este tem todas as peças, ele pode finalizar a construção da transacção, e submeter isso á rede. Ele pode também enviar a ID da transacção a cada participante para que estes confirmem que ela foi publicada.

Bibliografia

- [1] Add intervening v5 fork for increased min block size. <https://github.com/monero-project/monero/pull/1869> [Online; accessed 05/24/2018].
- [2] cryptography - What is a cryptographic oracle? <https://security.stackexchange.com/questions/10617/what-is-a-cryptographic-oracle> [Online; accessed 04/22/2018].
- [3] Cryptography Tutorial. <https://www.tutorialspoint.com/cryptography/index.htm> [Online; accessed 05/19/2018].
- [4] Cryptonote Address Tests. <https://xmr.llcoins.net/addresstests.html> [Online; accessed 04/19/2018].
- [5] Directed acyclic graph. https://en.wikipedia.org/wiki/Directed_acyclic_graph [Online; accessed 05/27/2018].
- [6] Extended Euclidean algorithm. https://en.wikipedia.org/wiki/Extended_Euclidean_algorithm [Online; accessed 03/19/2020].
- [7] hackerone #501585. <https://hackerone.com/reports/501585> [Online; accessed 12/28/2019].
- [8] How do payment ids work? <https://monero.stackexchange.com/questions/1910/how-do-payment-ids-work> [Online; accessed 04/21/2018].
- [9] Modular arithmetic. https://en.wikipedia.org/wiki/Modular_arithmetic [Online; accessed 04/16/2018].
- [10] Monero 0.13.0 “Beryllium Bullet” Release. <https://www.getmonero.org/2018/10/11/monero-0.13.0-released.html> [Online; accessed 10/12/2019].
- [11] Monero 0.14.0 “Boron Butterfly” Release. <https://web.getmonero.org/2019/02/25/monero-0.14.0-released.html> [Online; accessed 10/12/2019].
- [12] Monero inception and history. <https://monero.stackexchange.com/questions/475/monero-inception-and-history-how-did-monero-get-started-what-are-its-origins-a/476#476> [Online; accessed 05/23/2018].
- [13] Monero v0.9.3 - Hydrogen Helix - released! https://www.reddit.com/r/Monero/comments/4bgw4z/monero_v093_hydrogen_helix_released_urgent_and/ [Online; accessed 05/24/2018].
- [14] Randomx. <https://www.monerooutreach.org/stories/RandomX.php> [Online; accessed 10/12/2019].
- [15] Ring CT; Moneropedia. <https://getmonero.org/resources/moneropedia/ringCT.html> [Online; accessed 06/05/2018].

- [16] Tail emission. <https://getmonero.org/resources/moneropedia/tail-emission.html> [Online; accessed 05/24/2018].
- [17] Trust the math? An Update. <http://www.math.columbia.edu/~woit/wordpress/?p=6522> [Online; accessed 04/04/2018].
- [18] Useful for learning about Monero coin emission. https://www.reddit.com/r/Monero/comments/512kwh/useful_for_learning_about_monero_coin_emission/d78tpgi/ [Online; accessed 05/25/2018].
- [19] What is a premine? <https://www.cryptocompare.com/coins/guides/what-is-a-premine/> [Online; accessed 06/11/2018].
- [20] What is the block maturity value seen in many pool interfaces? <https://monero.stackexchange.com/questions/2251/what-is-the-block-maturity-value-seen-in-many-pool-interfaces> [Online; accessed 05/26/2018].
- [21] XOR – from Wolfram Mathworld. <http://mathworld.wolfram.com/XOR.html> [Online; accessed 04/21/2018].
- [22] Federal Information Processing Standards Publication (FIPS 197). Advanced Encryption Standard (AES), 2001. <http://csrc.nist.gov/publications/fips/fips197/fips-197.pdf> [Online; access 03/04/2020].
- [23] Fungible, July 2014. <https://wiki.mises.org/wiki/Fungible> [Online; accessed 03/31/2020].
- [24] NIST Releases SHA-3 Cryptographic Hash Standard, August 2015. <https://www.nist.gov/news-events/news/2015/08/nist-releases-sha-3-cryptographic-hash-standard> [Online; accessed 06/02/2018].
- [25] Base58Check encoding, November 2017. https://en.bitcoin.it/wiki/Base58Check_encoding [Online; accessed 02/20/2020].
- [26] Monero cryptonight variants, and add one for v7, April 2018. <https://github.com/monero-project/monero/pull/3253> [Online; accessed 05/23/2018].
- [27] Payment Reversal Explained + 10 Ways to Avoid Them, October 2018. <https://tidalcommerce.com/learn/payment-reversal> [Online; accessed 02/11/2020].
- [28] Diffie–Hellman problem, December 2019. https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_problem [Online; accessed 03/31/2020].
- [29] Kurt M. Alonso and Jordi Herrera Joancomartí. Monero - Privacy in the Blockchain. Cryptology ePrint Archive, Report 2018/535, 2018. <https://eprint.iacr.org/2018/535>.
- [30] Kurt M. Alonso and Koe. Zero to Monero - First Edition, June 2018. <https://web.getmonero.org/library/Zero-to-Monero-1-0-0.pdf> [Online; accessed 01/15/2020].
- [31] Kristov Atlas. Kristov Atlas Security Advisory 20140609-0, June 2014. <https://www.coinjoinsudoku.com/advisory/> [Online; accessed 01/26/2020].
- [32] Adam Back. Ring signature efficiency. BitcoinTalk, 2015. <https://bitcointalk.org/index.php?topic=972541.msg10619684#msg10619684> [Online; accessed 04/04/2018].
- [33] Daniel J. Bernstein. ChaCha, a variant of Salsa20, January 2008. <https://cr.yp.to/chacha/chacha-20080120.pdf> [Online; accessed 03/04/2020].
- [34] Daniel J. Bernstein, Peter Birkner, Marc Joye, Tanja Lange, and Christiane Peters. *Twisted Edwards Curves*, pages 389–405. Springer Berlin Heidelberg, Berlin, Heidelberg, 2008. <https://eprint.iacr.org/2008/013.pdf> [Online; accessed 02/13/2020].
- [35] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, Sep 2012. <https://ed25519.cr.yp.to/ed25519-20110705.pdf> [Online; accessed 03/04/2020].
- [36] Daniel J. Bernstein and Tanja Lange. *Faster Addition and Doubling on Elliptic Curves*, pages 29–50. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007. <https://eprint.iacr.org/2007/286.pdf> [Online; accessed 03/04/2020].
- [37] Karina Bjørnholdt. Dansk politi har knækket bitcoin-koden, May 2017. <http://www.dansk-politi.dk/artikler/2017/maj/dansk-politi-har-knaekket-bitcoin-koden> [Online; accessed 04/04/2018].

- [38] Dan Boneh and Victor Shoup. A Graduate Course in Applied Cryptography. <https://crypto.stanford.edu/~dabo/cryptobook/> [Online; accessed 12/30/2019].
- [39] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short Proofs for Confidential Transactions and More. <https://eprint.iacr.org/2017/1066.pdf> [Online; accessed 10/28/2018].
- [40] Francisco Cabañas. Francisco Cabanas - Critical Role of Min Block Reward Trail Emission - DEF CON 27 Monero Village, December 2019. <https://www.youtube.com/watch?v=IlghysBBuyU> [Online; accessed 01/15/2020].
- [41] Francisco Cabañas. Lightning talk: An Overview of Monero's Adaptive Blockweight Approach to Scaling, December 2019. <https://frab.riat.at/en/36C3/public/events/125.html> [Online; accessed 01/12/2020].
- [42] Francisco Cabañas. MoneroKon 2019 - Spam Mitigation and Size Control in Permissionless Blockchains, June 2019. https://www.youtube.com/watch?v=HbmOub3qWw4&list=LL2HXH-vq_sTPXMNP4fZNKrw&index=10&t=0s [Online; accessed 01/10/2020].
- [43] Miles Carlsten, Harry Kalodner, Matthew Weinberg, and Arvind Narayanan. On the Instability of Bitcoin Without the Block Reward, 2016. http://randomwalker.info/publications/mining_CCS.pdf [Online; accessed 01/12/2020].
- [44] Chainalysis. THE 2020 STATE OF CRYPTO CRIME, January 2020. <https://go.chainalysis.com/rs/503-FAP-074/images/2020-Crypto-Crime-Report.pdf> [Online; accessed 02/11/2020].
- [45] David Chaum and Eugène Van Heyst. Group Signatures. In *Proceedings of the 10th Annual International Conference on Theory and Application of Cryptographic Techniques*, EUROCRYPT'91, pages 257–265, Berlin, Heidelberg, 1991. Springer-Verlag. https://chaum.com/publications/Group_Signatures.pdf [Online; accessed 03/04/2020].
- [46] Michael Davidson and Tyler Diamond. On the Profitability of Selfish Mining Against Multiple Difficulty Adjustment Algorithms. Cryptology ePrint Archive, Report 2020/094, 2020. <https://eprint.iacr.org/2020/094> [Online; accessed 03/26/2020].
- [47] W. Diffie and M. Hellman. New Directions in Cryptography. *IEEE Trans. Inf. Theor.*, 22(6):644–654, September 2006. <https://ee.stanford.edu/~hellman/publications/24.pdf> [Online; accessed 03/04/2020].
- [48] Manu Drijvers, Kasra Edalatnejad, Bryan Ford, Eike Kiltz, Julian Loss, Gregory Neven, and Igors Stepanovs. On the Security of Two-Round Multi-Signatures. Cryptology ePrint Archive, Report 2018/417, 2018. <https://eprint.iacr.org/2018/417> [Online; accessed 02/07/2020].
- [49] Justin Ehrenhofer. Justin Ehrenhofer - Improving Monero Release Schedule - DEF CON 27 Monero Village, December 2019. <https://www.youtube.com/watch?v=MjNXmJUK2Jo> timestamp 22:15 [Online; accessed 03/20/2020].
- [50] Justin Ehrenhofer. Monero Adds Blockchain Pruning and Improves Transaction Efficiency, February 2019. <https://web.getmonero.org/2019/02/01/pruning.html> [Online; accessed 12/30/2019].
- [51] Justin Ehrenhofer and knacc. Advisory note for users making use of subaddresses, October 2019. <https://web.getmonero.org/2019/10/18/subaddress-janus.html> [Online; accessed 01/02/2020].
- [52] Serhack et al. Mastering Monero, December 2019. <https://masteringmonero.com/> [Online; accessed 01/10/2020].
- [53] Exantech. Methods of anonymous blockchain analysis: an overview, November 2019. <https://medium.com/@exantech/methods-of-anonymous-blockchain-analysis-an-overview-d700e27ea98c> [Online; accessed 01/26/2020].
- [54] Ittay Eyal and Emin Gün Sirer. Majority is not Enough: Bitcoin Mining is Vulnerable. *CoRR*, abs/1311.0243, 2013. <https://arxiv.org/pdf/1311.0243.pdf> [Online; accessed 03/04/2020].
- [55] Amos Fiat and Adi Shamir. How To Prove Yourself: Practical Solutions to Identification and Signature Problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology — CRYPTO' 86*, pages 186–194, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg. https://link.springer.com/content/pdf/10.1007/2F3-540-47721-7_12.pdf [Online; accessed 03/04/2020].

- [56] Ryo “fireice_uk” Cryptocurrency. On-chain tracking of Monero and other Cryptonotes, April 2019. https://medium.com/@crypto_ryo/on-chain-tracking-of-monero-and-other-cryptonotes-e0afc6752527 [Online; accessed 03/25/2020].
- [57] Riccardo “fluffypony” Spagni. Monero 0.12.0.0 “Lithium Luna” Release, March 2018. <https://web.getmonero.org/2018/03/29/monero-0.12.0.0-released.html> [Online; accessed 02/18/2020].
- [58] Riccardo “fluffypony” Spagni and luigi1111. Disclosure of a Major Bug in Cryptonote Based Currencies, May 2017. <https://getmonero.org/2017/05/17/disclosure-of-a-major-bug-in-cryptonote-based-currencies.html> [Online; accessed 04/10/2018].
- [59] Eiichiro Fujisaki and Koutarou Suzuki. *Traceable Ring Signature*, pages 181–200. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007. https://link.springer.com/content/pdf/10.1007/2F978-3-540-71677-8_13.pdf [Online; accessed 03/04/2020].
- [60] Brandon Goodell, Sarang Noether, and Arthur Blue. Concise linkable ring signatures and applications, MRL-0011, September 2019. <https://web.getmonero.org/resources/research-lab/pubs/MRL-0011.pdf> [Online; accessed 02/02/2020].
- [61] Thomas C Hales. The NSA back door to NIST. *Notices of the AMS*, 61(2):190–192. <https://www.ams.org/notices/201402/rnoti-p190.pdf> [Online; accessed 03/04/2020].
- [62] Darrel Hankerson, Alfred J. Menezes, and Scott Vanstone. *Guide to Elliptic Curve Cryptography*. Springer-Verlag New York, Inc., Secaucus, NJ, USA, 2003.
- [63] Howard “hyc” Chu. RandomX, Pull Request #5549, May 2019. <https://github.com/monero-project/monero/pull/5549> [Online; accessed 03/03/2020].
- [64] Aram Jivanyan. Lelantus: Towards Confidentiality and Anonymity of Blockchain Transactions from Standard Assumptions. Cryptology ePrint Archive, Report 2019/373, 2019. <https://eprint.iacr.org/2019/373.pdf> [Online; accessed 03/04/2020].
- [65] Don Johnson and Alfred Menezes. The Elliptic Curve Digital Signature Algorithm (ECDSA). Technical Report CORR 99-34, Dept. of C&O, University of Waterloo, Canada, 1999. <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.472.9475&rep=rep1&type=pdf> [Online; accessed 04/04/2018].
- [66] JollyMort. Monero Dynamic Block Size and Dynamic Minimum Fee, March 2017. <https://github.com/JollyMort/monero-research/blob/master/Monero%20Dynamic%20Block%20Size%20and%20Dynamic%20Minimum%20Fee/Monero%20Dynamic%20Block%20Size%20and%20Dynamic%20Minimum%20Fee%20-%20DRAFT.md> [Online; accessed 01/12/2020].
- [67] S. Josefsson, SJD AB, and N. Moeller. EdDSA and Ed25519. Internet Research Task Force (IRTF), 2015. <https://tools.ietf.org/html/draft-josefsson-eddsa-ed25519-03> [Online; accessed 05/11/2018].
- [68] Simon Josefsson and Ilari Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, January 2017. <https://rfc-editor.org/rfc/rfc8032.txt> [Online; accessed 03/04/2020].
- [69] kenshi84. Subaddresses, Pull Request #2056, May 2017. <https://github.com/monero-project/monero/pull/2056> [Online; accessed 02/16/2020].
- [70] Eike Kiltz, Daniel Masny, and Jiaxin Pan. Optimal Security Proofs for Signatures from Identification Schemes. In *Proceedings, Part II, of the 36th Annual International Cryptology Conference on Advances in Cryptology — CRYPTO 2016 - Volume 9815*, pages 33–61, Berlin, Heidelberg, 2016. Springer-Verlag. <https://eprint.iacr.org/2016/191.pdf> [Online; accessed 03/04/2020].
- [71] Alexander Klimov. ECC Patents?, October 2005. <http://article.gmane.org/gmane.comp.encryption.general/7522> [Online; accessed 04/04/2018].
- [72] koe and jtgrassie. Historical significance of FEE.PER.KB.OLD, December 2019. <https://monero.stackexchange.com/questions/11864/historical-significance-of-fee-per-kb-old> [Online; accessed 01/02/2020].
- [73] koe and jtgrassie. Complete extra field structure (standard interpretation), January 2020. <https://monero.stackexchange.com/questions/11888/complete-extra-field-structure-standard-interpretation> [Online; accessed 01/05/2020].

- [74] Mitchell Krawiec-Thayer. Numerical simulation for upper bound on dynamic blocksize expansion, January 2019. https://github.com/noncesense-research-lab/Blockchain_big_bang/blob/master/models/Isthmus_Bx_big_bang_model.ipynb [Online; accessed 01/08/2020].
- [75] Russell W. F. Lai, Viktoria Ronge, Tim Ruffing, Dominique Schröder, Sri Thyagarajan, and Jiafan Wang. Omniring: Scaling Private Payments Without Trusted Setup. pages 31–48, 11 2019. <https://eprint.iacr.org/2019/580.pdf> [Online; accessed 03/04/2020].
- [76] Joseph K. Liu, Victor K. Wei, and Duncan S. Wong. *Linkable Spontaneous Anonymous Group Signature for Ad Hoc Groups*, pages 325–335. Springer Berlin Heidelberg, Berlin, Heidelberg, 2004. <https://eprint.iacr.org/2004/027.pdf> [Online; accessed 03/04/2020].
- [77] Jan Macheta, Sarang Noether, Suraj Noether, and Javier Smooth. Counterfeiting via Merkle Tree Exploits within Virtual Currencies Employing the CryptoNote Protocol, MRL-0002, September 2014. <https://web.getmonero.org/resources/research-lab/pubs/MRL-0002.pdf> [Online; accessed 05/27/2018].
- [78] Ueli Maurer. Unifying Zero-Knowledge Proofs of Knowledge. In Bart Preneel, editor, *Progress in Cryptology – AFRICACRYPT 2009*, pages 272–286, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg. <https://www.crypto.ethz.ch/publications/files/Maurer09.pdf> [Online; accessed 03/04/2020].
- [79] Greg Maxwell. Confidential Transactions. https://people.xiph.org/~greg/confidential_values.txt [Online; accessed 04/04/2018].
- [80] Gregory Maxwell and Andrew Poelstra. Borromean Ring Signatures. 2015. <https://pdfs.semanticscholar.org/4160/470c7f6cf05ffc81a98e8fd67fb0c84836ea.pdf> [Online; accessed 04/04/2018].
- [81] Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. Simple Schnorr Multi-Signatures with Applications to Bitcoin. May 2018. <https://eprint.iacr.org/2018/068.pdf> [Online; accessed 03/01/2020].
- [82] Sarah Meiklejohn and Claudio Orlandi. Privacy-Enhancing Overlays in Bitcoin. https://fc15.ifca.ai/preproceedings/bitcoin/paper_5.pdf [Online; accessed 01/26/2020].
- [83] R. C. Merkle. Protocols for Public Key Cryptosystems. In *1980 IEEE Symposium on Security and Privacy*, pages 122–122, April 1980. <http://www.merkle.com/papers/Protocols.pdf> [Online; accessed 03/04/2020].
- [84] Andrew Miller, Malte Möser, Kevin Lee, and Arvind Narayanan. An Empirical Analysis of Linkability in the Monero Blockchain. *CoRR*, abs/1704.04299, 2017. <https://arxiv.org/pdf/1704.04299.pdf> [Online; accessed 03/04/2020].
- [85] Victor S Miller. Use of Elliptic Curves in Cryptography. In *Lecture Notes in Computer Sciences; 218 on Advances in cryptology—CRYPTO 85*, pages 417–426, Berlin, Heidelberg, 1986. Springer-Verlag. https://link.springer.com/content/pdf/10.1007/3-540-39799-X_31.pdf [Online; accessed 03/04/2020].
- [86] Nicola Minichiello. The Bitcoin Big Bang: Tracking Tainted Bitcoins, June 2015. <https://bravenewcoin.com/insights/the-bitcoin-big-bang-tracking-tainted-bitcoins> [Online; accessed 03/31/2020].
- [87] Ludwig Von Mises. Human Action: A Treatise on Economics; The Scholar’s Edition, 1949. https://cdn.mises.org/Human%20Action_3.pdf [Online; accessed 03/05/2020].
- [88] Monero Project. Monero source code, August 2026. Fotografia do código-fonte datada de 14 de Agosto de 2026, <https://github.com/monero-project/monero>.
- [89] Satoshi Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. <http://bitcoin.org/bitcoin.pdf> [Online; accessed 03/04/2020].
- [90] Arvind Narayanan and Malte Möser. Obfuscation in Bitcoin: Techniques and Politics. *CoRR*, abs/1706.05432, 2017. <https://arxiv.org/ftp/arxiv/papers/1706/1706.05432.pdf> [Online; accessed 03/04/2020].
- [91] Y. Nir, Check Point, A. Langley, and Google Inc. ChaCha20 and Poly1305 for IETF Protocols. Internet Research Task Force (IRTF), May 2015. <https://tools.ietf.org/html/rfc7539> [Online; accessed 05/11/2018].
- [92] Sarang Noether. Janus mitigation, Issue #62, January 2020. <https://github.com/monero-project/research-lab/issues/62> [Online; accessed 02/17/2020].
- [93] Sarang Noether. Multisignature implementation, Issue #67, January 2020. <https://github.com/monero-project/research-lab/issues/67> [Online; accessed 02/16/2020].

- [94] Sarang Noether. Transaction proofs (InProofV1 and OutProofV1) have incomplete Schnorr challenges, Issue #60, January 2020. <https://github.com/monero-project/research-lab/issues/60> [Online; accessed 02/20/2020].
- [95] Sarang Noether. WIP: Updated transaction proofs and tests, Pull Request #6329, February 2020. <https://github.com/monero-project/monero/pull/6329> [Online; accessed 02/20/2020].
- [96] Sarang Noether and Brandon Goodell. An efficient implementation of Monero subaddresses, MRL-0006, October 2017. <https://web.getmonero.org/resources/research-lab/pubs/MRL-0006.pdf> [Online; accessed 04/04/2018].
- [97] Sarang Noether and Brandon Goodell. Triptych: logarithmic-sized linkable ring signatures with applications. Cryptology ePrint Archive, Report 2020/018, 2020. <https://eprint.iacr.org/2020/018.pdf> [Online; accessed 03/04/2020].
- [98] Shen Noether. Understanding `ge_fromfe_frombytes_vartime`. https://web.getmonero.org/resources/research-lab/pubs/ge_fromfe.pdf [Online; accessed 01/20/2020].
- [99] Shen Noether, Adam Mackenzie, and Monero Core Team. Ring Confidential Transactions, MRL-0005, February 2016. <https://web.getmonero.org/resources/research-lab/pubs/MRL-0005.pdf> [Online; accessed 06/15/2018].
- [100] Shen Noether, Adam Mackenzie, and Monero Core Team. Ring Multisignature, April 2016. https://web.archive.org/web/20161023010706/https://shnoe.files.wordpress.com/2016/03/mrl-0008_april28.pdf [Online; accessed via WayBack Machine 01/21/2020].
- [101] Shen Noether and Sarang Noether. Monero is Not That Mysterious, MRL-0003, September 2014. <https://web.getmonero.org/resources/research-lab/pubs/MRL-0003.pdf> [Online; accessed 06/15/2018].
- [102] Shen Noether and Sarang Noether. Thring Signatures and their Applications to Spender-Ambiguous Digital Currencies, MRL-0009, November 2018. <https://web.getmonero.org/resources/research-lab/pubs/MRL-0009.pdf> [Online; accessed 01/15/2020].
- [103] Michael Padilla. Beating Bitcoin bad guys, August 2016. <http://www.sandia.gov/news/publications/labnews/articles/2016/19-08/bitcoin.html> [Online; accessed 04/04/2018].
- [104] Torben Pryds Pedersen. *Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing*, pages 129–140. Springer Berlin Heidelberg, Berlin, Heidelberg, 1992. <https://www.cs.cornell.edu/courses/cs754/2001fa/129.PDF> [Online; accessed 03/04/2020].
- [105] rbrunner7. 2/2 Multisig in CLI Wallet, January 2018. <https://taiga.getmonero.org/project/rbrunner7-really-simple-multisig-transactions/wiki/22-multisig-in-cli-wallet> [Online; accessed 01/21/2020].
- [106] René “rbrunner7” Brunner. Multisig transactions with MMS and CLI wallet. <https://web.getmonero.org/resources/user-guides/multisig-messaging-system.html> [Online; accessed 01/21/2020].
- [107] René “rbrunner7” Brunner. Project Rationale From the Initiator, January 2018. <https://taiga.getmonero.org/project/rbrunner7-really-simple-multisig-transactions/wiki/home> [Online; accessed 01/21/2020].
- [108] René “rbrunner7” Brunner. Basic Monero support ready for assessment, Issue #1638, June 2019. <https://github.com/OpenBazaar/openbazaar-go/issues/1638> [Online; accessed 02/11/2020].
- [109] Jamie Redman. Industry Execs Claim Freshly Minted ‘Virgin Bitcoins’ Fetch 20% Premium, March 2020. <https://news.bitcoin.com/industry-execs-freshly-minted-virgin-bitcoins/> [Online; accessed 03/31/2020].
- [110] Ronald L. Rivest, Adi Shamir, and Yael Tauman. How to Leak a Secret. C. Boyd (Ed.): ASIACRYPT 2001, LNCS 2248, pp. 552–565, 2001. <https://people.csail.mit.edu/rivest/pubs/RST01.pdf> [Online; accessed 04/04/2018].
- [111] SafeCurves. SafeCurves: choosing safe curves for elliptic-curve cryptography, 2013. <https://safecurves.cr.yt.to/rigid.html> [Online; accessed 03/25/2020].

- [112] C. P. Schnorr. Efficient Identification and Signatures for Smart Cards. In Gilles Brassard, editor, *Advances in Cryptology — CRYPTO' 89 Proceedings*, pages 239–252, New York, NY, 1990. Springer New York. https://link.springer.com/content/pdf/10.1007%2F0-387-34805-0_22.pdf [Online; accessed 03/04/2020].
- [113] Paola Scozzafava. Uniform distribution and sum modulo m of independent random variables. *Statistics & Probability Letters*, 18(4):313 – 314, 1993. <https://sci-hub.tw/https://www.sciencedirect.com/science/article/abs/pii/016771529390021A> [Online; accessed 03/04/2020].
- [114] Bassam El Khoury Seguias. Monero Building Blocks, 2018. <https://delfr.com/category/monero/> [Online; accessed 10/28/2018].
- [115] Seigen, Max Jameson, Tuomo Nieminen, Neocortex, and Antonio M. Juarez. CryptoNight Hash Function. CryptoNote, March 2013. <https://cryptonote.org/cns/cns008.txt> [Online; accessed 04/04/2018].
- [116] QingChun ShenTu and Jianping Yu. Research on Anonymization and De-anonymization in the Bitcoin System. *CoRR*, abs/1510.07782, 2015. <https://arxiv.org/ftp/arxiv/papers/1510/1510.07782.pdf> [Online; accessed 03/04/2020].
- [117] stoffu. Reserve proof, Pull Request #3027, December 2017. <https://github.com/monero-project/monero/pull/3027> [Online; accessed 02/24/2020].
- [118] thankful_for_today. [ANN][BMR] Bitmonero - a new coin based on CryptoNote technology - LAUNCHED, April 2014. Monero's actual launch date was April 18th, 2014. <https://bitcointalk.org/index.php?topic=563821.0> [Online; accessed 05/24/2018].
- [119] Manny Trillo. Visa Transactions Hit Peak on Dec. 23, January 2011. <https://www.visa.com/blogarchives/us/2011/01/12/visa-transactions-hit-peak-on-dec-23/index.html> [Online; accessed 01/10/2020].
- [120] Alicia Tuovila. Audit, July 2019. <https://www.investopedia.com/terms/a/audit.asp> [Online; accessed 02/24/2020].
- [121] UkoeHB. Proof an output has not been spent, Issue #68, January 2020. <https://github.com/monero-project/research-lab/issues/68> [Online; accessed 02/19/2020].
- [122] Maciej Ulas. Rational points on certain hyperelliptic curves over finite fields, 2007. <https://arxiv.org/pdf/0706.1448.pdf> [Online; accessed 03/03/2020].
- [123] user36100 and cooldude45. Which entities are related to Bytecoin and Minergate?, December 2016. <https://monero.stackexchange.com/questions/2930/which-entities-are-related-to-bytecoin-and-minergate> [Online; accessed 01/17/2020].
- [124] Nicolas van Saberhagen. CryptoNote V2.0. <https://cryptonote.org/whitepaper.pdf> [Online; accessed 04/04/2018].
- [125] David Wagner. A Generalized Birthday Problem. In Moti Yung, editor, *Advances in Cryptology — CRYPTO 2002*, pages 288–304, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg. https://link.springer.com/content/pdf/10.1007%2F3-540-45708-9_19.pdf [Online; accessed 02/07/2020].
- [126] Adam “waxwing” Gibson. From Zero (Knowledge) To Bulletproofs, March 2018. <https://github.com/AdamISZ/from0k2bp/blob/master/from0k2bp.pdf> [Online; accessed 03/01/2020].
- [127] Adam “waxwing” Gibson. Avoiding Wagnerian Tragedies, December 2019. <https://joinmarket.me/blog/blog/avoiding-wagnerian-tragedies/> [Online; accessed 03/01/2020].
- [128] Albert Werner, Montag, Prometheus, and Tereno. CryptoNote Transaction Extra Field. CryptoNote, October 2012. <https://cryptonote.org/cns/cns005.txt> [Online; accessed 04/04/2018].
- [129] Ursula Whitcher. The Birthday Problem. <http://mathforum.org/dr.math/faq/faq.birthdayprob.html> [Online; accessed 02/07/2020].
- [130] Wikibooks. Cryptography/Prime Curve/Standard Projective Coordinates, March 2011. https://en.wikibooks.org/wiki/Cryptography/Prime_Curve/Standard_Projective_Coordinates [Online; accessed 03/03/2020].

-
- [131] Tsz Hon Yuen, Shi-feng Sun, Joseph K. Liu, Man Ho Au, Muhammed F. Esgin, Qingzhao Zhang, and Dawu Gu. RingCT 3.0 for Blockchain Confidential Transaction: Shorter Size and Stronger Security. Cryptology ePrint Archive, Report 2019/508, 2019. <https://eprint.iacr.org/2019/508.pdf> [Online; accessed 03/04/2020].
- [132] zawy12. Summary of Difficulty Algorithms, Issue #50, December 2019. <https://github.com/zawy12/difficulty-algorithms/issues/50> [Online; accessed 02/11/2020].

Appendices

Espécime histórico: estrutura de transacção RCTTypeBulletproof2

Apresenta-se neste apêndice uma transacção de Monero real do tipo `RCTTypeBulletproof2`, juntamente com explicações dos seus campos relevantes. É um espécime histórico, incluído no bloco 2045821, em 2020, quando a versão 12 estava activa. Os seus anéis de onze membros, as assinaturas MLSAG, os Bulletproofs clássicos e os alvos sem etiquetas de visualização não descrevem uma nova transacção v16. O caminho activo é `RCTTypeBulletproofPlus`, explicado no capítulo 6.

Esta transacção foi obtida do explorador de blocos <https://xmchain.net>. Também se pode obter ao executar o comando `print_tx <TransactionID> +hex +json` no *daemon monerod* (sem o parâmetro *detached*). Aqui, `<TransactionID>` é o *hash* da transacção (secção 7.4.1). A primeira linha mostra o comando executado. Para replicar estes resultados, o leitor faz o seguinte:

1. É preciso o programa de linha de comandos de Monero, disponível no sítio oficial de descarregas.¹ Descarregue a versão para o seu sistema operativo e extraia o arquivo.
2. Abra um terminal e navegue para o directório criado pela extracção.
3. Corra `monerod` com `./monerod`. O nó descarrega e valida a lista de blocos na máquina local; o comando abaixo só poderá encontrar a transacção depois de a sincronização alcançar a altura necessária.
4. Depois do processo de sincronização estar feito, é possível executar comandos como `print_tx`. Use `help` para conhecer outros comandos.

¹ <https://web.getmonero.org/downloads/>

O componente `rctsig_prunable` é, como o nome indica, podável. Um nó podado pode omitir estes dados antigos e conservar a parte não podável e o *hash* da parte podável necessário para reconstruir o identificador da transacção. A raiz de Merkle do bloco compromete-se com os identificadores das transacções; não se substitui literalmente este campo pela raiz da árvore.

As imagens de chave ficam nas entradas do prefixo, na área não podável. São cruciais para detectar gastos duplos e não podem ser podadas.

Esta transacção exemplo tem duas entradas e duas saídas, e foi adicionada à lista de blocos com o carimbo temporal 2020-03-02 19:01:10 UTC.

```
1 print_tx 84799c2fc4c18188102041a74cef79486181df96478b717e8703512c7f7f3349
2 Found in blockchain at height 2045821
3 {
4   "version": 2,
5   "unlock_time": 0,
6   "vin": [ {
7     "key": {
8       "amount": 0,
9       "key_offsets": [ 14401866, 142824, 615514, 18703, 5949, 22840, 5572, 16439,
10      983, 4050, 320
11     ],
12     "k_image": "c439b9f0da76ca0bb17920ca1f1f3f1d216090751752b091bef9006918cb3db4"
13   }
14 }, {
15   "key": {
16     "amount": 0,
17     "key_offsets": [ 14515357, 640505, 8794, 1246, 20300, 18577, 17108, 9824, 581,
18     637, 1023
19   ],
20   "k_image": "03750c4b23e5be486e62608443151fa63992236910c41fa0c4a0a938bc6f5a37"
21 }
22 ],
23 "vout": [ {
24   "amount": 0,
25   "target": {
26     "key": "d890ba9ebfa1b44d0bd945126ad29a29d8975e7247189e5076c19fa7e3a8cb00"
27   }
28 }, {
29   "amount": 0,
30   "target": {
31     "key": "dbec330f8a67124860a9bfb86b66db18854986bd540e710365ad6079c8a1c7b0"
32   }
33 }
```

```
    }
  }
],
"extra": [ 1, 3, 39, 58, 185, 169, 82, 229, 226, 22, 101, 230, 254, 20, 143,
37, 139, 28, 114, 77, 160, 229, 250, 107, 73, 105, 64, 208, 154, 182, 158, 200,
73, 2, 9, 1, 12, 76, 161, 40, 250, 50, 135, 231
],
"rct_signatures": {
  "type": 4,
  "txnFee": 32460000,
  "ecdhInfo": [ {
    "amount": "171f967524e29632"
  }, {
    "amount": "5c2a1a9f54ccf40b"
  } ],
  "outPk": [ "fed8aded6914f789b63c37f9d2eb5ee77149e1aa4700a482aea53f82177b3b41",
    "670e086e40511a279e0e4be89c9417b4767251c5a68b4fc3deb80fdae7269c17" ]
},
"rctsig_prunable": {
  "nbp": 1,
  "bp": [ {
    "A": "98e5f23484e97bb5b2d453505db79caadf20dc2b69dd3f2b3dbf2a53ca280216",
    "S": "b791d4bc6a4d71de5a79673ed4a5487a184122321ede0b7341bc3fdc0915a796",
    "T1": "5d58cfa9b69ecdb2375647729e34e24ce5eb996b5275aa93f9871259f3a1aecd",
    "T2": "1101994fea209b71a2aa25586e429c4c0f440067e2b197469aa1a9a1512f84b7",
    "taux": "b0ad39da006404ccacee7f6d4658cf17e0f42419c284bdca03c0250303706c03",
    "mu": "cacd7ca5404afa28e7c39918d9f80b7fe5e572a92a10696186d029b4923fa200",
    "L": [ "d06404fc35a60c6c47a04e2e43435cb030267134847f7a49831a61f82307fc32",
      "c9a5932468839ee0cda1aa2815f156746d4dce79dab3013f4c9946fce6b69eff",
      "efdae043dcedb79512581480d80871c51e063fe9b7a5451829f7a7824bcc5a0b",
      "56fd2e74ac6e1766cfd56c8303a90c68165a6b0855fae1d5b51a2e035f333a1d",
      "81736ed768f57e7f8d440b4b18228d348dce1eca68f969e75fab458f44174c99",
      "695711950e076f54cf24ad4408d309c1873d0f4bf40c449ef28d577ba74dd86d",
      "4dc3147619a6c9401fec004652df290800069b776fe31b3c5cf98f64eb13ef2c"
    ],
    "R": [ "7650b8da45c705496c26136b4c1104a8da601ea761df8bba07f1249495d8f1ce",
      "e87789fbe99a44554871fcf811723ee350cba40276ad5f1696a62d91a4363fa6",
      "ebf663fe9bb580f0154d52ce2a6dae544e7f6fb2d3808531b0b0749f5152ddbf",
      "5a4152682a1e812b196a265a6ba02e3647a6bd456b7987adff288c5b0b556ec6",
      "dc0dcb2e696e11e4b62c20b6bfc6b182290748c5de254d64bf7f9e3c38fb46c9",
      "101e2271ced03b229b88228d74b36088b40c88f26db8b1f9935b85fb3ab96043",
      "b14aae1d35c9b176ac526c23f31b044559da75cf95bc640d1005bfcc6367040b"
    ]
  } ]
}
```



```
75     ],
76     "a": "4809857de0bd6becdb64b85e9dfbf6085743a8496006b72ceb81e01080965003",
77     "b": "791d8dc3a4ddde5ba2416546127eb194918839ced3dea7399f9c36a17f36150e",
78     "t": "aace86a7a1cbdec3691859fa07fdc83eed9ca84b8a064ca3f0149e7d6b721c03"
79   }
80 ],
81 "MGs": [ {
82   "ss": [ [ "d7a9b87cfa74ad5322eedd1bff4c4dca08bcff6f8578a29a8bc4ad6789dee106",
83     "f08e5dfade29d2e60e981cb561d749ea96ddc7e6855f76f9b807842d1a17fe00"],
84     ["de0a86d12be2426f605a5183446e3323275fe744f52fb439041ad2d59136ea0b",
85     "0028f97976630406e12c54094cbbe23a23fe5098f43bcae37339bfc0c4c74c07"],
86     ["f6eef1f99e605372cb1ec2b3dd4c6e56a550fec071c8b1d830b9fda34de5cc05",
87     "cd98fc987374a0ac993edf4c9af0a6f2d5b054f2af601b612ea118f405303306"],
88     ["5a8437575dae7e2183a1c620efbce655f3d6dc31e64c96276f04976243461e08",
89     "5090103f7f73a33024fbda999cd841b99b87c45fa32c4097cdc222fa3d7e9502"],
90     ["88d34246afbcbcd24d2af2ba29d835813634e619912ea4ca194a32281ac14e0e",
91     "eacdf59478f132dd8dbb9580546f96de194092558ffceeff410ee9eb30ce570e"],
92     ["571dab8557921bbae30bda9b7e613c8a0cff378d1ec6413f59e4972f30f2470d",
93     "5ca78da9a129619299304d9b0318623370023debfdaddcd49c1a338c1f0c50d"],
94     ["ac8dbe6bb28839cf98f02908bd1451742a10c713fdd51319f2d42a58bf1d7507",
95     "7347bf16cba5ee6a6f2d4f6a59d1ed0c1a43060c3a235531e7f1a75cd8c8530d"],
96     ["b8876bd3a5766150f0fbc675ba9c774c2851c04afc4de0b17d3ac4b6de617402",
97     "e39f1d2452d76521cbf02b85a6b626eeb5994f6f28ce5cf81adc0ff2b8adb907"],
98     ["1309f8ead30b7be8d0c5932743b343ef6c0001cef3a4101eae98ffde53f46300",
99     "370693fa86838984e9a7232bca42fd3d6c0c2119d44471d61eee5233ba53c20f"],
100    ["80bc2da5fc5951f2c7406fce37a7aa72ffef9cfa21595b1b68dfab4b7b9f9f0c",
101    "c37137898234f00bce00746b131790f3223f97960eefe67231eb001092f5510c"],
102    ["01c89e07571fd365cac6744b34f1b44e06c1c31cbf3ee4156d08309345fdb20e",
103    "a35c8786695a86c0a4e677b102197a11dad7171dd8c2e1de90d828f050ec00f"]],
104    "cc": "0d8b70c600c67714f3e9a0480f1ffc7477023c793752c1152d5df0813f75ff0f"
105  }, {
106    "ss": [ [ "4536e585af58688b69d932ef3436947a13d2908755d1c644ca9d6a978f0f0206",
107      "9aab6509f4650482529219a805ee09cd96bb439ee1766ced5d3877bf1518370b"],
108      ["5849d6bf0f850fcee7acbef74bd7f02f77ecfaaa16a872f52479ebd27339760f",
109      "96a9ec61486b04201313ac8687eaf281af59af9fd10cf450cb26e9dc8f1ce804"],
110      ["7fe5dcc4d3eff02fca4fb4fa0a7299d212cd8cd43ec922d536f21f92c8f93f00",
111      "d306a62831b49700ae9daad44fcd00c541b959da32c4049d5bdd49be28d96701"],
112      ["2edb125a5670d30f6820c01b04b93dd8ff11f4d82d78e2379fe29d7a68d9c103",
113      "753ac25628c0dada7779c6f3f13980dfc5b7518fb5855fd0e7274e3075a3410c"],
114      ["264de632d9cb867e052f95007dfdf5a199975136c907f1d6ad73061938f49c01",
115      "dd7eb6028d0695411f647058f75c42c67660f10e265c83d024c4199bed073d01"],
116      ["b2ac07539336954f2e9b9cba298d4e1faa98e13e7039f7ae4234ac801641340f",
```

```

117     "69e130422516b82b456927b64fe02732a3f12b5ee00c7786fe2a381325bf3004"],
118     ["49ea699ca8cf2656d69020492cdfa69815fb69145e8f922bb32e358c23cebb0f",
119     "c5706f903c04c7bed9c74844f8e24521b01bc07b8dbf597621ccee3b3afc1d0c"],
120     ["a1faf85aa942ba30b9f0511141fcab3218c00953d046680d36e09c35c04be905",
121     "7b6b1b6fb23e0ee5ea43c2498ea60f4fcf62f70c7e0e905eb4d9afa1d0a18800"],
122     ["785d0993a70f1c2f0ac33c1f7632d64e34dd730d1d8a2fb0606f5770ed633506",
123     "e12777c49ffc3f6c35d27a9ccb3d9b8fed7f0864a880f7bae7399e334207280e"],
124     ["ab31972bf1d2f904d6b0bf18f4664fa2b16a1fb2644cd4e6278b63ade87b6d09",
125     "1efb04fe9a75c01a0fe291d0ae00c716e18c64199c1716a086dd6e32f63e0a07"],
126     ["a6f4e21a27bf8d28fc81c873f63f8d78e017666adbf038da0b83c2ad04ef6805",
127     "c02103455f93c2d7ec4b7152db7de00d1c9e806b1945426b6773026b4a85dd03"]],
128     "cc": "d5ac037bb78db41cf924af713b7379c39a4e13901d3eac017238550a1a3b910a"
129   }],
130   "pseudoUts": [ "b313c1ae9ca06213684fbdefa9412f4966ad192bc0b2f74ed1731381adb7ab58",
131   "7148e7ef5cfd156c62a6e285e5712f8ef123575499ff9a11f838289870522423"]
132 }
133 }

```

Componentes de transacção

- (linha 2) - O comando `print_tx` iria reportar o bloco em que foi encontrada a transacção, o que é replicado aqui com o propósito de demonstração.
- `version` (linha 4) – A versão de serialização da transacção; ‘2’ é a versão usada pelos formatos RingCT, históricos e actuais.
- `unlock_time` (linha 5) – Impede que as saídas sejam gastas antes do limite comum. Um valor inferior a 500 000 000 é uma altura de bloco; um valor a partir daí é um tempo Unix. Zero não acrescenta bloqueio.
- `vin` (linha 6-23) - lista de entradas (existem duas aqui)
- `amount` (linha 8) – Campo de montante visível. É zero nestas entradas RingCT porque o montante está comprometido.
- `key_offsets` (linha 9) – Permitem encontrar na lista de blocos as chaves e os compromissos dos membros do anel. O primeiro índice é absoluto e os seguintes são diferenças relativas. Por exemplo, {7, 11, 15, 20} serializa-se como {7, 4, 4, 5}, pois $7 + 4 + 4 + 5 = 20$. Cada lista deste espécime tem onze elementos, o tamanho histórico; na versão 16 teria dezasseis (secção 6.2.1).
- `k_image` (linha 12) – Imagem de chave da primeira entrada. Neste espécime é a imagem ligada pela MLSAG; nas transacções actuais, a mesma posição do prefixo contém a imagem ligada pela CLSAG.

- **vout** (linhas 24-35) - Lista de saídas (existem duas aqui)
- **amount** (linha 25) – Campo de montante visível, zero numa saída RingCT ordinária.
- **key** (linha 27) – Chave do endereço oculto de destino para a saída $t = 0$, como descrito na secção 4.2. O alvo histórico não tem o *view tag* que passou a ser obrigatório nas saídas v16.
- **extra** (linhas 36–39) – Dados convencionais da carteira. Cada número mostrado é um byte. A etiqueta ‘1’ anuncia a chave pública de transacção rG , de 32 bytes, que permite ao destinatário derivar o segredo partilhado da secção 4.2; por ter tamanho fixo, não leva um comprimento. Segue-se a etiqueta ‘2’, um *nonce* de nove bytes: o primeiro, ‘1’, identifica um ID de pagamento cifrado, e os oito restantes contêm-no. As etiquetas dão significado convencional aos bytes; a fronteira de validação actual é explicada no capítulo 6. Vejam-se [73, 128].
- **rct_signatures** (linhas 40-50) - Primeira parte dos dados de assinatura
- **type** (linha 41) – Tipo RingCT; neste espécime, `RCTTypeBulletproof2` é ‘4’. `RCTTypeFull` e `RCTTypeSimple` são ‘1’ e ‘2’; as transacções de mineiro usam `RCTTypeNull`, ‘0’. O caminho ordinário v16 usa `RCTTypeBulletproofPlus`, ‘6’.
- **txnFee** (linha 42) - Taxa de transacção em texto claro, neste caso 0.00003246 xmr.
- **ecdhInfo** (linhas 43-47) - ‘elliptic curve diffie-hellman Info’: Montante oculto para cada uma das saídas $t \in \{0, \dots, p-1\}$; em que $p = 2$
- **amount** (linha 44) - O campo *amount* com $t = 0$ é descrito na secção 5.3.
- **outPk** (linhas 48-49) - Compromissos para cada saída, secção 5.4.
- **rctsig_prunable** (linhas 51-132) - Segunda parte dos dados da assinatura
- **nbp** (linha 52) – Número de provas de domínio *Bulletproof* neste espécime. A prova única cobre as duas saídas.
- **bp** (linhas 53-80) - elementos da prova *Bulletproof*

$$\Pi_{BP} = (A, S, T_1, T_2, \tau_x, \mu, \mathbb{L}, \mathbb{R}, a, b, t)$$

- **MGs** (linhas 81-129) – Duas assinaturas MLSAG, uma por entrada. Uma transacção v16 teria o vector **CLSAGs**.
- **ss** (linhas 82-103) - Componentes $r_{i,1}$ e $r_{i,2}$ da assinatura MLSAG para a primeira entrada.

$$\sigma_j(\mathbf{m}) = (c_1, r_{1,1}, r_{1,2}, \dots, r_{v+1,1}, r_{v+1,2})$$

- **cc** (linha 104) - Componente c_1 da assinatura MLSAG, para a primeira entrada.
- **pseudoOuts** (linhas 130-131) – Pseudo-compromissos de saída $\tilde{C}_j^a$, como descrito na secção 5.3. A sua soma é igual à soma dos dois compromissos de saída mais o compromisso da taxa fH .

```
src/
cryptonote_
basic/tx_
extra.h
```

APÊNDICE B

Conteúdo de bloco

Neste apêndice mostra-se a estrutura de um bloco, nomeadamente o bloco n° 1582196. Este bloco contém 5 transações, e foi minerado para a lista de blocos com o carimbo no tempo 2018-05-27 21:56:01 UTC, pelo seu mineiro.

```
1 print_block 1582196
2 timestamp: 1527458161
3 previous hash: 30bb9b475a08f2ea6fe07a1fd591ea209a7f633a400b2673b8835a975348b0eb
4 nonce: 2147489363
5 is orphan: 0
6 height: 1582196
7 depth: 2
8 hash: 50c8e5e51453c2ab85ef99d817e166540b40ef5fd2ed15ebc863091ca2a04594
9 difficulty: 51333809600
10 reward: 4634817937431
11 {
12   "major_version": 7,
13   "minor_version": 7,
14   "timestamp": 1527458161,
15   "prev_id": "30bb9b475a08f2ea6fe07a1fd591ea209a7f633a400b2673b8835a975348b0eb",
16   "nonce": 2147489363,
17   "miner_tx": {
18     "version": 2,
19     "unlock_time": 1582256,
```

```

20     "vin": [ {
21         "gen": {
22             "height": 1582196
23         }
24     }
25 ],
26     "vout": [ {
27         "amount": 4634817937431,
28         "target": {
29             "key": "39abd5f1c13dc6644d1c43db68691996bb3cd4a8619a37a227667cf2bf055401"
30         }
31     }
32 ],
33     "extra": [ 1, 89, 148, 148, 232, 110, 49, 77, 175, 158, 102, 45, 72, 201, 193,
34     18, 142, 202, 224, 47, 73, 31, 207, 236, 251, 94, 179, 190, 71, 72, 251, 110,
35     134, 2, 8, 1, 242, 62, 180, 82, 253, 252, 0
36 ],
37     "rct_signatures": {
38         "type": 0
39     }
40 },
41     "tx_hashes": [ "e9620db41b6b4e9ee675f7bfdeb9b9774b92aca0c53219247b8f8c7aecf773ae",
42                   "6d31593cd5680b849390f09d7ae70527653abb67d8e7fdca9e0154e5712591bf",
43                   "329e9c0caf6c32b0b7bf60d1c03655156bf33c0b09b6a39889c2ff9a24e94a54",
44                   "447c77a67adeb5dbf402183bc79201d6d6c2f65841ce95cf03621da5a6bffeefc",
45                   "90a698b0db89bbb0704a4ffa4179dc149f8f8d01269a85f46ccd7f0007167ee4"
46 ]
47 }

```

Componentes do bloco

- (linhas 2-10) - Informação dada pelo software cliente, isto não faz parte do bloco.
- **is orphan** (linha 5) - Significa que este bloco foi órfão. Os *nodes* usualmente guardam todos os ramos durante uma situação de garfo de rede, e descartam ramos desnecessários quando uma outra dificuldade cumulativa vencedora emerge, o que torna os blocos do outro garfo órfãos.
- **depth** (linha 7) - A profundidade de qualquer dado bloco, é a distância desse bloco ao último bloco na lista (distância ao bloco mais recente).
- **hash** (linha 8) - A ID deste bloco.

- **difficulty** (linha 9) - A dificuldade não é guardada num bloco, desde que os utilizadores podem calcular *todas* as dificuldades de bloco a partir dos carimbos no tempo e com as regras na secção 7.2.
- **major_version** (linha 12) - Corresponde á versão do protocolo usada para verificar este bloco.
- **minor_version** (linha 13) - Originalmente destinada para ser um mecanismo entre mineiros, agora simplesmente repete a **major_version**.
- **timestamp** (linha 14) - Uma representação com um inteiro do carimbo no tempo em UTC, posto pelo mineiro.
- **prev_id** (linha 15) - A ID do bloco anterior.
- **nonce** (linha 16) - A nonce usada pelo mineiro do bloco para passar o alvo de dificuldade. É possível verificar que esta prova de trabalho é válida para esta nonce.
- **miner_tx** (linhas 17-40) - A transacção de mineiro
- **version** (linha 18) - O formato/era da transacção; version ‘2’ corresponde a RingCT.
- **unlock_time** (linha 19) - A transacção de mineiro não pode ser gasta até ao bloco 1582256, até que mais 59 blocos tenham sido minerados (é um tempo de bloqueio de 60 blocos pois não pode ser gasto até que 60 intervalos de tempo entre blocos tenham passado, e.g. $2 * 60 = 120$ minutos).
- **vin** (linhas 20-25) - Entradas para as transacção de mineiro, não existem, pois a transacção de mineiro é usada para gerar o subsídio de bloco e coleccionar taxas de transacção.
- **gen** (linha 21) - Forma curta para ‘gerar’.
- **height** (linha 22) - A altura de bloco. Cada altura de bloco só pode gerar o subsídio de bloco, uma vez.
- **vout** (linhas 26-32) - Saídas da transacção de mineiro.
- **amount** (linha 27) - Montante da transacção de mineiro, contém o subsídio do bloco e as taxas de transacção deste bloco. Usa a unidade atómica *piconero*.
- **key** (linha 29) - Endereço oculto que é proprietário do montante na transacção de mineiro.
- **extra** (linhas 33-36) - Informação extra para a transacção de mineiro, inclui a chave pública de transacção.
- **type** (linha 38) - Tipo de transacção, neste caso ‘0’ para **RCTTypeNull**.
- **tx_hashes** (linhas 41-46) - Todas as ID’s de transacção incluídas neste bloco (mas não a ID da transacção de mineiro, que é :
06fb3e1cf889bb972774a8535208d98db164394ef2b14ecfe74814170557e6e9).

APÊNDICE C

Bloco de génese

Neste apêndice mostra-se a estrutura do bloco génese, o bloco tem zero transacções, só envia o primeiro subsídio de bloco para `thankful_for_today` [118]). O fundador de Monero não adicionou um carimbo no tempo, talvez uma marca de Bytecoin, a moeda da qual provêm o código fonte de Monero. Mais sobre a história de Bytecoin pode ser lida aqui [12], e aqui [123].

O bloco 1 foi adicionado á rede com o carimbo no tempo 2014-04-18 10:49:53 UTC, assume-se que o bloco 0, de génese tenha sido minerado no mesmo dia. Isto corresponde com a data de lançamento anunciada por `thankful_for_today` [118].

```
1 print_block 0
2 timestamp: 0
3 previous hash: 0000000000000000000000000000000000000000000000000000000000000000
4 nonce: 10000
5 is orphan: 0
6 height: 0
7 depth: 1580975
8 hash: 418015bb9ae982a1975da7d79277c2705727a56894ba0fb246adaabb1f4632e3
9 difficulty: 1
10 reward: 17592186044415
11 {
12   "major_version": 1,
13   "minor_version": 0,
14   "timestamp": 0,
```

```
15  "prev_id": "0000000000000000000000000000000000000000000000000000000000000000",
16  "nonce": 10000,
17  "miner_tx": {
18    "version": 1,
19    "unlock_time": 60,
20    "vin": [ {
21      "gen": {
22        "height": 0
23      }
24    }
25  ],
26  "vout": [ {
27    "amount": 17592186044415,
28    "target": {
29      "key": "9b2e4c0281c0b02e7c53291a94d1d0cbff8883f8024f5142ee494ffbbd088071"
30    }
31  }
32 ],
33  "extra": [ 1, 119, 103, 170, 252, 222, 155, 224, 13, 207, 208, 152, 113, 94, 188,
34  247, 244, 16, 218, 235, 197, 130, 253, 166, 157, 36, 162, 142, 157, 11, 200, 144,
35  209
36 ],
37  "signatures": [ ]
38 },
39  "tx_hashes": [ ]
40 }
```

Componentes do bloco de g nese

Como foi usado o mesmo software para imprimir o bloco de g nese e o bloco do ap ndice B, a estrutura parece ser b sicamente a mesma. S o dadas algumas diferen as.

- **difficulty** (linha 9) - A dificuldade   1, o que significa que qualquer nonce serve.
- **timestamp** (linha 14) - O bloco de g nese n o tem um carimbo no tempo significativo.
- **prev_id** (linha 15) - Usam-se 32 bytes a zero para o ID anterior, por conven  o.
- **nonce** (linha 16) - $n = 10000$ por conven  o.
- **amount** (linha 27) - Isto corresponde exactamente ao c lculo para o primeiro subs dio de bloco (17.592186044415 xmr) da sec  o 7.3.1.

- **key** (linha 29) - Os primeiros moneroj foram dispersados para o fundador de Monero `thankful_for_today`.
- **extra** (linhas 33-36) - Usa-se a codificação discutida no apêndice o campo **extra** da transacção de mineiro, só contém uma chave pública de transacção.
- **signatures** (linha 37) - Não existem assinaturas no bloco de génese. Isto aparece aqui devido á função `print_block`. O mesmo é verdade para `tx_hashes` na linha 39.

APÊNDICE D

Uma nota sobre notações

Neste apêndice explica-se a notação dentro de bulletproofs. Existem pois duas operações básicas $a + b = c$ é sobre os escalares em F_p , e $A + B = C$ é sobre todos os pontos de CE disponíveis em ed25519.

sobre os pontos de CE :

$$a\underline{B} = \{aB_1, aB_2, aB_3, \dots, aB_n\}$$

em que a operação $\circ$ não é relevante (hadamard).

$$\langle \underline{a}, \underline{B} \rangle = \sum_{j=1}^n a_j B_j$$

existem otimizações específicas para o seguinte :

$$\langle \underline{a}, \underline{B} \rangle + \langle \underline{b}, \underline{C} \rangle + \langle \underline{c}, \underline{D} \rangle \dots$$

para mais informações sobre algoritmos de factorização sobre pontos de CE (Pippenger et al.) veja-se[].

sobre os escalares :

$$a\underline{b} = \{ab_1, ab_2, ab_3, \dots, ab_n\} = a\underline{1} \circ \underline{b}$$

$$a\underline{1} = \underline{a}$$

$$\underline{a}^n = \{a^0, a^1, a^2, \dots, a^{n-1}\}$$

$$\underline{a} \circ \underline{b} = \{a_1 b_1, a_2 b_2, a_3 b_3, \dots, a_n b_n\}$$

$$\langle \underline{a}, \underline{b} \rangle = \sum_{j=1}^n a_j b_j$$

$$\langle \underline{a}, \underline{b} \rangle = \langle \underline{1}, \underline{a} \circ \underline{b} \rangle$$

$$\langle \underline{a}, \underline{b} \rangle + \langle \underline{a}, \underline{c} \rangle = \langle \underline{a}, \underline{b} + \underline{c} \rangle$$

para extraír $\delta(y, z)$ de :

$$\begin{aligned} vz^2 &= \langle \underline{a}_L, \underline{2}^n \rangle z^2 \\ 0z &= \langle \underline{a}_L - \underline{1} - \underline{a}_R, \underline{y}^n \rangle z \\ 0 &= \langle \underline{a}_L \circ \underline{a}_R, \underline{y}^n \rangle \end{aligned}$$

$$vz^2 + 0 + 0 = \langle \underline{a}_L, \underline{2}^n \rangle z^2 + \langle \underline{a}_L - \underline{1} - \underline{a}_R, \underline{y}^n \rangle z + \langle \underline{a}_L \circ \underline{a}_R, \underline{y}^n \rangle$$

$$vz^2 = \langle \underline{a}_L, \underline{2}^n \rangle z^2 + \langle \underline{a}_L, \underline{y}^n \rangle z + \langle -\underline{1}, \underline{y}^n \rangle z + \langle -\underline{a}_R, \underline{y}^n \rangle z + \langle \underline{a}_L, \underline{a}_R \circ \underline{y}^n \rangle$$

$$vz^2 + \langle \underline{1}, \underline{y}^n \rangle z = \langle \underline{a}_L, \underline{2}^n \rangle z^2 + \langle \underline{a}_L, \underline{y}^n \rangle z + \langle -\underline{1}, \underline{a}_R \circ \underline{y}^n \rangle z + \langle \underline{a}_L, \underline{a}_R \circ \underline{y}^n \rangle$$

$$vz^2 + \langle \underline{1}, \underline{y}^n \rangle z = \langle \underline{a}_L, \underline{2}^n z^2 + \underline{y}^n z + \underline{a}_R \circ \underline{y}^n \rangle + \langle -\underline{1}, \underline{a}_R \circ \underline{y}^n \rangle z$$

$$\begin{aligned} vz^2 + \langle \underline{1}, \underline{y}^n \rangle z - \langle \underline{1}, \underline{2}^n z^2 + \underline{y}^n z \rangle z &= \langle -\underline{1}, \underline{2}^n z^2 + \underline{y}^n z \rangle z \\ &\quad + \langle \underline{a}_L, \underline{2}^n z^2 + \underline{y}^n z + \underline{a}_R \circ \underline{y}^n \rangle \\ &\quad + \langle -\underline{1}, \underline{a}_R \circ \underline{y}^n \rangle z \end{aligned}$$

$$\begin{aligned} vz^2 + \langle \underline{1}, \underline{y}^n \rangle z - \langle \underline{1}, \underline{y}^n z \rangle z - \langle \underline{1}, \underline{2}^n z^2 \rangle z &= \langle -z\underline{1}, \underline{2}^n z^2 + \underline{y}^n z + \underline{a}_R \circ \underline{y}^n \rangle \\ &\quad + \langle \underline{a}_L, \underline{2}^n z^2 + \underline{y}^n z + \underline{a}_R \circ \underline{y}^n \rangle \end{aligned}$$

$$vz^2 + (z - z^2)\langle \underline{1}, \underline{y}^n \rangle - z^3\langle \underline{1}, \underline{2}^n \rangle = \langle \underline{a}_L - z\underline{1}, \underline{2}^n z^2 + \underline{y}^n z + \underline{a}_R \circ \underline{y}^n \rangle$$

$$vz^2 + \underbrace{(z - z^2)\langle \underline{1}, \underline{y}^n \rangle - z^3\langle \underline{1}, \underline{2}^n \rangle}_{\delta(y, z)} = \langle \underline{a}_L - z\underline{1}, \underline{y}^n \circ (\underline{1}z + \underline{a}_R) + \underline{2}^n z^2 \rangle$$